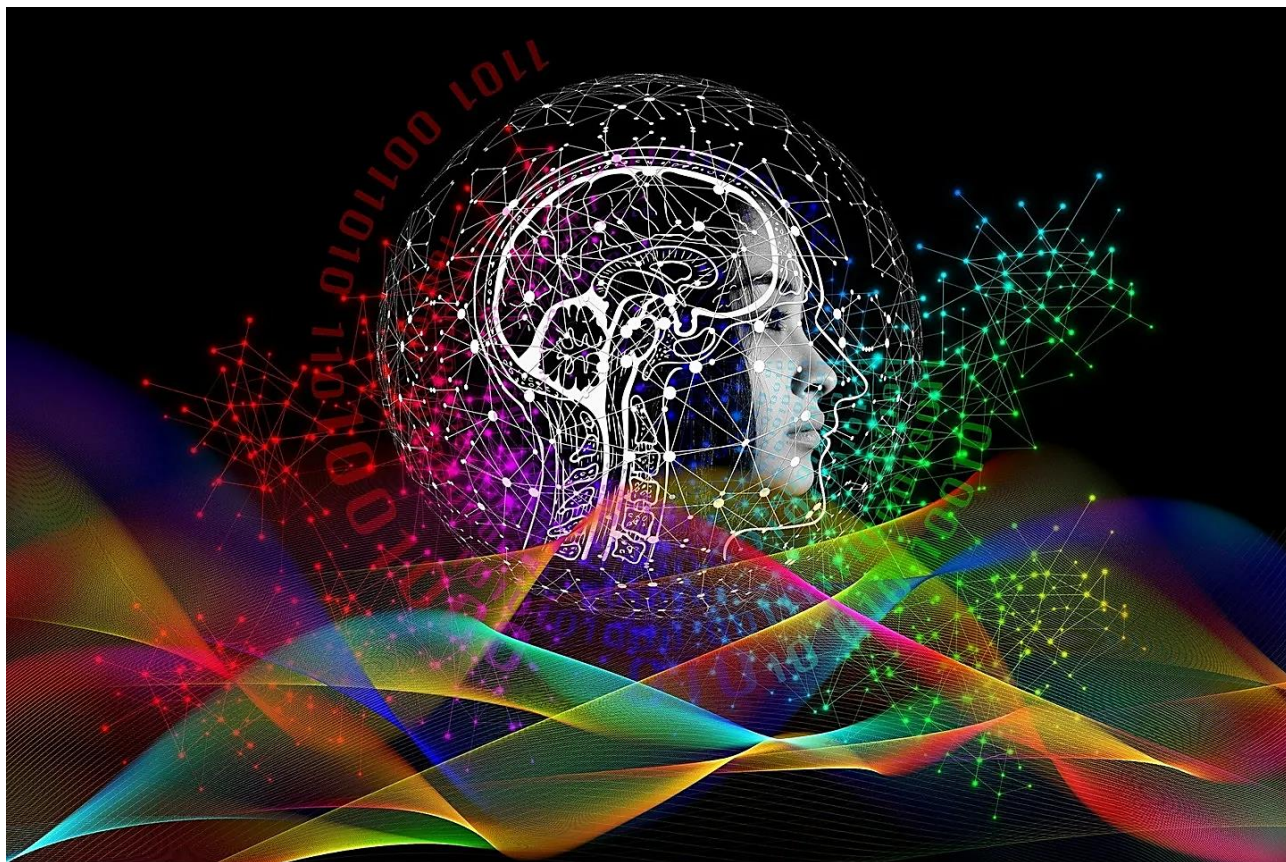


Пензенский государственный университет

кафедра "Компьютерные технологии"



Горбаченко В. И.

МАШИННОЕ ОБУЧЕНИЕ

КОНСПЕКТ ЛЕКЦИЙ

Пенза, 2025

СОДЕРЖАНИЕ

ЧАСТЬ 1. КЛАССИЧЕСКОЕ МАШИННОЕ ОБУЧЕНИЕ	5
1. ОСНОВНЫЕ ПОНЯТИЯ И СОСТАВНЫЕ ЧАСТИ МАШИННОГО ОБУЧЕНИЯ	5
Литература к разделу 1	13
2. ПОДГОТОВКА ДАННЫХ ДЛЯ МАШИННОГО ОБУЧЕНИЯ.....	15
2.1. Виды операций подготовки данных для машинного обучения	15
2.2. Заполнение пропусков в данных	17
2.3. Очистка данных	21
2.4. Кодирование качественных признаков	22
2.5. Масштабирование признаков.....	23
2.6. Отбор признаков.....	25
2.7. Декорреляция признаков	27
2.8. Понижение размерности признаков методом главных компонент	32
2.9. Ядерный метод главных компонент	36
2.10. Использование автокодировщиков для преобразования	41
признаков	41
2.11. Аугментация и сэмплинг	43
Литература к разделу 2	48
3. РЕГРЕССИЯ.....	52
3.1. Линейная регрессия.....	52
3.1.1. Линейная регрессия. Определение параметров регрессии методом наименьших квадратов	52
3.1.2. Оценка качества регрессионной модели	59
3.1.3. Регуляризованная регрессия.....	64
3.2. Логистическая регрессия	65
3.3. Нелинейная регрессия.....	70
Литература к разделу 3	71
4. КЛАССИФИКАЦИЯ И КЛАСТЕРИЗАЦИЯ	73
4.1. Байесовский алгоритм классификации	73
4.1.1. Теорема Байеса	73
4.1.2. Наивный байесовский классификатор.....	74
4.2. Алгоритм ближайших соседей.....	77
4.3. Деревья решений	79
4.3.1. Понятие дерева решений	79

4.3.2. Построение дерева решений.....	83
4.3.3. Регуляризация деревьев решений	86
4.4. Метод опорных векторов.....	88
4.4.1. Случай линейно разделимой выборки.....	88
4.4.2. Метод опорных векторов с мягким зазором	95
4.4.3. Ядерный метод опорных векторов.....	98
4.5. Кластеризация.....	100
4.5.1. Постановка задачи кластеризации	100
4.5.2. Меры близости, используемые в кластеризации.....	102
4.5.3. Алгоритмы кластеризации.....	105
4.6. Оценивание классификаторов.....	110
4.6.1. Перекрестная проверка моделей	110
4.6.2. Метрики качества в задачах классификации	111
Литература к разделу 4	120
5. АНСАМБЛИ МОДЕЛЕЙ МАШИННОГО ОБУЧЕНИЯ	123
5.1. Предпосылки объединения моделей машинного обучения в ансамбли.....	123
5.2. Классификаторы с голосованием	124
5.3. Бэггинг и перестановка.....	125
5.4. Бустинг	127
5.4.1. AdaBoost	127
5.4.2. Градиентный бустинг	129
Литература к разделу 5	131
6. МНОГОСЛОЙНЫЙ ПЕРСЕПТРОН.....	133
6.1. Математическая модель искусственного нейрона.....	133
6.2. Функции активации.....	134
6.3. Инициализация нейронных сетей.....	142
6.4. Структура многослойного персептрона.....	145
6.5. Обучение многослойного персептрона.....	149
6.5.1. Алгоритм обратного распространения ошибки.....	149
6.5.2. Представление данных нейронных сетей в форме тензоров.....	156
6.5.3. Понятие автоматического дифференцирования	158
6.5.4. Общие подходы к градиентным алгоритмам обучения нейронных сетей.....	161
6.5.5. Алгоритмы обучения первого порядка.....	164
6.5.6. Алгоритмы обучения второго порядка.....	169
а) алгоритм сопряженных градиентов	169
б) квазиньютоновские алгоритмы	172

в) алгоритмы Левенберга-Марквардта и доверительных областей	176
6.5.7. Регуляризация в обучении нейронных сетей.....	180
6.6. Пакетная нормализация	184
Литература к разделу 6	188

ЧАСТЬ 1. КЛАССИЧЕСКОЕ МАШИННОЕ ОБУЧЕНИЕ

1. ОСНОВНЫЕ ПОНЯТИЯ И СОСТАВНЫЕ ЧАСТИ МАШИННОГО ОБУЧЕНИЯ

В настоящее время много говорят об искусственном интеллекте. Это весьма расплывчатый термин, так как нет даже общепринятого определения естественного (человеческого) интеллекта. Например, в [1] приведен обзор более 70 определений искусственного интеллекта. Примем определение искусственного интеллекта, приведенное в ГОСТ Р 59277-2020 "Системы искусственного интеллекта. Классификация систем искусственного интеллекта" [2]: "*Искусственный Интеллект* (artificial intelligence) — это комплекс технологических решений, позволяющий имитировать когнитивные функции человека (включая самообучение, поиск решений без заранее заданного алгоритма и достижение инсайта) и получать при выполнении конкретных практически значимых задач обработки данных результаты, сопоставимые, как минимум, с результатами интеллектуальной деятельности человека. *Примечание* — Комплекс технологических решений включает в себя информационно-коммуникационную инфраструктуру, программное обеспечение (в том числе, в котором используются методы машинного обучения), процессы и сервисы по обработке данных, анализу и синтезу решений".

В искусственном интеллекте принято выделять *сильный ИИ* (Strong AI), или общий искусственный интеллект (Artificial general intelligence, AGI) — интеллектуальный алгоритм, способный решать широкий спектр интеллектуальных задач как минимум наравне с человеческим разумом" и *слабый ИИ* (Weak AI), или узкий ИИ (Narrow AI), *прикладной ИИ* — интеллектуальный алгоритм, имитирующий человеческий разум в решении конкретных узкоспециализированных задач. Сильный ИИ в настоящее время — это, скорее цель для науки. В области прикладного ИИ достигнуты огромные результаты: распознавание лиц, общение на естественном языке, поиск информации и т.п.).

Структура современного прикладного ИИ показана на рис. 1.1 Основным направлением современного искусственного интеллекта является *машинное обучение* (Machine Learning) — технологии автоматического обучения алгоритмов ИИ на примерах, в результате чего качество работы алгоритмов повышается [3]. Под качеством работы алгоритмов понимаются определенные числовые характеристики — метрики качества.

Важнейшей (но не единственной) частью современного машинного обучения являются *искусственные нейронные сети* (Artificial Neural Networks) — математические модели, состоящие из слоёв "нейронов", построенных по принципу организации и функционирования биологических нейронных сетей [2].

Новым направлением нейронных сетей являются *глубокие нейронные сети* (*Deep Neural Networks*) — сети, содержащие большое число слоев.

С искусственным интеллектом пересекается *наука о данных* (*Data Science*) — раздел информатики, изучающий проблемы анализа, обработки и представления данных в цифровой форме. Data Science объединяет методы по обработке данных в условиях больших объёмов и высокого уровня параллелизма, статистические методы, методы интеллектуального анализа данных и приложения искусственного интеллекта для работы с данными, а также методы проектирования и разработки баз данных [4].

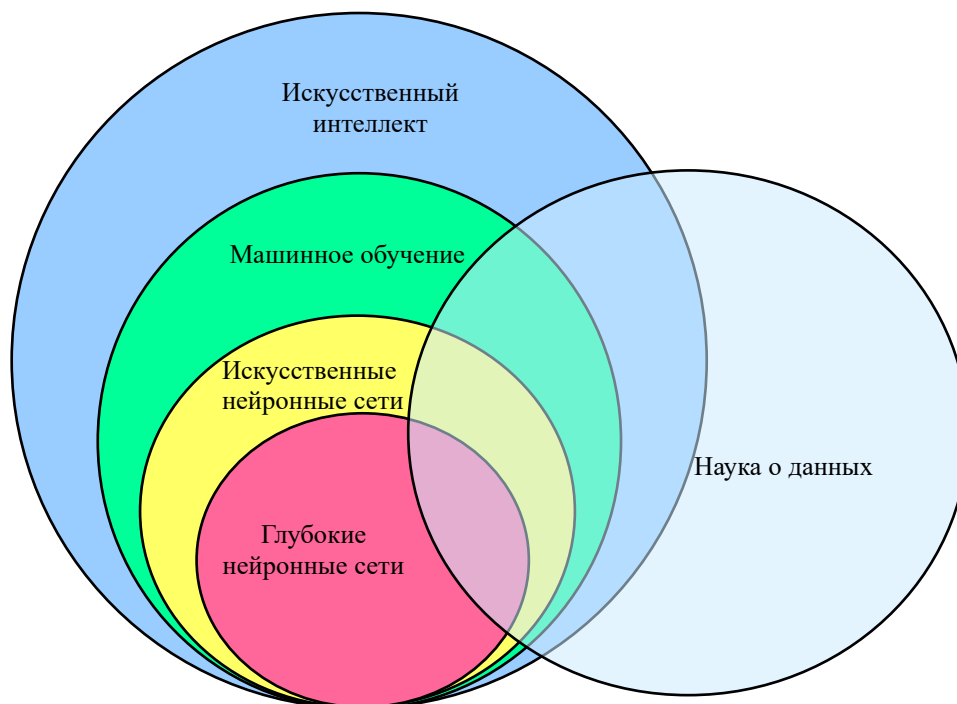


Рис. 1.1. Структура прикладного искусственного интеллекта

Существуют различные определения *машинного обучения*. Например, в [3] дается такое определение: "Машинным обучением называется систематическое обучение алгоритмов и систем, в результате которого их знания или качество работы возрастают по мере накопления опыта".

В классической работе [5] дано более строгое определение: компьютерная программа обучается на основе опыта E по отношению к некоторому классу задач T и меры качества P , если качество решения задач из T , измеренное на основе P , улучшается с приобретением опыта E .

Машины не учатся как люди [6]. Машинное обучение заключается в поиске математической модели, применение которой к набору входных данных (обучающему набору) дает желаемые результаты. Эта модель должна также обладать обобщающей способностью — формировать правильные выходные данные для входных данных, отличающихся от обучающих данных, при условии, что входные данные имеют такое же или близкое статистическое распределение,

что и обучающие данные. Пока машинное обучение уступает обучению людей и животных. Например, если модель машинного обучения не обучена распознавать изображения при их повороте, то она не распознает изображение при повороте, а человек легко решает эту задачу. Термин "машинное обучение" был придуман в фирме IBM в 1959 году из маркетинговых соображений для привлечения клиентов и талантливых сотрудников.

Машинное обучение делится на следующие основные виды:

Обучение с учителем, или обучение по прецедентам (supervised learning). Обучение с учителем называют еще *обучением на размеченных данных*. При *обучении с учителем* имеется набор обучающих (тренировочных) примеров (learning set, или training set). Обучающие примеры — это наборы признаков, факторов (feature), описывающих отдельные объекты, например, показатели, описывающие состояние больного, оцифрованная картинка. Каждому примеру соответствует известный ответ (метка, отклик, реакция, целевое значение). Задача обучения — так настроить на имеющихся примерах модель, чтобы на тех данных, на которых модель не обучалась, она выдавала достаточно точный ответ. Естественно, что обучающий набор и данные, на которых будет применяться модель, должны быть из одного множества.

Основными задачами, решаемыми с помощью обучения с учителем, являются *задачи классификации и регрессии*. При решении *задачи классификации* необходимо отнести объект к одному из заранее известных классов. Типичные задачи классификации — задачи распознавания образов. Например, множество фотографий необходимо разделить на фотографии определенных объектов; множество клиентов банка разделить на надежных заемщиков, которым можно давать кредит, и ненадежных; разделить пациентов больницы на подозреваемых в заболевании и здоровых. При решении задач регрессии необходимо предсказать значение некоторой функции, например, по характеристикам квартиры оценить ее стоимость, предсказать курс акций.

Обучение без учителя (unsupervised learning), или обучением на неразмеченных данных — это обучение на неразмеченных данных, когда целевые значения неизвестны. Типичная задача обучения без учителя — это *кластеризация*, когда необходимо разделить объекты на заранее неизвестные группы (кластеры) в зависимости от близости их признаков. Например, пользователей мобильного оператора можно разделить на кластеры в зависимости от особенностей пользователей. Другая задача обучения без учителя — снижение размерности, то есть количества признаков, описывающих объект.

Обучение с частичным привлечением учителя (semi-supervised learning) использует обучающий набор данных, включающий как размеченные, так и не размеченные данные. Разметка данных представляет собой трудоемкий процесс,

выполняемый, как правило людьми. Известны различные подходы к снижению трудоемкости разметки данных. Например, при активном обучении (active learning) модель сначала обучается на относительно небольшом наборе размеченных данных. Затем модель пытается классифицировать примеры неразмеченной части набора. Если классификация некоторых примеров вызывает у модели проблемы, то модель запрашивает разметку этих примеров человеком. Модель обучается, используя расширенное множество размеченных примеров и процесс многократно повторяется.

В обучении с подкреплением (reinforcement learning) модель "живет" в некотором окружении и выполняет некоторые действия. Разные действия приносят разные вознаграждения. Цель алгоритма обучения с подкреплением — построить поведение, приносящее максимальные вознаграждения.

Определим основные понятия машинного обучения согласно [7]. Математически задача обучения по прецедентам (с учителем) описывается следующим образом. Задано множество объектов X , множество допустимых ответов Y , и существует целевая функция (target function) $y^* : X \rightarrow Y$, значения которой $y_i = y^*(x_i)$ известны только на конечном подмножестве объектов $\{x_1, \dots, x_l\} \subset X$. Пары "объект-ответ" (x_i, y_i) называются *прецедентами* (обучающими примерами). Совокупность пар $X^l = (x_i, y_i)_{i=1}^l$ называется *обучающей выборкой* (training sample, training set, learning set).

Задача обучения по прецедентам заключается в том, чтобы по выборке X^l восстановить зависимость y^* , то есть построить *решающую функцию* (decision function) $a : X \rightarrow Y$, которая приближала бы целевую функцию $y^*(x)$, причём не только на объектах обучающей выборки, но и на всём множестве X .

Решающая функция a должна допускать эффективную компьютерную реализацию; по этой причине в [7] ее называют алгоритмом.

Признак (feature) f объекта x — это результат измерения некоторой характеристики объекта. Формально признаком называется отображение $f : X \rightarrow D_f$, где D_f — множество допустимых значений признака. В зависимости от природы множества D_f признаки делятся на несколько типов.

- Если $D_f = \{0, 1\}$, то f — *бинарный признак*, например, 1 соответствует признаку "хороший", а 0 — "плохой";
- Если D_f — конечное множество, то f — *номинальный (качественный) признак*, например, "красный", "зеленый";

- Если D_f — конечное упорядоченное множество, то f — *порядковый признак*, или упорядоченный качественный признак, когда между признаками установлено отношение порядка, например, "малый размер", "средний размер", "большой размер";
- Если $D_f = \mathbb{R}$, где $\mathbb{R}$ — множество вещественных чисел, f — *количественный признак*.

Рассмотрим модели для структурированных данных, представленных в форме таблицы. Пусть имеется набор признаков $f_1, \dots, f_n$. Вектор $[f_1(x), \dots, f_n(x)]$ называют *признаковым описанием объекта* $x \in X$. Совокупность признаковых описаний всех объектов выборки X^l , записанную в виде таблицы размера $l \times n$, называют *матрицей объектов-признаков*. В математической статистике такую матрицу принято называть *матрицей плана* (*design matrix*) [8].

$$F = \begin{bmatrix} f_1(x_1) & \cdots & f_n(x_1) \\ \cdots & \cdots & \cdots \\ f_1(x_l) & \cdots & f_n(x_l) \end{bmatrix}.$$

В зависимости от природы множества допустимых ответов Y задачи обучения по прецедентам делятся на следующие типы.

Если $Y = \{1, \dots, M\}$, то это *задача классификации* (classification) на M непересекающихся классов. В этом случае алгоритм $a(x)$ должен давать ответ на вопрос "какому классу принадлежит x ?". В некоторых приложениях классы называют *образами* и говорят о *задаче распознавания образов* (pattern recognition).

Если $Y = \mathbb{R}$, где $\mathbb{R}$ — множество действительных чисел, то это задача *регрессионного анализа, восстановления регрессии* (regression estimation).

Задачи прогнозирования (forecasting) являются частными случаями классификации или восстановления регрессии, когда $x \in X$ — описание прошлого поведения объекта x , а $y \in Y$ — описание некоторых характеристик его будущего поведения.

Моделью алгоритма a называется параметрическое семейство функций $g(x, \Theta): X \rightarrow Y$, где Θ — параметры.

Процесс подбора оптимальной функции g и оптимальных параметров Θ по обучающей выборке называют *обучением* (learning, training), или *настройкой* (fitting, tuning) *алгоритма a* .

В задачах обучения по прецедентам чётко различаются два этапа: этап обучения по обучающей выборке и этап применения метода для новых объектов. Этап обучения наиболее сложен.

Функция потерь (loss function) — это неотрицательная функция $L(a, x)$, характеризующая величину ошибки алгоритма a на объекте x . Если $L(a, x) = 0$, то ответ $a(x)$ называется корректным.

Функционал качества (функционал ошибки) алгоритма a на выборке X^l определяется следующим образом

$$Q(a, X^l) = \frac{1}{l} \sum_{i=1}^l L(a, x_i). \quad (1.1)$$

Функционал Q называют также *функционалом средних потерь*, или *эмпирическим риском*, так как он вычисляется по эмпирическим (экспериментальным данным) данным. На практике часто функционал качества называют функцией потерь, что нарушает математическую строгость определения функционала качества.

Функция потерь, принимающая только значения 0 и 1, называется *бинарной*. В этом случае $L(a, x) = 1$ означает, что алгоритм a допускает ошибку на объекте x , а функционал Q называется *частотой ошибок алгоритма a на выборке X^l* .

Наиболее часто используются следующие функции потерь при $Y \subseteq \mathbb{R}$:

- $L(a, x) = [a(x) \neq y^*(x)]$ — *индикатор ошибки*, обычно применяется в задачах классификации (здесь квадратные скобки переводят логическое значение в число по правилу $[ложь] = 0$, $[истина] = 1$);
- $L(a, x) = |a(x) - y^*(x)|$ — *отклонение от правильного ответа*; функционал Q называется *средней ошибкой алгоритма a на выборке X^l* ;
- $L(a, x) = (a(x) - y^*(x))^2$ — *квадратичная функция потерь*; функционал Q называется *средней квадратичной ошибкой алгоритма a на выборке X^l* .

Классический метод обучения, называемый *минимизацией эмпирического риска* (empirical risk minimization, ERM), заключается в том, чтобы найти в заданной модели A алгоритм a , доставляющий минимальное значение функционалу качества Q на заданной обучающей выборке X^l

$$\mu(X^l) = \arg \min_{a \in A} Q(a, X^l).$$

В машинном обучении возможны недообучение и переобучение. *Недообучение* (англ. underfitting) — негативное явление, при котором алгоритм обучения не обеспечивает достаточно малой величины ошибки на обучающей выборке. Недообучение возникает при использовании недостаточно сложных моделей. Когда качество работы алгоритма на новых примерах, не входящих в обучающую выборку, оказывается существенно хуже, чем на обучающей выборке, говорят об эффекте *переобучения* (overtraining), или *переподгонки* (overfitting). При

решении практических задач с этим явлением приходится сталкиваться очень часто. *Обобщающая способность* (generalization ability) метода характеризует качество работы алгоритма на данных, не вошедших в обучающую выборку.

Рассмотрим простейшую задачу полиномиальной аппроксимации данных полиномами различной степени M (рис. 1.2).

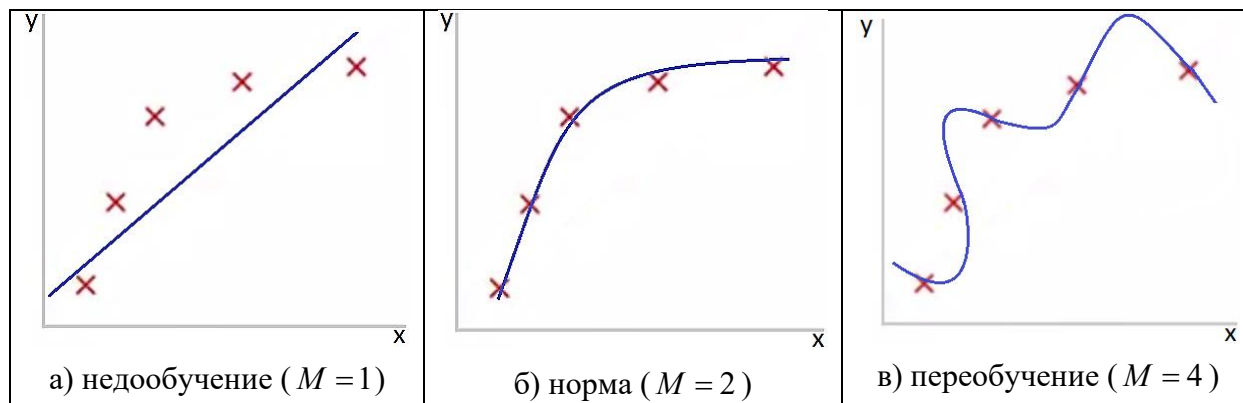


Рис. 1.2. Иллюстрация недообучения и переобучения

Как видно из рис. 1.2, данные не подчиняются линейной зависимости и модель недообучается. Полином второй степени хорошо описывает данные. И в то же время не повторяет мелкие особенности данных. Полином четвертой степени слишком сложен и повторяет все особенности конкретного набора данных. Для близкого другого набора модель даст плохой результат.

Оценивать обобщающую способность алгоритма следует на тех данных, на которых алгоритм не обучался. Для этого из исходных данных выделяют два набора данных:

- *обучающий набор* (training set), которое используется для настройки параметров алгоритма;
- *тестовый, или контрольный набор* (test set), которое используется для оценки обобщающей способности алгоритма.

На обучающем множестве наборе алгоритм обучается, а затем на тестовом наборе вычисляются показатели качества работы алгоритма.

При относительно небольшом размере обучающего набора невозможно выделить под тестовый набор достаточное количество примеров, обеспечивающих достоверные оценка качества алгоритма. Тогда используется *скользящий контроль* (cross-validation). Обучающий набор разбивается на блоки равной (или почти равной) длины, каждый блок по очереди становится тестовым набором, а объединение всех остальных блоков — обучающим. Для каждой контрольной выборки вычисляется эмпирический риск, затем вычисляется средний эмпирический риск по всем контрольным наборам, который характеризует обобщающую способность алгоритма.

Ошибку обобщения модели можно рассматривать как сумму двух ошибок (см., например, [9]): смещения и дисперсии. *Смещение* (Bias) — часть ошибки обобщения, связанная с неверными предположениями в алгоритме обучения, например, считают, что данные линейные, когда они на самом деле квадратичные. Модель с высоким смещением, скорее всего, недообучится на обучаемых данных. Вторая составляющая — *дисперсия* (Variance) — объясняется чрезмерной чувствительностью модели к небольшим изменениям в обучающих данных. Модель со многими параметрами (например, нейронная сеть с большим количеством весов), вероятно, будет иметь высокую дисперсию и потому может переобучаться.

Иногда выделяют еще *неустранимую погрешность*, которая является следствием низкого качества обучающих данных. Например, обучающая выборка неточно отражает существенные особенности исследуемого процесса.

Возрастание сложности модели обычно увеличивает ее дисперсию и уменьшает смещение. И наоборот, сокращение сложности модели увеличивает ее смещение и уменьшает дисперсию. Это называется дилеммой смещения-дисперсии (рис. 1.3). Добиваться компромисса между смещением и дисперсией, как правило, весьма сложно.

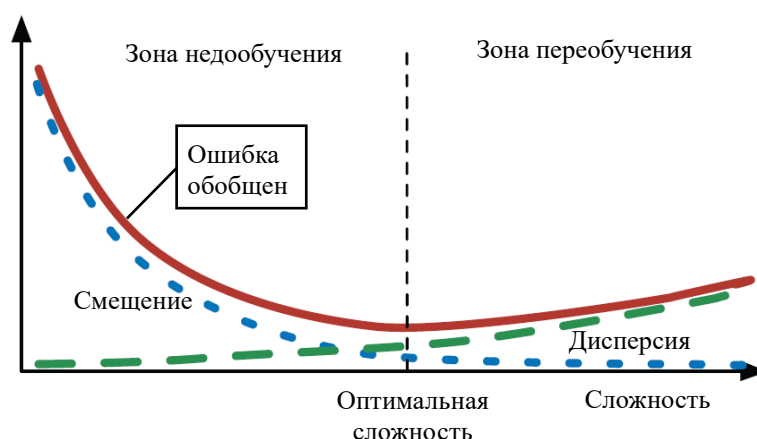


Рис. 1.3. Дилемма смещение-дисперсия

Для преодоления переобучения используется *регуляризация* — это введение ограничений, накладываемых на алгоритм машинного обучения. Например, в функционал качества вводится член, ограничивающий рост параметров алгоритма. При использовании итерационных алгоритмов обучения, например в нейронных сетях, часто используется *ранняя остановка обучения* (early stopping). Для этого кроме обучающего и тестового наборов выделяют *валидационный*, или *проверочный набор* (validation set), на котором на каждой итерации обучения вычисляется ошибка алгоритма. В начале процесса обучения ошибки на обучающем и валидационном наборах уменьшается. Затем наступает момент, когда из-за излишне точной настройки параметров алгоритма ошибка на

валидационном наборе перестает уменьшаться, а затем начинает расти. Как только ошибка валидации не уменьшается на заданном числе итераций, процесс обучения останавливается. Из-за нерегулярного, как правило, изменения ошибки останавливать процесс нужно только при росте ошибки на нескольких итерациях.

В заключение рассмотрим примеры типовых задач машинного обучения. При всем многообразии задач, решаемых с помощью машинного обучения, можно выделить ряд основных типовых классов задач:

1. *Задачи классификации*. Например, в медицинской диагностике пациенты в зависимости от их анализов относятся к классам здоровых или больных. В кредитном скоринге клиенты банка относятся к классам "хороших" или "плохих заемщиков". Распознавание изображений также относится к задачам классификации, так как распознанное изображение относится к одному из классов, например, к фотографии определенного объекта.

2. *Задачи регрессии*, например, прогнозирование потребительского спроса, прогнозирование курса валют.

3. *Задачи ранжирования*, например, упорядочения результатов информационного поиска по степени релевантности, то есть соответствия запросу.

4. *Задачи кластеризации*, то есть объединения объектов со близкими признаками в группы. Например, выделение разных групп пользователей неким сервисом.

Литература к разделу 1

1. Legg S., Hutter M. A Collection of Definitions of Intelligence (2007). [Электронный ресурс]. URL: <https://arxiv.org/abs/0706.3639> (дата обращения: 01.09.2024).

2. ГОСТ Р 59277-2020 Системы искусственного интеллекта. Классификация систем искусственного интеллекта. — М.: Стандартинформ, 2021. — 16 с.

3. Флах П. Машинное обучение. Наука и искусство построения алгоритмов, которые извлекают данные из знаний. — М.: ДМК Пресс, 2015. — 400 с.

4. Наука о данных. [Электронный ресурс]. URL: https://ru.wikipedia.org/wiki/Наука_о_данных (дата обращения: 01.09.2024).

5. Mitchell T. M. Machine Learning. — McGraw-Hill, 1997. — 414 p.

6. Бурков А. Машинное обучение без лишних слов. — СПб.: Питер, 2020. — 192 с.

7. Воронцов К. В. Вычислительные методы обучения по прецедентам. [Электронный ресурс].

URL: <http://www.machinelearning.ru/wiki/images/6/6d/Voron-ML-1.pdf> (дата обращения: 01.09.2024).

8. Эверитт Б. С. Большой словарь по статистике. М.: Проспект, 2012. — 736 с.

9. Жерон О. Прикладное машинное обучение с помощью Scikit Learn, Keras и TensorFlow: концепции, инструменты, и техники для создания интеллектуальных систем. — СПб.: ООО "Диалектика", 2020. — 1040 с.

2. ПОДГОТОВКА ДАННЫХ ДЛЯ МАШИННОГО ОБУЧЕНИЯ

2.1. Виды операций подготовки данных для машинного обучения

Машинное обучение не сводится только к построению модели. Анализ данных состоит из нескольких этапов. Только грамотное выполнение всех этапов может обеспечить успех. Наиболее распространенной методологией анализа данных является *CRISP-DM (Cross-Industry Standard Process for Data Mining)* [1]. Модель жизненного цикла исследования данных в методологии состоит из шести фаз, стрелки обозначают зависимости между фазами. Последовательность этих фаз строго не определена. Как правило в большинстве проектов приходится возвращаться к предыдущим этапам, а затем снова двигаться вперед (рис. 2.1).

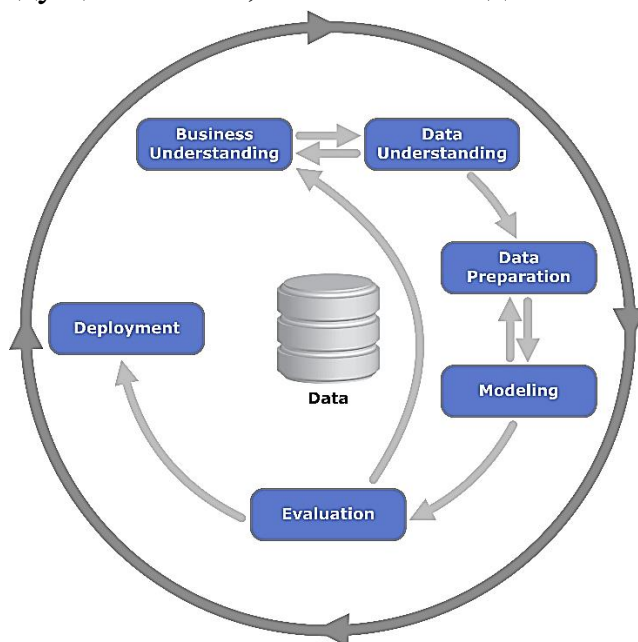


Рис. 2.1. Жизненный цикл исследования данных

Жизненный цикл исследования данных состоит из следующих фаз:

1. Понимание бизнес-целей (Business Understanding)

На этой фазе определяются бизнес-цели, оценивается ситуация, определяются цели анализа данных и составляется план проекта.

2. Начальное изучение данных (Data Understanding)

На этой фазе производится сбор и описание исходные данных, исследование и проверка качества данных. Для исследования и проверки качества данных применяется *разведочный (разведывательный) анализ данных* [2], основанный на методах статистического анализа данных и графического представления информации.

3. Подготовка данных (Data Preparation)

Данные для машинного обучения как правило бывают "грязными" — содержат пропуски, выбросы, представлены в форме, неудобной для обработки. Поэтому

необходима предварительная обработка данных, т. е. представление в форме, удобной для обработки.

4. Моделирование (Modeling)

Моделирование — это выбор, обучение и оценка одной или нескольких моделей машинного обучения, например, регрессии, деревьев решения, нейронных сетей. Моделирование — это центральная часть машинного обучения. Моделирование в настоящее время чаще всего производится на языке Python с использованием различных библиотек.

5. Оценка (Evaluation)

Анализ количественных характеристик качества модели с точки зрения достижения бизнес-целей.

6. Внедрение (Deployment)

Это внедрение построенных моделей в прикладную сферу. Необходимо сделать модель доступной конечному пользователю. Для этого зачастую необходимо интегрировать модель в какую-то программу. Для развертывания используют или программный интерфейс между программой и моделью или даже переписать модель на том языке, на котором разработана программа. После развертывания работа с моделью не заканчивается — необходимо отслеживать качество работы модели на реальные данные и при необходимости совершенствовать модель.

Рассмотрим подготовку данных. Подготовка данных, в основном, используется при применении *признакового описания объектов*, когда обучающие данные представляются в виде табличных данных. Когда в качестве обучающих примеров используются изображения, то применяют обработку изображений с целью повышения качества изображений, приведения к одному размеру и увеличения числа примеров (аугментацию) с помощью поворотов, отражений и т. п. изображений.

Выделяют три основных задачи работы с признаками [3–4]:

- feature extraction и feature engineering — выявление признаков и преобразование в допустимые для модели машинного обучения данные;
- feature transformation — преобразование признаков для повышения точности алгоритма;
- feature selection — отбор признаков, оказывающих наибольшее влияние на результат.

Можно выделить следующие операции предварительной обработки информации.

1. *Восстановление пропущенных данных*. Некоторые данные в обучающей выборке могут быть пропущены. Зачастую исключить данные с пропусками нельзя, так как можно потерять важную информацию. Тогда необходимо заполнить пропуски правдоподобными значениями.

2. *Обработка аномалий, противоречий и дубликатов в данных.* Обучающие данные могут содержать ошибки, выбросы, дубликаты, снижающие качество данных. Их необходимо удалить или заменить правдоподобными значениями. Причем эти отклонения не всегда очевидны. Необходим тщательный анализ, что является аномалией, а что — возможным значением, важным для анализа.

3. *Трансформация данных*, то есть представление данных в виде, удобном для обработки. Это, прежде всего, *кодирование данных* — то есть представление всех данных в виде чисел, и *шкалирование (масштабирование)* — приведение к определенному диапазону.

4. *Понижение размерности входных данных.* Чем выше размерность входных данных (входных векторов), тем сложнее должна быть модель машинного обучения. Чем сложнее модель, тем труднее ее обучить. Уменьшение размерности входов модели позволяет использовать меньше данных, обеспечивая приемлемый уровень сложности модели. Понижение размерности входных данных производится или отбором наиболее важных признаков или математическим преобразованием признаков в пространство меньшей размерности.

2.2. Заполнение пропусков в данных

Во многих случаях обучающие выборки содержат пропуски (отсутствие) данных. Например, в медицине и социальных науках данные часто содержат пропуски. Рассмотрим матрицу объектов-признаков, часто называемую матрицей исходных данных. Пусть строки таблицы представляют собой примеры для обучения моделей машинного обучения. Столбцы соответствуют различным признакам. Ячейки таблицы — это значения признаков в разных примерах. Возможно и другое представление таблицы — примерами являются столбцы, а признаки образуют строки. Пример матрицы $\mathbf{X}$ объектов-признаков представлен в табл. 2.1. Тогда x_{ij} означает i -й компонент вектора $\mathbf{x}_j$. Матрицу можно представить состоящей из векторов признаков $\mathbf{X} = [\mathbf{x}_1 \ \mathbf{x}_2 \ \dots \ \mathbf{x}_n]$.

Таблица 2.1. Матрица объектов-признаков

$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	...	$\mathbf{x}_{n-1}$	$\mathbf{x}_n$
x_{11}	x_{12}	x_{13}	...	$x_{1,(n-1)}$	x_{1n}
x_{21}	x_{22}	x_{23}	...	$x_{2,(n-1)}$	x_{2n}
...	...	...	...	...	...
$x_{(m-1),1}$	$x_{(m-1),2}$	$x_{(m-1),3}$	...	$x_{(m-1),(n-1)}$	$x_{(m-1),n}$
x_{m1}	x_{m2}	x_{m3}	...	$x_{m,(n-1)}$	x_{mn}

По разным причинам многие наборы данных реального мира содержат пропущенные значения. Пропущенные значения кодируются оговоренными символами, например, пробелами, знаками вопроса, *NaN* или другими символами. Однако такие наборы с пропусками данных не могут использоваться в алгоритмах машинного обучения.

Одной из стратегий использования неполных наборов данных является *исключение некомплектных объектов*, заключающееся в удалении целых строк и/или столбцов, содержащих пропущенные значения. Однако, в таком случае происходит потеря данных, которые могут быть гораздо важнее оставшихся признаков (хоть и имеют пропуски). Лучшим подходом считается восстановление пропущенных значений правдоподобными данными. Известны различные подходы к восстановлению пропущенных данных [5–10].

Выделяют *одномерные и многомерные заполнители пропусков*. Разница между этими двумя разновидностями алгоритмов заключается в следующем. Одномерный заполнитель вычисляет значения в i -м измерении таблицы признаков (мы приняли, что признаки представляют собой столбцы таблицы, поэтому заполнение будем проводить в столбцах), используя только непропущенные значения в этом измерении признака. Многомерный заполнитель используют весь набор доступных измерений признаков для оценки отсутствующих значений.

Одномерный заполнитель заполняет пропуски постоянным значением или с использованием средств математической статистики (среднее значение, медиана или наиболее частое значение) для каждого столбца. Этот класс также допускает различные типы пропущенных значений, например, строки. Заполнение средним или медианой допустимо только для числовых значений. Часто используемое заполнение пропусков средним значением исходит из предположения о нормальном распределении значений признака. Если распределение не является нормальным, то многие значения признака будут далеки от среднего значения. В этом случае лучше использовать заполнение медианой. Одномерное заполнение признаков уменьшает дисперсию признака, что отрицательно сказывается на обучении модели машинного обучения. Возможно заполнение отсутствующих значений наиболее частым значением по столбцу или постоянным значением. Такие заполнители могут использоваться как со строками, так и с числовыми данными.

Одномерный заполнитель применим только в случае однородных данных. В задаче классификации пропущенные значения признака в столбце нельзя заполнять средним значением признака, относящегося к разным классам. Например, в задаче диагностики заболевания заполнение пропущенных значений температуры пациентов приведет к заполнению "средним значением по больнице". В задачах классификации следует примеры, соответствующие различным классам, вынести в

отдельные таблицы, заполнить пропуски данных, а затем сформировать единую таблицу.

Неплохие результаты дает замена пропущенного значения случайным числом, распределенным по закону распределения соответствующего признака (по закону распределения присутствующих значений столбца таблицы). Причем можно использовать эмпирический закон распределения. Подход применим, если имеется достаточно большое количество примеров, позволяющее получить достоверные оценки параметров распределения.

Хорошие результаты дает кластерный метод заполнения пропусков [11]. Идея подхода заключается в группировке векторов выборки в кластеры по некоторым признакам и замене пропусков данных локально в каждой группе. Пропущенные данные заменяются средними значениями в данном кластере или случайными значениями по эмпирическому распределению интересующего параметра в кластере.

Подбор ближайшего соседа [12] основан на предположении, что пропущенное значение близко к заполненным значениям строк, близких к строке, содержащей пропуск. Для простоты предположим, что в таблице отсутствует единственный элемент x_{ij} . Рассматривая строки таблицы как векторы, в качестве меры близости между строками таблицы возьмем евклидово расстояние между элементами i -ой и всех остальных строк таблицы. Тогда расстояние между i -ой и k -ой ($k = 1, 2, \dots, m$) строками определится формулой

$$d_{ki} = \sqrt{\sum_{\substack{l=1 \\ l \neq j}}^n (x_{kl} - x_{il})^2}, \quad k = 1, 2, \dots, i-1, i+1, \dots, m.$$

Среди d_{ki} выбирается K минимальных значений (K назначается), тем самым определяются K ближайших соседей i -ой строки. Обозначим множество номеров строк, ближайших к строке i , через m_K . Для получения x_{ij} элементы j -го столбца ближайших соседей усредняются с весами, обратно пропорциональными их евклидову расстоянию до строки, содержащей пропуск

$$x_{ij} = \frac{\sum_{l \in m_K} x_{lj} C_l}{\sum_{l=1}^K C_l},$$

где $C_l = \frac{1}{1 + d_{li}}$ — вес l -ой строки.

Заполнение с помощью прогноза. В этом случае исходные данные разделяются на две группы: первая группа содержит все признаки, вторая группа содержит данные с пропусками. Пропущенные значения прогнозируются с

использованием данных первой группы. По своей сути заполнение с помощью прогноза использует моделирование, то есть строится модель зависимости отсутствующего фактора от комплектных данных. Применяются различные методы прогнозирования: линейная регрессия, аппроксимация полиномами, нейросетевые методы.

Популярным методом, основанным на регрессии, является метод *многомерного заполнения с применением цепочки уравнений* (MICE — Multivariate Imputation by Chained Equations) [13–16]. Название "цепочка уравнений" подразумевает следующую особенность алгоритма заполнения. Метод MICE реализуется как объединение процедур для заполнения недостающих данных в отдельных столбцах. Алгоритм многомерного заполнения пропусков состоит из следующих действий.

Предварительно все пропущенные значения в таблице заполняются одним из одномерных заполнителей (средним значением, медианой, наиболее часто встречающимся значением, константой). Далее повторяются шаги многомерного заполнения. Одна итерация заполнения пропусков состоит из следующих шагов:

Шаг 1. Выбирается один из столбцов таблицы (выбор возможен последовательно или случайным образом). Данные, заполненные в этом столбце одномерным заполнителем, удаляются. Этот столбец рассматривается как выход y , а другие столбцы рассматриваются как входные данные X регрессионной модели. Форма регрессионной модели зависит от характера целевой переменной y (для количественных переменных используются модели линейной регрессии для прогнозирования их недостающих значений, а для качественных переменных потребуется модель логистической регрессии). Алгоритм регрессии обучается на (X, y) для известных y .

Шаг 2. С использованием обученной регрессионной модели предсказываются отсутствующие значения в выбранном столбце.

Шаги 1–2 повторяются для всех оставшихся столбцов таблицы. Причем для построения регрессионных моделей в качестве входные данные X используются как наблюдаемые значения (присутствовавшие в исходной таблице), так и спрогнозированные значения. При этом к спрогнозированным значениям рекомендуется добавлять небольшие случайные значения. После обработки всех столбцов первая итерация считается завершенной. После итерации все пропущенные значения будут заменены прогнозами регрессионных моделей, которые отражают отношения, наблюдаемые в данных между этими переменными.

Шаги 1–2 циклически повторяются в течение нескольких циклов: выбирается столбец, удаляются предсказанные значения, строится регрессионная модель и предсказываются пропущенные значения. Эти действия повторяются для всех столбцов. Так как регрессионные модели в каждой итерации строятся по

уточненным данным, то предсказанные значения точнее отражают зависимости между признаками каждого примера. Итерационный процесс выполняется заданное число итераций.

В результате для каждого столбца набора данных строится своя регрессионная модель. При заполнении пропусков в регрессионных моделях используются только присутствующие значения. Пропущенные значения в столбце будут заменяться значениями, вычисленными с помощью регрессионной модели для столбца с входными данными, равными присутствующим значениям для соответствующей строки.

2.3. Очистка данных

Очистка данных — это удаление дубликатов, противоречий и выбросов. *Дубликаты* — это примеры с одинаковыми, а точнее, с *близкими значениями*. *Противоречивые данные* содержат близкие значения признаков, но различные целевые значения. *Аномальные значения (выбросы)* — примеры, сильно отличающиеся от окружающих данных или несовместимые с ними.

Дубликаты — одинаковые примеры. Они могут дублировать информацию об одном и том же объекте, а могут содержать идентичную информацию о двух различных, но похожих объектах. В первом случае дубликаты должны быть удалены, а во втором требуется более тонкая обработка.

Особенно сложным является вопрос *выявления аномалий (выбросов)*. Резко отличающиеся примеры могут нести полезную информацию. Некоторые методы анализа данных основаны на обнаружении аномальных значений. Например, аномально большая сумма, снимаемая со счета, может рассматриваться как признак мошенничества с кредитной картой.

Для обнаружения выбросов, дубликатов и противоречий данных удобно использовать представление данных в многомерном пространстве признаков. Рассмотрим такое представление на примере обучающей выборки для задачи классификации. Обучающая выборка представляет собой множество примеров $X = \{\mathbf{x}_i, y_i\}$, $i = 1, 2, \dots, l$ где $\mathbf{x}_i$ — вектор признаков объекта i ; y_i — метка (обозначение) класса, к которому принадлежит i -й объект. На множестве векторов признаков объектов задана *метрика*, такая, что выполняется *гипотеза компактности*. В задачах классификации гипотеза компактности — это предположение о том, что схожие объекты гораздо чаще лежат в одном классе, чем в разных классах.

В качестве метрики можно рассматривать, например, *евклидово расстояние*

$$d_{ij} = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2},$$

где d_{ij} — расстояние между векторами $\mathbf{x}_i$ и $\mathbf{x}_j$, соответственно, между объектами i и j ; n — размерность пространства (число компонентов векторов); x_{ik} и x_{jk} — компоненты векторов.

Пример $(\mathbf{x}_i, y_i)$ является выбросом, если расстояние от i -го объекта до ближайшего к нему примера из "своего" класса значительно превышает максимальное расстояние между остальными примерами данного класса. Два примера, принадлежащих одному классу, можно считать близкими (дубликатами), если расстояние между ними мало. Противоречивые примеры близки по векторам входных данных, но принадлежат разным классам.

Аномальные значения в обучающей выборке могут быть заменены на некоторые значения, полученные теми же методами, что и при восстановлении пропущенных данных (фактически считаем, что аномальные значения пропущены). Ещё раз напомним, что удаление или замена аномальных значений могут исказить результаты анализа.

2.4. Кодирование качественных признаков

Признаки качественного типа — это признаки с конечным числом значений, причем нельзя ввести расстояние между значениями. Примером может служить состояние больного — тяжелое, среднее, легкое. Нельзя сказать, что расстояние от легкого состояния больного до среднего больше, меньше или равно расстоянию от среднего состояния до тяжелого. Другой пример — оценки студента "отлично" и "хорошо". Нельзя сказать, на какую величину "отлично" отличается от "хорошо". В наборе данных категориальные признаки имеют строковые значения.

Модели машинного обучения, как правило, работают только с числами. Поэтому качественные признаки необходимо закодировать. Качественные признаки *не рекомендуется кодировать целыми числами* (Ordinal Encoder, Label Encoder), например, "белый" — 1, "черный" — 2 и т.д., так как кодирование категориальных признаков числами вводит расстояние между значениями признаков. А такого понятия для категориальных признаков не существует.

У бинарных признаков (возможен только один из вариантов — истина или ложь) ложь кодируется как "-1" или "0", а "истина" — как "1".

Для кодирования качественных признаков широко используется прием, известный в современном машинном обучении как *One-Hot Encoding* (*унитарный код*) [17]. Кодирование One-Hot Encoding заключается в следующем. Если столбец набора данных представляет собой качественный признак, принимающий n

значений, то он заменяется n столбцами. В строках таблицы рассматриваемый признак заменяется n значениям, все из которых равны нулю за исключением одного. На позиции, соответствующей номеру значения признака, размещается 1.

Кодирование унитарным кодом создает большое количество бинарных признаков, особенно в случае большого количества категорий. Известны альтернативные подходы к кодированию качественных переменных [18]. Например, в *частотном кодировании (Frequency Encoding)* значения качественной переменной кодируются частотой встречаемости этого значения для данной переменной.

Среднее, или целевое кодирование (Mean Encoding, Target Encoding) применимо, если целевые значения представляют собой числа, и реализуется по следующему алгоритму:

1. Выберите категориальную переменную, которую вы хотите преобразовать.
2. Получите сумму значений целевой переменной для каждого значения выбранной качественной переменной.
3. Подсчитайте количество значений выбранной качественной переменной.
4. Разделите результаты шага 2 на результаты шага 3 и используйте как значения качественной переменной.

Например, в таблице представлены качественные значения температуры тела пациента и целевые значения: 1 — пациент боле, 0 — пациент здоров (табл. 2.2). Результаты выполнения шагов кодирования представлены в таблицах 2.3–2.5.

Таблица 2.2. Исходная таблица

Температура	Целевое значение
Горячий	1
Холодный	1
Очень горячий	1
Теплый	0
Горячий	1
Теплый	0
Теплый	1
Горячий	0
Горячий	1
Холодный	1

Таблица 2.3. Шаг 2

Температура	Сумма целевых значений
Горячий	3
Холодный	2
Очень горячий	1
Теплый	1

Таблица 2.4. Шаг 3

Температура	Количество значений признака
Горячий	4
Холодный	2
Очень горячий	1
Теплый	3

Таблица 2.5. Шаг 4

Температура	Кодированные значения признака
Горячий	0,75
Холодный	1
Очень горячий	1
Теплый	0,33

2.5. Масштабирование признаков

Многие алгоритмы машинного обучения критичны к масштабированию признаков. Числовые признаки рассматриваются как безразмерные величины. Обычно числовые признаки изменяются в разных диапазонах значений. Признаки, принимающие большие значения и изменяющимися в широком диапазоне, будут

сильнее влиять на обучение модели, чем признаки, изменяющихся в узком диапазоне. Обеспечив изменение каждого признака в пределах одного диапазона, мы обеспечим равное влияние каждого признака на модель в процессе обучения. При масштабировании признаки рассматриваются как безразмерные величины. Векторы признаков масштабируются независимо друг от друга.

Известны два способа масштабирования [19]: *масштабирование по минимаксу* (min-max scaling), или *нормализация* (normalization) и *стандартизация* (standardization, StandardScaling, называемая также Z-score normalization). *Нормализация* — это приведение переменных к одному диапазону. *Стандартизация* — преобразование переменных к значениям с нулевым математическим ожиданием и единичной дисперсией.

Стандартизированное значение признака рассчитывается следующим образом

$$p_{ij} = \frac{x_{ij} - \bar{x}_j}{\bar{\sigma}_j},$$

где $\bar{x}_j = \frac{1}{m} \sum_{i=1}^m x_{ij}$ — выборочная оценка математического ожидания j -го признака,

m — количество обучающих примеров, $\bar{\sigma}_j^2 = \frac{1}{m-1} \sum_{i=1}^m (x_{ij} - \bar{x}_j)^2$ — несмещенная выборочная оценка дисперсии j -го признака.

Стандартизация хотя и не делает распределение нормальным, но в какой-то мере защищает от выбросов.

Альтернативным подходом к масштабированию является *масштабирование по минимаксу* — MinMax Scaling. Данный подход масштабирует данные в указанный диапазон (Scaling features to a range). Если $[a, b]$ — допустимый диапазон изменения масштабируемых признаков, то масштабирование описывается формулой

$$x_{ij\text{norm}} = \frac{(x_{ij} - x_{j\min})(b - a)}{x_{j\max} - x_{j\min}} + a, \quad (2.1)$$

где $x_{ij\text{norm}}$ — нормализованное значение j -го признака в i -ом примере, x_{ij} — исходное значение j -го признака в i -ом примере, $[a, b]$ — допустимый диапазон изменения нормализованных признаков, $[x_{j\min}, x_{j\max}]$ — диапазон изменения исходных значений j -го признака.

Чаще всего признаки масштабируются в 0 до 1. Реже, например, например, при использовании некоторых нейронных сетей масштабирование производится в диапазон от -1 до 1.

2.6. Отбор признаков

Отбор признаков — это сокращение количества признаков в обучающем наборе (датасете). Имеется как минимум две важные причины избавляться от неинформативных признаков. Первая причина очевидна: чем больше данных, тем выше вычислительная сложность. Пока мы работаем с простыми датасетами, размер данных — это не проблема, но для огромного набора данных время обработки датасета и обучение модели может существенно увеличиться. Другая причина — некоторые алгоритмы принимают шум (данные с ошибками или неинформативные признаки) за корректный сигнал и переобучаются, т.е. теряют способность к обобщению.

Процедура уменьшения количества признаков называется понижением размерности, так как в результате этой процедуры уменьшается размерность входных векторов сети.

Размерность признаков может быть понижена двумя способами:

- выбором признаков (feature selection), которые оказывают наибольшее влияние на результат;
- проецированием данных с высокой размерностью в пространство меньшей размерности. Этот метод включает преобразование набора признаков в меньшее количество новых признаков, которые как можно полнее описывают исходный набор признаков.

Рассмотрим выбор признаков. Проецирование признаков в пространство меньшей размерности рассмотрим позже. Для выбора признаков можно использовать содержательный анализ признаков, но такой анализ является сложной задачей, так как исследование может показать, что некоторые, казалось бы, полезные признаки, можно исключить, а малополезные на первый взгляд признаки в совокупности с другими признаками могут нести полезную информацию. Поэтому целесообразно применять формализованные методы.

Можно выделить три классы формальных методов выбора признаков [3, 20–21]:

- методы фильтрации (filter methods);
- методы "обёртки" (wrapper methods);
- встроенные методы (embedded methods);
- отбор с использованием моделей.

Методы фильтрации основаны на статистических методах и позволяют удалить малополезные признаки, например, признаки, значения которых практически не изменяются (дисперсия которых мала), и признаки, слабо коррелированные с целевым значением.

При решении задач классификации с числовыми признаками вычисляется F -критерий Фишера дисперсионного анализа (ANOVA — *ANalysis Of VAriance*) [22] между каждым признаком и вектором целей. Значения F -критерия показывают, различаются ли значимо средние значения для каждой группы, если числовой признак сгруппировать по вектору целей [23]. Например, если имеется бинарный вектор целей — пол и количественный признак — экзаменационные баллы, то критерий показывает, отличается ли средний экзаменационный балл для мужчин от среднего экзаменационного балла для женщин. Если нет, то экзаменационный балл не помогает нам предсказывать пол, и поэтому этот признак не имеет значения.

При решении задачи классификации с категориальными признаками применяется критерий хи-квадрат между каждым признаком и вектором целей [23]. Критерий хи-квадрат является разницей между полученным числом наблюдений в каждом классе категориального признака и тем, что мы ожидали бы, если бы этот признак был независимым от вектора целей (т. е. не связанным с ним). Если признак не связан с вектором целей, то его можно удалить.

Имеет смысл удалить те признаки, которые статистически связаны с другими признаками. Для этого вычисляется ковариационная матрица, содержащая ковариации между всеми парами признаков. Пары признаков с высокими значениями ковариации являются статистически зависимыми, поэтому один признак из пары может быть исключен. Целесообразно удалять признаки с малым значение дисперсии, так как практически постоянные признаки слабо влияют на обучение.

Методы фильтрации эффективны с точки зрения времени вычислений. Но методы фильтрации независимы от модели, поэтому не учитывают особенности модели. Например, нейросетевая модель является нелинейной, а корреляция линейна. Поэтому выбор признаков может быть не оптимальным.

Суть *методов обертки* состоит в том, что процедура отбора признаков "обертывается" процедурой, реализующей модель. Модель обучается на разных наборах признаков, которые выбирает процедура отбора. Выбирается набор признаков, обеспечивающий лучший показатель качества модели. Существуют четыре подхода в этом классе методов: методы включения (forward selection), исключения (backwards selection) признаков, пошаговый подход (step wise), исчерпывающий метод (exhaustive feature selection). Методы включения стартуют с пустого подмножества признаков, куда постепенно добавляются разные признаки. Методы исключения стартуют с подмножества, равного исходному множеству признаков, и из него постепенно удаляются признаки. В пошаговом подходе на каждом шаге после включения новой переменной в модель осуществляется проверка на значимость переменных, которые уже были введены в

нее ранее. В случае, если будут обнаружены малозначащие переменные, то они выводятся из состава модели. Исчерпывающий метод использует полный перебор признаков. Методы обертки учитывают особенности модели и взаимосвязь признаков. Но число подмножеств множества признаков выражается экспоненциальной зависимостью. Поэтому перебрать все подмножества обычно невозможно. Большинство подходов используют "жадные" алгоритмы [24], на каждом шаге которых выбирается лучшее по некоторому критерию подмножество, но не пересматривается ранее сделанный выбор.

Встроенные методы не разделяют отбор признаков и обучение модели, а производят отбор внутри процесса обучения модели. Эти методы основаны на *регуляризации*, то есть введении ограничений на параметры модели. Идея регуляризации состоит в том, чтобы построить алгоритм, минимизирующий не только ошибку модели, но и количество используемых переменных. Например, при построении линейной регрессии применяют методы регуляризации: метод LASSO, гребневая регрессия и эластичные сети. Эти методы используют регуляризацию для удаления избыточных признаков за счет уменьшения их регрессионных коэффициентов до нуля. Метод LASSO (Least Absolute Shrinkage and Selection Operator) [25] заключается во введении ограничения на сумму модулей коэффициентов модели. Ридж-регрессия или гребневая регрессия (ridge regression) [26] вводит ограничение на квадрат евклидовой нормы вектора коэффициентов модели. Эластичная сеть [27] — модель регрессии с двумя регуляризаторами: на квадрат нормы вектора коэффициентов и на сумму модулей коэффициентов. В процессе работы алгоритма величины коэффициентов будут пропорциональны важности соответствующих переменных, а перед теми переменными, которые дают наименьший вклад в устранение ошибки, станут близкими к нулю.

Отбор признаков с использованием моделей основан на применении простых моделей, явно показывающих важность использования тех или иных признаков, например, деревьев решений или регрессии с регуляризацией. Если признак бесполезен в простой модели, то вряд ли надо его использовать в сложной модели.

2.7. Декорреляция признаков

Известно [28], что при статистической независимости признаков будет достигнута максимальная информационная насыщенность каждого из входных признаков в отдельности. Действительно, если значения признака зависят от значений других признаков, то они могут быть получены из этих признаков. Для достижения статистической независимости признаков используется *линейное преобразование*, которое осуществляет *декорреляцию* векторов признаков [29–31]. Алгоритм декорреляции называется ещё "*отбеливание*" (*whitening*).

Декорреляция может быть основана на различных подходах [31]. Рассмотрим наиболее популярный подход, основанный на применении метода главных компонент. *Метод главных компонент* (РСА — principal component analysis) [32–34] основан на линейном преобразовании признаков в новую систему координат. Вычисление главных компонент может быть сведено к вычислению собственных векторов и собственных значений ковариационной матрицы признаков или к сингулярному разложению матрицы исходных данных. Рассмотрим вычислительную сущность метода главных компонент, доказательства можно найти, например, в [32–34].

Рассмотрим матрицу объектов-признаков (табл. 2.1). Векторы признаков будем рассматривать как случайные коррелированные векторы. Преобразуем векторы признаков в центрированные, то есть в векторы с нулевым математическим ожиданием. Для этого вычислим матрицу $\dot{X} = [\dot{x}_1 \ \dot{x}_2 \ \dots \ \dot{x}_n]$, где $\dot{x}_{ij} = x_{ij} - \bar{x}_j$, $i = 1, 2, \dots, n$, $j = 1, 2, \dots, m$, то есть вычитаем из элементов каждого столбца его среднее значение.

Сначала рассмотрим пример [30], когда $n = 2$, то есть матрица объектов-признаков содержит два столбца. Если на плоскости отобразить объекты, характеризуемые двумя признаками, то получим *диаграмму рассеяния*. В случае коррелированных столбцов матрицы объектов-признаков диаграмма рассеяния будет иметь вид, например, как на рис. 2.2 [30]. Несложно увидеть, что можно выбрать ортогональные главные направления u_1 и u_2 изменения переменных (рис. 2.3). Если перейти в новую ортогональную систему координат (u_1, u_2) , то в новой системе координат признаки будут некоррелированными (рис. 2.4). Но *признаки потеряют первоначальный физический смысл*.

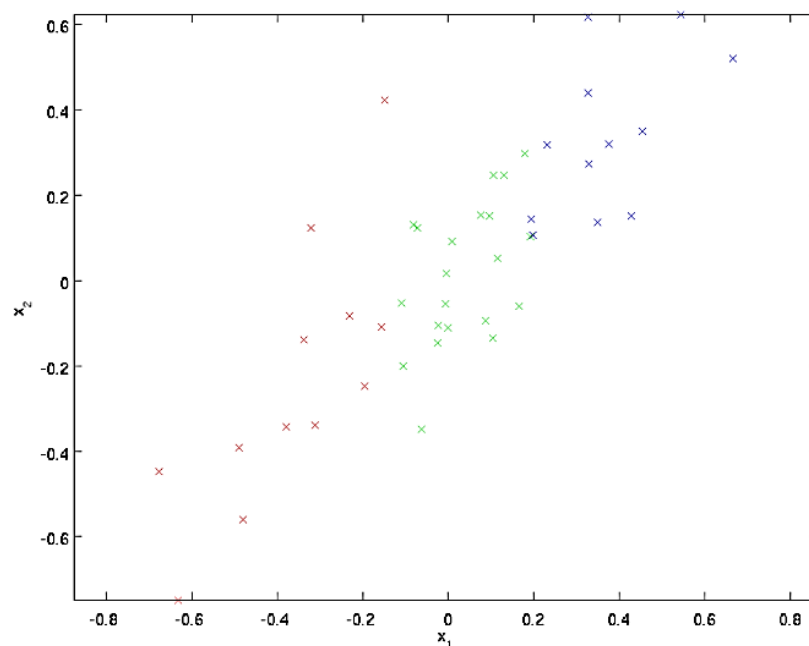


Рис. 2.2. Пример диаграммы рассеяния коррелированных признаков

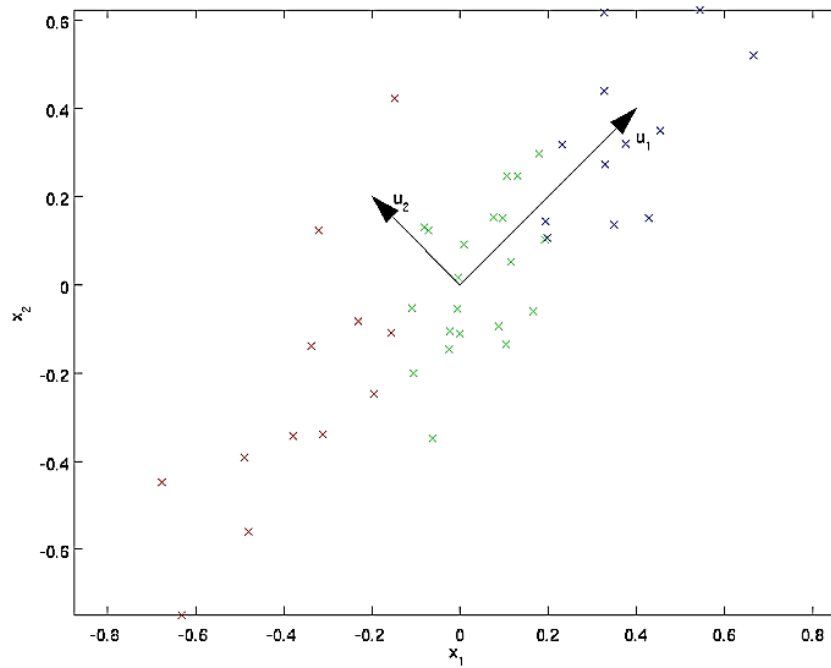


Рис. 2.3. Главные направления изменения признаков

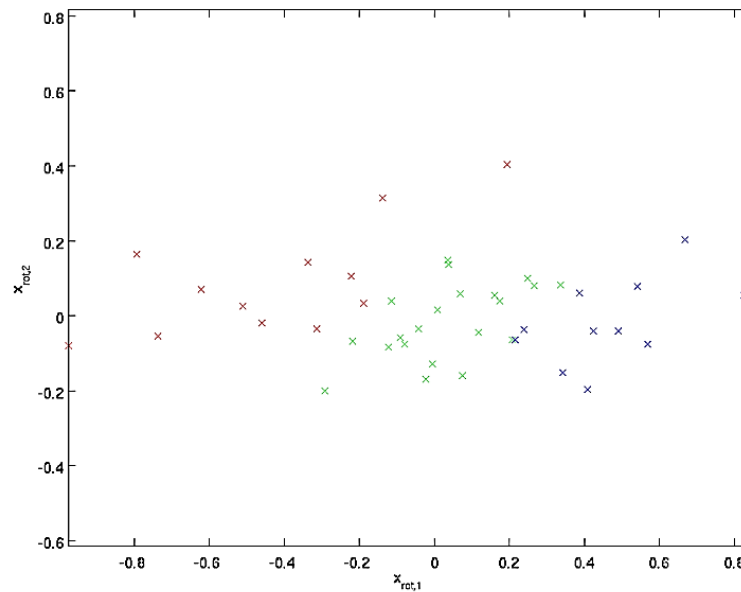


Рис. 2.4. Диаграмма рассеяния некоррелированных признаков

Математически это преобразование описывается следующим образом. Исходя из матрицы объектов-признаков (табл.2.1) вычислим ковариационную матрицу. *Ковариационная матрица* **K** — это квадратная матрица размера $n \times n$, образованная из попарных ковариаций m компонентов каждого вектора. Элементы ковариационной матрицы равны (используем несмещенную оценку)

$$k_{ij} = \frac{1}{m-1} \sum_{s=1}^m (x_{si} - \bar{x}_i)(x_{sj} - \bar{x}_j) = \frac{1}{m-1} \sum_{s=1}^m \dot{x}_{si} \dot{x}_{sj}, \quad i, j = 1, 2, \dots, n, \quad (2.2)$$

где n — число векторов признаков (столбцов матрицы объектов-признаков), m — число компонентов векторов признаков (число строк матрицы), s — номер входного примера, i и j — номера признаков, между которыми вычисляется ковариация.

Элементы ковариационной матрицы удобно рассматривать, используя скалярные произведения центрированных векторов признаков

$$k_{ij} = \frac{1}{m-1} (\dot{\mathbf{x}}_i, \dot{\mathbf{x}}_j) = \frac{1}{m-1} \dot{\mathbf{x}}_i^T \dot{\mathbf{x}}_j,$$

где $(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$ — скалярное произведение векторов.

Тогда ковариационная матрица запишется в виде

$$\mathbf{K} = \frac{1}{m-1} \dot{\mathbf{X}}^T \dot{\mathbf{X}}, \quad (2.3)$$

где $\dot{\mathbf{X}} = [\dot{\mathbf{x}}_1 \ \dot{\mathbf{x}}_2 \ \dots \ \dot{\mathbf{x}}_n]$ — матрица центрированных векторов признаков.

Матрица $\mathbf{K}$ является симметричной и положительно определенной матрицей размером $n \times n$.

Матрица $\mathbf{T}$, составленная из преобразованных *некоррелированных* векторов признаков, получается из исходной матрицы $\dot{\mathbf{X}}$ линейным преобразованием [32–34]

$$\mathbf{T} = \dot{\mathbf{X}} \cdot \mathbf{P}, \quad (2.4)$$

где матрицы в математической статистике имеют специальные названия: $\dot{\mathbf{X}}$ — исходная матрица $m \times n$ называется *матрицей данных*, $\mathbf{T}$ — матрица $m \times n$ преобразованных данных называется *матрицей счетов* (scores); $\mathbf{P}$ — матрица $n \times n$ коэффициентов преобразований называется *матрица нагрузок* (loading).

Матрица $\mathbf{P}$ имеет вид: $\mathbf{P} = [\mathbf{p}_1 \ \mathbf{p}_2 \ \dots \ \mathbf{p}_n]$, где $\mathbf{p}_i$, $i = 1, 2, \dots, n$ — собственные векторы ковариационной матрицы $\mathbf{K}$. Причем векторы $\mathbf{p}_i$ являются ортогональными нормированными векторами, т. е. $\mathbf{P}^T \mathbf{P} = \mathbf{E}$, где $\mathbf{E}$ — единичная матрица.

Задача на собственные значения для матрицы $\mathbf{K}$ имеет вид

$$\mathbf{K} \mathbf{p}_i = \lambda_i \mathbf{p}_i, \quad i = 1, 2, \dots, n.$$

где λ_i и $\mathbf{p}_i$ — собственные числа и соответствующие собственные векторы матрицы $\mathbf{K}$.

В результате преобразования (2.4) столбцы матрицы $\dot{\mathbf{X}}$ преобразуются в *некоррелированные столбцы* матрицы $\mathbf{T}$. Матрица ковариации для $\mathbf{T}$ представляет собой диагональную матрицу, с диагональю из дисперсий столбцов матрицы $\mathbf{T}$. Известно [32–34], что эти дисперсии равны соответствующим собственным числам матрицы $\mathbf{K}$.

Обычно векторы преобразованных исходных данных нормируют — делят каждое значение преобразованной переменной на стандартное отклонение. В результате получается матрица *Z-счетов* (Z-scores) — матрица *отбеленных* (whiting) признаков (рис. 2.5).

$$\mathbf{Z} = \mathbf{S}^{-1} \dot{\mathbf{X}} \mathbf{P}, \quad (2.5)$$

где $\mathbf{S} = \text{diag}[\sqrt{\lambda_1}, \sqrt{\lambda_2}, \dots, \sqrt{\lambda_n}]$.

На рис. 2.5 видно, что отбеленные признаки располагаются как бы внутри некоторой окружности. В случае трех переменных признаки расположатся внутри сферы, в общем случае — внутри гиперсферы. Поэтому в англоязычных источниках отбеливание иногда называют *sphering* [30] — *сферическое преобразование*.

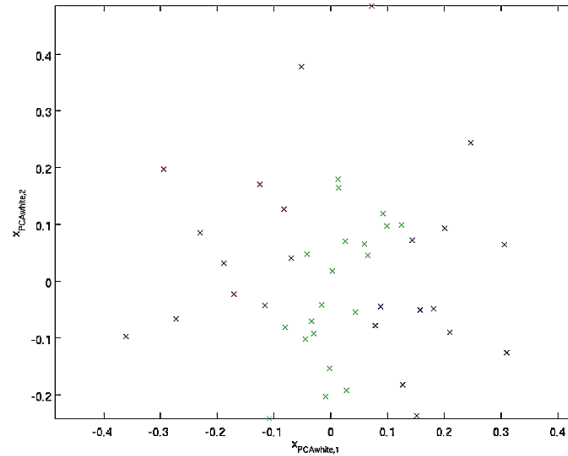


Рис. 2.5. Диаграмма рассеяния отбеленных признаков

При работе с обученной моделью машинного обучения вектор-строка признаков входа модели должна центрироваться и преобразовываться по (2.4).

Матрицы $\mathbf{P}$ и $\mathbf{S}$ могут быть получены также в результате *спектрального разложения матрицы* $\mathbf{K}$

$$\mathbf{K} = \mathbf{P} \cdot \mathbf{S} \cdot \mathbf{P}^T,$$

где $\mathbf{P}$ — матрица, столбцы которой являются ортонормированными собственными векторами матрицы $\mathbf{K}$, $\mathbf{S}$ — диагональная матрица с соответствующими собственными значениями на главной диагонали.

В машинном обучении метод главных компонент обычно реализуют с помощью *сингулярного разложения* матрицы данных $\dot{\mathbf{X}}$. Сингулярное разложение матрицы $\dot{\mathbf{X}}$ имеет вид [35]

$$\dot{\mathbf{X}} = \mathbf{U} \mathbf{S}_1 \mathbf{V}^T,$$

где $\mathbf{U}$ — матрица, образованная ортонормированными собственными векторами матрицы $\dot{\mathbf{X}} \dot{\mathbf{X}}^T$, $\mathbf{V}$ — матрица, образованная ортонормированными собственными векторами матрицы $\dot{\mathbf{X}}^T \dot{\mathbf{X}}$, $\mathbf{S}_1$ — диагональная матрица, на диагонали которой стоят сингулярные числа $\sigma_i = \sqrt{\lambda_i}$, где λ_i — собственные числа матриц $\dot{\mathbf{X}} \dot{\mathbf{X}}^T$ и $\dot{\mathbf{X}}^T \dot{\mathbf{X}}$ (собственные числа этих матриц равны).

Если обозначить $\mathbf{T} = \mathbf{U} \mathbf{S}_1$, $\mathbf{P} = \mathbf{V}$, то получаем $\dot{\mathbf{X}} = \mathbf{T} \cdot \mathbf{P}^T$. Учитывая ортогональность матрицы $\mathbf{P}$, получаем

$$\mathbf{T} = \dot{\mathbf{X}} \cdot \mathbf{P}. \quad (2.6)$$

То есть (2.6) совпадает с (2.4). Нормирование преобразованных данных производится по формуле $\mathbf{Z} = \mathbf{S}_1^{-1} \dot{\mathbf{X}} \mathbf{P}$, где $\mathbf{S}_1 = \text{diag}[\sqrt{\lambda_1}, \sqrt{\lambda_2}, \dots, \sqrt{\lambda_m}]$, λ_i — равные собственным числам матриц $\dot{\mathbf{X}} \dot{\mathbf{X}}^T$ и $\dot{\mathbf{X}}^T \dot{\mathbf{X}}$.

Покажем, что в результате преобразования (2.6) получаются некоррелированные векторы. Найдем произведение матриц $\mathbf{T}^T \mathbf{T}$, используемое при вычислении ковариационной матрицы, учтем, что $\mathbf{T} = \mathbf{U} \mathbf{S}_1$ и в силу ортогональности матрицы $\mathbf{U}^T = \mathbf{U}^{-1}$

$$\mathbf{T}^T \mathbf{T} = (\mathbf{U} \mathbf{S}_1)^T \mathbf{U} \mathbf{S}_1 = \mathbf{S}_1^T \mathbf{U}^T \mathbf{U} \mathbf{S}_1 = \mathbf{S}_1^T \mathbf{S}_1.$$

Так как матрица $\mathbf{S}_1$ диагональная и её элементы равны среднеквадратическим отклонениям матрицы $\mathbf{T}$, то $\mathbf{T}^T \mathbf{T}$ будет диагональной матрицей — у ковариационной матрицы будут отличны от нуля только диагональные элементы, следовательно столбцы матрицы не коррелированы.

Столбцы матрицы счетов будут не только некоррелированными, но и ортогональными. Действительно, два случайных вектора $\mathbf{x}_i$ и $\mathbf{x}_j$ называют некоррелированными, если $M(\mathbf{x}_i^T \mathbf{x}_j) = M(\mathbf{x}_i^T) M(\mathbf{x}_j)$ и ортогональными, если $M(\mathbf{x}_i^T \mathbf{x}_j) = 0$ (здесь M обозначает вычисление математического ожидания) [36]. В нашем случае центрированных векторов, у которых математическое ожидание равно нулю, некоррелированность векторов означает их ортогональность.

2.8. Понижение размерности признаков методом главных компонент

Популярным и мощным средством понижения размерности входных данных является *метод (анализ) главных компонент* (МГК), или по-английски *PCA — Principal-Component Analysis*. Это метод в данных большой размерности находит направления с максимальной дисперсией данных и проецирует данные на часть этих направлений, то есть проецирует данные с высокой размерностью в пространство с меньшей размерностью [32]. Происходит сжатие информации с потерей наименее значимой информации. Снижение размерности применяется не только для преобразования табличных признаков, но и при работе с изображениями [30]. Например, алгоритм обучается на распознавании изображений в градациях серого размером 16×16 пикселей. Очевидно, что значения соседних пикселей в изображении сильно коррелированы. Можно рассматривать изображения как векторы, содержание 256 компонентов, представляющих собой значения интенсивности пикселей. Из-за корреляции между соседними пикселями метод главных компонент позволит нам аппроксимировать входной сигнал гораздо меньшим размером, при этом допуская очень небольшую ошибку.

Метод главных компонент использует линейное преобразование, аналогичное преобразованию (2.4), использовавшемуся для ортогонализации входных векторов. Но в отличие от преобразования (2.4), в котором матрица $\mathbf{P}$ имеет размерность $n \times n$, в МГК используется матрица размерностью $n \times k$, где $k < n$. В результате такого преобразования размерность преобразованных векторов будет меньше размерности исходных векторов. Для построения матрицы преобразования МГК упорядочим положительные собственные значения ковариационной матрицы (2.3) в порядке убывания: $\lambda_1 > \lambda_2 > \dots \lambda_n$ (собственные значения матрицы $\mathbf{K}$ являются вещественными и положительными). В аналогичной последовательности расположим соответствующие собственные векторы $\mathbf{p}_i$. Если ограничиться k максимальными собственными значениями, то матрица преобразования получит вид $\mathbf{P} = [\mathbf{p}_1 \mathbf{p}_2 \dots \mathbf{p}_k]$. Далее производится преобразование (2.4). Первый преобразованный вектор $\mathbf{t}_1$ называется первым главным компонентом. Первый главный компонент имеет наибольшую дисперсию. Второй главный компонент $\mathbf{t}_2$ имеет меньшую дисперсию и т. д. Для рассмотренного ранее примера [31] с $m = 2$ и $k = 1$ диаграмма рассеяния первого компонента представлена на рис. 2.6 (в оригинале $\tilde{\mathbf{x}}$ — главный компонент).

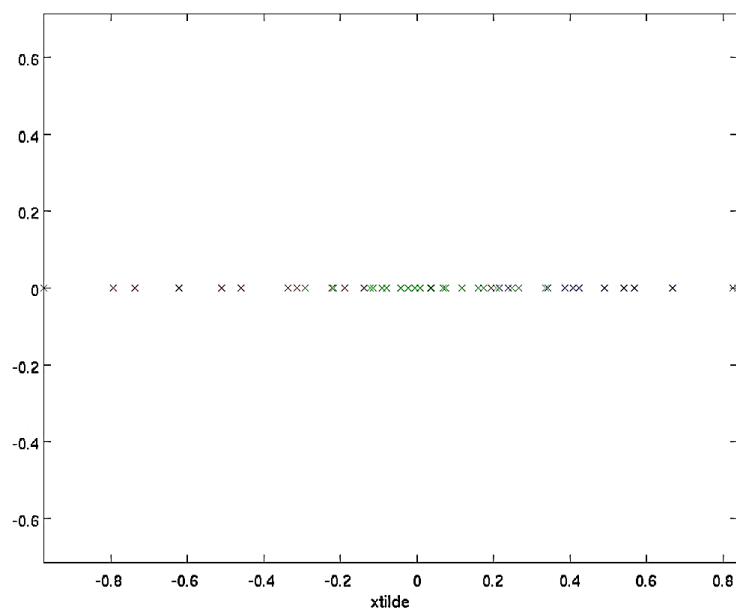


Рис. 2.6. Диаграмма рассеяния главного компонента

Так как дисперсия определяет ценность преобразованных признаков для машинного обучения, необходимо отобрать признаки с наибольшей дисперсией и пренебречь признаками с малой дисперсией. Известны разные критерии определения числа преобразованных признаков (количества столбцов матрицы счетов). Количество векторов k можно оценить по относительному вкладу преобразованных векторов в общую дисперсию (по доле *объясненной дисперсии*)

[32], учитывая, что дисперсии преобразованных признаков равны соответствующим собственным числам матрицы $\mathbf{K}$. Доля дисперсии, объясненной первым вектором $\mathbf{t}_1$, равна

$$d_1 = \lambda_1 / \sum_{i=1}^m \lambda_i, \text{ доля дисперсии, объясненной векторами } \mathbf{t}_1 \text{ и } \mathbf{t}_2 \text{ равна } d_2 = (\lambda_1 + \lambda_2) / \sum_{j=1}^m \lambda_j,$$

объясненной k первыми векторами — $d_k = \sum_{i=1}^k \lambda_i / \sum_{i=1}^m \lambda_i$. Берется такое количество векторов (главных компонент) k , чтобы доля d_k была достаточно большой.

При использовании *критерия Кайзера* (Kaiser) [32] — выбираются те главные компоненты, которым соответствуют дисперсии, превосходящие среднюю дисперсию: $\lambda_i > \frac{1}{m} \sum_{i=1}^m \lambda_i$. Критерий Кайзера хорошо работает, когда имеется

несколько главных компонент с дисперсиями, намного превосходящими среднее значение. В противном случае критерий дает слишком многоглавных компонент.

Критерий каменистой осыпи Кэттелла (Cattell) [32] является эвристическим (не имеющим строгого обоснования) критерием. Отсортированные собственные значения изображаются в виде графика "номер собственного значения–величина собственного значения". На графике находится точка, где убывание собственных значений слева направо максимально замедляется. Признаки, которым соответствуют собственные числа, расположенные правее этой точки, отбрасываются. Название произошло от аналогии с геологическим термином "осыпь", обозначающим обломки горных пород, скапливающиеся в нижней части скалистого склона. Как и критерий Кайзера критерий каменистой осыпи дает хорошие результаты, когда имеется несколько главных компонент с дисперсиями, намного превосходящими среднее значение. В противном случае критерий выделяет слишком мало главных компонент.

Другим эвристическим критерием является *правило сломанной трости* (Broken stick model) [37]. Правило наглядно иллюстрируется следующим образом. Трость единичной длины в $m-1$ случайных равномерно и независимо распределенных точках ломается на куски. Куски трости нумеруются в порядке длины кусков. Математическое ожидание длины i -го куска трости равно

$$l_i = \frac{1}{m} \sum_{j=i}^m \frac{1}{j}. \text{ Выбирается } k \text{ главных компонент, если}$$

$$\frac{\lambda_1}{\sum_{i=1}^m \lambda_i} > l_1, \frac{\lambda_2}{\sum_{i=1}^m \lambda_i} > l_2, \dots, \frac{\lambda_k}{\sum_{i=1}^m \lambda_i} > l_k, \frac{\lambda_{k+1}}{\sum_{i=1}^m \lambda_i} < l_{k+1}.$$

Правило сломанной трости занижает количество значимых главных компонент.

Отобранные главные компоненты часто нормируют. Тогда матрица главных компонент имеет вид

$$\mathbf{Z} = \left[\frac{1}{\sqrt{\lambda_1}} \mathbf{t}_1 \quad \frac{1}{\sqrt{\lambda_2}} \mathbf{t}_2 \quad \dots \quad \frac{1}{\sqrt{\lambda_k}} \mathbf{t}_k \right].$$

Таким образом, метод главных компонент преобразует векторы признаков в ортогональные и оставляет только наиболее изменчивые признаки. Метод главных компонент производит сжатие данных. Но как уже указывалось, преобразованные векторы не несут физического смысла.

Точное и эффективное сингулярное разложение для больших матриц является сложной вычислительной задачей. Для работы с большими матрицами были разработаны методы, основанные на *рандомизированной линейной алгебре* [38], в частности, *рандомизированное сингулярное разложение* (*Randomized Singular Value Decomposition*). Рандомизированное сингулярное разложение позволяет приближенно вычислить k главных компонент [38]. Рассмотрим алгоритм вычисления, теоретическое обоснование можно найти в [38]. Схема алгоритма представлена на рис. 2.7.

Шаг 1. Строится случайная матрица $\mathbf{P}$, элементы которой равны нормально распределенным случайным числам с нулевым математическим ожиданием и подбираемой дисперсией. С помощью матрицы $\mathbf{P}$ исходная матрица $\mathbf{X}$ проецируется в пространство меньшей размерности

$$\mathbf{Z} = \mathbf{XP}.$$

Матрица $\mathbf{Z}$ аппроксимирует пространство столбцов с высокой вероятностью. Далее вычисляется QR-разложение

$$\mathbf{Z} = \mathbf{QR}.$$

Шаг 2. С использованием матрицы $\mathbf{Q}$ производится проецирование $\mathbf{X}$ на меньшее подпространство

$$\mathbf{Y} = \mathbf{Q}^T \mathbf{X}.$$

Строится сингулярное разложение

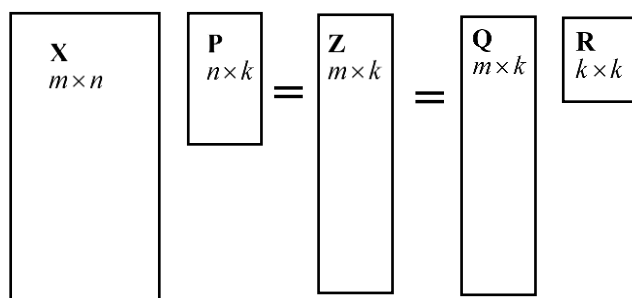
$$\mathbf{Y} = \mathbf{U}_Y \mathbf{\Sigma} \mathbf{V}^T.$$

Доказано, что матрица сингулярных чисел $\mathbf{\Sigma}$ и матрица правых сингулярных векторов $\mathbf{V}$ одинаковы для разложения матриц $\mathbf{Y}$ и $\mathbf{X}$.

Шаг 3. Строится матрица левых сингулярных векторов

$$\mathbf{U} = \mathbf{Q} \mathbf{U}_Y.$$

Шаг 1



Шаг 2

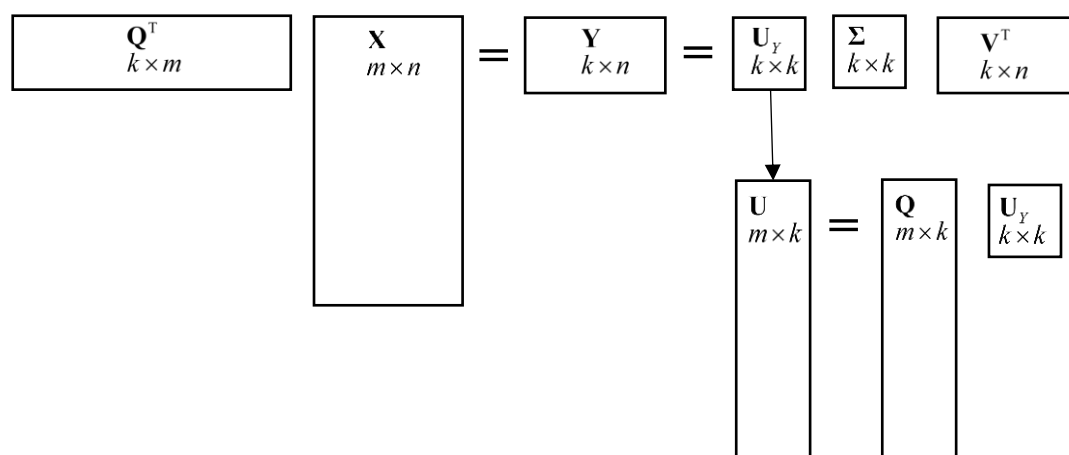


Рис. 2.7. Схема рандомизированного алгоритма SVD

2.9. Ядерный метод главных компонент

Известно большое число методов машинного обучения, использующих линейные разделяющие функции (линейную сепарабельность). Например, линейная разделимость на два класса объектов обучающей выборки с двумя признаками означает, что на диаграмме рассеяния можно провести прямую, разделяющую два класса. Например, персептрон может производить классификацию только в случае линейно разделимых классов. Но зачастую данные не являются линейно разделимыми. Оказывается, нелинейное преобразование образов (признаковых описаний объектов) в пространство большей размерности повышает вероятность линейного разделения признаков [39]. Нелинейное преобразование в пространство большей размерности означает, что с помощью нелинейной функции из имеющихся признаков формируется большее количество признаков. Например, после преобразования матрицы объектов-признаков (табл. 2.1) вместо n признаков получится $l \gg n$ признаков. Примеры функций преобразования рассмотрим позже. Теоретической основой такого подхода служит *теорема Ковера* [40], утверждающая, что нелинейное преобразование задачи классификации образов в пространство более высокой размерности повышает

вероятность линейной разделимости образов¹. Пространство высокой размерности, в котором возможно линейное разделение образов, называется *спрямляющим пространством* [39]. На теореме Ковера основан эмпирический прием, основанный на расширении набора признаков путем добавления нелинейно преобразованных некоторых признаков [20].

Рассмотренный линейный метод главных компонент хорошо работает в случае линейной разделимости классов. В случае, если классы линейно не разделены, метод строго говоря, не применим. Выходом является нелинейное преобразование признаков и использование в спрямляющем пространстве метода главных компонент, что позволит перейти в пространство меньшей размерности. При этом преобразованные данные будут линейно разделимы. Обозначим образы (*строки* матрицы "объекты-признаки") как $\mathbf{x}_i$, $i = 1, 2, \dots, m$. Применяя нелинейное преобразование в пространство бóльшей размерности, получаем спрямляющее пространство, в котором образы описываются векторами $\phi(\mathbf{x}_i)$, бóльшей размерности $l \gg n$, чем исходные векторы $\mathbf{x}_i$. *Центрированную* матрицу объектов-признаков в спрямляющем пространстве размером $m \times l$ обозначим Φ (матрица Φ обязательно должна быть центрирована, иначе нельзя представить ковариационную матрицу в виде (2.3)). Использование ковариационной матрицы $\frac{1}{m-1}\Phi^T\Phi$ (см. (2.3)) для реализации метода главных компонент может потребовать очень больших затрат из-за большого числа признаков в спрямляющем пространстве. Однако известен прием, известный как "*ядерный трюк*" (*kernel trick*) [42–43], резко сокращающий трудоемкость применения метода главных компонент в спрямляющем пространстве.

Рассмотрим этот прием [42]. В "ядерном трюке" работают не с матрицей $\Phi^T\Phi$, а с *матрицей Грама* (см., например, [44]) $\mathbf{G} = \Phi\Phi^T$. Матрица Грама размерами $(m \times m)$, где m — количество примеров (строк матрицы "объекты-признаки") имеет вид

$$\mathbf{G} = \begin{bmatrix} (\phi(\mathbf{x}_1), \phi(\mathbf{x}_1)) & (\phi(\mathbf{x}_1), \phi(\mathbf{x}_2)) & \cdots & (\phi(\mathbf{x}_1), \phi(\mathbf{x}_m)) \\ (\phi(\mathbf{x}_2), \phi(\mathbf{x}_1)) & (\phi(\mathbf{x}_2), \phi(\mathbf{x}_2)) & \cdots & (\phi(\mathbf{x}_2), \phi(\mathbf{x}_m)) \\ \cdots & \cdots & \cdots & \cdots \\ (\phi(\mathbf{x}_m), \phi(\mathbf{x}_1)) & (\phi(\mathbf{x}_m), \phi(\mathbf{x}_2)) & \cdots & (\phi(\mathbf{x}_m), \phi(\mathbf{x}_m)) \end{bmatrix}, \quad (2.7)$$

¹ Перевод по [41]. Наглядная иллюстрация эффекта от нелинейного преобразования признаков представлена на видео <https://www.youtube.com/watch?v=9NrALgHFwTo> и <https://www.youtube.com/watch?v=3liCbRZPrZA>

где $(\varphi(\mathbf{x}_i), \varphi(\mathbf{x}_j))$ — скалярное произведение преобразованных векторов примеров $\varphi(\mathbf{x}_i)$ и $\varphi(\mathbf{x}_j)$, φ — функция нелинейного преобразования в пространство большей размерности.

Допустим, что признаки центрированы. Пусть $\mathbf{v}_j$ — собственный вектор матрицы Грама, соответствующий собственному значению λ_j , т. е. $\Phi\Phi^T\mathbf{v}_j = \lambda_j\mathbf{v}_j$. Рассмотрим выражение

$$\Phi^T\Phi(\Phi^T\mathbf{v}_j) = \Phi^T(\Phi\Phi^T\mathbf{v}_j) = \lambda_j\Phi^T\mathbf{v}_j,$$

или $\Phi^T\Phi(\Phi^T\mathbf{v}_j) = \lambda_j\Phi^T\mathbf{v}_j$, $\Phi^T\Phi\mathbf{w}_j = \lambda_j\mathbf{w}_j$, откуда следует, что вектор $\mathbf{w}_j = \Phi^T\mathbf{v}_j$ является собственным вектором матрицы $\Phi^T\Phi$, соответствующим собственному значению λ_j , а λ_j — одно из собственных значений матрицы Грама.

Вспомним, что матрица $\Phi^T\Phi$ используется для вычисления ковариационной матрицы преобразованных признаков (сравни с (2.3)). Множитель $\frac{1}{m-1}$ при вычислении собственных чисел и векторов ковариационной матрицы можно не учитывать, так как каждому собственному числу матрицы соответствует бесчисленное множество собственных векторов, отличающихся лишь скалярным множителем. Размер матрицы Грама $m \times m$, размер матрицы $\Phi^T\Phi$ — $l \times l$, поэтому у матрицы Грама будет значительно больше собственных значений, чем у $\Phi^T\Phi$, но в МГК собственные числа матрицы Грама ранжируются и используется только их часть, меньшая l .

Найдем квадрат евклидовой нормы вектора $\mathbf{w}_j$. Будем считать, что собственный вектор $\mathbf{v}_j$ матрицы Грама нормирован (в силу упомянутого свойства собственных векторов, их, как правило, нормируют). Т. е.

$$\begin{aligned}\|\mathbf{v}_j\|^2 &= (\mathbf{v}_j, \mathbf{v}_j) = \mathbf{v}_j^T \mathbf{v}_j = 1 \\ \|\mathbf{w}_j\|^2 &= \|\Phi^T\mathbf{v}_j\|^2 = (\Phi^T\mathbf{v}_j)^T \Phi^T\mathbf{v}_j = \mathbf{v}_j^T \Phi\Phi^T\mathbf{v}_j = \mathbf{v}_j^T (\Phi\Phi^T\mathbf{v}_j) = \\ &= \lambda_j \mathbf{v}_j^T \mathbf{v}_j = \lambda_j \|\mathbf{v}_j\|^2 = \lambda_j.\end{aligned}\tag{2.8}$$

То есть квадрат евклидовой нормы вектора $\mathbf{w}_j$ равен собственному значению матрицы $\Phi^T\Phi$.

В разделе 2.7 мы проводили нормирование матрицы преобразованных признаков (матрицы счетов), делением на среднеквадратические отклонения столбцов этой матрицы. В результате получалась матрица Z-счетов. Причем эти среднеквадратические отклонения равны квадратным корням из собственных чисел ковариационной матрицы (2.5). Собственные числа ковариационной

матрицы находятся из (2.8). Тогда векторы $\mathbf{u}_j = \lambda_j^{-1/2} \Phi^T \mathbf{v}_j$ являются ортонормированными собственными векторами матрицы $\Phi^T \Phi$, а вследствие известных свойств собственных векторов матриц (умножение матрицы на скаляр не изменяет собственное значение матрицы) — и собственными векторами ковариационной матрицы $\frac{1}{n-1} \Phi^T \Phi$, т. е. являются векторами главных компонент.

Преобразуем выражение для вектора $\mathbf{u}_j$ с учетом таблицы 2.1

$$\begin{aligned} \mathbf{u}_j &= \lambda_j^{-1/2} \Phi^T \mathbf{v}_j = \lambda_j^{-1/2} \cdot \begin{bmatrix} \varphi(x_{11}) & \varphi(x_{21}) & \cdots & \varphi(x_{m1}) \\ \varphi(x_{12}) & \varphi(x_{22}) & \cdots & \varphi(x_{m2}) \\ \cdots & \cdots & \cdots & \cdots \\ \varphi(x_{1l}) & \varphi(x_{2l}) & \cdots & \varphi(x_{ml}) \end{bmatrix} \cdot \begin{bmatrix} v_{j1} \\ v_{j2} \\ \cdots \\ v_{jm} \end{bmatrix} = \\ &= \lambda_j^{-1/2} \cdot \begin{bmatrix} \varphi(x_{11})v_{j1} + \varphi(x_{21})v_{j2} + \cdots + \varphi(x_{m1})v_{jm} \\ \varphi(x_{12})v_{j1} + \varphi(x_{22})v_{j2} + \cdots + \varphi(x_{m2})v_{jm} \\ \cdots \\ \varphi(x_{1l})v_{j1} + \varphi(x_{2l})v_{j2} + \cdots + \varphi(x_{ml})v_{jm} \end{bmatrix} = \lambda_j^{-1/2} \sum_{i=1}^m v_{ji} \varphi(\mathbf{x}_i)^T = \quad (2.9) \\ &= \sum_{i=1}^m \alpha_{ji} \varphi(\mathbf{x}_i)^T, \end{aligned}$$

где $\alpha_{ji} = \lambda_j^{-1/2} v_{ji}$.

Таким образом, мы выразили главные компоненты через описания объектов в спрямляющем пространстве.

Матрица преобразованных данных (матрица Z-счетов) формируется следующим образом

$$\mathbf{Z} = \Phi \mathbf{P}, \quad (2.10)$$

где Φ — матрица объектов-признаков в спрямляющем пространстве, строки этой матрицы имеют вид $\varphi(\mathbf{x}_i)$, $\mathbf{x}_i$ — строки исходной матрица объектов-признаков, матрица $\mathbf{P} = [\mathbf{u}_1 \mathbf{u}_2 \dots \mathbf{u}_l]$ состоит из столбцов (2.9).

Вспомним, что элементы матрицы, полученной в результате произведения двух матриц, равны скалярным произведениям соответствующих строк и столбцов матриц-сомножителей. Тогда компоненты матрицы преобразованных данных (матрицы Z-счетов) (2.10) формируются как скалярные произведения

$$z_{ij} = (\varphi(\mathbf{x}_i), \mathbf{u}_j) = \left(\varphi(\mathbf{x}_i), \sum_{i=1}^m \alpha_{ji} \varphi(\mathbf{x}_i) \right) = \sum_{i=1}^m \alpha_{ji} (\varphi(\mathbf{x}_i), \varphi(\mathbf{x}_i)). \quad (2.9)$$

Проблема состоит в сложности вычисления скалярного произведения векторов в спрямляющем пространстве большой размерности. Для преодоления

этой проблемы применим "ядерный трюк". "Ядерный трюк" основан на использовании функции ядра (kernel function), или ядра скалярного произведения (inner-product kernel) [42, 45]. Ядром скалярного произведения называется функция, представимая в виде скалярного произведения в некотором пространстве: $K(\mathbf{x}, \mathbf{z}) = \varphi(\mathbf{x})^T \varphi(\mathbf{z})$, где $\varphi: R^n \rightarrow R^l$ — отображение из исходного пространства признаков в некоторое спрямляющее пространство. Ядро позволяет вычислять скалярные произведения в спрямляющем пространстве, не производя в явном виде отображения в это пространство. Тогда (2.9) запишется в виде

$$z_{ij} = \sum_{i=1}^m \alpha_{ji} K(\mathbf{x}_i, \mathbf{x}_i^T). \quad (2.10)$$

При вычислении матрицы Грама также следует использовать функцию ядра. Из (2.7) получаем

$$\mathbf{G} = \begin{bmatrix} K(\mathbf{x}_1, \mathbf{x}_1) & K(\mathbf{x}_1, \mathbf{x}_2) & \cdots & K(\mathbf{x}_1, \mathbf{x}_m) \\ K(\mathbf{x}_2, \mathbf{x}_1) & K(\mathbf{x}_2, \mathbf{x}_2) & \cdots & K(\mathbf{x}_2, \mathbf{x}_m) \\ \cdots & \cdots & \cdots & \cdots \\ K(\mathbf{x}_m, \mathbf{x}_1) & K(\mathbf{x}_m, \mathbf{x}_2) & \cdots & K(\mathbf{x}_m, \mathbf{x}_m) \end{bmatrix}. \quad (2.11)$$

Тогда для вычисления проекций образов на главные компоненты спрямляющего пространства необходимо вычислить и центрировать матрицу Грама (2.11). Необходимо найти собственные векторы и собственные числа центрированной матрицы Грама, ранжировать собственные векторы в порядке убывания собственных чисел и выбрать k главных компонент (как и в классическом методе главных компонент). Далее вычислить проекции образов на отобранные главные компоненты по (2.10).

Ядра можно выбирать множеством способов. Проверить, является ли функция $K(\mathbf{x}, \mathbf{z})$ ядром, можно с помощью теоремы Мёрсера (Mercer J.) [46]: функция $K(\mathbf{x}, \mathbf{z})$ является ядром тогда и только тогда, когда она симметрична: $K(\mathbf{x}, \mathbf{z}) = K(\mathbf{z}, \mathbf{x})$ и неотрицательно определена: для любой конечной выборки $X = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n\}$ матрица $[K(\mathbf{x}_i, \mathbf{x}_j)]$ неотрицательно определена. Обычно используются типовые функции ядра, из которых наиболее популярно гауссово ядро, или радиальная базисная функция (RBF — Radial Basis Function)

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2a^2}\right),$$

где $\mathbf{x}_j$ — центр RBF, a — подбираемый параметр формы (ширина).

Можно говорить, что RBF реализует сравнение образов $\mathbf{x}_i$ и $\mathbf{x}_j$: если образы близки, т.е. значение расстояния $\|\mathbf{x}_i - \mathbf{x}_j\|$ мало, то функция близка к единице, в противном случае функция близка к нулю. Параметр формы регулирует оценку степени близости. Суть "ядерного трюка" состоит в том, что не требуется явное вычисление скалярных произведений согласно (2.7). Требуется вычисление матрицы (2.11), элементы которой $K(\mathbf{x}_i, \mathbf{x}_j)$ — результат применения RBF с центром $\mathbf{x}_j$ к образу $\mathbf{x}_i$. При использовании радиальных базисных функций в качестве ядер $K(\mathbf{x}_i, \mathbf{x}_i^T) = 1$ в (2.10) и для вычисления проекций образов на главные компоненты спрямляющего пространства достаточно вычислить собственные векторы и числа матрицы Грама (2.11).

Примерами других ядер являются:

Линейное ядро: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$ — это скалярное произведение в исходном пространстве. *Полиномиальное ядро:* $K(\mathbf{x}_i, \mathbf{x}_j) = (\gamma \mathbf{x}_i^T \mathbf{x}_j + r)^d$, где параметры γ , r и d подбираются. *Сигмоидальное ядро (гиперболический тангенс):* $K(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\gamma \mathbf{x}_i^T \mathbf{x}_j + r)$, параметры γ и r подбираются. Сигмоидальное ядро не соблюдает все условия теорем Мерсера, но на практике работает хорошо. Недостатком ядерных методов является большой объем вычислений при большом размере обучающей выборки.

При использовании ядерного трюка матрица Грама должна быть центрирована. Рекомендуется вместо вычитания из столбцов элементов преобразованной матрицы признаков средних значений столбцов использовать *двойное центрирование* [42]. Приведем без доказательства формулу двойного центрирования

$$\dot{\mathbf{G}} = \mathbf{C}_m \mathbf{G} \mathbf{C}_m,$$

где $\mathbf{C}_m = \mathbf{E}_{m \times m} - \frac{1}{m} \mathbf{1}_{m \times m}$ — центрирующая матрица размером $m \times m$, $\mathbf{E}_{m \times m}$ — единичная матрица размером $m \times m$, $\mathbf{1}_{m \times m}$ — матрица, состоящая из единиц, размером $m \times m$.

2.10. Использование автокодировщиков для преобразования признаков

Автокодировщики или *автоэнкодеры* (*Autoencoder*) широко применяются в современных нейронных сетях глубокой архитектуры [19, 47]. В данном разделе мы кратко рассмотрим применение автокодировщиков для снижения размерности признаков. Автокодировщики (рис. 2.8) — это нейронные сети прямого

распространения, которые восстанавливают входной сигнал на выходе. Входной и выходной слои автокодировщика имеют одинаковое количество нейронов. Для сжатия входных данных скрытый, или латентный (latent) слой должен иметь меньшее количество нейронов, чем входной и выходной слои.

Автокодировщик состоит из двух частей:

1. *Кодировщик* (Encoder), отвечающий за сжатие входа в скрытый слой. Кодировщик переводит входной сигнал в его представление (код) и реализует функцию $h = g(x)$.

2. *Дешифратор* (Decoder) предназначен для декодирования сигнала по его коду: $\tilde{x} = f(h)$.

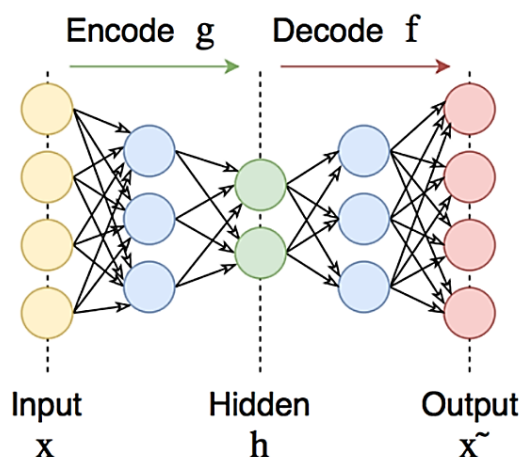


Рис. 2.8. Структура автокодировщика

Обучение автоэнкодера производится без учителя и заключается в получении $\tilde{x} \approx x$. То есть автокодировщик производит сжатие данных с потерями. Такие автокодировщики называются *понижающими* (undercomplete).

Если в автокодировщике использовать линейные нейроны и функцию потерь в виде среднеквадратической ошибки, то такой автокодировщик реализует линейный анализ главных компонент (доказательство см. ссылки в [42]).

На практике, обычно, применяют *многослойные автокодировщики* (stacked autoencoder) с нелинейными функциями активации. Применение многих слоев расширяет возможности автокодировщика по сжатию данных. Но имеется опасность переобучения, когда автокодировщик будет ставить в соответствие каждому входу единственное значение скрытого слоя (в пределе — единственное число). Такое автокодировщик будет идеально реконструировать обучающие данные, но не сможет работать с новыми примерами. Так как в автокодировщиках применяют нелинейные функции активации, то их можно рассматривать как нелинейное обобщение метода главных компонент. Рассмотрим пример многослойного автокодировщика для набора данных MNIST, содержащего черно-

белые изображения рукописных цифр размером 28x28 пикселей. Изображения преобразуются в векторы из 784 элементов. Скрытый слой содержит 30 нейронов.

Представление изображений в виде векторов эффективно работает только для небольших изображений. Для сжатия больших изображений используют *сверточные автокодировщики*. В сверточных автокодировщиках кодировщик представляет собой сверточную нейронную сеть, содержащую сверточные слои и слои пуллинга.

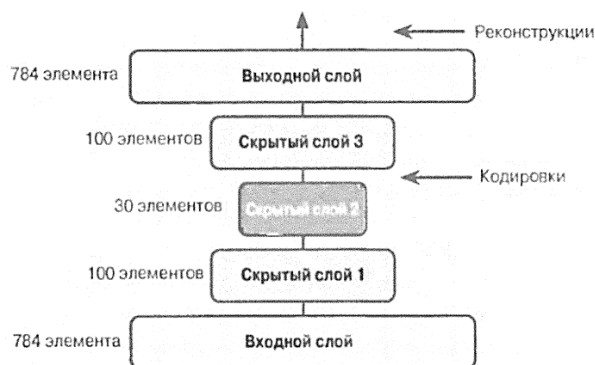


Рис. 2.9. Многослойный автокодировщик [19]

Кодировщик уменьшает пространственные размеры (высоту и ширину) изображений и увеличивает глубину (количество карт признаков). Декодировщик увеличивает пространственные размеры и уменьшает глубину до первоначальных измерений. Декодировщик является сверточной сетью, содержащей *транспонированные сверточные слои* [48]. При использовании транспонированной свертки между элементами входных карт признаков вставляются нули. Это увеличивает размер выходной карты после свертки.

2.11. Аугментация и сэмплинг

Данных для обучения моделей машинного обучения часто не хватает. В этой ситуации можно применить *аугментацию* (augmentation — увеличение, приращение) данных.

При распознавании изображения для аугментации используются различные преобразования изображений [49–50]: масштабирование, смещение, повороты, изменение яркости, изменение фона, зашумление изображений и т. п. К обучающим данным, представленным в виде таблиц чисел, такие приемы не применимы. Более универсальным является генерация *синтетических данных* [51–52]. Синтетические данные — это данные, созданные с помощью компьютерного алгоритма в качестве замены реальных данных. Известны следующие основные методы генерации синтетических данных:

- статистические методы;
- методы, основанные на машинном обучении.

Статистическими методами можно генерировать синтетические данные, определив статистическое распределение для имеющегося набора числовых данных. Но из-за недостаточного объема обучающих данных часто бывает сложно определить статистические характеристики данных.

Методы, основанные на машинном обучении, позволяют генерировать наиболее реалистичные синтетические данные, включая изображения и тексты. Наибольшую популярность получили три метода машинного обучения генерирования синтетических данных:

- использование вариационных автокодировщиков;
- использование генеративно-состязательных сетей;
- использование диффузионных моделей.

Вариационный автокодировщик (Variational Autoencoder — VAE) [19, 53] похож на сжимающий автокодировщик (рис. 2.10).

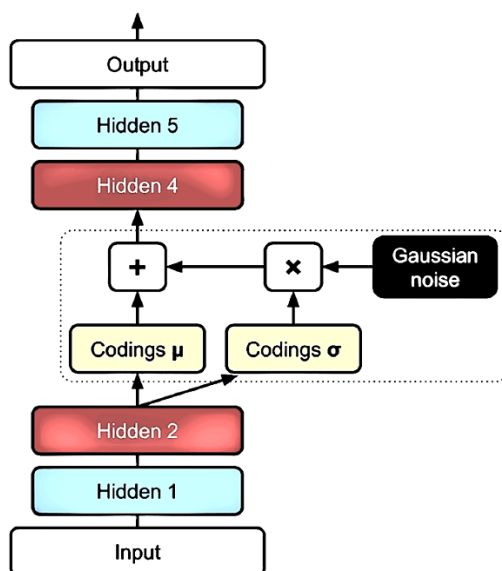


Рис. 2.10. Вариационный автокодировщик

В отличие от сжимающего автокодировщика вариационный автокодировщик в процессе обучения формирует не значения элементов скрытого (латентного) слоя, а два вектора — вектор μ средних значений элементов скрытого слоя и вектор σ стандартных отклонений элементов. Вектор h входа декодера формируется из случайного вектора z , элементы которого распределены по нормальному закону с нулевым математическим ожиданием и единичным стандартным отклонением, по формуле

$$h = z \odot \sigma + \mu,$$

где $\odot$ — операция поэлементного умножения.

Обучение вариационного автокодировщика производится методом обратного распространения ошибки. Функция потерь включает потерю от реконструкции входного вектора, которая заставляет автокодировщик

воспроизводить свои входы (например, перекрестную энтропию) и регуляризатор — *латентную потерю* (latent loss), вынуждающую автокодировщик формировать выходы, которые выглядят так, будто они были выбраны из простого гауссова распределения. Латентная потеря является *расстоянием Кульбака-Лейблера* [42] между случайным скрытым вектором с параметрами μ , σ и случайного вектора, компоненты которого распределены по нормальному закону с нулевым математическим ожиданием и единичным стандартным отклонением [19]

$$L = -\frac{1}{2} \sum_{i=1}^n \left(1 + \log(\sigma_i^2) - \sigma_i^2 - \mu_i^2 \right),$$

где L — латентная потеря, n — размерность скрытого слоя, μ_i и σ_i — средняя величина и стандартное отклонение i -го компонента скрытого слоя, основание логарифма особого значения не имеет.

Вариационные автокодировщики позволяют генерировать изображения [19]. Но изображения получаются размытыми. Изображения высокого качества способны генерировать генеративно-состязательные нейронные сети и диффузионные модели (диффузионные нейронные сети). *Генеративно-состязательные сети* (Generative Adversarial Network — GAN) [54] — это алгоритм машинного обучения без учителя, использующий две нейронных сети. Одна сеть (генератор) генерирует образцы, а другая сеть (дискриминатор) учится отличать правильные образцы от неправильных.

В основе различных *диффузионных моделей* (Diffusion Models) [55–57]) лежат схожие идеи. Диффузионные модели сначала искажают обучающие данные, постепенно добавляя гауссовский шум, стирая детали, пока данные не превратятся в чистый шум, а затем нейронная сеть обучается, чтобы восстановить обучающие данные.

Генеративно-состязательные сети и диффузионные модели широко используются для синтеза реалистических изображений, но сложны в обучении и требуют больших вычислительных затрат.

При использовании некоторых моделей машинного обучения, в частности, ансамблей машинного обучения, которые мы рассмотрим позже, необходимо сформировать несколько "псевдовыборок", или "повторных выборок", необходимых для обучения модели. Для формирования "псевдовыборок" используют методы *ресэмплинга* (*resampling* — повторная выборка) [58–59] (в русской литературе используются разные термины: "ресамплинг", "ресэмплинг", "ресемплинг", "методы генерации повторных выборок"). Ресэмплинг объединяет несколько близких по сути подходов: рандомизация, или перестановочный тест (*permutation*), метод "складного ножа" (*jackknife*), бутстреп (*bootstrap*).

Перестановочный тест применяется, когда необходимо сравнить работу модели машинного обучения на двух выборках: обучающей и тестовой. Модель

обучается на обучающей выборке, а обобщающие возможности модели проверяются на тестовой выборке. При недостатке примеров можно использовать перестановочный тест: выборку разделяют на обучающую и тестовую. Модель многократно обучается на обучающей выборке и тестируется на тестовой выборке. При каждом обучении происходит случайный обмен примерами между обучающей и тестовой выборками. Показатели качества работы модели усредняются по всем результатам обучения. Часто применяется *перекрестная проверка* (*Cross-validation*) без случайного обмена данными. При оценке модели имеющиеся в наличии данные разбиваются на K частей. Затем на $K - 1$ частях данных производится обучение модели, а оставшаяся часть данных используется для тестирования. Процедура повторяется K раз. При этом для тестирования используются разные части. В итоге каждая из K частей данных используется для тестирования.

Метод "*складного ножа*" [59] состоит в том, чтобы из одной выборки сделать много, исключая по одному примеру и возвращая ранее исключенные.

Бутстреп [58–59] был предложен Б. Эфроном (*B. Efron*) как обобщение метода "*складного ножа*", чтобы не уменьшать число элементов по сравнению с исходной совокупностью. По одной из версий [58] слово "*bootstrap*" означает кожаную полоску в виде петли, прикрепляемую к заднику ботинка для облегчения его натягивания на ногу. Благодаря этому термину появилась английская поговорка: "*Lift oneself by the bootstrap*", которую можно перевести как "Пробить себе дорогу благодаря собственным усилиям". В бутстреп берется исходная выборка из n элементов, из которой за n шагов формируется новая выборка, содержащая n элементов. На каждом шаге с равной вероятностью из элементов исходной выборки выбирается очередной элемент новой выборки (выбранный элемент не удаляется из исходной выборки и может быть выбран на других шагах). В результате n шагов формируется выборка, содержащая n элементов, но отобранных в случайном порядке. При этом некоторые элементы исходной выборки могут оказаться многократно повторенными в новой выборке, а другие — не появиться ни разу. Таким образом, полученные с помощью бутстрэпа выборки, скорее всего, будут иметь различное распределение данных, что позволит обеспечить независимое обучение некоторого количества моделей на единственном наборе или получить более достоверные оценки качества работы модели.

Мы рассматривали ситуации, когда не хватает данных для обучения и тестирования модели. Но в аналитике [60] возможны ситуации, когда в хранилище данных накапливаются такие объемы информации, что их использование для анализа затруднено. В этой ситуации решается задача сэмплинга. *Сэмплинг* (*sampling* — выборка) — это процесс отбора из исходной совокупности данных

выборки, представляющей интерес для анализа. При реализации сэмплинга используются специальные методы отбора, которые должны обеспечить репрезентативность выборки с точки зрения решаемой аналитической задачи. Различают равномерный случайный, стратификационный и другие виды сэмплинга [60].

При *равномерном случайном сэмплинге (Uniform Random Sampling)* все элементы исходной выборки разделяются на группы, в каждой из которых содержится одинаковое количество элементов. Из каждой группы случайным образом выбирается один элемент и помещается в результирующую выборку. Количество групп, на которые следует разбить исходную совокупность, зависит от размера исходной выборки и размера результирующей выборки. Размер результирующей выборки зависит от особенностей решаемой задачи. Возможен вариант случайного выбора элементов из всей исходной выборки без разбиения на группы. Равномерный случайный сэмплинг отличается простотой, но при неоднородной исходной совокупности результирующая выборка может оказаться не репрезентативной. Например, могут быть нарушены вероятности появления разных элементов.

В *стратифицированном сэмплинге* исходная выборка разбивается на несколько непересекающихся страт (слоев — от англ. *Strata* — слой). В качестве признака деления на страты используются значения одного или нескольких полей элементов выборки. Из каждой страты элементы извлекаются случайным образом. Размер выборки, извлекаемой из каждой страты, может выбираться двумя способами:

- пропорционально доли элементов страты в исходной выборке (из большей страты выборка больше);
- размер выборки пропорционален стандартному отклонению распределения некоторой переменной (наибольшие выборки берутся в стратах с большей вариабельностью).

Кластерный сэмплинг отличается тем, что предварительно по степени близости элементов (по некоторому расстоянию между элементами) строятся кластеры элементов, а затем так же, как и в стратифицированном сэмплинге, производится случайный выбор из кластеров.

Литература к разделу 2

1. CRISP-DM. [Электронный ресурс]. URL: <http://www.machinelearning.ru/wiki/index.php?title=Crisp-dm> (дата обращения: 01.09.2025).
2. Брюс П., Брюс Э., Гедек П. Практическая статистика для специалистов Data Science. — СПб.: БХВ-Петербург, 2021. — 352 с.
3. Чжен Э., Казари А. Машинное обучение. Конструирование признаков: принципы и техника для аналитиков. — М.: Эксмо, 2022. — 240 с.
4. Dong G., Liu H. Feature Engineering for Machine Learning and Data Analytics. — Chapman & Hall, 2018. — 418 p.
5. Литтл Р. Дж. А., Рубин Д. Б. Статистический анализ данных с пропусками. — М.: Финансы и статистика, 1991. — 336 с.
6. Снитюк В. Е. Прогнозирование. Модели, методы, алгоритмы. — Киев: "Маклаут", 2008. — 364 с.
7. Handbook of Missing Data Methodology / Molenberghs G., Fitzmaurice G., Kenward M. G., Tsiatis A., Verbeke G. — Chapman and Hall/CRC, 2014. — 600 p.
8. Allison P. D. Missing Data. — SAGE Publications, 2001. — 104 p.
9. Craig E. K. Applied Missing Data Analysis. — Guilford Press, 2010 — 377 p.
10. Graham J. W. Missing Data: Analysis and Design. — Springer, 2012. — 360 p.
11. Кластерный метод восстановления пропусков в данных для обучения ИНС / В. В. Аюев, З. Е. Аунг, Чжо Мин Тейн, М. Б. Логинова // Нейрокомпьютеры: разработка, применение. — 2009. — № 7. — С. 23–34.
12. Злоба Е., Яцкив И. Статистические методы восстановления пропущенных данных // Computer Modelling & New Technologies. — 2002, vol. 6. — № 1. — Р. 51–61.
13. Van Buuren S., Groothuis-Oudshoorn K. mice: Multivariate Imputation by Chained Equations in R // Journal of Statistical Software. — 2011, Vol. 45, No. 3. — Р. 1–67.
14. Multiple imputation by chained equations: what is it and how does it work? / M. J. Azur, E. A. Stuart, C. F., P. Leaf // International Journal of Methods in Psychiatric Research. — 2011, Vol. 20. — No. 1. — Р. 40–49.
15. Wulff J., Ejlskov L. Multiple Imputation by Chained Equations in Praxis: Guidelines and Review // The Electronic Journal of Business Research Methods. — 2017, Vol. 15. — No. 1. — Р. 41–56.
16. Van Buuren S. Flexible Imputation of Missing Data. — Chapman & Hall, 2018. — 444 p.
17. Brownlee J. Why One-Hot Encode Data in Machine Learning?

- URL: <https://machinelearningmastery.com/why-one-hot-encode-data-in-machine-learning/> (дата обращения 01.09.2025).
18. Baijaya R. All about Categorical Variable Encoding. [Электронный ресурс]. URL: <https://towardsdatascience.com/all-about-categorical-variable-encoding-305f3361fd02> (дата обращения: 01.09.2025).
19. Жерон О. Прикладное машинное обучение с помощью Scikit-Learn, Keras и TensorFlow: концепции, инструменты и техники для создания интеллектуальных систем. — СПб.: ООО "Альфа-книга", 2020. — 1040 с.
20. Мюллер А., Гвидо С. Введение в машинное обучение с помощью Python. — СПб.: Вильямс, 2019. — 480 с.
21. Raza M. S., Qamar U. Understanding and Using Rough Set Based Feature Selection: Concepts, Techniques and Applications. — Springer, 2019. — 236 p.
22. Кобзарь А. И. Прикладная математическая статистика. Для инженеров и научных работников. — М.: ФИЗМАТЛИТ, 2006. — 816 с.
23. Элбон К. Машинное обучение с использованием Python. Сборник рецептов. — СПб.: БХВ-Петербург, 2019. — 384 с.
24. Алгоритмы: построение и анализ / Т. Кормен., Ч. Лейзерсон, Р. Ривест, К. Штайн. — М.: Вильямс, 2005. — 1296 с.
25. Лассо. [Электронный ресурс].
URL: <http://www.machinelearning.ru/wiki/index.php?title=Лассо> (дата обращения: 01.09.2025).
26. Ридж-регрессия. [Электронный ресурс].
URL: <http://www.machinelearning.ru/wiki/index.php?title=Ридж-регрессия> (дата обращения: 01.09.2025).
27. Эластичная сеть. [Электронный ресурс].
URL: http://www.machinelearning.ru/wiki/index.php?title=Эластичная_сеть (дата обращения: 01.09.2025).
28. Ежов А. А., Шумский С. А. Нейрокомпьютинг и его применения в экономике и бизнесе. — М.: МИФИ, 1998. — 222 с.
29. Bishop C. M. Neural Networks for Pattern Recognition. — Oxford University Press, 1995. — 496 p.
30. PCA Whitening. [Электронный ресурс].
URL: <http://ufldl.stanford.edu/tutorial/unsupervised/PCAWhitening/> (дата обращения: 01.09.2025).
31. Kessy A., Lewin A., Strimmer K. Optimal whitening and decorrelation. [Электронный ресурс]. URL: <https://arxiv.org/pdf/1512.00809.pdf> (дата обращения: 01.09.2025).
32. Большаков А. А., Каримов Р. Н. Методы обработки многомерных данных и временных рядов. — М.: Горячая линия-Телеком, 2007. — 522 с.

33. Померанцев А. Метод Главных Компонент (PCA). [Электронный ресурс]. URL: <https://www.chemometrics.ru/books/%D0%9C%D0%B5%D1%82%D0%BE%D0%B4-%D0%B3%D0%BB%D0%B0%D0%B2%D0%BD%D1%8B%D1%85-%D0%BA%D0%BE%D0%BC%D0%BF%D0%BE%D0%BD%D0%B5%D0%BD%D1%82.pdf> (дата обращения: 01.09.2025).
34. Прикладная статистика. Классификация и снижение размерности / С. А. Айвазян, В. М. Бухштабер, И. С. Енюков, Л. Д. Мешалкин. — М.: Финансы и статистика, 1989. — 607 с.
35. Горбаченко В. И. Вычислительная линейная алгебра с примерами на MATLAB. — СПб.: БХВ-Петербург, 2011. — 320 с.
36. Фукунага К. Введение в статистическую теорию распознавания образов. — М.: Наука. Главная редакция физико-математической литературы, 1979. — 368 с.
37. Cangelosi R., Goriely A. Component retention in principal component analysis with application to cDNA microarray data // Biology Direct 2007, 2:2. [Электронный ресурс]. URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC1797006/> (дата обращения: 01.09.2025).
38. Брайтон С. Л., Куц Дж. Н. Анализ данных в науке и технике. — М.: ДМК Пресс, 2021. — 574 с.
39. Сирота А. А. Методы и алгоритмы анализа данных и их моделирование в MATLAB. — СПб.: БХВ-Петербург, 2016. — 384 с.
40. Cover T. M. Geometrical and Statistical Properties of Systems of Linear Inequalities with Applications in Pattern Recognition // IEEE Transactions on Electronic Computers. — 1965, Vol. EC-14. Issue 3. — P. 326–334.
41. Хайкин С. Нейронные сети: полный курс. — М.: Вильямс, 2006. — 1104 с.
42. Мэрфи К. П. Вероятностное машинное обучение: введение. — М.: ДМК Пресс, 2002. — 990 с.
43. Соколов Е. А. Ядра в машинном обучении. [Электронный ресурс]. URL: <https://github.com/esokolov/ml-course-hse/blob/master/2017-spring/lecture-notes/lecture16-kernels.pdf> (дата обращения: 01.09.2025).
44. Воеводин В. В., Воеводин Вл. В. Энциклопедия линейной алгебры. Электронная система ЛИНЕАЛ. — СПб.: БХВ-Петербург, 2006. — 544 с.
45. Scholkopf B., Smola A., Muller K.-R. Nonlinear component analysis as a kernel eigenvalue problem // Artificial Neural Networks — ICANN '97 / 7th International Conference Lausanne, Switzerland, October 8–10, 1997. Proceedings. — Springer, 1997. — P. 583–588.

46. Смирнов В. И. Курс высшей математики, Т.4. Ч.1. — М.: Изд-во "Наука", 1974. — 336 с.
47. Аггарвал Ч. Нейронные сети и глубокое обучение. — СПб.: ООО "Диалектика", 2020. — 752 с.
48. Рашка С., Мирджалили В. Python и машинное обучение: машинное и глубокое обучение с использованием Python, scikit-learn и TensorFlow 2. — СПб.: ООО "Диалектика", 2020. — 848 с.
49. Методы аугментации обучающих выборок в задачах классификации изображений / С. О. Емельянов, А. А. Иванова, Е. А. Швеи, Д. П. Николаев // Сенсорные системы, 2018, том 32. — № 3. — С. 236–245.
50. Shorten C., Khoshgoftaar T. M. A survey on Image Data Augmentation for Deep Learning // Journal of Big Data, 2019, vol. 1. — no 1. — P. 1–48.
51. El Emam K., Mosquera L., Hoptroff R. Practical Synthetic Data Generation: Balancing Privacy and the Broad Availability of Data. — Sebastopol: O'Reilly Media, 2020. — 166 p.
52. Synthetic Data Generation.
URL: <https://www.pdpc.gov.sg/-/media/files/pdpc/pdf-files/other-guides/proposed-guide-on-synthetic-data-generation.pdf> (дата обращения: 01.09.2025).
53. Kingma D. P., Welling M. Auto-Encoding Variational Bayes".
URL: <https://arxiv.org/abs/1312.6114?context=cs.LG/> (дата обращения: 01.09.2025).
54. Goodfellow I. J. et al. Generative Adversarial Networks.
URL: <https://arxiv.org/abs/1406.2661> (дата обращения: 01.09.2025).
55. Sohl-Dickstein J., Maheswaranathan N., Ganguli S. Deep Unsupervised Learning using Nonequilibrium Thermodynamics.
URL: <https://arxiv.org/abs/1503.03585/> (дата обращения: 01.09.2025).
56. Song Y., Ermon S. Generative Modeling by Estimating Gradients of the Data Distribution. URL: <https://arxiv.org/abs/1907.05600/> (дата обращения: 01.09.2025).
57. Hi J., Jain A., Abbeel P. Denoising Diffusion Probabilistic Models. URL: <https://arxiv.org/abs/2006.11239/> (дата обращения: 01.09.2025).
58. Шитиков В. К., Розенберг Г. С. Рандомизация и бутстреп: статистический анализ в биологии и экологии с использованием R. — Тольятти: "Кассандра", 2013. — 305 с. [Электронный ресурс].
URL: <https://elibrary.ru/item.asp?id=26615659> (дата обращения: 01.09.2025).
59. Эфрон Б. Нетрадиционные методы многомерного статистического анализа. — М.: Финансы и статистика, 1988. — 263 с.
60. Паклин Н. Б., Орешков В. И. Бизнес-аналитика: от данных к знаниям. — СПб.: Питер, 2013. — 704 с.

3. РЕГРЕССИЯ

3.1. Линейная регрессия

3.1.1. Линейная регрессия. Определение параметров регрессии методом наименьших квадратов

Регрессия — математическое выражение, отражающее зависимость зависимой переменной y от независимых переменных x при условии, что это выражение будет иметь статистическую значимость [1]. Такую зависимость представим в виде

$$y = \psi(\mathbf{x}, \mathbf{a}),$$

где $\mathbf{a}$ — вектор неопределенных параметров модели.

В *линейной регрессии* значение y предсказывается на основе обучающих примеров (x_{ij}, y_j) с помощью линейной функции

$$y_i = a_0 + \sum_{j=1}^n a_j x_{ji}, \quad (3.1)$$

где y_i — зависимая переменная (отклик) для наблюдения i , $i = 1, 2, \dots, n$, x_{ij} — j -я независимая переменная (факторный признак, предиктор) $j = 1, 2, \dots, k$ для наблюдения i , a_0 — свободный член (может отсутствовать), a_i , $i = 1, 2, \dots, n$ — коэффициенты регрессии.

Перепишем (3.1) в виде

$$y_i = \sum_{j=0}^n a_j x_{ji}, \quad (3.2)$$

где $x_{0i} = 1$.

В матричной форме выражение (3.2) имеет вид

$$\mathbf{X}\mathbf{a} = \mathbf{y}, \quad (3.3)$$

где $\mathbf{y} = [y_1 \ y_2 \ \dots \ y_n]^T$ — вектор значений зависимой переменной, T — символ транспонирования, $\mathbf{a} = [a_0 \ a_1, \dots, a_k]^T$ — вектор коэффициентов регрессии,

$$\mathbf{X} = \begin{bmatrix} 1 & x_{11} & \dots & x_{1k} \\ 1 & x_{21} & \dots & x_{2k} \\ \vdots & \vdots & & \vdots \\ 1 & x_{n1} & \dots & x_{nk} \end{bmatrix} \quad (3.4)$$

— матрица независимых переменных, которую можно представить в виде

$$\mathbf{X} = [\mathbf{1} \ \mathbf{x}_1 \ \mathbf{x}_2 \ \dots \ \mathbf{x}_k], \quad (3.5)$$

где $\mathbf{x}_i = [x_{1i} \ x_{2i} \ \dots \ x_{ji} \ \dots \ x_{ni}]^T$, $i = 1, 2, \dots, k$, $j = 1, 2, \dots, n$.

Так как $n \gg k$ (число обучающих примеров значительно больше числа слагаемых в модели (3.2)), система (3.3) является переопределенной и найти вектор $\mathbf{a}$, точно удовлетворяющий системе невозможно. Для нахождения приближенного вектора коэффициентов $\tilde{\mathbf{a}}$ традиционно используется *метод наименьших квадратов* [2], минимизирующий квадрат нормы невязки — разности между левой и правой частями (3.3)

$$I = \|\mathbf{r}\|^2 = \|\mathbf{X}\tilde{\mathbf{a}} - \mathbf{y}\|^2 = \sum_{i=1}^n \left(\sum_{j=1}^k \tilde{a}_j x_{ji} - y_i \right)^2. \quad (3.6)$$

Используем необходимое условие экстремума функции (3.6)

$$\frac{\partial I}{\partial \tilde{a}_p} = 0, \quad p = 1, 2, \dots, k. \quad (3.7)$$

Вычисляя частные производные (3.7) с учетом (3.6), получаем

$$\begin{aligned} \sum_{i=1}^n 2 \left(\sum_{j=1}^k \tilde{a}_j x_{ji} - y_i \right) \cdot \frac{\partial}{\partial \tilde{a}_p} \left(\sum_{j=1}^k \tilde{a}_j x_{ji} - y_i \right) &= 0, \\ \sum_{i=1}^n \left(\sum_{j=1}^k \tilde{a}_j x_{ji} - y_i \right) x_{pi} &= 0, \\ \sum_{i=1}^n \sum_{j=1}^k \tilde{a}_j x_{ji} x_{pi} &= \sum_{j=1}^k x_{pj} y_j, \quad p = 1, 2, \dots, k \end{aligned} \quad (3.8)$$

Изменяя порядок суммирования в (3.8), получаем систему линейных алгебраических уравнений — *нормальную систему метода наименьших квадратов*

$$\sum_{i=1}^n \left(\sum_{j=1}^k x_{ji} x_{pi} \right) \tilde{a}_j = \sum_{j=1}^k x_{pj} y_j, \quad p = 1, 2, \dots, k,$$

которая в матричной форме имеет вид

$$\mathbf{X}^T \mathbf{X} \tilde{\mathbf{a}} = \mathbf{X}^T \mathbf{y} \quad (3.9)$$

Матрица системы (3.9) является квадратной с размерами $k \times k$, симметричной. Если все столбцы матрицы признаков $\mathbf{X}$ линейно независимы, то матрица $\mathbf{X}$ имеет полный ранг² $\text{rank } \mathbf{X} = k$ и матрица $\mathbf{X}^T \mathbf{X}$ является *положительно определенной*. Поэтому система (3.9) может быть решена многими известными методами. Однако нормальная система может быть плохо обусловленной, а полученное решение сильно искажено погрешностями. Однако при небольших k

² Рангом системы строк (столбцов) матрицы $\mathbf{A}$ с m строками и n столбцами называется максимальное число линейно независимых строк (столбцов). Несколько строк (столбцов) называются линейно независимыми, если ни одна из них не выражается линейно через другие. Ранг системы строк всегда равен рангу системы столбцов, и это число называется рангом матрицы.

Ранг матрицы — наивысший из порядков всевозможных ненулевых миноров этой матрицы. Ранг нулевой матрицы любого размера равен нулю. Если все миноры второго порядка равны нулю, то ранг равен единице, и т.д.

получается приемлемый результат. Если среди столбцов матрицы признаков $\mathbf{X}$ имеются линейно зависимы, то матрица $\mathbf{X}$ будет матрицей неполного ранга $\text{rank } \mathbf{X} < k$, определитель матрицы равен нулю и система не имеет решения.

В современном машинном обучении для нахождения приближенного вектора коэффициентов $\tilde{\mathbf{a}}$ применяется *сингулярное разложение* (SVD-разложение). Получение сингулярного разложения выходит за рамки нашего курса. Будем считать, что сингулярное разложение получено с использованием одной из библиотек программ.

Рассмотрим использование SVD-разложения для *решения переопределенной задачи наименьших квадратов* (3.3) в общем случае, когда $\text{rank } \mathbf{X} = r < k$. Пусть получено сингулярное разложение матрицы $\mathbf{X}$

$$\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T. \quad (3.10)$$

Известно [3], что *ранг матрицы* совпадает с числом ее ненулевых сингулярных чисел. Действительно, если $\mathbf{X}$ — квадратная матрица $n \times n$, то из (3.10) следует, что $|\det \mathbf{X}| = \sigma_1 \sigma_2 \dots \sigma_n$, так как определитель произведения матриц равен произведению определителей, определитель ортогональной матрицы по модулю равен единице, а определитель диагональной матрицы равен произведению диагональных элементов. Если $\text{rank } \mathbf{X} < k$, то $\det \mathbf{X} = 0$, так как на диагонали матрицы $\mathbf{\Sigma}$ будут нулевые элементы. Тогда матрица $\mathbf{\Sigma}$ имеет вид

$$\mathbf{\Sigma} = \left[\begin{array}{c|c} \hat{\mathbf{\Sigma}} & \mathbf{0} \\ \hline \mathbf{0} & \mathbf{0} \end{array} \right],$$

где $\hat{\mathbf{\Sigma}} = \text{diag}\{\sigma_1 \sigma_2 \dots \sigma_r\}$.

Так как умножение на ортогональную матрицу не изменяет евклидову норму вектора, то с учетом сингулярного разложения (3.10) представим евклидову норму невязки в виде

$$\|\mathbf{y} - \mathbf{X}\mathbf{a}\| = \|\mathbf{U}^T(\mathbf{y} - \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T\mathbf{a})\| = \|\mathbf{U}^T\mathbf{y} - \mathbf{\Sigma}(\mathbf{V}^T\mathbf{a})\|.$$

Обозначим $\mathbf{c} = \mathbf{U}^T\mathbf{y}$ и $\mathbf{z} = \mathbf{V}^T\mathbf{a}$. Тогда

$$\|\mathbf{y} - \mathbf{X}\mathbf{a}\|^2 = \|\mathbf{c} - \mathbf{\Sigma}\mathbf{z}\|^2 = \sum_{i=1}^r (c_i - \sigma_i z_i)^2 + \sum_{i=r+1}^m c_i^2. \quad (3.11)$$

В (3.11) ненулевые подматрицы $\hat{\mathbf{\Sigma}}$ матрицы $\mathbf{\Sigma}$ имеет размер $r \times r$, а вектор $\mathbf{c}$ имеет размер $m \times 1$. Выражение (3.11) принимает минимальное значение при

$$z_i = \frac{c_i}{\sigma_i}, \quad i = 1, 2, \dots, r. \quad (3.12)$$

При этом компоненты решения z_i , $i = r+1, \dots, m$ могут принимать произвольные значения. Выбор $z_i = 0$, $i = r+1, \dots, m$ дает решение $\mathbf{a}$ с минимальной нормой, называемое *псевдорешением*. Действительно, решение системы (3.3) равно $\mathbf{a} = \mathbf{V}\mathbf{z}$,

матрица $\mathbf{V}$ ортогональна, поэтому $\|\mathbf{a}\| = \|\mathbf{z}\|$. Таким образом, задача минимальных квадратов имеет единственное псевдорешение.

Введя блочное представление векторов

$$\mathbf{c} = \begin{bmatrix} \hat{\mathbf{c}} \\ \mathbf{d} \end{bmatrix}, \quad \mathbf{z} = \begin{bmatrix} \hat{\mathbf{z}} \\ \mathbf{w} \end{bmatrix},$$

где $\mathbf{c}$ и $\mathbf{z}$ — векторы размером $m \times 1$, а $\hat{\mathbf{c}}$ и $\hat{\mathbf{z}}$ — векторы размером $r \times 1$, запишем алгоритм решения неполноранговой задачи наименьших квадратов с использованием SVD-разложения [3].

1. Вычисляем $\begin{bmatrix} \hat{\mathbf{c}} \\ \mathbf{d} \end{bmatrix} = \mathbf{U}^T \mathbf{y}$.
2. Находим $\hat{\mathbf{z}} = \hat{\Sigma}^{-1} \hat{\mathbf{c}}$.
3. Полагаем $\mathbf{w} = 0$, размерностью $(m - r) \times 1$.
4. Полагаем $\mathbf{z} = \begin{bmatrix} \hat{\mathbf{z}} \\ \mathbf{w} \end{bmatrix}$.
5. Полагаем $\mathbf{a} = \mathbf{V} \mathbf{z}$.

Понятие сингулярного разложения позволило ввести обобщение обратной матрицы — *псевдообратную матрицу Мура-Пенроуза* (Moor-Penrose Pseudoinverse) [3]. Для прямоугольной матрицы $\mathbf{A}$, возможно, неполного ранга псевдообратная матрица имеет вид

$$\mathbf{A}^+ = (\mathbf{U} \Sigma \mathbf{V}^T)^{-1} = \mathbf{V} \Sigma^+ \mathbf{U}^T,$$

где $\Sigma^+ = \begin{bmatrix} \hat{\Sigma}^{-1} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}$.

Зачастую, например, в [3] используют следующее обозначение псевдообратной матрицы: $\mathbf{A}^\dagger$. С помощью псевдообратной матрицы решение произвольной СЛАУ $\mathbf{A} \mathbf{x} = \mathbf{b}$ запишется в виде

$$\mathbf{x} = \mathbf{A}^+ \mathbf{b} = \mathbf{V} \Sigma^+ \mathbf{U}^T \mathbf{b}. \quad (3.13)$$

Выражение (3.13) приводит к тому же результату, что и описанный алгоритм решения неполноранговой задачи наименьших квадратов с использованием SVD-разложения. То есть $\mathbf{A}^+$ играет роль обратной матрицы.

Как видно из (3.12), наличие малых сингулярных чисел приводит к большим погрешностям z_i . На практике все сингулярные числа, меньшие некоторой малой величины, заменяются нулями, а соответствующие значения z_i полагаются равными нулю. Такой прием называется *TSVD — Truncated Singular Value Decomposition* [4]. В простейшем случае сингулярные числа, меньшие абсолютной величины погрешности задания элементов матрицы, считаются равными нулю.

В машинном обучении для вычисления вектора коэффициентов регрессии применяют *градиентные алгоритмы* минимизации функции потерь. В качестве функции потерь для линейной регрессии обычно используется *среднеквадратическая функция потерь* (MSE — *mean-square error*)

$$L = \frac{1}{n} \|\mathbf{r}\|^2 = \frac{1}{n} \|\mathbf{X}\mathbf{a} - \mathbf{y}\|^2 = \frac{1}{n} \sum_{i=1}^n \left(\sum_{j=1}^k a_j x_{ji} - y_i \right)^2$$

Обучение модели простейшим *алгоритмом градиентного спуска* производится итерационным путем. На итерации p вектор параметров корректируется по формуле

$$\mathbf{a}^{(p+1)} = \mathbf{a}^{(p)} - \eta \nabla_{\mathbf{a}^{(p)}},$$

где $\nabla_{\mathbf{a}^{(p)}}$ — вектор градиента функции потерь по вектору параметров, вычисленному на итерации p ; η — подбираемый коэффициент (скорость обучения).

Вектор градиента направлен в сторону скорейшего возрастания функции потерь. Т. к. необходимо минимизировать функцию потерь, то вектор весов корректируется в направлении, противоположном градиенту.

Вектор градиента вычисляется по формуле

$$\nabla_{\mathbf{a}^{(p)}} = \begin{bmatrix} \frac{\partial L(\mathbf{a}^{(p)})}{\partial a_0} \\ \frac{\partial L(\mathbf{a}^{(p)})}{\partial a_1} \\ \vdots \\ \frac{\partial L(\mathbf{a}^{(p)})}{\partial a_k} \end{bmatrix} = \frac{2}{n} \mathbf{X}^T (\mathbf{X}\mathbf{a}^{(p)} - \mathbf{y}), \quad (3.14)$$

$$\text{где } \frac{\partial L(\mathbf{a}^{(p)})}{\partial a_j} = \frac{2}{n} \sum_{i=1}^n \left(\sum_{j=1}^k a_j^{(p)} x_{ji} - y_i \right) x_{ji}.$$

Вычисления по (3.14) используют весь набор обучающих данных, поэтому алгоритм называется *пакетным градиентным спуском*. В случае большого набора обучающих данных этот алгоритм работает очень медленно. Противоположной крайностью является *стохастический градиентный спуск* (SGD — Stochastic Gradient Descent) — на каждом шаге обучения из обучающего набора случайным образом выбирается один пример и вычисляется градиент на основе одного примера. Алгоритм работает быстро, но функция потерь ведет себя крайне нерегулярно. Компромиссом является *мини-пакетный градиентный спуск* (Mini Batch Gradient Descent), когда на каждом шаге обучения из общего обучающего набора выбирается относительно небольшой мини-пакет.

Среднеквадратическая функция потерь является выпуклой функцией. У такой функции отсутствуют локальные минимумы и существует только глобальный минимум. При обучении моделей машинного обучения приходится подбирать скорость обучения η — при малом значении скорости обучения процесс обучения будет идти очень медленно, а при большом значении алгоритм проскочит минимум.

При решении задач регрессии среднеквадратическая ошибка принимает большое значение при наличии выбросов в данных. В задачах регрессии не всегда можно исключить выбросы или заменить правдоподобными значениями (в реальных наблюдениях могут быть резкие изменения значений). Выбросы меньше сказываются при использовании *средней абсолютной ошибки* (MAE — Mean Absolute Error)

$$L_{MAE} = \frac{1}{N} \sum_{i=1}^N |e_i|.$$

Однако MAE имеет серьезную проблему — градиент этой функции не зависит от величины ошибки. Действительно:

$$\frac{\partial L_{MAE}}{\partial a_i} = \frac{1}{n} \sum_{i=1}^n \frac{\partial |\tilde{y}_i - y_i|}{\partial a_i} = \frac{1}{n} \sum_{i=1}^n \frac{\partial |\tilde{y}_i - y_i|}{\partial (\tilde{y}_i - y_i)} \cdot \frac{\tilde{y}_i - y_i}{\partial a_i} = \frac{1}{n} \sum_{i=1}^n \frac{\partial |\tilde{y}_i - y_i|}{\partial (\tilde{y}_i - y_i)} \cdot x_i,$$

где $\tilde{y}_i$ — предсказанное значение,

$$\frac{\partial |\tilde{y}_i - y_i|}{\partial (\tilde{y}_i - y_i)} = \begin{cases} +1, & \text{если } a > 0, \\ -1, & \text{если } a < 0, \\ \text{не определено,} & \text{если } a = 0. \end{cases}$$

Поясним последнее соотношение.

Для $a > 0$: $|a| = a \Rightarrow \frac{\partial |a|}{\partial a} = \frac{\partial a}{\partial a} = 1.$

Для $a < 0$: $|a| = -a \Rightarrow \frac{\partial |a|}{\partial a} = \frac{\partial (-a)}{\partial a} = -1.$

Для $a = 0$ используем определение производной:

$$f'(0) = \lim_{h \rightarrow 0} \frac{|0 + h| - |0|}{h} = \lim_{h \rightarrow 0} \frac{|h|}{h}$$

Правосторонний предел: $\lim_{h \rightarrow 0^+} \frac{|h|}{h} = \lim_{h \rightarrow 0^+} \frac{h}{h} = 1$

Левосторонний предел: $\lim_{h \rightarrow 0^-} \frac{|h|}{h} = \lim_{h \rightarrow 0^-} \frac{-h}{h} = -1$

Пределы не равны, следовательно, производная в нуле не существует.

Так как градиент функции MAE не зависит от величины ошибки, то даже для небольших значений потерь градиент велик. MSE убывает с уменьшением потерь, что обеспечивает сходимость градиентных алгоритмов обучения. Чтобы устранить этот дефект, мы можем использовать переменную скорость обучения, чтобы уменьшить скорость обучения, когда потери близки к минимуму.

В задачах регрессии указанные недостатки решаются использованием функции потерь Хьюбера (Huber loss) [5]

$$L_{\delta} = \begin{cases} \frac{1}{2} e_i^2 & \text{для } |e_i| \leq \delta \\ \delta |e_i| - \frac{1}{2} \delta & \text{иначе} \end{cases}, \quad (3.15)$$

Функция потерь Хьюбера не так чувствительна к выбросам в данных, как квадрат ошибки. Функция дифференцируема в точке 0. По сути, функция потерь Хьюбера является абсолютной ошибкой, но она становится квадратной ошибкой, когда ошибка мала. На рис. 3.7 показан график функции потерь Хьюбера.

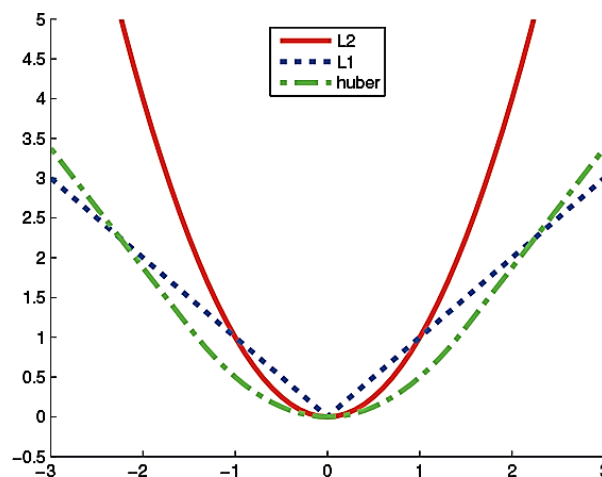


Рис. 3.1. Графики функций потерь [5]. Прогнозируемые значения — ось X, истинное значение 0. L1 — средняя абсолютная ошибка, L2 — средняя квадратическая ошибка

Для малых ошибок функции потерь Хьюбера ведет себя как MSE. Она дифференцируема во всех точках, и ее градиент пропорционален ошибке, что обеспечивает плавную и точную сходимость к минимуму. Для больших ошибок функция ведет себя как MAE. Градиент становится кусочно-постоянным, что ограничивает влияние выбросов на процесс обучения.

Недостатком функции потерь Хьюбера является необходимость подбора параметра δ .

3.1.2. Оценка качества регрессионной модели

Из-за приближенного определения коэффициентов получаем модель

$$\tilde{y}_i = \sum_{j=0}^n \tilde{a}_j x_{ji}.$$

Величины

$$e_i = y_i - \tilde{y}_i$$

называются *остатками* (невязками). Анализ остатков важен с точки зрения адекватности полученной модели. Регрессионный остаток можно выразить в векторной форме: $\mathbf{y} - \tilde{\mathbf{y}}$.

Чем лучше регрессионная модель согласуется с данными, тем меньше величина остатков. Одной из важных оценок качества регрессионной зависимости является *сумма квадратов остатков (ошибок)* (SSE — Sum Squared Errors)

$$SSE = \sum_{i=1}^n (y_i - \tilde{y}_i)^2 = \|\mathbf{y} - \tilde{\mathbf{y}}\|^2 = ((\mathbf{y} - \tilde{\mathbf{y}}), (\mathbf{y} - \tilde{\mathbf{y}})) = (\mathbf{y} - \tilde{\mathbf{y}})^T (\mathbf{y} - \tilde{\mathbf{y}}), \quad (3.16)$$

где $\|\cdot\|$ — евклидова норма, $((\mathbf{y} - \tilde{\mathbf{y}}), (\mathbf{y} - \tilde{\mathbf{y}})) = (\mathbf{y} - \tilde{\mathbf{y}})^T (\mathbf{y} - \tilde{\mathbf{y}})$ — скалярное произведение векторов.

Используется также средняя сумма квадратов ошибки (MSE — Mean Square Error) $MSE = \frac{1}{n} SSE$.

Для практического использования удобна *среднеквадратическая ошибка* модели (Root Mean Squared Error) $RMSE = \sqrt{\frac{SSE}{n}}$.

Полезной характеристикой качества регрессионной модели является *коэффициент детерминации*

$$R^2 = 1 - \frac{\|\mathbf{y} - \tilde{\mathbf{y}}\|^2}{\|\mathbf{y} - \bar{\mathbf{y}}\|^2},$$

где $\bar{\mathbf{y}}$ — среднее значение $\mathbf{y}$.

Если $R^2 = 0$, то регрессия не улучшает качества предсказания $\mathbf{y}$ по сравнению с тривиальным предсказанием $\tilde{\mathbf{y}} = \bar{\mathbf{y}}$, где $\bar{\mathbf{y}}$ — среднее значение компонентов вектора $\mathbf{y}$. Если $R^2 = 1$, то это означает, что вектор отклика регрессионной модели $\tilde{\mathbf{y}}$ равен вектору $\mathbf{y}$ наблюдений.

Коэффициент детерминации, обозначаемый как R^2 — это статистическая мера, которая представляет *долю дисперсии* зависимой переменной (отклика) $\tilde{\mathbf{y}}$, которая предсказуема по независимым переменным (предикторам) $\mathbf{x}$ в модели линейной регрессии.

Проще говоря, он отвечает на вопрос: "Какой процент общего варьирования результата может быть объяснен нашей регрессионной моделью?"

Формула для R^2 выводится из фундаментального разложения общей дисперсии: общая сумма квадратов (SST) = Объясненная сумма квадратов (SSR) + Сумма квадратов ошибок (SSE)

$$SST = SSR + SSE ,$$

где $SST = \sum_{i=1}^n (y_i - \bar{y})^2$ — общая сумма квадратов (Total Sum of Squares, SST), измеряет общую вариацию в наблюдаемых данных y_i вокруг их среднего значения $\bar{y}$;

$SSR = \sum_{i=1}^n (\tilde{y}_i - \bar{y})^2$ — объясненная сумма квадратов (Sum of Squares due to Regression, SSR), измеряет вариацию, объясненную линией регрессии, это разница между предсказанными значениями $\tilde{y}_i$ и средним значением $\bar{y}$;

$SSE = \sum_{i=1}^n (y_i - \tilde{y}_i)^2$ — сумма квадратов ошибок (Sum of Squares of Errors/Residuals, SSE) — измеряет необъясненную вариацию, то есть расхождение между наблюдаемыми y_i и предсказанными $\tilde{y}_i$ значениями.

Исходя из этого, R^2 вычисляется как:

$$R^2 = \frac{SSR}{SST} = \frac{SST - SSE}{SST} = 1 - \frac{SSE}{SST} = 1 - \frac{\sum_{i=1}^n (y_i - \tilde{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} = 1 - \frac{\|\mathbf{y} - \tilde{\mathbf{y}}\|^2}{\|\mathbf{y} - \bar{\mathbf{y}}\|^2}.$$

R^2 характеризует несколько ключевых свойств регрессионной модели:

1. *Качество аппроксимации.* Это основное свойство. R^2 количественно определяет, насколько хорошо линия регрессии приближает реальные точки данных. R^2 , равный 0,80, означает, что 80% дисперсии исходных данных объясняется моделью, а оставшиеся 20% — необъяснимый случайный шум.

2. *Сила линейной связи.* Высокий R^2 указывает на сильную линейную связь, а низкий — на слабую.

3. *Прогностическая сила.* Модель с более высоким R^2 будет в среднем иметь предсказания, которые ближе к фактическим наблюдаемым значениям для данных, на которых она обучалась. Это известно как соответствие на обучающей выборке (in-sample fit — внутривыборочная аппроксимация).

R^2 является показателем *объясняющей силы*, но не обязательно *прогностической точности* или общей корректности модели.

Хотя высокий R^2 часто связан с точной моделью, опасно полагаться только на него. Вот почему R^2 может быть несовершенным и неполным индикатором точности:

1. R^2 не указывает на причинно-следственную связь. Высокий R^2 не означает, что изменения в независимых переменных вызывают изменения в выходной переменной. Связь может быть ложной или обусловлена скрытой переменной, не включенной в модель.

2. R^2 может быть искусственно завышен за счет добавления независимых переменных (предикторов). Добавление любой переменной, даже совершенно нерелевантной, никогда не уменьшит R^2 . Это приводит к переобучению (overfitting), когда модель подстраивается под шум в обучающих данных, а не под основную закономерность. Переобученная модель имеет высокий R^2 на обучающих данных, но будет плохо работать (быть неточной) на новых, ранее не встречавшихся данных. Для снижения влияния неинформативных признаков используется скорректированный (adjusted) коэффициент детерминации

$$R_{adj}^2 = 1 - \frac{\|y - \tilde{y}\|^2 / (n - k - 1)}{\|y - \bar{y}\|^2 / (n - 1)} = 1 - (1 - R^2) \frac{(n - 1)}{(n - k - 1)},$$

который штрафует модель за добавление неинформативных предикторов.

3. Высокий R^2 не означает, что модель верна. Модель может быть неверно определена за по таким причинам:

– *Нелинейность*. R^2 измеряет силу линейной связи. Если истинная связь нелинейна (например, квадратичная, циклическая), линейная модель все равно может дать неплохой R^2 , будучи неточным представлением реальности. Остатки (невязки) будут большими.

– *Гетероскедастичность*. R^2 предполагает постоянную дисперсию ошибок (гомоскедастичность). Если разброс остатков изменяется вдоль диапазона изменения независимых переменных (гетероскедастичность), надежность модели и интерпретация R^2 ставятся под сомнение.

– *Выбросы*. Один единственный экстремальный выброс может резко завысить или занижить значение R^2 , создавая ложное впечатление об общем качестве модели.

4. Низкий R^2 не означает, что модель бесполезна. В некоторых областях (например, в социальных науках, медицине) изучаемые процессы по своей природе зашумлены, и даже низкий R^2 (например, 0,10–0,20) может иметь смысл, и модель может иметь значительную прогностическую ценность. Ключевой момент — является ли связь статистически значимой (низкое р-значение для F-статистики или коэффициентов).

5. R^2 — относительная, а не абсолютная мера. R^2 — это доля объясненной дисперсии относительно простой модели, использующей только среднее значение. Он не говорит вам о абсолютной величине ошибок предсказания. Две модели с

одинаковым R^2 могут иметь совершенно разные уровни точности в своих прогнозах.

Надежная оценка модели включает:

1. *Использование R^2 и скорректированного R^2 для оценки объясняющей силы, особенно при сравнении моделей на одном наборе данных.*

2. *Анализ остатков — необходима проверка на случайность и гомоскедастичность. Любая структура в остатках указывает на неверно специфицированную модель, независимо от значения R^2 .*

3. *Проверка статистической значимости — модель в целом (F-тест) и ключевые коэффициенты (t-тесты) должны быть статистически значимы.*

4. *Использование абсолютных метрик ошибки RMSE (Среднеквадратичная ошибка) или MAE (Средняя абсолютная ошибка) для оценки величины ошибок предсказания.*

5. *Валидация на тестовых данных. Главный тест на точность и способность модели к обобщению (generalizability) — это ее показатели на отложенной выборке (holdout sample) или с помощью перекрестной проверки (cross-validation). R^2 модели на тестовых данных — гораздо более честный показатель ее истинной прогностической силы.*

Таким образом, коэффициент детерминации R^2 характеризует долю дисперсии, объясненную моделью, и является показателем ее качества аппроксимации на обучающей выборке и силы линейной связи. Однако он не является достаточным показателем общей точности, прогностической силы или корректности модели и должен интерпретироваться в сочетании с другими диагностиками и метриками ошибок.

Наглядное представление о качестве линейной регрессии дает *график рассеяния предсказанных значений относительно тестовых* (рис. 3.1).

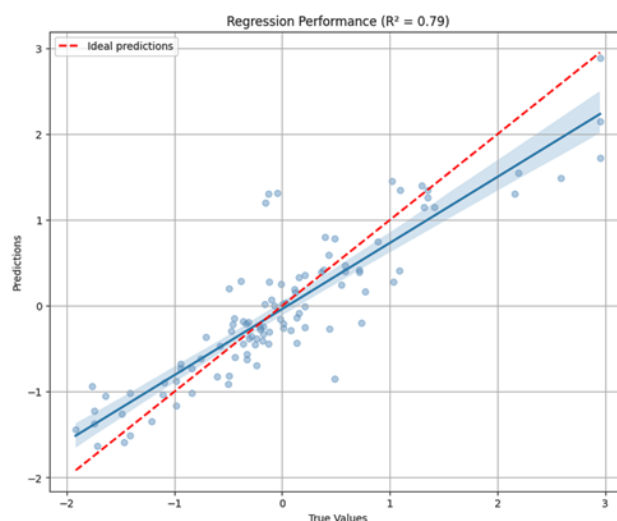


Рис. 3.1. Пример графика рассеяния предсказанных значений относительно тестовых (построен с использованием библиотеки seaborn)

Строится диаграмма рассеяния (scatter-plot) показывает разброс предсказанных значений относительно тестовых значений. Красная линия на рис. 3.1 — это эталонная линия, означающая идеальные предсказания, когда предсказанные значения равны тестовым. Это аппроксимация зависимости между истинными значениями и предсказанными. Синяя линия показывает, как в среднем предсказания соотносятся с тестовыми значениями. Если синяя линия совпадает с красной, это означает идеальные предсказания. Если синяя линия отклоняется, то модель систематически завышает/занижает предсказания. Синяя линия — не предсказания модели, а аппроксимация связи $y_{\text{test}} \sim y_{\text{pred}}$. Синяя линия строится как линейная регрессия методом наименьших квадратов и задается уравнением: $y_{\text{pred}} = a * y_{\text{test}} + b$, где: a — угол наклона (slope), b — смещение (intercept). Вокруг линии часто рисуется полупрозрачная область — доверительный интервал. Это 95% доверительный интервал для предсказаний (параметр `ci=95` в `regplot()`). Широкая полупрозрачная область вокруг синей линии означает большую неопределенность в предсказаниях.

Полезно также строить *гистограммы распределения остатков* (невязок) рис. 3.2.

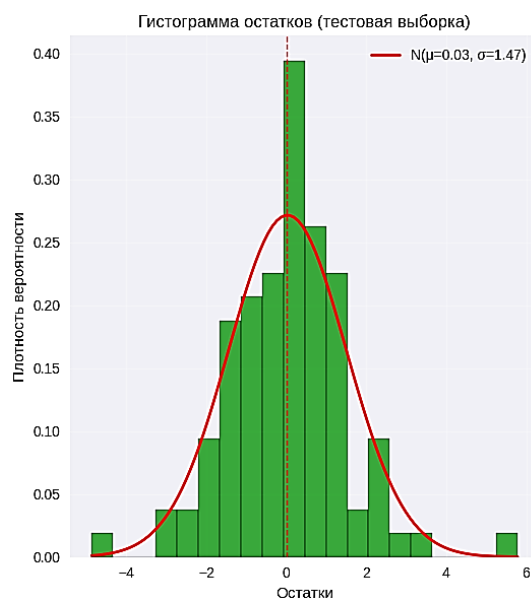


Рис. 3.2. Гистограмма распределения остатков

Гистограмма остатков показывает:

- Форму распределения ошибок
- Наличие выбросов
- Симметричность относительно нуля
- Соответствие нормальному распределению

Идеальная гистограмма:

- Колоколообразная форма
- Симметрична относительно нуля

- Нет выраженных выбросов
- Соответствует нормальному распределению

Возможные проблемы:

- Скошенность — нелинейность зависимости
- Тяжелые хвосты — наличие выбросов
- Смещение от нуля — систематическая ошибка

Полезно также провести статистический анализ остатков.

3.1.3. Регуляризованная регрессия

Регрессия по методу наименьших квадратов может быть неустойчивой, то есть сильно зависеть от обучающих данных. Неустойчивость возникает за счет плохой обусловленности матрицы $\mathbf{X}^T \mathbf{X}$ в (3.9). Плохая обусловленность $\mathbf{X}^T \mathbf{X}$ возникает в случае *мультиколлинеарности* [6], когда столбцы матрицы $\mathbf{X}$ почти линейно зависимы, а среди собственных чисел имеются очень маленькие значения. В результате решение системы (3.9) очень сильно зависит от погрешностей исходных данных, что приводит к большим погрешностям вычисления коэффициентов регрессии.

В этом случае применяется *метод регуляризации* [7] — накладываются дополнительные условия. Например, в функционал (3.6) добавляется регуляризирующее слагаемое

$$I = \|\mathbf{r}\|^2 + \beta \|\tilde{\mathbf{a}}\|^2 = \|\mathbf{X}\tilde{\mathbf{a}} - \mathbf{y}\|^2 + \beta \|\tilde{\mathbf{a}}\|^2, \quad (3.17)$$

где β — коэффициент регуляризации, малая подбираемая величина.

Тогда для получения коэффициентов регрессии в отличие от (3.9) необходимо решить систему

$$(\mathbf{X}^T \mathbf{X} + \beta \mathbf{E}) \tilde{\mathbf{a}} = \mathbf{X}^T \mathbf{y}, \quad (3.18)$$

где $\mathbf{E}$ — единичная матрица.

Известно [8] оценка для числа обусловленности матрицы

$$\text{cond}(\mathbf{X}^T \mathbf{X}) \geq \frac{|\lambda(\mathbf{X}^T \mathbf{X})|_{\max}}{|\lambda(\mathbf{X}^T \mathbf{X})|_{\min}} = \frac{\lambda(\mathbf{X}^T \mathbf{X})_{\max}}{\lambda(\mathbf{X}^T \mathbf{X})_{\min}},$$

так как матрица $\mathbf{X}^T \mathbf{X}$ является положительно определенной.

Также известно [8], что у матриц $\mathbf{X}^T \mathbf{X}$ и $\mathbf{X}^T \mathbf{X} + \beta \mathbf{E}$ собственные вектора совпадают, а собственным значением различаются на β . Поэтому число обусловленности для матрицы $\mathbf{X}^T \mathbf{X} + \beta \mathbf{E}$ равно

$$\text{cond}(\mathbf{X}^T \mathbf{X} + \beta \mathbf{E}) \geq \frac{\lambda(\mathbf{X}^T \mathbf{X})_{\max} + \beta}{\lambda(\mathbf{X}^T \mathbf{X})_{\min} + \beta}. \quad (3.19)$$

Плохая обусловленность матрицы $\mathbf{X}^T\mathbf{X} + \beta\mathbf{E}$ возникает из-за малого значения $\lambda(\mathbf{X}^T\mathbf{X})_{\min}$. Прибавление к малому знаменателю малого коэффициента регуляризации существенно увеличивает знаменатель, а большой числитель увеличивается несущественно. Это уменьшает число обусловленности.

Полученная таким образом регрессия называется *гребневой регрессией* (ridge regression, ридж-регрессия) [1, 9], так как к матрице $\mathbf{X}^T\mathbf{X}$ добавляется *гребень* $\beta\mathbf{E}$. Если $\beta = 0$, то гребневая регрессия переходит в обычную линейную регрессию. Увеличение β делает решение более гладким и устойчивым, но при этом невязка увеличивается и точность решения падает.

Метод лассо, LASSO (Least Absolute Shrinkage and Selection Operator — Наименьшее абсолютное уменьшение и оператор выбора), лассо Тибширани (R. Tibshirani) [10] заключается во введении ограничения на сумму абсолютных величин коэффициентов модели

$$I = \|\mathbf{r}\|^2 + \beta \sum_{j=1}^n |\tilde{a}_j| = \|\mathbf{X}\tilde{\mathbf{a}} - \mathbf{y}\|^2 + \beta \sum_{j=1}^n |\tilde{a}_j|,$$

где n — количество слагаемых в регрессии.

Необходимо отметить, что гребневая регрессия очень чувствительна к параметру регуляризации β .

Эластичная сеть [11] — модель регрессии с двумя регуляризаторами: на квадрат нормы вектора коэффициентов и на сумму абсолютных величин коэффициентов

$$I = \|\mathbf{r}\|^2 + \beta_1 \|\tilde{\mathbf{a}}\|^2 + \beta_2 \sum_{j=1}^n |\tilde{a}_j| = \|\mathbf{X}\tilde{\mathbf{a}} - \mathbf{y}\|^2 + \beta_1 \|\tilde{\mathbf{a}}\|^2 + \beta_2 \sum_{j=1}^n |\tilde{a}_j|.$$

Гребневая регрессия и метод лассо являются частными случаями эластичной сети.

3.2. Логистическая регрессия

Логистическая регрессия — это разновидность *множественной регрессии* [1], общее назначение которой состоит в анализе связи между несколькими независимыми переменными (называемыми также регрессорами или предикторами) и зависимой переменной. *Бинарная логистическая регрессия* применяется в случае, когда зависимая переменная является бинарной (т.е. может принимать только два значения). Иными словами, с помощью логистической регрессии можно оценивать вероятность того, что событие наступит для конкретного испытуемого (больной/здоровый, возврат кредита/дефолт и т.д.).

Рассмотрим логистическую регрессию, следуя [12–13]. Как известно, все регрессионные модели могут быть записаны в виде формулы

$$y = F(x_1, x_2, \dots, x_n),$$

где y — зависимая переменная, $x_1, x_2, \dots, x_n$ — независимые переменные.

Например, во множественной линейной регрессии предполагается, что зависимая переменная является линейной функцией независимых переменных

$$y = a_0 + a_1x_1 + a_2x_2 + \dots + a_nx_n = \mathbf{a}^T \mathbf{x}, \quad (3.20)$$

где $\mathbf{a} = [a_0, a_1, a_2, \dots, a_n]^T$ — вектор-столбец коэффициентов регрессии (параметров), $\mathbf{x} = [1, x_1, x_2, \dots, x_n]^T$ — вектор-столбец независимых переменных.

Можно ли использовать линейную регрессию для задачи оценки вероятности исхода события? Да, можно, вычислив стандартные коэффициенты регрессии. Например, если рассматривается исход по займу, задается переменная y со значениями 1 и 0, где 1 означает, что соответствующий заемщик расплатился по кредиту, а 0, что не расплатится. Однако здесь возникает проблема: множественная регрессия не "знает", что переменная отклика бинарна по своей природе. Это неизбежно приведет к модели с предсказываемыми значениями большими 1 и меньшими 0. Но такие значения вообще не допустимы для первоначальной задачи. Таким образом, множественная регрессия просто игнорирует ограничения на диапазон значений для y .

Для решения проблемы задача регрессии может быть сформулирована иначе: вместо предсказания бинарной переменной мы предсказываем непрерывную переменную со значениями на отрезке $[0,1]$ при любых значениях независимых переменных. Это достигается применением следующего регрессионного уравнения (*логит-преобразование*)

$$P = \frac{1}{1 + e^{-y}}, \quad (3.21)$$

где P — вероятность того, что произойдет интересующее событие, e — основание натуральных логарифмов, y — значение стандартного уравнения линейной регрессии (3.20).

Зависимость, связывающая вероятность события и величину y , показана на рис. 3.3.

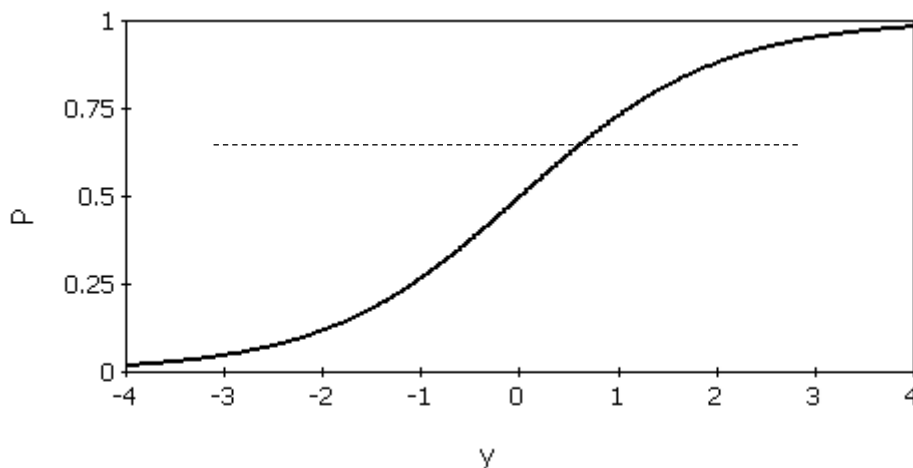


Рис. 3.3. Логистическая кривая [1]

С использованием (3.21) логистическую регрессию можно записать в виде

$$\ln\left(\frac{P}{1-P}\right) = a_0 + a_1x_1 + a_2x_2 + \dots + a_nx_n. \quad (3.22)$$

В справедливости (3.22) несложно убедиться, если вместо P подставить (3.21).

Для отнесения каждого примера к одному из двух классов выход логистической регрессии должен принимать значения 0 или 1. Но выход логистической регрессии никогда не будет точно равен 0 или 1. Чтобы отнести пример к одному из двух классов применяют *порог отсечения*. Если выходное значение логистической регрессии больше порога отсечения, то пример относится к классу, если меньше порогового значения — не относится к классу. Часто принимают порог отсечения равным 0,5.

Для нахождения коэффициентов логистической регрессии часто применяют *метод максимального правдоподобия* [13–14]. Для этого используется обучающая выборка, состоящая из обучающих примеров $\{\mathbf{x}^{(i)}, y^{(i)}\}$, $i = 1, 2, \dots, m$, где $\mathbf{x}^{(i)}$ — векторы независимых переменных, $y^{(i)}$ — соответствующие значения зависимой переменной. Необходимо найти вектор $\mathbf{a}$ коэффициентов регрессии, максимизирующий значение *функции правдоподобия* (*likelihood function*), представляющей собой вероятность совместного появления результатов обучающей выборки $y^{(1)}, y^{(2)}, \dots, y^{(m)}$. При независимых событиях функция правдоподобия имеет вид

$$L(\mathbf{a}) = \prod_{i=1}^m P\{y = y^{(i)} | \mathbf{x} = \mathbf{x}^{(i)}\} = \prod_{i=1}^m P_i(\mathbf{a}, \mathbf{x}^{(i)}). \quad (3.23)$$

В (3.23) предполагается, что вероятность наступления события $y^{(i)} = 1$ при $\mathbf{x}^{(i)}$ равна (3.21)

$$P_i = P_i(\mathbf{a}, \mathbf{x}^{(i)}) = \frac{1}{1 + e^{-\mathbf{a}^T \mathbf{x}^{(i)}}}.$$

Так как $y^{(i)}$ принимает лишь значения 1 или 0, то вероятность второго возможного значения равна $P\{y=0|\mathbf{x}^{(i)}\}=1-P_i$. Для краткости при заданном $\mathbf{x}^{(i)}$ можно записать

$$P\{y_i|\mathbf{x}^{(i)}\}=P_i^{y_i}(1-P_i)^{1-y_i}, \quad y_i \in \{0,1\}. \quad (3.24)$$

Задачу нахождения оптимального значения вектора $\mathbf{a}$ можно записать в следующем виде

$$\hat{\mathbf{a}} = \arg \max_{\mathbf{a}} L(\mathbf{a}),$$

эта запись означает, что необходимо найти аргумент $\mathbf{a}$, обеспечивающий максимум функции $L(\mathbf{a})$.

Нахождение вектора $\hat{\mathbf{a}}$ упрощается, если максимизировать не функцию L (3.23), а натуральный логарифм функции $L^*(\mathbf{a}) = \ln(L(\mathbf{a}))$, т. к. максимум обеих функций достигается при одном и том же значении $\mathbf{a}$. Очевидно, что максимум функции L равен 1, а максимум функции L^* равен нулю (функция L^* отрицательная при неоптимальных значениях $\mathbf{a}$).

Логарифмическая функция правдоподобия с учетом (3.24) равна

$$L^*(\mathbf{a}) = \sum_{i=1}^m [y_i \ln P_i + (1-y_i) \ln(1-P_i)]. \quad (3.25)$$

Для нахождения оптимального вектора $\hat{\mathbf{a}}$ в простейшем случае может быть применен *метод градиентного спуска*

$$\mathbf{a}^{(k+1)} = \mathbf{a}^{(k)} - \alpha \mathbf{g}^{(k)},$$

где k — номер итерации (цикла обучения), $\alpha > 0$ — подбираемый параметр, $\mathbf{g}^{(k)}$ — вектор градиента функции $L^*(\mathbf{a})$, равный [13]

$$\mathbf{g}^{(k)} = \sum_{i=1}^m (y^{(i)} - P_i(\mathbf{a}^{(k)}, \mathbf{x}^{(i)})) \mathbf{x}^{(i)}.$$

Начальное значение $\mathbf{a}^{(0)}$ представляет собой вектор с малыми случайными значениями. Процесс продолжается до тех пор, пока абсолютная величина L^* не станет меньше некоторой малой величины.

Логистическая регрессия реализует двухклассовую (двоичную) классификацию, т. е. может относить пример к одному из двух классов. Используя двоичные классификаторы, можно реализовать многоклассовую классификацию, т. е. разделить примеры на несколько классов. Реализация многоклассовой классификации с помощью двоичных классификаторов использует две стратегии [12]. Если проводится классификация на K классов, то стратегия "один против остальных" (one-vs-rest) предусматривает создание K двоичных классификаторов. Первый классификатор производит классификацию объектов на

два класса: относящихся к первому классу, и всем остальным. Вторым классификатором — относящихся ко второму классу и всем остальным и т.д. При классификации нового объекта признаки объекта подаются на все классификаторы и выбирается класс, чей классификатор дал наибольший результат. Стратегия "один против одного" (one-versus-one) для классификации на K классов создает $\frac{K(K-1)}{2}$ двоичных классификаторов, выделяющих по два класса. Например, при классификации на три класса первый классификатор выделяет первый и второй классы, второй — первый и третий, третий — второй и третий. Во время прогнозирования образец подается на каждый из классификаторов. В качестве класса образца принимается класс, спрогнозированный большинством классификаторов.

Логистическая регрессия позволяет проводить многоклассовую классификацию без построения множества двоичных классификаторов [15]. Для этого используется *мультиномиальная (многопеременная) логистическая регрессия* (Softmax Regression), или *полиномиальная логистическая регрессия* (Multinomial Logistic Regression). Мультиномиальная логистическая регрессионная модель для примера $\mathbf{x}$ сначала, аналогично (3.20) вычисляет сумму очков для каждого класса k

$$s_k(\mathbf{x}) = \mathbf{a}_k^T \mathbf{x},$$

где каждый класс имеет свой вектор параметров $\mathbf{a}_k$. Эти векторы обычно хранят в виде матрицы.

Вероятность принадлежности примера $\mathbf{x}_i$ классу k вычисляется с помощью функции *softmax* (как и в нейронных сетях)

$$p_k^{(i)} = \frac{\exp(s_k(\mathbf{x}_i))}{\sum_{j=1}^K \exp(s_j(\mathbf{x}_i))},$$

где K — количество классов.

Для обучения мультиномиальной логистической регрессии используется функция потерь в форме перекрестной энтропии (кросс энтропии) [15]

$$J = -\frac{1}{m} \sum_{i=1}^m \sum_{k=1}^K y_k^{(i)} \ln(p_k^{(i)}), \quad (3.26)$$

где $y_k^{(i)}$ — целевая вероятность того, что пример $\mathbf{x}_i$ принадлежит классу k , m — количество примеров.

Для обучения мультиномиальной логистической регрессии одним из градиентных алгоритмов для каждого вектора параметров $\mathbf{a}_k$ вычисляется вектор градиента [15]

$$\mathbf{g}_k = \frac{1}{m} \sum_{i=1}^m (p_k^{(i)} - y_k^{(i)}) \mathbf{x}^{(i)}. \quad (3.27)$$

Процесс обучения является итерационным — на каждой итерации вычисляется функция потерь (3.26). Если функция потерь меньше некоторой заданной малой величины, то процесс заканчивается. Иначе для каждого вектора параметров $\mathbf{a}_k$ вычисляется градиент (3.27), с помощью векторов градиента корректируются векторы параметров. После чего происходит переход к новой итерации.

3.3. Нелинейная регрессия

Данные не всегда удобно аппроксимировать линейной регрессией, так как при этом могут возникать большие ошибки. В таких случаях используют нелинейную регрессию. Различают *два класса нелинейных регрессий*:

- регрессии, нелинейные относительно включенных в анализ объясняющих переменных, но линейные по оцениваемым параметрам;
- регрессии, нелинейные по оцениваемым параметрам.

Примером нелинейной регрессии по включаемым в нее объясняющим переменным, но линейной по оцениваемым параметрам может служить одномерная *полиномиальная регрессия*

$$y = a_0 + a_1x + a_2x^2 + \dots + a_nx^n.$$

Примером множественной нелинейной регрессии по включаемым в нее объясняющим переменным может служить регрессия вида

$$y = \Phi_m(x) = a_1\varphi_1(x) + a_2\varphi_2(x) + \dots + a_m\varphi_m(x), \quad (3.28)$$

где $\varphi_1(x), \dots, \varphi_m(x)$ — заданные *базисные функции*, например, треугольные функции, функция Гаусса, $a_1, \dots, a_m$ — искомые параметры (модель является *линейной*, так как искомые параметры входят в нее линейно).

Особенностью таких регрессий является то, что параметрами базисных функций задаются, а определяют только линейные коэффициенты.

Если потребовать, чтобы $\Phi_m(x_i) = y_i$, $i = 1, 2, \dots, n$, то искомые параметры $a_1, a_2, \dots, a_m$ могут быть найдены из решения системы линейных уравнений

$$\mathbf{P}\mathbf{a} = \mathbf{y},$$

где

$$\mathbf{P} = \begin{bmatrix} \varphi_1(x_1) & \varphi_2(x_1) & \cdots & \varphi_m(x_1) \\ \varphi_1(x_2) & \varphi_2(x_2) & \cdots & \varphi_m(x_2) \\ \cdots & \cdots & \cdots & \cdots \\ \varphi_1(x_n) & \varphi_2(x_n) & \cdots & \varphi_m(x_n) \end{bmatrix}, \quad \mathbf{a} = \begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_m \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}.$$

Для нахождения неизвестного вектора $\mathbf{a}$ используется метод наименьших квадратов.

Следует учитывать, что линейная модель может давать большую ошибку, если данные плохо аппроксимируются прямой, т. е. модель не дообучается. Полиномиальная модель высокой степени будет хорошо аппроксимировать конкретный набор обучающих данных, но склонна к переобучению, так как будет адаптироваться под особенности конкретного набора данных, включая погрешности. Для исключения переобучения можно применить *перекрестную проверку* — обучать модели полиномиальной регрессии разной степени на одном наборе данных, а ошибку оценивать на другом (проверочном) наборе.

Примером нелинейной регрессии по оцениваемым параметрам является экспоненциальная функции

$$y = e^{a_0 + a_1 x}.$$

Обучение нелинейной регрессии по оцениваемым параметрам требует настройки параметров, входящих в модель как линейно, так и нелинейно, и применения достаточно сложных алгоритмов обучения, например, градиентного спуска.

Литература к разделу 3

1. Дрейпер Н., Смит Г. Прикладной регрессионный анализ. — М.: Вильямс, 2007. — 912 с.
2. Лоусон Ч., Хенсон Р. Численное решение задач метода наименьших квадратов. — М.: Наука, 1986. — 232 с.
3. Уоткинс Д. С. Основы матричных вычислений. — М.: БИНОМ. Лаборатория знаний, 2006. — 664 с.
4. Каханер Д., Моулер К., Нэш С. Численные методы и программное обеспечение. — М.: Мир, 2001. — 575 с.
5. Мэрфи К. П. вероятностное машинное обучение: введение. — М.: ДМК Пресс, 2002. — 990 с.
6. Большаков А. А., Керимов Р. Н. Методы обработки многомерных данных и временных рядов. — М.: Горячая линия–Телеком, 2007. — 522 с.
7. Тихонов А. Н., Арсенин В. Я. Методы решения некорректных задач. — М.: Наука, 1986. — 288 с.
8. Горбаченко В. И. Вычислительная линейная алгебра с примерами на MATLAB. — СПб.: БХВ-Петербург, 2011. — 320 с.
9. Ридж-регрессия. [Электронный ресурс].
URL: <http://www.machinelearning.ru/wiki/index.php?title=Ридж-регрессия>
(дата обращения: 10.10.2025).
10. Лассо. [Электронный ресурс].
URL: <http://www.machinelearning.ru/wiki/index.php?title=Лассо>
(дата обращения: 10.10.2025).

11. Эластичная сеть. [Электронный ресурс].

URL: http://www.machinelearning.ru/wiki/index.php?title=Эластичная_сеть

(дата обращения: 10.10.2025).

12. Паклин Н. Б., Орешков В. И. Бизнес-аналитика: от данных к знаниям. — 2-е издание. — СПб.: Питер, 2013. — 704 с.

13. Логистическая регрессия и ROC-анализ — математический аппарат [Электронный ресурс].

URL: <http://www.basegroup.ru/library/analysis/regression/logistic/> (дата обращения: 10.10.2025).

14. Магнус Я. Р., Катышев П. К., Пересецкий А. А. Эконометрика. Начальный курс. — М.: Дело, 2004. — 576 с.

15. Жерон О. Прикладное машинное обучение с помощью Scikit-Learn, Keras и TensorFlow: концепции, инструменты и техники для создания интеллектуальных систем. — СПб.: ООО "Диалектика", 2020. — 1040 с.

4. КЛАССИФИКАЦИЯ И КЛАСТЕРИЗАЦИЯ

4.1. Байесовский алгоритм классификации

4.1.1. Теорема Байеса

Вспомним *формулу Байеса* (*Thomas Bayes*) [1–2]. Рассмотрим формулу Байеса применительно к задаче классификации [3]. Пусть имеется объект X , класс которого неизвестен. Пусть также имеется гипотеза H , согласно которой X относится к некоторому классу C . Для задачи классификации можно определить условную вероятность $P(H|X)$, то есть вероятность того, что гипотеза H для X справедлива. Вероятность $P(H|X)$ называется *условной вероятностью* того, что гипотеза H верна при условии, что классифицируется объект X , или *апостериорной вероятностью*.

Предположим, что объектами классификации являются фрукты, которые описываются их цветом и формой. Определим объект X как красный и круглый и выдвинем гипотезу H , что это яблоко. Тогда условная вероятность $P(H|X)$ отражает вероятность того, что объект X является яблоком при условии, что он красный и круглый. Кроме условной (апостериорной) вероятности, рассмотрим так называемую *априорную вероятность* $P(H)$. В нашем примере это вероятность того, что любой наблюдаемый объект является яблоком, безотносительно к тому, как он выглядит. Таким образом, апостериорная вероятность основана на бóльшей информации, чем априорная, не предполагающая зависимость от свойств объекта X . Аналогично $P(X|H)$ — апостериорная вероятность X при условии H , или вероятность того, что X является красным и круглым, если известно, что это яблоко. $P(H)$ — *априорная вероятность* того, что объект является яблоком. $P(X)$ — *априорная вероятность* X , то есть вероятность встретить красный и круглый объект. Вероятности $P(X)$, $P(H)$ и $P(X|H)$ могут быть оценены на основе наблюдаемых данных.

Для вычисления апостериорной вероятности $P(H|X)$ на основе $P(X)$, $P(H)$ и $P(X|H)$ используется формула Байеса

$$P(H|X) = \frac{P(X|H)P(H)}{P(X)}. \quad (4.1)$$

Можно сказать, что формула Байеса позволяет переставить местами причину и следствие. Зная с какой вероятностью, причина приводит к некоему событию, эта

теорема позволяет рассчитать вероятность того, что именно эта причина привела к наблюдаемому событию.

Рассмотрим машинное обучение [4]. Пусть X — набор признаков, характеризующих некоторый объект, c — некоторый класс объектов. Тогда вероятность принадлежности этого объекта классу c вычисляется по формуле Байеса

$$P(c|X) = \frac{P(X|c)P(c)}{P(X)}, \quad (4.2)$$

где $P(c|X)$ — апостериорная вероятность того, что объект с набором признаков X принадлежит классу c ; $P(X|c)$ — вероятность встретить объект с набором признаков X среди всех объектов класса c ; $P(c)$ — априорная вероятность встретить среди всех объектов объект класса c ; $P(X)$ — вероятность встретить объект с набором признаков X среди всех объектов.

Чтобы определить, к какому классу относится объект с набором признаков X , необходимо найти класс $c \in C$, для которого вероятность $P(c|X)$ максимальна. Математически это можно записать так

$$c = \arg \max_{c \in C} P(c|X).$$

Такая гипотеза называется *максимальной апостериорной гипотезой* (*maximum a posteriori hypothesis*, *MAP*).

Перепишем это по формуле Байеса (4.2)

$$c = \arg \max_{c \in C} P(c|X) = \arg \max_{c \in C} \frac{P(X|c)P(c)}{P(X)} = \arg \max_{c \in C} P(X|c)P(c),$$

потому что вероятность $P(X)$ встретить объект от класса c не зависит.

4.1.2. Наивный байесовский классификатор

Казалось бы, для поиска класса, к которому относится объект, необходимо для каждого класса вычислить апостериорную вероятность $P(c|X)$ и выбрать тот класс, для которой эта вероятность максимальна. Но такой подход практически нереализуем, так как очень сложно вычислить $P(X|c)$ — набор признаков X может встречаться у многих классов, а классов может быть очень много. Поэтому на практике применяют "наивный" (простой) байесовский классификатор (*naïve Bayesian classifier*, иногда даже пишут *Idiot's Bayes*). В наивном байесовском классификаторе принимают допущение, что признаки объекта независимы. На самом деле признаки могут быть зависимыми. В этом состоит "наивность"

предположения. Наивный байесовский классификатор, как правило, дает неплохие результаты и легко реализуется.

Если объект описывается набором признаков $X = \{x_1, x_2, \dots, x_n\}$, то при независимых признаках вероятности, входящие в (4.2) легко вычисляются:

$$P(X|c) = P(x_1|c) \cdot P(x_2|c) \cdot \dots \cdot P(x_n|c),$$

$$P(X) = P(x_1) \cdot P(x_2) \cdot \dots \cdot P(x_n).$$

Алгоритм простого байесовского классификатора имеет вид [3].

1. Пусть исходное множество данных S содержит атрибуты $A_1, A_2, \dots, A_n$. Тогда каждый объект $X \in S$ будет представлен своим набором значений этих атрибутов $x_1, x_2, \dots, x_n$, где x_i — значение, которое принимает атрибут A_i в данном наблюдении.

2. Предположим, что задано m классов $C = \{C_1, C_2, \dots, C_m\}$ и наблюдение X , для которого класс неизвестен. Классификатор должен определить, что X относится к классу, который обеспечивает *наибольшую апостериорную вероятность* $P(H|X)$. Говорят, что байесовский классификатор использует оценку *апостериорного максимума* (*Maximum a posteriori estimation*) для определения наиболее вероятного класса. Простой байесовский классификатор относит наблюдение X к классу C_k , где $k = 1, 2, \dots, m$ тогда и только тогда, когда выполняется условие $P(C_k|X) > P(C_j|X)$ для любых $1 \leq j \leq m, k \neq j$.

Вероятность $P(C_k|X)$ определяется по формуле Байеса, вытекающей из (4.1)

$$P(C_k|X) = \frac{P(X|C_k)P(C_k)}{P(X)}. \quad (4.3)$$

3. Поскольку вероятность $P(X)$ для всех классов одинакова, максимизировать требуется только числитель формулы (4.3). Если априорная вероятность класса $P(C_k)$ неизвестна, то можно предположить, что классы равновероятны: $P(C_1) = P(C_2) = \dots = P(C_m)$ и, следовательно, мы должны выбрать максимальную вероятность $P(X|C_k)$. В общем виде априорные вероятности классов могут быть оценены как $P(C_k) = s_k/s$, где s_k — число наблюдений обучающей выборки, которые относятся к классу C_k , а s — общее число обучающих примеров.

4. Если исходное множество данных содержит большое количество атрибутов, то определение $P(X|C_k)$ может потребовать значительных вычислительных

затрат. Чтобы их уменьшить, используется "наивное" предположение о независимости признаков. То есть для набора атрибутов $X = (x_1, x_2, \dots, x_n)$ можно записать

$$P(X|C_k) = P(x_1|C_k) \cdot P(x_2|C_k) \cdot \dots \cdot P(x_n|C_k). \quad (4.4)$$

Вероятности, стоящие в правой части формулы (4.4), могут быть определены из обучающего набора данных для следующих случаев:

- Атрибут A_i является категориальным (качественным). Тогда $P(x_i|C_k) = s_{ik}/s_k$, где s_{ik} — число наблюдений класса C_k , в которых атрибут A_i принимает значение x_i , а s_k — общее число наблюдений, относящихся к классу C_k .
- Атрибут A_i является непрерывным, тогда, обычно, предполагается, что его значения распределены по нормальному закону

$$P(x_i|C_k) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x_i-m)^2}{2\sigma^2}},$$

где m и σ^2 — математическое ожидание и дисперсия значения атрибута A_i для наблюдений, относящихся к классу C_k .

При классификации неизвестного наблюдения объект X будет относиться к классу, для которого $P(X|C_k)P(C_k)$ принимает наибольшее значение.

Рассмотрим классификационные правила на примере классификации фруктов [5]. Имеется обучающий набор данных о 1000 фруктах. Фрукт может быть бананом, апельсином или каким-нибудь другим (это классы). Фрукт может быть длинным, сладким или желтым (это признаки) — таблица.

Таблица 4.1. Обучающий набор

Класс	Длинный	Сладкий	Желтый	Всего
Банан	400	350	450	500
Апельсин	0	150	300	300
Другие	100	150	50	200
Всего	500	650	800	1000

Что мы видим в этом обучающем наборе данных?

- Из 500 бананов 400 длинные, 350 сладкие и 450 желтые;
- Среди 300 апельсинов нет ни одного длинного, но оказалось 150 сладких и 300 желтых.
- Из оставшихся 200 фруктов 100 оказались длинными, 150 сладкими и 50 желтыми.

Если мы получим только параметры — длину, сладость и цвет фрукта (не зная его класса), то сможем вычислить вероятность того, что фрукт окажется бананом, апельсином или чем-то другим. Предположим, что неизвестный фрукт длинный, сладкий и желтый. Вычислим по (4.2) вероятность принадлежности этого фрукта к одному из трех классов, используя предположение о независимости признаков. Будем вычислять только числитель (4.2), так как знаменатель для всех трех классов одинаков.

Для класса "банан" получаем:

- $P(\text{Длинный}|\text{Банан}) = 400/500 = 0.8$,
- $P(\text{Сладкий}|\text{Банан}) = 350/500 = 0.7$,
- $P(\text{Желтый}|\text{Банан}) = 450/500 = 0.9$,
- $P(\text{Банан}) = 500/1000 = 0.5$.

Перемножив эти значения (согласно уравнению), мы получим:

$$P(\text{Банан}|\text{Длинный, Сладкий, Желтый}) = 0.8 \times 0.7 \times 0.9 \times 0.5 = 0.252.$$

Аналогично для классов апельсин и другие получаем:

- $P(\text{Апельсин}|\text{Длинный, Сладкий, Желтый}) = 0$,
- $P(\text{Другие}|\text{Длинный, Сладкий, Желтый}) = 0.01875$.

Наивный байесовский классификатор классифицирует этот длинный, сладкий и желтый фрукт как банан.

4.2. Алгоритм ближайших соседей

Алгоритм k-ближайших соседей (k-nearest neighbor algorithm, KNN) основан на сходстве объектов [6–7]. Алгоритм находит среди всех примеров обучающей выборки *k* примеров (*k*-ближайших соседей), похожих на новый неизвестный ранее объект. На основе классов ближайших соседей выносится решение касательно нового объекта. Алгоритм использует некоторую *метрику* — расстояние между векторами признаков неизвестного объекта и объектов обучающего множества. Поэтому алгоритм относят к *метрическим алгоритмам*. Алгоритм *k*-ближайших соседей относится к *непараметрическим "ленивым" моделям* [6]. В случае параметрических моделей в процессе обучения, используя обучающий набор, находится фиксированный набор параметров заранее выбранной модели. При работе с обученной моделью обучающий набор не используется. Непараметрические методы не используют фиксированный набор параметров [7]. В случае "ленивой" модели обучение вообще не производится.

Чтобы различные признаки оказывали одинаковое влияние на расчет расстояния, перед расчетом расстояния необходима *нормализация количественных признаков*. Используется *минимаксная нормализация*

$$p_{ij} = \frac{x_{ij} - x_{j\min}}{x_{j\max} - x_{j\min}},$$

где p_{ij} — нормализованное значение j -го признака в i -ом примере, x_{ij} — исходное значение j -го признака в i -ом примере (см. табл. 2.1).

Другим подходом является *стандартизация* — приведение к нулевому среднему значению и единичному стандартному отклонению

$$p_{ij} = \frac{x_{ij} - \bar{x}_j}{\bar{\sigma}_j},$$

где $\bar{x}_j = \frac{1}{m} \sum_{i=1}^m x_{ij}$ — выборочная оценка математического ожидания j -го признака,

m — количество обучающих примеров, $\bar{\sigma}_j^2 = \frac{1}{m-1} \sum_{i=1}^m (x_{ij} - \bar{x}_j)^2$ — несмещенная выборочная оценка дисперсии j -го признака.

В случае количественных признаков в качестве расстояния часто используется *евклидово расстояние*

$$d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sqrt{\sum_k |x_k^{(i)} - x_k^{(j)}|^2},$$

где $\mathbf{x}^{(i)}$ и $\mathbf{x}^{(j)}$ — векторы признаков, $x_k^{(i)}$ и $x_k^{(j)}$ — k -е компоненты векторов $\mathbf{x}^{(i)}$ и $\mathbf{x}^{(j)}$, $|\dots|$ — модуль числа (число может быть комплексным).

Часто используется *расстояние Минковского*

$$d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sqrt[p]{\sum_k |x_k^{(i)} - x_k^{(j)}|^p}.$$

При $p=2$ расстояние Минковского становится евклидовым расстоянием, а при $p=1$ — *расстоянием Манхэттена*

$$d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sum_k |x_k^{(i)} - x_k^{(j)}|.$$

При нахождении расстояния иногда учитывают значимость признаков. Значимость определяется аналитиком субъективно, полагаясь на собственный опыт. В таком случае при нахождении расстояния, например, евклидова расстояния, вводятся веса w_k , отражающие значимость признаков

$$d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sqrt{\sum_k w_k |x_k^{(i)} - x_k^{(j)}|^2}.$$

В случае качественных признаков в качестве расстояния применяется *функция отличия*, принимающая значение 1, если признаки равны, и ноль, если признаки не равны.

Алгоритм k ближайших соседей строится следующим образом.

1. Выбрать k — количество ближайших соседей и метрику расстояния. Если принять $k = 1$, то алгоритм потеряет обобщающую способность (то есть способность выдавать правильный результат для данных, не встречавшихся ранее в алгоритме) так как новой записи будет присвоен класс самой близкой к ней. Если установить слишком большое значение, то многие локальные особенности не будут выявлены. Какую выбрать метрику расстояния? Обычно, выбирают евклидово расстояние. В случае большого количества признаков (большой размерности признаков) и наличия выбросов может оказаться полезным расстояние Манхэттена, не возводящее в квадрат разность значений признаков.

2. Найти k ближайших соседей образца, который нужно классифицировать.

3. Определить класс образца по большинству классов соседей. Это невзвешенное голосование. В этом случае может возникнуть проблема, если имеются одинаковые количества примеров, относящихся к разным классам. В этой ситуации полезно взвешенное голосование. Для каждого класса рассчитывается вес

$$v = \sum_i^n \frac{1}{d(\mathbf{x}, \mathbf{y}_i)^2},$$

где $\mathbf{x}$ — классифицируемый образец, $\mathbf{y}_i$ — вектор признаков соседа, принадлежащего классу, n — количество примеров в классе.

Образцу присваивается класс, набравшим наибольшее количество голосов.

Алгоритм ближайших соседей позволяет решать задачи регрессии: искомое значение для образца вычисляется как среднее или взвешенное среднее целевых значений примеров.

4.3. Деревья решений

4.3.1. Понятие дерева решений

Деревья решений (decision trees) [3, 6, 8] — это способ представления правил в иерархической, последовательной структуре. Обычно каждый узел дерева включает проверку одного атрибута (независимой переменной). Иногда в узле две независимые переменные сравниваются друг с другом или вычисляется некоторая функция от одной или нескольких переменных.

Если в узле проверяется *категориальная переменная* (принимаяющая качественные значения), то каждому возможному значению переменной соответствует ветвь, выходящая из узла. Если проверяется *числовая переменная*, то проверяется, больше или меньше это значение некоторой константы. Можно применить *квантование* числовой переменной (разбить область числовой переменной на несколько интервалов) и проверять попадание числовой переменной в один из интервалов.

Листья (конечные узлы дерева) соответствуют зависимой переменной, т. е. классам. Объект принадлежит определенному классу, если значения его независимых переменных удовлетворяют условиям, записанным в узлах дерева на пути от корня к листу, соответствующему этому классу.

Введем некоторые понятия из теории деревьев решений [3, 6, 8]. *Дерево* — граф, в котором нет циклов даже без учета направления ребер. *Объект* — это пример (шаблон, наблюдение). *Атрибут (признак)* — признак (независимая переменная, свойство). *Метка класса* — зависимая переменная (целевая переменная, признак, определяющий класс объекта). *Узел* — внутренний узел дерева (узел проверки). *Лист* — конечный узел дерева (узел решения). *Проверка* — условие в узле. *Правило* — логическая конструкция, представленная в виде "если ... то ...". Пример *структуры дерева решений* показана на рис. 4.1.

При построении дерева решений формируются решающие правила и для каждого из них создается узел. Для каждого узла необходимо выбрать атрибут (*атрибут ветвления*, или *атрибут разбиения* — *splitting attribute*), по которому будет производиться проверка правила. Метод, в соответствии с которым производится выбор атрибута ветвления на каждом шаге, называется *алгоритмом построения дерева решений*. При выборе атрибута ветвления должны обеспечиваться максимальная *чистота (purity)*. Большая чистота означает, что в узле будут представлены в основном объекты, относящиеся к одному классу. В идеальном случае в узле должны быть представлены объекты одного класса (не должно быть "примесей"). Кроме того, разбиение не должно создавать узлы, содержащие мало объектов (это значит, что правило применимо для малого числа объектов).

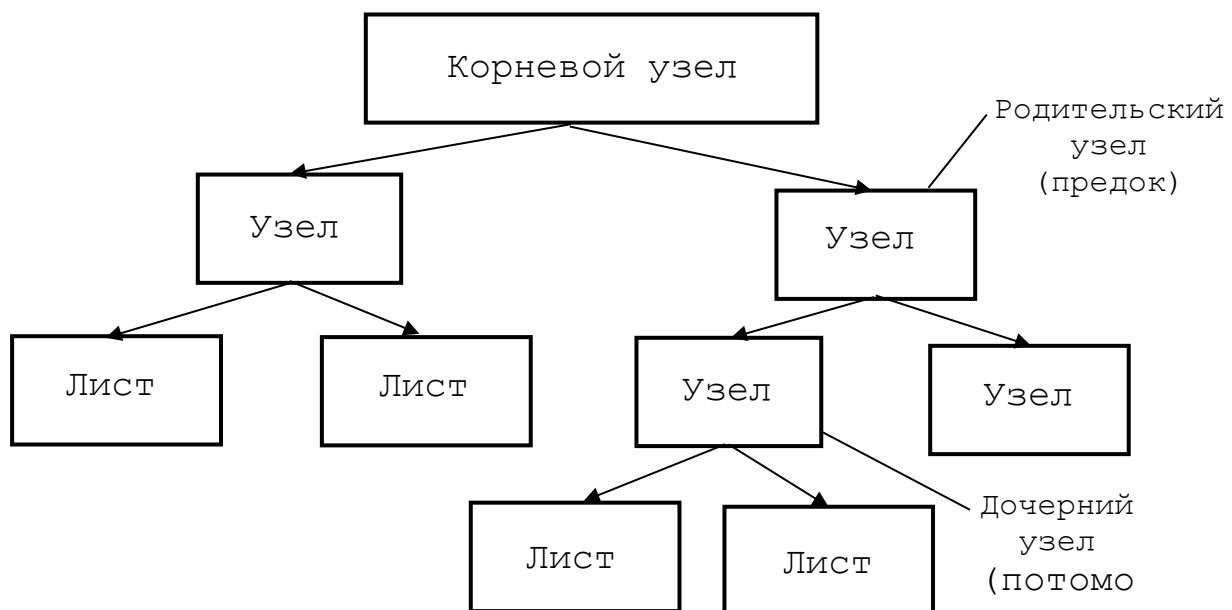


Рис. 4.1. Структура дерева решений

При построении деревьев решений применяют так называемые *жадные алгоритмы (Greedy Algorithm)* [9]. Жадный алгоритм — это алгоритм, заключающийся в принятии локально оптимальных решений на каждом этапе, допуская, что конечное решение также окажется оптимальным. При построении дерева решений жадный алгоритм для каждого узла ищет оптимальный атрибут разбиения. Построение деревьев решений основано на *машинном обучении*.

Для выбора атрибута ветвления используются различные критерии, основанные на теории машинного обучения или на математической статистике. Критерии, основанные на теории машинного обучения, нацелены на повышение чистоты множеств, получающихся в узлах. Критерии, основанные на математической статистике, нацелены на обеспечение статистической значимости различий между распределением значений выходной переменной в узлах. Основными являются следующие *критерии* [3, 6, 8]:

- коэффициент Джини (*Gini Coefficient*), примесь, неопределенность Джини (*Gini impurity*);
- уменьшение энтропии, или прирост информации;
- отношение прироста информации;
- тест хи-квадрат;
- уменьшение дисперсии;
- *F*-критерий Фишера.

Коэффициент Джини (Corrado Gini) — это статистический показатель, с помощью которого можно описывать характер изменения одной величины относительно изменения другой. Основным применением индекса Джини является оценка неравномерности распределения изучаемого признака (например, годового дохода) для различных социальных групп. Этот метод был разработан итальянским статистиком и демографом Коррадо Джини (1884–1965) и впервые опубликован в 1912 г. В настоящее время индекс Джини широко применяется в экономических, социальных и демографических исследованиях. При построении деревьев решений индекс Джини применяется в случае категориальной целевой переменной. Для числовых переменных следует произвести квантование. Индекс Джини определяет вероятность того, что два объекта, случайным образом выбранные из одной совокупности, относятся к одному классу. Для идеально чистой выборки данная вероятность равна 1. *Коэффициент Джини (Gini Coefficient)* представляет собой сумму квадратов долей классов в узле

$$G_i = \sum_{k=1}^s \left(\frac{n_k}{N_i} \right)^2.$$

В машинном обучении обычно *применяют примесь, неопределенность Джини (Gini impurity)*

$$G_i = 1 - \sum_{k=1}^s \left(\frac{n_k}{N_i} \right)^2,$$

где N_i — число объектов в узле i , $\frac{n_k}{N_i}$ — доля примеров класса k среди примеров в i -узле, s — число классов в узле i .

Если в узле содержится равное число объектов, относящихся к двум классам, то $G_i = 0.5^2 + 0.5^2 = 0.5$. Действительно, в этом случае вероятность того, что случайно дважды будет выбран объект, относящийся к одному классу, равна 0.5. Идеально чистый узел имеет значение меры $G_i = 0$.

Критерий уменьшения энтропии (entropy reduction), или прироста информации (information gain) применим в случае категориальной целевой переменной. Для числовых переменных следует произвести квантование. Из теории информации известно, что чем больше состояний может принимать некоторая система, тем больше информации требуется для ее описания. Степень неупорядоченности системы отражает *энтропия* [10]. Если рассмотреть построение дерева решений, то в результате разбиения узлов должны получаться дочерние узлы с более однородными объектами. Поэтому в результате разбиения энтропия должна падать, а количество внутренней информации в узле должно расти. Уменьшение энтропии эквивалентно приросту информации. *Энтропия* узла i определяется формулой

$$H_i = - \sum_{\substack{k=1, \\ p_{i,k} \neq 0}}^n p_{ik} \log_2 p_{ik}, \quad (4.5)$$

где p_{ik} — доля (вероятность) объектов, относящихся к классу k в узле i .

Пусть в произвольном узле i содержится множество объектов, мощность этого множества, т. е. количество примеров, равна N . По формуле (4.5) рассчитаем энтропию H_i . В результате разбиения S для данного узла создано s потомков $T_1, T_2, \dots, T_s$, каждый из которых содержит N_i объектов $i = 1, 2, \dots, s$. Для каждого потомка по (4.5) рассчитывается энтропия. Энтропия разбиения S представляет собой сумму энтропий всех узлов, умноженных на долю записей каждого узла в числе записей исходного множества

$$H(S) = \sum_{j=1}^s \frac{N_j}{N} H_j. \quad (4.6)$$

Мера уменьшения энтропии (entropy reduction), или прироста информации (information gain) в результате разбиения S вычисляется по формуле

$$Gain(S) = H_i - H(S). \quad (4.7)$$

В качестве наилучшего атрибута в разбиении S следует выбирать тот атрибут, который обеспечивает наибольший прирост информации.

Критерий отношения прироста информации (gain-ratio) применим в случае категориальной целевой переменной. Для числовых переменных следует произвести квантование. В критерии используется отношение полного прироста информации после разбиения к оценке потенциальной информации, созданной при разбиении. Потенциальная информация, созданная при разбиении, оценивается по (4.6)

$$SplitInfo(S) = \sum_{j=1}^s \frac{N_j}{N} H_j.$$

Критерий отношения прироста информации (gain-ratio) строится по формуле

$$GainRatio(S) = \frac{Gain(S)}{SplitInfo(S)},$$

где $Gain(S)$ определяется по (4.7).

При разбиении выбирается атрибут, обеспечивающий максимум $GainRatio$.

Тест хи-квадрат используется для категориальных переменных или квантованных непрерывных переменных. Это тест статистической значимости различия между распределениями двух выборок. Тест представляет собой оценку вероятности того, что статистические законы распределения для двух неупорядоченных совокупностей наблюдений значимо различаются. При разбиении узла оценивается значимость различия между распределениями объектов в получившихся узлах. Большое значение теста говорит о значимости разбиения. Следовательно, следует выбрать тот атрибут, который обеспечивает максимум теста хи-квадрат.

Аналогична идея применения *F-критерий Фишера*. Это критерий применяется при построении разбиений непрерывных переменных.

Критерий уменьшение дисперсии используется для непрерывных переменных. Согласно этому критерию, разбиение следует проводить по атрибуту, уменьшающему дисперсию после разбиения.

4.3.2. Построение дерева решений

Одним из простейших алгоритмов построения дерева решений является алгоритм ID3 (*Iterative Dichotomizer 3*) [3]. Алгоритм начинает работу со всеми обучающими примерами в корневом узле дерева. Для разделения множества примеров корневого узла выбирается один из атрибутов, и для каждого значения, принимаемого этим атрибутом, строится ветвь и создается дочерний узел. Затем все примеры распределяются по дочерним узлам в соответствии со значением атрибута. Например, атрибут X принимает три значения: A, B, C . Тогда при разбиении исходного множества T по атрибуту X алгоритм создаст три дочерних узла, в первый из которых будут помещены все записи со значением A , во второй — со значением B , а в третий — со значением C .

Алгоритм повторяется рекурсивно до тех пор, пока в узлах не останутся только примеры одного класса, после чего узлы будут объявлены листьями и разбиение прекратится. Наиболее сложным этапом алгоритма является выбор атрибута, по которому будет производиться разбиение в каждом узле. Для выбора атрибута разбиения *ID3* использует критерий, называемый *приростом информации* (*information gain*), или *уменьшением энтропии* (*entropy reduction*). В качестве атрибута разбиения используется атрибут, который обеспечивает наибольший прирост информации $Gain(S)$

Практическое применение классического алгоритма *ID3* сопряжено с рядом проблем, характерных для моделей, основанных на обучении вообще и деревьев решений, в частности. Основными из них являются переобучение и наличие пропусков в данных. Для их эффективного преодоления *ID3* был доработан, в результате чего появилось его расширение, названное алгоритм *C4.5*. Основной недостаток алгоритма *ID3* — тенденция к *переобучению* и как следствие — *сверхчувствительность* (*overfitting*) к часто повторяющимся значениям. Данная проблема решается в алгоритме *C4.5* использованием рассмотренного ранее критерия отношения прироста информации (*gain-ratio*). При разбиении выбирается атрибут, обеспечивающий максимум $GainRatio$.

Поясним эффект от введения $GainRatio$. Если в результате разбиения получается большое число подмножеств с небольшим числом примеров, что характерно для переобучения, то параметр $SplitInfo$ растет. Поскольку $SplitInfo$ стоит в знаменателе выражения для $GainRatio$, то $GainRatio$ для такого разбиения уменьшается. Благодаря этому атрибут, для которого параметр $SplitInfo$ растет, имеет меньше шансов быть выбранным для разбиений, чем при использовании обычного $Gain$ -критерия.

В настоящее время в машинном обучении наиболее популярным является алгоритм *CART* (*Classification and Regression Tree* — *деревья классификации и регрессии*) [3, 8]. Деревья решений, построенные с помощью алгоритма *CART*, являются бинарными, то есть содержат только два потомка в каждом узле. Алгоритм в каждом узле t производит расщепление примеров на два поднабора с применением единственного признака k . Алгоритм ищет пару (k, t) , которая производит самые чистые поднаборы. Функция потерь, которую пытается минимизировать алгоритм, имеет вид

$$J(k, t) = \frac{m_L}{m} G_L + \frac{m_R}{m} G_R,$$

где m_L и m_R — количество примеров в левом и правом дочерних узлах узла t , m — количество примеров в узле t , G_L и G_R — меры загрязненности левого и правого дочерних узлов.

В качестве меры загрязненности узла используется мера загрязненности Джини или сокращение энтропии.

Рассмотрим алгоритм построения дерева CART *на примере классификации цветов ириса по длине и ширине лепестка*. Алгоритм перебирает все возможные разбиения по всем признакам (длина и ширина лепестка) и находит то единственное, которое лучше всего разделяет данные в конкретном узле. В качестве критерия используем неопределенность Джини (Gini impurity).

Представим, что у нас есть небольшой набор данных для обучения:

Длина лепестка (cm)	Ширина лепестка (cm)	Вид (Класс)
2.5	0.8	setosa
3.0	1.5	versicolor
5.0	1.8	virginica
5.5	2.0	virginica
3.5	1.0	setosa
6.0	2.5	virginica

Алгоритм строит правило для корневого узла (корня). Алгоритм берет первый признак — "Длина лепестка". Он сортирует все его уникальные значения по возрастанию: 2.5, 3.0, 3.5, 5.0, 5.5, 6.0 и вычисляет пороги (thresholds) как средние арифметические между соседними значениями в отсортированном списке.

- Между 2.5 и 3.0: $(2.5 + 3.0) / 2 = 2.75$
- Между 3.0 и 3.5: $(3.0 + 3.5) / 2 = 3.25$
- Между 3.5 и 5.0: $(3.5 + 5.0) / 2 = 4.25$
- Между 5.0 и 5.5: $(5.0 + 5.5) / 2 = 5.25$
- Между 5.5 и 6.0: $(5.5 + 6.0) / 2 = 5.75$

Теперь у нас есть 5 порогов-кандидатов для признака "Длина лепестка".

Для корневого узла для *каждого порога* алгоритм проверяет правило: Длина лепестка $\leq$ <значение порога> и вычисляет неопределенность Джини (Gini impurity). Затем эти действия для второго признака "Ширина лепестка". После того как алгоритм перебрал все пороги для всех признаков, у него есть таблица со всеми возможными вариантами. Он выбирает тот один-единственный признак и порог, которые дали значение неопределенности Джини.

В нашем примере, скорее всего, победит правило "Длина лепестка" ≤ 4.25 , так как оно создало один идеально чистый узел (правый).

Процесс повторяется рекурсивно для каждого дочернего узла, пока не будет выполнено одно из условий остановки (например, достигнута максимальная глубина дерева, в узле остался всего один образец, или разбиение больше не дает значимого снижения неопределенности).

Важно отметить, что *каждое правило будет повторяться только один раз*. Причина состоит в том, что дерево строится рекурсивно, и каждый узел работает только с подмножеством данных, которое до него "дошло". В нашем примере правило "Длина лепестка" ≤ 4.25 в корневом узле данные делятся на две непересекающиеся группы:

- Левый узел: Все образцы, где "Длина лепестка" ≤ 4.25
- Правый узел: Все образцы, где "Длина лепестка" > 4.25

В правом узле все примеры удовлетворяют условию "Длина лепестка" > 4.25 . Для них правило "Длина лепестка" ≤ 4.25 никогда не выполняется и алгоритм будет рассматривать другие правила.

В левом узле все примеры уже удовлетворяют условию "Длина лепестка" ≤ 4.25 . Это правило всегда дает положительный ответ и алгоритм будет искать другое правило, которое разделит данные этого узла.

CART может быть ресурсоемким, особенно на больших datasets. Поэтому применяется целый ряд приемов, снижающих ресурсоемкость алгоритма. Например, вместо копирования самих признаков и меток для каждого узла, дерево хранит только индексы примеров, принадлежащих узлу. Это экономит память и позволяет быстро создавать дочерние узлы отбором индексов. Вместо сортировки в каждом узле используется предварительная сортировка признаков. Применяются также оптимизированные переборы порогов.

Алгоритм CART позволяет строить не только классификационные, но и *регрессионные модели*. При построении регрессионной модели алгоритм CART работает так же, как раньше, но только вместо попытки расщеплять обучающий набор способом, сводящим к минимуму загрязненность, он пробует расщеплять его способом, который минимизирует среднеквадратическую ошибку MSE.

4.3.3. Регуляризация деревьев решений

Формально дерево решений будет строиться до тех пор, пока могут быть найдены новые разбиения, пока не будут получены абсолютно чистые листья или пока листья не будут содержать только одно наблюдение. В результате будет построено так называемое *полное дерево*. Полное дерево может быть сложным, те есть содержать много узлов и листьев. Полное дерево, как правило, является результатом *переобучения (overfitting)*, то есть адаптации модели к частным случаям, нетипичным примерам, шумам в данных и т. п. Такое дерево идеально

работает на обучающем множестве, но может давать много ошибок на тестовом множестве, на котором модель не обучалась.

Другой крайностью является *недообучение* (*underfitting*). Недообученное дерево получается слишком простым, содержит мало разбиений и не обеспечивает высокую точность обучения.

Во избежание переобучения обучающими данными необходимо ограничивать свободу дерева принятия решений во время обучения. Такой прием называется *регуляризацией*. *Гиперпараметры регуляризации* зависят от используемого алгоритма, но обычно можно, по крайней мере, ограничить максимальную глубину дерева принятия решений. Также можно ограничивать форму дерева принятия решений: задавать минимальное число примеров, которые должны присутствовать в узле, прежде чем его можно будет расщепить, минимальное количество примеров, которое должен иметь листовой узел, максимальное количество листовых узлов.

Альтернативными подходами являются ранняя остановка и отсечение ветвей [10]. *Ранняя остановка* — алгоритм будет остановлен, как только будет достигнуто заданное значение некоторого критерия, например процентной доли правильно распознанных примеров. Единственным преимуществом подхода является снижение времени обучения. Главным недостатком является то, что ранняя остановка всегда делается в ущерб точности дерева, поэтому многие авторы рекомендуют отдавать предпочтение отсечению ветвей. *Алгоритм отсечения ветвей* реализуется в 3 шага:

1. Строительство полного дерева, в котором листья содержат примеры одного класса.
2. Определение двух показателей: относительную точность модели (отношение числа правильно распознанных примеров к общему числу примеров) и абсолютную ошибку (число неправильно классифицированных примеров).
3. Удаление листов и узлов, потеря которых минимально скажется на точности модели и увеличении ошибки.

Отсечение ветвей проводят противоположно росту дерева, то есть снизу вверх, путем последовательного преобразования узлов в листья. Главное отличие метода отсечения ветвей от ранней остановки — получается найти оптимальное соотношение между точностью и понятностью. При этом уходит больше времени на обучение, потому что в рамках этого подхода изначально строится полное дерево.

4.4. Метод опорных векторов

4.4.1. Случай линейно разделимой выборки

В 60–70-е годы двадцатого века коллективом советских математиков под руководством В. Н. Вапника был разработан *метод обобщённого портрета*, основанный на построении оптимальной разделяющей гиперплоскости [11]. Требование оптимальности заключалось в том, что обучающие объекты должны быть удалены от разделяющей поверхности настолько далеко, насколько это возможно. В 90-е годы метод получил широкую мировую известность и после некоторой переработки и серии обобщений стал называться *машиной опорных векторов* (*support vector machine, SVM*) [12]. В настоящее время он считается одним из лучших методов классификации. Его современное изложение можно найти в [13–17]. Примеры применения SVM в машинном обучении можно найти в [6–8].

Метод SVM обладает несколькими замечательными свойствами. Во-первых, обучение SVM сводится к задаче квадратичного программирования, имеющей единственное решение, которое вычисляется достаточно эффективно даже на выборках в сотни тысяч объектов. Во-вторых, решение обладает свойством разреженности: положение оптимальной разделяющей гиперплоскости зависит лишь от небольшой доли обучающих объектов. Они и называются *опорными векторами*; остальные объекты фактически не задействуются. Наконец, с помощью изящного математического приёма — введения *функции ядра* (*kernel trick* — ядерного трюка) — метод обобщается на случай нелинейных разделяющих поверхностей. Вопрос о выборе ядра, оптимального для данной прикладной задачи, до сих пор остаётся открытой теоретической проблемой.

Случай линейно разделимой выборки рассмотрим на простом примере. Даны точки на плоскости, разбитые на два класса (рис. 4.2). Разделяющая линия (красная линия на рис. 4.2) делит точки на два класса. Новые точки (не из обучающей выборки) классифицируются следующим образом: точки, расположенные выше прямой, попадают в класс **A**, точка ниже прямой — в класс **B**. В пространствах высоких размерностей вместо прямых необходимо рассматривать гиперплоскости — пространства, размерность которых на единицу меньше, чем размерность исходного пространства.

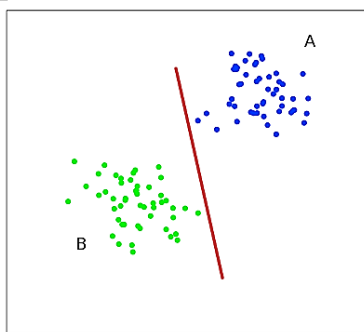


Рис. 4.2. Пример разделяющей прямой

В нашем примере можно построить несколько разделяющих прямых (рис. 4.3).

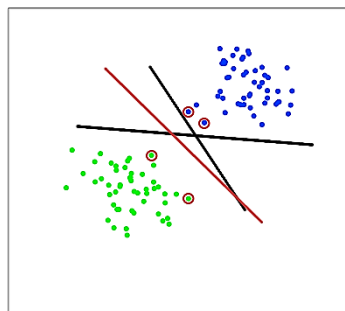


Рис. 4.3. Пример вариантов разделяющей прямой

С точки зрения точности классификации лучше всего выбрать прямую, расстояние от которой до каждого класса максимально. Такая разделяющая линия будет лучше классифицировать новые точки. Выберем ту прямую, которая разделяет классы наилучшим образом (красная прямая на рис.4.3). Такая прямая, а в общем случае — гиперплоскость, называется *оптимальной разделяющей гиперплоскостью*. Векторы, лежащие ближе всех к разделяющей гиперплоскости, называются *опорными векторами (support vectors)*. На рис.4.3. они помечены красным. Положение разделяющей гиперплоскости определяется не всеми обучающими примерами, а только опорными векторами.

Перейдем к более формализованному рассмотрению метода опорных векторов. Рассмотрим множество обучающих примеров $\{(\mathbf{x}_i, d_i)\}$, $i = 1, 2, \dots, N$, где $\mathbf{x}_i$ — вектор признаков примера, d_i — соответствующий желаемый отклик (целевой выход). Эти примеры будем рассматривать как векторы (точки) многомерного пространства. Объекты из обучающей выборки относятся к *двум классам*, для одного класса $d_i = +1$, для другого $d_i = -1$. Допустим, что эти два класса являются линейно разделимыми. *Линейная разделимость* означает, что существует *гиперплоскость*, разделяющая эти два класса. Из аналитической геометрии известно уравнение гиперплоскости в векторной форме (аналогичное уравнению плоскости)

$$\mathbf{w}^T \mathbf{x} + b = 0, \quad (4.8)$$

где $\mathbf{w}^T \mathbf{x} = (\mathbf{w}, \mathbf{x})$ — скалярное произведение настраиваемого вектора весов $\mathbf{w}$ и вектора входных переменных $\mathbf{x}$; b — порог.

Из аналитической геометрии известно, что вектор $\mathbf{w}$ перпендикулярен гиперплоскости (4.8), т. е. является вектором нормали к плоскости.

Для пояснения смысла формулы (4.8) рассмотрим двумерный случай. В этом случае разделяющая гиперплоскость превращается в разделяющую линию

$$x_2 = ax_1 + b, \quad (4.9)$$

или

$$-ax_1 + x_2 - b = 0,$$

или

$$[-a, 1] \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - b = \begin{bmatrix} -a \\ 1 \end{bmatrix}^T \cdot \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} - b = 0. \quad (4.10)$$

Эквивалентность выражений (4.9) и (4.10) очевидна.

Линейная разделимость означает:

$$\mathbf{w}^T \mathbf{x} + b \geq 0 \quad \text{для } d_i = +1,$$

$$\mathbf{w}^T \mathbf{x} + b < 0 \quad \text{для } d_i = -1.$$

Заметим, если вектор $\mathbf{x}$ попадает на гиперплоскость, то $\mathbf{w}^T \mathbf{x} + b = 0$.

Для данного вектора весов $\mathbf{w}$ и порога b расстояние между гиперплоскостью (4.8) и ближайшей точкой из набора данных называется *зазором* (margin). Потребуем, чтобы разделяющая гиперплоскость максимально далеко отстояла от ближайших к ней точек обоих классов. То есть *оптимальная гиперплоскость должна обеспечивать максимальный зазор*.

Если $\mathbf{w}_0$ и b_0 — оптимальные значения вектора весов и порога, то *оптимальная гиперплоскость* описывается уравнением

$$\mathbf{w}_0^T \mathbf{x} + b_0 = 0. \quad (4.11)$$

Дискриминантная функция

$$g(\mathbf{x}) = \mathbf{w}_0^T \mathbf{x} + b_0 \quad (4.12)$$

определяет расстояние от точки $\mathbf{x}$ до оптимальной гиперплоскости.

Рассмотрим расстояние от точки до оптимальной гиперповерхности на примере двумерного случая (рис. 4.4).

Вектор $\mathbf{x}$ представим как сумму векторов (рис. 4.4)

$$\mathbf{x} = \mathbf{x}_p + \mathbf{r} = \mathbf{x}_p + \frac{r}{\|\mathbf{w}_0\|} \mathbf{w}_0, \quad (4.13)$$

где $\mathbf{x}_p$ — нормальная проекция точки $\mathbf{x}$ на оптимальную гиперплоскость,

$\mathbf{r} = \frac{r}{\|\mathbf{w}_0\|} \mathbf{w}_0$, r — расстояние от точки до оптимальной гиперплоскости, $\|\mathbf{w}_0\|$ —

евклидова норма вектора нормали $\mathbf{w}_0$ оптимальной гиперплоскости, $\frac{\mathbf{w}_0}{\|\mathbf{w}_0\|}$ —

нормированный вектор нормали к гиперплоскости (т. е. имеющий единичную длину).

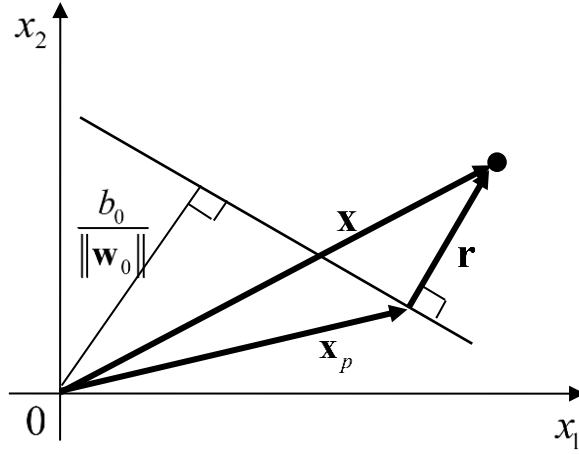


Рис. 4.4. Геометрическая интерпретация расстояния от точки до оптимальной гиперплоскости

Так как $\mathbf{x}_p$ лежит на оптимальной гиперплоскости, то $g(\mathbf{x}_p) = 0$. Учитывая это, а также определение евклидовой нормы $\|\mathbf{w}_0\| = \sqrt{\mathbf{w}_0^T \mathbf{w}_0}$, после подстановки (4.13) в (4.12) получаем

$$g(\mathbf{x}) = \mathbf{w}_0^T \mathbf{x}_p + b_0 + \frac{r}{\|\mathbf{w}_0\|} \mathbf{w}_0^T \mathbf{w}_0 = g(\mathbf{x}_p) + r \|\mathbf{w}_0\|,$$

откуда

$$r = \frac{g(\mathbf{x})}{\|\mathbf{w}_0\|}. \quad (4.14)$$

В частности, расстояние от начала координат, где $\mathbf{x} = 0$ до оптимальной гиперплоскости равно $b_0 / \|\mathbf{w}_0\|$.

Заметим, что параметры линейного порогового классификатора определены с точностью до *нормировки*: значения $\mathbf{w}_0$ и b_0 в (4.11) можно масштабировать. Удобно предположить, что проведено масштабирование таким образом, что $\mathbf{w}_0$ и b_0 удовлетворяют ограничениям

$$\begin{aligned} \mathbf{w}_0^T \mathbf{x}_i + b_0 &\geq 1 \quad \text{для } d_i = +1, \\ \mathbf{w}_0^T \mathbf{x}_i + b_0 &< 1 \quad \text{для } d_i = -1. \end{aligned} \quad (4.15)$$

Точки $(\mathbf{x}_i, d_i)$, для которых ограничения (4.15) выполняются со знаком равенства, называются *опорными векторами (support vector)*. Отсюда получил свое название метод. Опорные векторы расположены ближе всего к разделяющей гиперплоскости и являются наиболее сложными для классификации. Проведем через точки, ближайšie к разделяющей гиперплоскости и расположенные по обе стороны от неё, две гиперплоскости, параллельные разделяющей гиперплоскости. Эти гиперплоскости будут иметь вектор нормали $\mathbf{w}_0$. Между этими

гиперплоскостями располагается так называемая *разделяющая полоса*, в которую не попадает ни одна точка. Считается, что разделяющая гиперплоскость расположена посередине разделяющей полосы. Чтобы разделяющая гиперплоскость как можно дальше отстояла от точек выборки, *ширина полосы должна быть максимальной*.

Как видно из (4.15), дискриминантная функция для опорных векторов $\mathbf{x}^{(s)}$ равна $g(\mathbf{x}^{(s)}) = +1$, если опорный вектор $\mathbf{x}^{(s)}$ расположен с положительной стороны от разделяющей гиперплоскости, и $g(\mathbf{x}^{(s)}) = -1$, если вектор $\mathbf{x}^{(s)}$ расположен с отрицательной стороны от разделяющей гиперплоскости. Расстояние (4.14) от опорного вектора $\mathbf{x}^{(s)}$ до разделяющей гиперплоскости равно

$$r = \frac{g(\mathbf{x}^{(s)})}{\|\mathbf{w}_0\|} = \begin{cases} \frac{1}{\|\mathbf{w}_0\|}, & \text{если } d^{(s)} = +1, \\ -\frac{1}{\|\mathbf{w}_0\|}, & \text{если } d^{(s)} = -1. \end{cases}$$

Тогда *ширина разделяющей полосы* равна

$$\rho = 2|r| = \frac{2}{\|\mathbf{w}_0\|}.$$

Ширина разделяющей полосы максимальна, когда норма вектора $\mathbf{w}$ минимальна.

Оптимальная гиперплоскость должна также удовлетворять условиям (4.15), которые запишем в виде одного неравенства

$$d_i \cdot (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \text{для } i = 1, 2, \dots, N. \quad (4.16)$$

Тогда задача нахождения оптимальной разделяющей гиперплоскости можно сформулировать следующим образом: для данного обучающего множества $\{(\mathbf{x}_i, d_i)\}$, $i = 1, 2, \dots, N$ найти оптимальные значения вектора весовых коэффициентов $\mathbf{w}$ и порога b , удовлетворяющие условию (4.16) и минимизирующие функцию

$$\Phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w}. \quad (4.17)$$

Масштабирующий множитель $1/2$ включен в (4.17) для удобства вычислений.

Задача (4.17) с ограничениями (4.16) представляет собой задачу *квадратичного программирования* [18–20].

Функция $\Phi(\mathbf{w})$ является *выпуклой*³, ограничения на $\mathbf{w}$ являются линейными.

Для решения задачи (4.17), (4.16) можно использовать *метод множителей Лагранжа* [19–20]. По этому методу необходимо найти минимум функции *Лагранжа*

$$J(\mathbf{w}, b, \alpha) = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^N \alpha_i \left[d_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 \right], \quad (4.18)$$

где $\mathbf{w}$ — вектор весов, b — порог, $\mathbf{x}_i$ — i -й вектор входных переменных, d_i — целевой выход, соответствующий входному вектору $\mathbf{x}_i$, α_i — *множители Лагранжа*.

Пусть m — размерность пространства признаков, w_j — j -й компонент вектора весов. Минимум функции (4.18) найдем, приравнявая нулю производные

$$\frac{\partial J(\mathbf{w}, b, \alpha)}{\partial w_j} = 0, \quad j = 1, 2, \dots, m, \quad (4.19)$$

$$\frac{\partial J(\mathbf{w}, b, \alpha)}{\partial b} = 0. \quad (4.20)$$

Из (4.19) получаем

$$\begin{aligned} \frac{\partial J(\mathbf{w}, b, \alpha)}{\partial w_j} &= \frac{1}{2} \frac{\partial}{\partial w_j} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^N \alpha_i \left[d_i \frac{\partial}{\partial w_j} (\mathbf{w}^T \mathbf{x}_i + b) - 1 \right] = \\ &= \frac{1}{2} \frac{\partial}{\partial w_j} (w_1^2 + w_2^2 + \dots + w_j^2 + \dots + w_m^2) - \\ &- \sum_{i=1}^N \alpha_i \left[d_i \frac{\partial}{\partial w_j} (w_1 x_{1i} + w_2 x_{2i} + \dots + w_j x_{ji} + \dots + w_m x_{mi} + b) - 1 \right] = \\ &= w_j - \sum_{i=1}^N \alpha_i d_i x_{ji}. \end{aligned} \quad (4.21)$$

Приравнявая нулю (4.21), получаем значения весов

$$w_j = \sum_{i=1}^N \alpha_i d_i x_{ji}.$$

Для вектора весов можно записать

$$\mathbf{w} = \sum_{i=1}^N \alpha_i d_i \mathbf{x}_i. \quad (4.22)$$

Из (4.20) получаем

³ Функция называется выпуклой на некотором интервале, если для любых двух точек x_1 и x_2 из этого интервала и для любого числа $t \in [0, 1]$, выполняется неравенство $f(tx_1 + (1-t)x_2) \leq t \cdot f(x_1) + (1-t)f(x_2)$.

$$\sum_{i=1}^N \alpha_i d_i = 0. \quad (4.23)$$

В точке минимума функции Лагранжа (4.18) для каждого множителя Лагранжа выполняются *условия Куна-Таккера*, или условия *Каруша-Куна-Таккера*⁴ (англ. *Karush-Kuhn-Tucker conditions, KKT*) — необходимые условия решения задачи нелинейного программирования [20]

$$\alpha_i \left[d_i (\mathbf{w}^T \mathbf{x}_i + b) - 1 \right] = 0 \quad \text{для } i = 1, 2, \dots, N. \quad (4.24)$$

Таким образом, ненулевые значения могут иметь только те множители Лагранжа, которые в точности удовлетворяют условию (4.24).

Для нахождения множителей Лагранжа перейдем к *двойственной задаче*. Известно [19, 21–22], что при условиях нашей задачи может быть сформулирована двойственная задача, которая будет иметь то же оптимальное решение, что и прямая задача, но будет находиться максимум целевой функции, зависящей только от множителей Лагранжа.

Для вывода целевой функции двойственной задачи (см., например, [15]) раскроем целевую функцию прямой задачи (4.18)

$$J(\mathbf{w}, b, \alpha) = \frac{1}{2} \mathbf{w}^T \mathbf{w} - \sum_{i=1}^N \alpha_i d_i \mathbf{w}^T \mathbf{x}_i - b \sum_{i=1}^N \alpha_i d_i + \sum_{i=1}^N \alpha_i. \quad (4.25)$$

Третье слагаемое в (4.25) равно нулю вследствие условия (4.23). Из (4.22) получаем

$$\mathbf{w}^T \mathbf{w} = \sum_{i=1}^N \alpha_i d_i \mathbf{w}^T \mathbf{x}_i = \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j d_i d_j \mathbf{x}_i^T \mathbf{x}_j. \quad (4.26)$$

Тогда, представив целевую функцию как $J(\mathbf{w}, b, \alpha) = Q(\alpha)$, учитывая (4.26), выражение (4.25) можно представить в виде

$$Q(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j d_i d_j \mathbf{x}_i^T \mathbf{x}_j. \quad (4.27)$$

Двойственная задача формулируется следующим образом: для обучающего множества $\{(\mathbf{x}_i, d_i)\}$, $i = 1, 2, \dots, N$ найти множители Лагранжа α_i , $i = 1, 2, \dots, N$, максимизирующие целевую функцию (4.27) при ограничениях

⁴ Необходимо найти $\min_{\mathbf{x}} f(\mathbf{x})$ при условиях $g_i(\mathbf{x}) \leq 0$, $i = 1, 2, \dots, m$. Если $\mathbf{x}_0$ — решение задачи при наложенных ограничениях, то найдётся ненулевой вектор множителей Лагранжа такой, что для функции Лагранжа выполняются условия:

– стационарности: $\min_{\mathbf{x}} L(\mathbf{x}) = L(\mathbf{x}_0)$;

– дополняющей нежесткости: $\lambda_i g_i(\mathbf{x}_0) = 0$, $i = 1, 2, \dots, m$, $L(\mathbf{x}) = f(\mathbf{x}) + \sum_{i=1}^m \lambda_i g_i(\mathbf{x})$;

– неотрицательности: $\lambda_i \geq 0$, $i = 1, 2, \dots, m$.

$$\sum_{i=1}^N \alpha_i d_i = 0,$$

$$\alpha_i \geq 0 \text{ для } i = 1, 2, \dots, N.$$

Двойственная задача содержит только данные обучения. Для решения двойственной задачи известны алгоритмы [20–22].

Определив оптимальные множители Лагранжа, с помощью выражения (4.22) находится оптимальный вектор весов $\mathbf{W}_0$. Причем *суммирование идёт не по всей выборке, а только по опорным векторам, для которых $\alpha_i \neq 0$* , все остальные примеры можно убрать из обучающей выборки. Это свойство называют *разреженностью* (*sparsity*). Ненулевыми α_i обладают граничные опорные объекты.

Оптимальное значение порога b_0 получается из (4.15)

$$b_0 = 1 - \mathbf{w}_0^T \mathbf{x}^{(s)} \text{ для } d^{(s)} = 1.$$

На практике линейно разделимые классы встречаются довольно редко. Поэтому рассмотрим случай линейно неразделимой выборки.

4.4.2. Метод опорных векторов с мягким зазором

Для обобщения задачи на случай линейно неразделимой выборки позволим алгоритму допускать ошибки на обучающих объектах, но при этом постараемся, чтобы ошибок было поменьше. То есть допустим, что некоторая точка $(\mathbf{x}_i, d_i)$ нарушает условие (4.16)

$$d_i \cdot (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \text{ для } i = 1, 2, \dots, N.$$

При этом возможны два варианта:

- точка $(\mathbf{x}_i, d_i)$ попадает в область разделения с нужной стороны поверхности решений (рис. 4.5, а);
- точка $(\mathbf{x}_i, d_i)$ попадает в область разделения с другой стороны поверхности решений (рис. 4.5, б).

В первом случае классификация будет правильной, а во втором — ошибочной.

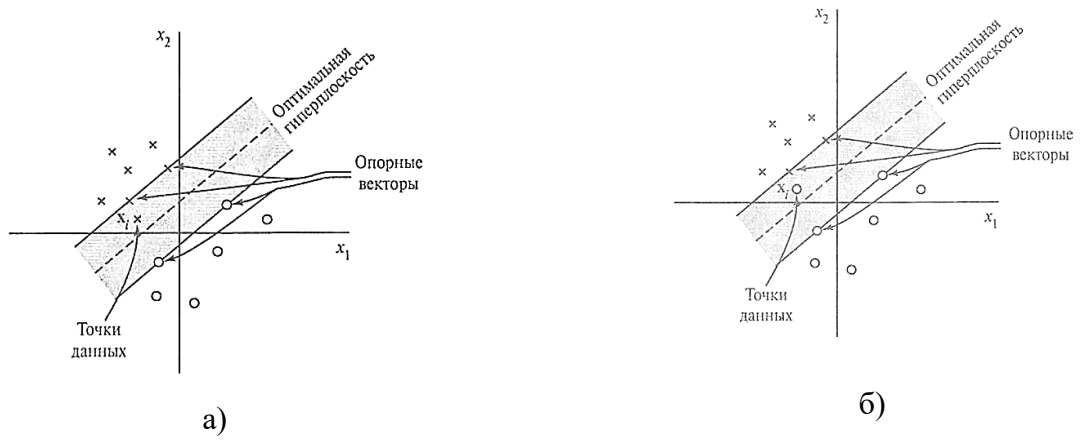


Рис. 4.5. Варианты попадания точки в область разделения [3]

Введем в определение разделяющей гиперплоскости *дополнительные* (*ослабляющие переменные* — *slack variable*) переменные ξ_i , определяющие отклонение точки от идеального состояния линейной разделимости (сравни с (4.16))

$$d_i \cdot (\mathbf{w}^T \mathbf{x}_i + b) \geq 1 + \xi_i \quad \text{для } i = 1, 2, \dots, N. \quad (4.28)$$

Если $\xi_i = 0$, то ошибки нет. Если $0 \leq \xi_i \leq 1$, то точка попадает в область разделения с нужной стороны гиперплоскости (рис. 4.5, а). Если $\xi_i > 1$, то точка попадает в область с неверной стороны разделяющей гиперплоскости (рис. 4.5, б). Опорные векторы являются точками, которые удовлетворяют неравенству (4.28) при $\xi_i > 0$.

Такой подход называется *методом опорных векторов с мягким зазором* (soft margin) [15].

В случае линейно неразделимой выборки минимизируемый функционал принимают в виде [15]

$$\Phi(\mathbf{w}, \xi) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i, \quad (4.29)$$

где $\sum_{i=1}^N \xi_i$ — *регуляризатор*, $C > 0$ — задаваемый пользователем *параметр регуляризации*.

Введение регуляризатора ограничивает рост ошибок и повышает устойчивость решения. Таким образом, принцип оптимальной разделяющей гиперплоскости или максимизации ширины разделяющей полосы тесно связан с *регуляризацией* некорректно поставленных задач по А. Н. Тихонову [23]. Если все $\xi_i = 0$, то приходим к задаче линейно разделимых множеств.

Таким образом, *задача для линейно неразделимых множеств* формулируется следующим образом: для заданного обучающего множества $\{(\mathbf{x}_i, d_i)\}$, $i = 1, 2, \dots, N$ найти оптимальные значения вектора весов $\mathbf{w}$ и порога b , удовлетворяющие ограничениям

$$d_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 + \xi_i \text{ для } i = 1, 2, \dots, N,$$

$$\xi_i \geq 0 \text{ для всех } i$$

и минимизирующие функцию

$$\Phi(\mathbf{w}, \xi) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i,$$

где $C > 0$ – задаваемый пользователем параметр.

Функция Лагранжа для случая линейно неразделимой выборки имеет вид

$$J(\mathbf{w}, b, \alpha) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_{i=1}^N \xi_i - \sum_{i=1}^N \alpha_i [d_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i] + \sum_{i=1}^N \eta_i \xi_i, \quad (4.30)$$

где η_i — множители Лагранжа для фиктивных переменных.

Вычисляя производные от (4.30) и приравнявая их нулю, получаем [15]

$$\frac{\partial J(\mathbf{w}, b, \alpha)}{\partial \mathbf{w}} = \mathbf{w} - \sum_{i=1}^N \alpha_i d_i \mathbf{x}_i = 0, \quad (4.31)$$

$$\frac{\partial J(\mathbf{w}, b, \alpha)}{\partial b} = -\sum_{i=1}^N \alpha_i d_i = 0, \quad (4.32)$$

$$\frac{\partial J(\mathbf{w}, b, \alpha)}{\partial \xi_i} = C - \alpha_i - \eta_i = 0. \quad (4.33)$$

Переходя к двойственной задаче, как было проделано для линейно разделимых множеств, с учетом (4.31)–(4.33) получим целевую функцию, совпадающую с (4.27). Таким образом, двойственная задача для неразделимых множеств формулируется следующим образом: для обучающего множества $\{(\mathbf{x}_i, d_i)\}$, $i = 1, 2, \dots, N$ найти множители Лагранжа, максимизирующие целевую функцию

$$Q(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j d_i d_j \mathbf{x}_i^T \mathbf{x}_j \quad (4.34)$$

при ограничениях $\sum_{i=1}^N \alpha_i d_i = 0$, $0 \leq \alpha_i \leq C$ для $i = 1, 2, \dots, N$, C – задаваемый пользователем параметр.

В двойственной задаче отсутствуют фиктивные переменные ξ_i и их множители Лагранжа. Важным отличием двойственной задачи для линейно неразделимых множеств от задачи для линейно разделимых множеств является более строгое ограничение для α_i . Вычисления весов и порога полностью совпадают с вычислениями для линейно разделимой выборки.

Серьезную проблему представляет выбор параметра регуляризации C . Обычно C подбирают, сравнивая качество обучения по тестовой выборке.

4.4.3. Ядерный метод опорных векторов

Существует подход к решению проблемы линейной неразделимости, основанный на *нелинейном отображении* входных векторов в *пространство признаков*, имеющее бóльшую размерность, чем входное пространство. В этом *спрямляющем пространстве* [16] образы входных векторов будут линейно разделимыми. Этот подход основан на *теореме Ковера* (*Cover's theorem on separability of pattern*) о разделимости образов [24]: нелинейное преобразование сложной задачи классификации образов в пространство признаков более высокой размерности повышает вероятность линейной разделимости образов. Далее в пространстве признаков строится оптимальная разделяющая гиперплоскость. Построение проводится, как показано в предыдущем подразделе, но построение проводится не в пространстве входных векторов, а в пространстве преобразованных признаков.

Пусть $\mathbf{x}$ — некоторый вектор из входного пространства, $\varphi_i(\mathbf{x})$, $i=1, 2, \dots, m_1$ — множество нелинейных преобразований из входного пространства в пространство признаков, m_1 — размерность пространства признаков. Разделяющую гиперплоскость в пространстве признаков можно представить в виде

$$\sum_{i=1}^{m_1} w_i \varphi_i(\mathbf{x}) + b = 0, \quad (4.35)$$

где w_i , $i=1, 2, \dots, m_1$ — множество весовых коэффициентов, b — порог.

Выражение (4.35) можно упростить

$$\sum_{i=0}^{m_1} w_i \varphi_i(\mathbf{x}) = 0, \quad (4.36)$$

где $\varphi_0(\mathbf{x}) = 1$ для всех $\mathbf{x}$, $w_0 = b$ — порог.

Введем вектор $\boldsymbol{\varphi}(\mathbf{x}) = [\varphi_0(\mathbf{x}), \varphi_1(\mathbf{x}), \dots, \varphi_{m_1}(\mathbf{x})]^T$. Предполагая, что в пространстве признаков определено скалярное произведение, поверхность решений (4.36) можно представить в виде

$$\mathbf{w}^T \boldsymbol{\varphi}(\mathbf{x}) = 0. \quad (4.37)$$

Если предположить, что описаниями объектов являются векторы $\boldsymbol{\varphi}(\mathbf{x}_i)$, а не векторы $\mathbf{x}_i$, то построение SVM проводится точно так же, как и ранее. В частности, (4.22) запишется в виде

$$\mathbf{w} = \sum_{i=1}^N \alpha_i d_i \boldsymbol{\varphi}(\mathbf{x}_i). \quad (4.38)$$

Подставляя (4.38) в (4.37), запишем уравнение поверхности решения в виде

$$\sum_{i=1}^N \alpha_i d_i \boldsymbol{\varphi}^T(\mathbf{x}) \boldsymbol{\varphi}(\mathbf{x}_i) = 0. \quad (4.39)$$

Использование (4.39) приводит к большим вычислительным затратам из-за большой размерности пространства, в которое преобразуются признаки. Кроме того, заранее невозможно определить какое нелинейное преобразование и с какими параметрами применить, что требует многократных экспериментов. Проблема решается использованием так называемого *ядерного трюка (kernel trick)* [25]. Ядерный трюк основан на использовании *ядерных функций (ядер)*. Вместо вычисления скалярных произведений в пространстве большой размерности вычисляю ядра в пространстве исходной размерности

$$\sum_{i=1}^N \alpha_i d_i \boldsymbol{\varphi}^T(\mathbf{x}) \boldsymbol{\varphi}(\mathbf{x}_i) = \sum_{i=1}^N \alpha_i d_i K(\mathbf{x}, \mathbf{x}_i) = 0$$

где $K(\mathbf{x}_i, \mathbf{x}_i) = \boldsymbol{\varphi}(\mathbf{x}_i)^T \boldsymbol{\varphi}(\mathbf{x}_i)$ — *ядро скалярного произведения (inner-product kernel)*.

Постановка двойственной задачи зависит только от скалярных произведений объектов, но не от самих признаков описаний. Это означает, что скалярное произведение $(\mathbf{x}, \mathbf{x}_i)$ можно формально заменить ядром $K(\mathbf{x}, \mathbf{x}_i)$.

Требования к функциям, которые могут выступать в качестве ядра, дает известная в функциональном анализе *теорема Мёрсера (Mercer's theorem)* [15, 26–27]. Рассмотрим теорему Мерсера в формулировке книги [15]. Пусть $K(\mathbf{x}, \mathbf{x}')$ — непрерывное симметричное ($K(\mathbf{x}, \mathbf{x}') = K(\mathbf{x}', \mathbf{x})$) *ядро*, определенное на закрытом интервале $\mathbf{a} \leq \mathbf{x} \leq \mathbf{b}$ (это же касается и $\mathbf{x}'$). Ядро может быть представлено в виде ряда

$$K(\mathbf{x}, \mathbf{x}') = \sum_{i=1}^{\infty} \lambda_i \varphi_i(\mathbf{x}) \varphi_i(\mathbf{x}') \quad (4.40)$$

с положительными коэффициентами $\lambda_i > 0$ для всех i . Для обеспечения корректности этого разложения и его абсолютной и равномерной сходимости необходимо и достаточно выполнение условия неотрицательной определенности

$$\int_a^b \int_a^b K(\mathbf{x}, \mathbf{x}') \psi(\mathbf{x}) \psi(\mathbf{x}') d\mathbf{x} d\mathbf{x}' \geq 0$$

для всех функций ψ , для которых $\int_a^b \psi^2(\mathbf{x}) d\mathbf{x} < \infty$.

Функции $\varphi_i(\mathbf{x})$ из (4.40) называются *собственными функциями разложения*, а числа λ_i — *собственными значениями разложения*.

Теорема Мерсера позволяет лишь определить, является ли функция ядром скалярного произведения. Но теорема ничего не говорит, как строить ядра. Проверка неотрицательной определённости функции в практических ситуациях

может оказаться сложной задачей. Обычно ограничиваются перебором конечного числа функций, про которые известно, что они являются ядрами. На сегодняшний день проблема выбора ядра, оптимального для данной конкретной задачи, остаётся открытой.

С использованием ядра скалярного произведения можно построить двойственную задачу условной оптимизации, формально заменив в (4.34) скалярные произведения $\mathbf{x}_i^T \mathbf{x}_j$ ядром скалярного произведения $K(\mathbf{x}_i, \mathbf{x}_j)$:

$$Q(\alpha) = \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N \alpha_i \alpha_j d_i d_j K(\mathbf{x}_i, \mathbf{x}_j)$$

при ограничениях $\sum_{i=1}^N \alpha_i d_i = 0$, $0 \leq \alpha_i \leq C$ для $i = 1, 2, \dots, N$, C — задаваемый пользователем параметр.

При построении SVM чаще всего используют несколько проверенных ядер скалярного произведения [8, 15–16]. Наиболее популярными являются три типа ядер.

Линейное ядро: $K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^T \mathbf{b}$ — это скалярное произведение в исходном пространстве.

Полиномиальное ядро $K(\mathbf{a}, \mathbf{b}) = (\gamma \mathbf{a}^T \mathbf{b} + r)^d$, где параметры γ , r и d подбираются.

Гауссово ядро (радиальная базисная функция, RBF — Radial Basis Function):

$K(\mathbf{a}, \mathbf{b}) = e^{-\frac{\|\mathbf{a}-\mathbf{b}\|^2}{2a^2}} = e^{-\gamma \|\mathbf{a}-\mathbf{b}\|^2}$, где a часто называют шириной RBF, так как этот параметр характеризует "остроту" графика Гауссиана. Параметр $\gamma = \frac{1}{2a^2}$ подбирают.

Сигмоидальное ядро (гиперболический тангенс):

$K(\mathbf{a}, \mathbf{b}) = \tanh(\gamma \mathbf{a}^T \mathbf{b} + r)$, параметры γ и r подбираются. Сигмоидальное ядро не соблюдает все условия теорем Мерсера, но на практике работает хорошо. Наглядно идея линейной разделимости при переходе в пространство большей размерности показана в видеоролике [28].

4.5. Кластеризация

4.5.1. Постановка задачи кластеризации

Кластеризация — это разделение исследуемого множества объектов на группы "похожих" объектов, называемых *кластерами* [6, 8, 29]. Синонимами

термина "кластер" (англ. Cluster — сгусток, пучок, группа) являются термины класс, *таксон*, сгущение. Задача кластеризации принципиально отличается от задачи классификации. Решением задачи *классификации* является отнесение каждого из объектов к одному (или нескольким) из *заранее определенных* классов. В задаче кластеризации происходит отнесение объекта к одному (или нескольким) из *заранее неопределенных* классов. Разбиение объектов по кластерам осуществляется *при одновременном формировании кластеров*. Кластеризация относится к методам *обучения без учителя*, так как метки, указывающие, к какому классу относится каждый пример, отсутствуют.

Кластеризация позволяет сгруппировать сходные данные, что облегчает решение ряда задач Data Science:

- *Изучение данных, облегчение анализа.* Содержательный анализ полученных кластеров позволяет обнаружить закономерности. Например, можно выявить группы клиентов сети сотовой связи, для которых можно предложить новый тарифный план. Другие примеры — выявление групп покупателей торговой сети, сегментация рынка. Анализ содержания кластера позволяет применить к объектам различных кластеров разные методы анализа.
- *Прогнозирование.* Относя новый объект к одному из кластеров, можно прогнозировать поведение объекта, поскольку его поведение будет схожим с поведением объектов кластера.
- *Обнаружение аномалий.* Содержательный анализ кластеров помогает выявить аномалии. Обычно, это кластеры, в которые попадает мало объектов.

Важно отметить роль *содержательной интерпретации каждого кластера*. Каждому кластеру необходимо присвоить содержательное название, отражающее суть объектов кластера. Для этого необходимо выявить, признаки, объединяющие объекты в кластер. Это может потребовать статистического анализа свойств объекта кластера.

Ограничимся кластеризацией объектов, описываемых количественными характеристиками. Кластеризация объектов, описываемых качественными (категорийными) данными, рассматривается, например, в [30].

Формально *задача кластеризации* описывается следующим образом [31]. Дано множество объектов I , каждый из которых представлен набором атрибутов (параметров). Требуется построить множество кластеров C и отображение F множества I на множество C , то есть $F: I \rightarrow C$. Отображение F задает модель, являющуюся решением задачи.

Множество I объектов определяется следующим образом

$$I = \{i_1, i_2, \dots, i_n\},$$

где $i_j, j=1,2,\dots,n$ — исследуемый объект.

Каждый из объектов характеризуется набором *признаков (параметров)*

$$i_j = \{x_1, x_2, \dots, x_h, \dots, x_m\}.$$

Каждая *переменная* x_h может принимать значения из некоторого множества

$$x_h = \{v_h^1, v_h^2, \dots\}.$$

Мы будем рассматривать в качестве значений действительные числа.

Задача кластеризации состоит в построении множества

$$C = \{c_1, c_2, \dots, c_k, \dots, c_g\},$$

где c_k — *кластер*, содержащий "похожие" объекты из множества I

$$c_k = \{i_j, i_p \mid i_j \in I, i_p \in I \text{ и } d(i_j, i_p) < \sigma\},$$

σ — величина, определяющая меру близости для включения объектов в один кластер, $d(i_j, i_p)$ — мера близости между объектами, называемая *расстоянием*.

Если расстояние $d(i_j, i_p)$ меньше некоторого значения σ , то объекты считаются близкими и помещаются в один кластер. В противном случае считается, что объекты отличны друг от друга и их помещают в разные кластеры.

4.5.2. Меры близости, используемые в кластеризации

Рассмотрим основные понятия [31]. Неотрицательное число $d(i_j, i_p)$ называется *расстоянием* (метрикой) между объектами i_j и i_p , если выполняются следующие условия:

1. $d(i_j, i_p) \geq 0$ для всех i_j и i_p .
2. $d(i_j, i_p) = 0$, тогда и только тогда, когда $i_j = i_p$.
3. $d(i_j, i_p) = d(i_p, i_j)$.
4. $d(i_j, i_p) \leq d(i_j, i_r) + d(i_r, i_p)$ — неравенство треугольника.

Алгоритмы кластеризации используют *матрицу расстояний (отличия)* D , строки и столбцы которой соответствуют элементам множества I . Элементами матрицы D являются значения $d(i_j, i_p)$ в строке j и столбце p . Это расстояние между объектами i_j и i_p . На главной диагонали стоят нули.

$$\mathbf{D} = \begin{bmatrix} 0 & d(i_1, i_2) & \dots & d(i_1, i_n) \\ d(i_2, i_1) & 0 & \dots & d(i_2, i_n) \\ \dots & \dots & \dots & \dots \\ d(i_n, i_1) & d(i_n, i_2) & \dots & 0 \end{bmatrix}. \quad (4.41)$$

Объекты представляются в виде точек m -мерного пространства R^m . Координатами точек (объектов) являются свойства (признаки) объектов.

При рассмотрении расстояний между объектами используем следующие обозначения.

$X_Q \subseteq R^m$ — множество объектов, являющееся подмножеством m -мерного пространства R^m .

$$\mathbf{x}_i = [x_{i1}, \dots, x_{im}] \in X_Q, \quad i = 1, \dots, Q \quad (4.42)$$

— элементы множества объектов. Каждый объект представляется как вектор m -мерного пространства R^m .

Наиболее известными *мерами близости (расстояниями)* между объектами являются следующие.

Евклидово расстояние

$$d_E(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{k=1}^m (x_{ik} - x_{jk})^2}. \quad (4.43)$$

В (4.43) учтено, что признаками являются действительные числа. Множество точек, равноудаленных от некоторого центра, при использовании евклидовой метрики образуют сферу. Поэтому кластеры, полученные с использованием евклидова расстояния, имеют форму, близкую к сферической.

Часто используется *расстояние Минковского*

$$d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sqrt[p]{\sum_k |x_k^{(i)} - x_k^{(j)}|^p}.$$

При $p = 2$ расстояние Минковского становится евклидовым расстоянием, а при $p = 1$ — *расстоянием Манхэттена*

$$d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) = \sum_k |x_k^{(i)} - x_k^{(j)}|.$$

Это расстояние между двумя точками, пройденное по линиям, параллельным осям координат. Кластеры, построенные с использованием этой метрики, стремятся к кубической форме. Использование линейного расстояния снижает влияние аномальных значений (выбросов), т. к. расстояния не возводятся в квадрат.

Для качественных (*категориальных*) признаков используется *функция отличия*

$$d(x_i, x_j) = \begin{cases} 0, & \text{если } x_i = x_j, \\ 1, & \text{в остальных случаях.} \end{cases}$$

Известны и другие меры близости.

Кластер имеет следующие *математические характеристики*: *центр*, *радиус*, *среднеквадратическое отклонение*, *размер кластера*.

Центр кластера (центроид) — это среднее геометрическое место точек в пространстве переменных. Координаты центроида вычисляются как среднее значение для каждого признака всех объектов. Например, в кластер вошли три объекта с параметрами $[x_{11}, x_{12}]$, $[x_{21}, x_{22}]$, $[x_{31}, x_{32}]$, тогда координаты его центроида рассчитываются по формуле

$$m(x_1, x_2) = \left[\frac{x_{11} + x_{21} + x_{31}}{3}, \frac{x_{12} + x_{22} + x_{32}}{3} \right]$$

Радиус кластера — максимальное расстояние точек от *центра кластера*.

Размер кластера может быть определен либо по *радиусу кластера*, либо по *среднеквадратичному отклонению* расстояния объектов от центра кластера.

Объект относится к кластеру, если расстояние от объекта до *центра кластера* меньше *радиуса кластера*. Если это условие выполняется для двух и более кластеров, объект является *спорным*.

В процессе кластерного анализа необходимо определять *меры близости (расстояния) между кластерами*. Чаще всего используются следующие меры близости.

Метод ближнего соседа или одиночная связь (Nearest Neighbor). Здесь расстояние между двумя кластерами определяется расстоянием между двумя наиболее близкими объектами (ближайшими соседями) в различных кластерах. Этот метод позволяет выделять кластеры сколь угодно сложной формы при условии, что различные части таких кластеров соединены цепочками близких друг к другу элементов. В результате работы этого метода кластеры представляются длинными "цепочками" или "волокнистыми" кластерами, "сцепленными вместе" только отдельными элементами, которые случайно оказались ближе остальных друг к другу.

Метод наиболее удаленных соседей (Furthest Neighbor) или полная связь. Здесь расстояния между кластерами определяются наибольшим расстоянием между любыми двумя объектами в различных кластерах (т.е. "наиболее удаленными соседями"). Метод хорошо использовать, когда объекты действительно происходят из хорошо разделенных кластеров (говорят, что кластеры принадлежат разным "родам"). Если же кластеры имеют в некотором роде удлиненную форму или их естественный тип является "цепочечным", то этот метод не следует использовать.

Метод невзвешенного попарного среднего (метод невзвешенного попарного арифметического среднего — *Unweighted Pair-Group Method Using Arithmetic Averages, UPGMA*). В качестве расстояния между двумя кластерами берется среднее расстояние между всеми парами объектов в них. Этот метод используется, если объекты действительно происходят из различных "рощ", в случаях присутствия кластеров "цепочного" типа, при предположении неравных размеров кластеров.

4.5.3. Алгоритмы кластеризации

Перед кластеризацией данные подвергаются нормировке (масштабированию) или стандартизации. Нормировка (стандартизация) необходимы, чтобы на результат кластеризации не оказывали влияние различные области значений атрибутов. Масштабирование производится независимо по каждому атрибуту всех объектов.

Перед проведением кластеризации необходимо определить количество кластеров. Это трудно формализуемая задача. Обычно используют перебор числа кластеров и определения оптимального числа кластеров в процессе перебора.

Рассмотрим основные алгоритмы кластеризации [6, 8].

Алгоритм K-Means требует задания количества кластеров и строится по алгоритму:

1. Случайным образом выбрать k центроидов из примеров (образцов) в качестве начальных центров кластеров.
2. Назначить каждый образец ближайшему центроиду $\mu^{(j)}$, $j \in \{1, 2, \dots, k\}$.
3. Вычислить новые центроиды кластеров. Новые центроиды можно вычислять как средние значений координат объектов, отнесенных к каждому кластеру.
4. Повторять шаги 2 и 3 до тех пор, пока координаты центроидов кластеров практически перестанут изменяться или не будет достигнуто максимальное количество итераций.

Алгоритм K-Means можно описать как задачу оптимизации — итерационный подход для минимизации *внутрикластерной суммы квадратичных ошибок* (*Sum of Squared Errors — SSE*), которую иногда называют инерцией кластера

$$SSE = \sum_{i=1}^n \sum_{j=1}^k w^{(i,j)} \left\| \mathbf{x}^{(i)} - \mu^{(j)} \right\|^2, \quad (4.44)$$

где $\mu^{(j)}$ — центроид кластера j , $w^{(i,j)} = 1$, если пример $\mathbf{x}^{(i)}$ находится в кластере j , и $w^{(i,j)} = 0$ в противном случае.

Инерция кластера является метрикой качества кластеризации — чем меньше инерция, тем лучше кластеризация. Другой характеристикой является *анализ силуэтов* [6], отражающий меру плотности группирования примеров в кластерах.

Коэффициент силуэта (silhouette coefficient) *одного примера* в наборе данных вычисляется по следующему алгоритму:

1. Вычислить *связность кластера* $a^{(i)}$ как среднее расстояние между примером и всеми остальными примерами в том же самом кластере.

2. Вычислить *отделение кластера* $b^{(i)}$ от следующего ближайшего кластера как среднее расстояние между примерами кластера и всеми примерами в ближайшем кластере.

3. Вычислить *коэффициент силуэта* $s^{(i)}$ как разность между связностью и отделением кластера, деленную на большее среди них значение:

$$s^{(i)} = \frac{b^{(i)} - a^{(i)}}{\max[b^{(i)}, a^{(i)}]}$$

Коэффициент силуэта лежит в диапазоне от -1 до 1. Коэффициент силуэта равен 0 в случае равенства отделения и связности кластера. Чем ближе коэффициент силуэта к 1, тем лучше. Большой коэффициент силуэта получается, если $b^{(i)} \gg a^{(i)}$, так как $b^{(i)}$ показывает, насколько пример не похож на примеры из других кластеров, и $a^{(i)}$ показывает, в какой степени пример похож на остальные примеры в своем кластере.

При оценке качества кластеризации обязателен анализ содержания кластеров, т. е. анализа содержания кластеров с точки зрения поставленной задачи.

Алгоритм K-Means для размещения исходных центроидов применяет случайные начальные значения, что может привести к плохой кластеризации или медленной сходимости в случае, если начальные центроиды выбраны неудачно. Один из способов решения проблемы предусматривает прогон алгоритма K-Means несколько раз на наборе данных и выбор модели, лучше всех выполняющейся с точки зрения SSE (4.44).

В *модификации алгоритма K-Means++* использован выбор начального расположения центроидов такой, что центроиды находятся далеко друг от друга. Алгоритм строится следующим образом:

1. Инициализировать пустой набор $\mathbf{M}$ для хранения k центроидов.

2. Случайным образом выбрать из входных образцов первый центроид $\mu^{(j)}$ и присвоить его $\mathbf{M}$.

3. Для каждого примера $\mathbf{x}^{(i)}$, отсутствующего в $\mathbf{M}$, найти минимальное квадрат расстояния $d(\mathbf{x}^{(i)}, \mathbf{M})^2$ до всех центроидов в $\mathbf{M}$.

4. Следующий центроид $\mu^{(p)}$ выбирается с наибольшей вероятностью

$$\frac{d(\mathbf{x}^{(p)}, \mathbf{M})^2}{\sum_i d(\mathbf{x}^{(i)}, \mathbf{M})^2}.$$

5. Повторять шаги 2 и 3 до тех пор, пока не будет выбрано k центроидов.

6. Продолжить выполнение согласно классическому алгоритму K-Means.

Для выбора количества кластеров строят график изменения SSE (4.44) для различного количества кластеров. Значение кластеров, при котором значение SSE начинает расти быстрее всего (эта точка называется "локтем") является лучшим количеством кластеров.

Другая группа алгоритмов кластеризации относится к *иерархической кластеризации*. Суть иерархической кластеризации состоит в последовательном объединении меньших кластеров в большие или разделении больших кластеров на меньшие. Соответственно выделяют две группы методов.

Иерархические агломеративные методы (Agglomerative Nesting, AGNES). Эта группа методов характеризуется последовательным объединением исходных элементов и соответствующим уменьшением числа кластеров. Алгоритм агломеративной кластеризации следующий [6]:

1. Рассчитать матрицу расстояний (4.41) для всех образцов.
2. Представить каждый пример как одноэлементный кластер.
3. Объединить два ближайших кластера, основываясь на расстоянии между самыми непохожими (дальними) членами.
4. Обновить матрицу расстояний.
5. Повторять шаги 2–4 до тех пор, пока не останется заданное количество кластеров.

Иерархические дивизимные (делимые) методы (Dlvisive ANALysis, DIANA) являются логической противоположностью агломеративным методам. В начале работы алгоритма все объекты принадлежат одному кластеру, который на последующих шагах делится на меньшие кластеры, в результате образуется последовательность расщепляющих групп. Этот метод применяют, когда необходимо разделить все множество объектов I на относительно небольшое число кластеров.

Рассмотрим алгоритм подробнее. На первом шаге все элементы помещаются в один кластер $C_1 = I$. Затем находится элемент, у которого среднее значение расстояния от других элементов в этом кластере наибольшее. Найденный элемент удаляется из кластера C_1 и образует первый элемент нового кластера C_2 .

На каждом последующем шаге элемент в кластере C_1 , для которого разница между средним расстоянием до элементов кластера C_2 и средним расстоянием до других элементов кластера C_1 наибольшая, переносится в кластер C_2 . То есть переносятся элементы, расположенные ближе к элементам C_2 , чем к элементам C_1 . Процесс продолжается, пока существуют в C_1 элементы, расположенные ближе к элементам C_2 , чем к остальным элементам C_1 .

В результате один кластер делится на два. Каждый полученный кластер расщепляется на следующем уровне иерархии. Часто на последующих уровнях расщепляют только один из полученных кластеров, например, кластер с наибольшим диаметром, определяемым как наибольшее расстояние между элементами в кластере. Процесс расщепления продолжается до тех пор, не будет получено нужное количество кластеров. Теоретически алгоритм позволяет получить одноэлементные кластеры.

Алгоритм кластеризации *DBSCAN* (*DBSCAN* — density based spatial clustering of applications with noise — пространственная кластеризация приложений с шумом на основе плотности). Подход *DBSCAN* не выдвигает допущений о сферических кластерах, как делает алгоритм *K-Means*. Кластеризация на базе плотности назначает метки кластеров, основываясь на плотных областях точек. В *DBSCAN* понятие плотности определяется как количество точек внутри указанного радиуса ε .

Согласно алгоритму *DBSCAN* каждому примеру (точке данных) назначается специальная метка с применением перечисленных ниже критериев [6] (рис. 4.6).

- Точка считается *ядерной* (core point), если, по крайней мере, указанное количество (MinPts) соседних точек попадают внутрь заданного радиуса ε .
- *Граничная точка* (border point) — это точка, имеющая число соседей меньше MinPts в пределах ε , но лежащая внутри радиуса ε ядерной точки.
- Все остальные точки, которые не являются ядерными или граничными, трактуются как *шумовые* (noise point).

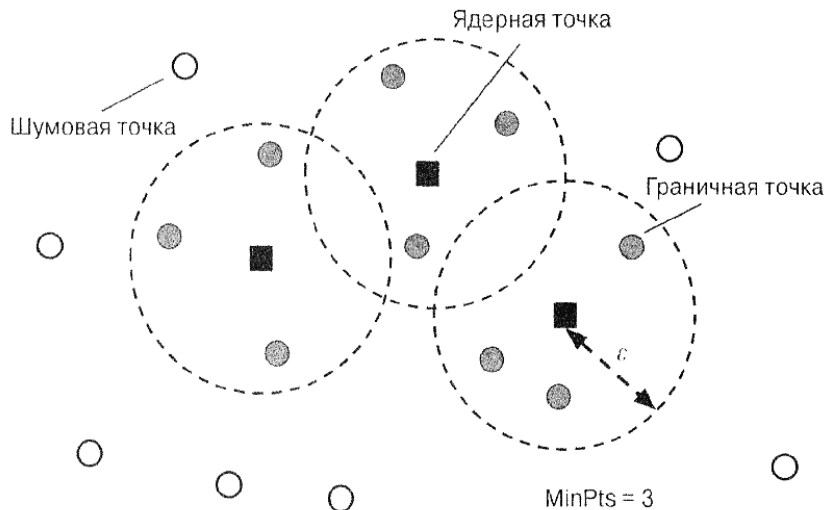


Рис. 4.6. Ядерные, граничные и шумовые точки [6]

После пометки точек как ядерных, граничных или шумовых алгоритм DBSCAN сводится к выполнению двух простых шагов.

1. Сформировать отдельный кластер для каждой ядерной точки или связной группы ядерных точек. (Ядерные точки считаются связными, если расположены не дальше, чем ϵ .)

2. Назначить каждую граничную точку кластеру, к которому принадлежит соответствующая ей ядерная точка.

Пример результатов кластеризации алгоритмом DBSCAN показан на рис. 4.7. Другими алгоритмами кластеризацию такого набора примеров провести нельзя.

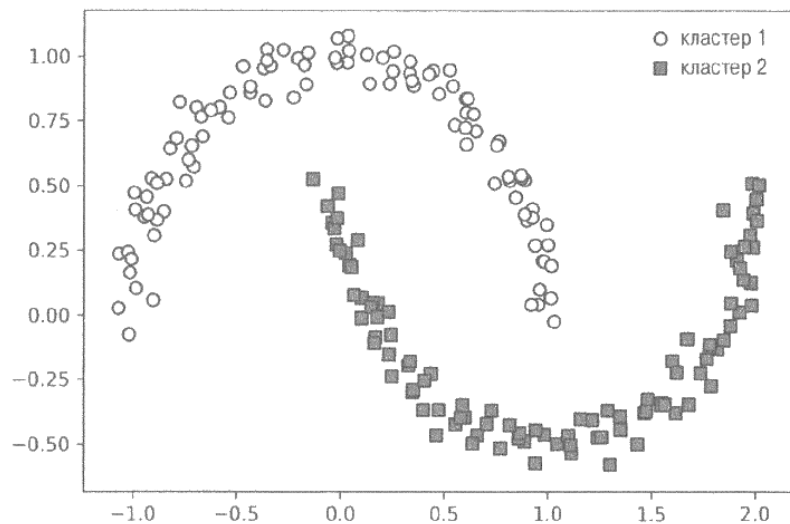


Рис. 4.7. Пример кластеризации по алгоритму DBSCAN [6]

4.6. Оценивание классификаторов

4.6.1. Перекрестная проверка моделей

Оценивание качества модели должно проводиться на тех данных, на которых модель не обучалась. Для этого используется *перекрестная проверка* (*CV*, *Cross-Validation* — *кросс-валидация*), или *скользящий контроль* [6]. *Перекрестная проверка* — процедура экспериментального оценивания обобщающей способности модели машинного обучения, обучаемой с учителем. Общая идея перекрестной проверки состоит в том, что исходное множество обучающих примеров разбивается на два подмножества: обучающее и тестовое. Разбиение может производиться один или несколько раз. Для каждого разбиения выполняется обучение модели на обучающем множестве, затем оценивается его ошибка на объектах тестового множества. *Оценкой перекрестной проверки* называется средняя по всем разбиениям величина ошибки на тестовых множествах. Если примеры исходного множества независимы, то средняя ошибка перекрестной проверки даёт несмещённую оценку вероятности ошибки.

Проще всего реализуется так называемый *валидация на отложенных данных* (*hold-out CV*). Зачастую такой метод проверки рассматривают как самостоятельный подход, противопоставляя его перекрестной проверке. Исходное множество разбивается в общем случае на два подмножества:

- *обучающее множество* (*Training Set*), которое используется для настройки параметров модели;
- *тестовое множество* (*Test Set*), которое используется для проверки качества обучения;

Метод *Hold-out* применяется в случаях больших наборов, т.к. требует меньше вычислительных мощностей по сравнению с другими методами кросс-валидации. Недостатком метода является то, что оценка существенно зависит от разбиения (например, в тестовое множество может попасть мало примеров определенного класса), тогда как желательно, чтобы она характеризовала только алгоритм обучения. Этот метод неприменим для малых наборов данных, так как в этом случае контрольные и тестовые множества будут маленькими и не обеспечат достоверных оценок.

При малом количестве обучающих примеров чаще всего используются *кросс-валидация по k блокам* (*k -fold CV*). В этом методе исходное множество примеров разбивается на k непересекающихся частей. Как правило, $k=10$ (5 в случае малого размера выборки). Производится k итераций. На каждой итерации производится следующие:

- Модель обучается на $k-1$ частях выборки;

- Модель тестируется на части обучающей выборки, которая не участвовала в обучении.

Каждая из k частей единожды используется для тестирования. Итоговая оценка точности получается усреднением оценок на каждой итерации. На рис. 4.8 показан пример для $k=5$. Используемые для тестирования блоки показаны заливкой.

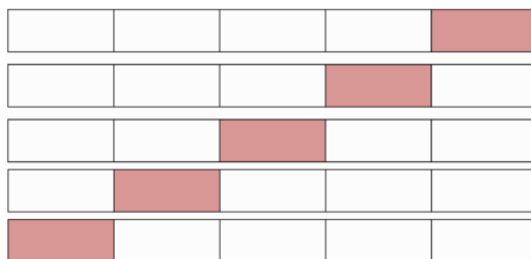


Рис. 4.8. Разбивка на блоки

Известны и другие подходы. Например, при реализации $t \times q$ -fold CV процедура проверки выполняется t раз. При каждом выполнении исходный набор данных случайным образом разбивается на k непересекающихся одинаковых по объему частей и производится k итераций k -fold CV. Кросс-валидация по отдельным объектам (Leave-One-Out) является вариантом кросс-валидации по k блокам (k -fold CV) при размере блока, равном одному примеру. То есть каждый пример по очереди используется для проверки, а остальные примеры используются для обучения.

4.6.2. Метрики качества в задачах классификации

Рассмотрим метрика оценивания моделей классификации [8, 32]. Ошибки классификатора должны определяться на данных, на которых классификатор не обучался, то есть определяется ошибка обобщения. Сначала рассмотрим задачу бинарной классификации, в которой метки (целевые значения) принадлежат множеству $\{0, 1\}$. Единица означает, что пример принадлежит классу, ноль — не принадлежит. Например, если стоит задача выявить больных пациентов, то метка, равная 1, стоит в примерах, соответствующих больным. Если необходимо выявить здоровых пациентов, то метка 1 стоит у примеров, соответствующих здоровым пациентам. Объекты с меткой 1 называются положительными, а с меткой 0 — отрицательными. Модель машинного обучения зачастую вычисляет вероятность отнесения примера к одному из классов. С помощью порогового значения (точки отсечения — Cut-Off Point) ответ классификатора превращается в бинарный. Если

выход классификатора превышает порог отсечения, то это означает положительный исход, в противном случае — отрицательный.

Ошибки классификатора удобно представлять в форме таблицы — в виде *матрицы ошибок*, или *несоответствий* (*Confusion Matrix*) — матрицы, в которой для каждого класса наблюдений приводятся результаты отнесения наблюдений к тому или иному классу. Столбцы матрицы соответствуют предсказанным классам, а строки — истинным классам. На рис. 4.9 показан пример матриц ошибок, полученной при классификации набора данных "Ирисы Фишера" с помощью логистической регрессии библиотеки Scikit-Learn [7].

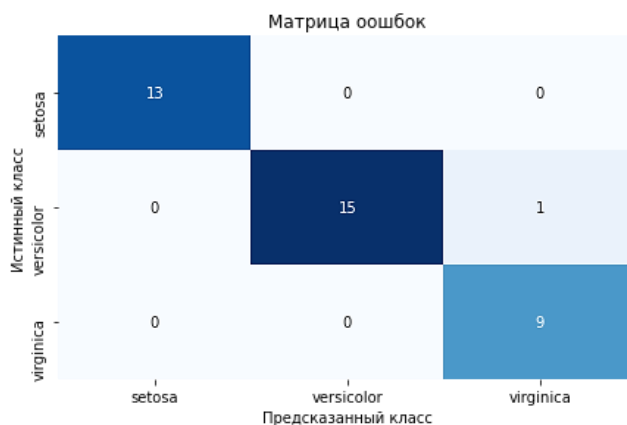


Рис. 4.9. Пример матриц ошибок

На рис. 4.9 видно, что 13 примеров *Iris setosa* распознаны правильно как *Iris setosa*. Из 16 примеров *Iris versicolor* 15 распознаны правильно, один пример распознан неправильно.

Ошибки, допускаемые моделью при ее практическом применении, могут приводить к ложным выводам, что влечет за собой материальные и, следовательно, финансовые потери от неправильно принятых решений. Такие потери называются *издержками ошибки классификации* (*Classification Cost Error, Misclassification Cost*). Например, при разработке модели кредитного скоринга (оценки кредитоспособности лица) ставится цель разделить всех клиентов банка на добросовестных и недобросовестных. Возникает вопрос: "Что лучше — принять добросовестного клиента за недобросовестного или наоборот?" Ответ очевиден: в первом случае мы теряем только проценты по кредиту, который не был выдан, а во втором — всю сумму, выданную недобросовестному заемщику. То есть издержки ошибок второго вида больше. При построении модели необходимо минимизировать вероятность появления ошибок, которые вызывают наибольшие издержки классификации. Такая методика называется *классификацией с учетом издержек* (*Cost-Sensitive Classification*) [3].

В бинарной классификации каждое отдельное предсказание может иметь четыре исхода:

- истинно положительный (*True Positive*, T_P);
- истинно отрицательный (*True Negative*, T_N);
- ложно положительный (*False Positive*, F_P);
- ложно отрицательный (*False Negative*, F_N).

Истинно положительным исход будет, когда фактический класс данного примера **1** и модель на выходе выдаст **1**. Истинно отрицательным исход будет, когда фактический класс наблюдения **0** и модель выдаст **0**. Ложно положительное значение имеет место, когда класс наблюдения **0**, а модель для него сформирует выход **1**. При ложно отрицательном выходе целевая переменная принимает значение **1**, а на выходе модель выдаст **0**. Результаты представлены на рис. 4.10 в виде матрицы ошибок.

		Результат классификации	
		1	0
Фактический класс	1	N_{TP}	N_{FN}
	0	N_{FP}	N_{TN}

Рис. 4.10. Матрица ошибок

На рис. 4.10 обозначено: N_{TP} — количество истинно положительных исходов (классификаций), определенных классификатором; N_{FN} — количество ложно негативных исходов; N_{FP} — количество ложно позитивных исходов; N_{TN} — количество истинно отрицательных исходов.

Рассмотренные понятия связаны с известными в математической статистике понятиями *ошибок первого и второго рода* [33]. Если бинарная классификационная модель строится на основе обучающей выборки, то все входящие в нее примеры соответствуют либо положительному, либо отрицательному исходам. Тогда в процессе работы модели могут возникнуть следующие ошибки.

- Пример соответствует положительному исходу, но был распознан как отрицательный: заемщик кредитоспособен, но модель распознает его как некредитоспособного; пациент болен, но модель распознает его как здорового; радиолокационная система не смогла обнаружить цель по ее сигналу, принятому локатором (пропуск цели), и т. д. Иными словами, интересующее событие ошибочно не обнаружено. Такие ошибки называются *ошибками I рода*.

- Пример соответствует отрицательному исходу, но был распознан как положительный: заемщик некредитоспособен, но модель определяет его как кредитоспособного; пациент здоров, но модель определяет его как больного; радиолокационная цель отсутствует, но система определяет ее наличие (ложная тревога) и т. д. Иными словами, интересующее событие не произошло, но было обнаружено. Такие ошибки называются *ошибками II рода*.

Рассмотрим основные показатели, позволяющих оценить качество бинарной классификационной модели [8, 32].

Точность (Accuracy) — это число правильно классифицированных наблюдений, отнесенное к общему числу наблюдений

$$R_{AC} = \frac{N_{TP} + N_{TN}}{N},$$

где N_{TP} — число истинно положительных наблюдений, N_{TN} — число истинно отрицательных наблюдений, N_{FP} — число ложно положительных наблюдений, N_{FN} — число ложно отрицательных наблюдений, $N = N_{TP} + N_{TN} + N_{FP} + N_{FN}$ — общее число наблюдений.

Этот показатель имеет большой недостаток. Например, диагностируется некоторое заболевание (положительный исход — наличие заболевания). Выборка является *несбалансированной* и содержит 1000 примеров, из них 50 положительных (наличие заболевания) и 950 отрицательных (отсутствие заболевания). Классификатор не обнаружил ни одного положительного исхода, а точность будет 0,95, т. е. очень хорошая.

От этого недостатка свободен показатель *прецизионность (Precision)* — точность предсказания положительного исхода, т. е. какая доля объектов, выделенных классификатором как положительные, действительно является положительными

$$R_{PR} = \frac{N_{TP}}{N_{TP} + N_{FP}}.$$

В русском языке *Accuracy* и *Precision*, обычно, переводятся как "точность", что вызывает путаницу.

Полнота (Recall), или *чувствительность (Sensitivity)* — это доля истинно положительных примеров, правильно определенных классификатором

$$S_R = \frac{N_{TP}}{N_{TP} + N_{FN}}.$$

В машинном обучении чаще используется термин "полнота", а в медицинской диагностике и биостатистике — "чувствительность". Также этот показатель называют *доля истинно положительных исходов (True Positive Rate)*

Специфичность (Specificity) оценивает способность классификатора предсказывать отрицательный исход

$$S_P = \frac{N_{TN}}{N_{TN} + N_{FP}}.$$

Если, например, диагностируется наличие болезни у пациентов, то чувствительность — это доля больных пациентов, для которых диагностическое правило верно диагностирует наличие болезни. А специфичность — доля здоровых пациентов, для которых диагностическое правило верно диагностирует отсутствие болезни.

Хороший классификатор должен обеспечивать высокие показатели точности и полноты. Но достичь одновременно максимальных значений точности и полноты нельзя. Поэтому для экспериментов с классификаторами необходима единая метрика, объединяющая точность и полноту. Такой метрикой является *F-мера* (*F-score*). Используется два вида F-меры. F_1 является гармоническим средним между прецизионностью и полнотой

$$F_1 = 2 \frac{R_{PR} * S_R}{R_{PR} + S_R}.$$

Мера F_1 будет близка к нулю, если хотя бы из ее аргументов близок к нулю. Причем прецизионность и полнота оказывают одинаковое влияние.

Можно задать различный вес прецизионности и полноте, используя меру

$$F_\beta = (\beta^2 + 1) \frac{R_{PR} * S_R}{\beta^2 R_{PR} + S_R}.$$

если $0 < \beta < 1$, то приоритет отдается прецизионности, если $\beta > 1$, то приоритет отдается полноте. При $\beta = 1$ получается мера F_1 .

Минимизировать издержки ошибок бинарной классификации можно с помощью выбора *точки отсечения* (*Cut-Off Point*) — порогового значения, разделяющего классы. При уменьшении порога отсечения увеличивается вероятность ошибочного распознавания положительных наблюдений (ложноположительных исходов), а при увеличении возрастает вероятность неправильного распознавания отрицательных наблюдений (ложноотрицательных исходов). Цель заключается в том, чтобы подобрать такое значение точки отсечения, которое дает наибольшую точность распознавания заданного класса, а какого именно — определяется постановкой задачи.

Для оценки качества бинарной классификации применяется *ROC-анализ* [8, 32], основанный на построении *характеристической кривой обнаружения*, или *кривой ошибок* — *ROC-кривой*. Этот термин (*Receiver Operating Characteristic* — операционная характеристика приёмника) пришел из теории обработки сигналов. В радиолокации задача заключается в обнаружении некоторой цели (морской, воздушной и т. д.). При этом положительный исход — цель обнаружена, отрицательный исход — цель не обнаружена. Во время второй мировой войны

термин возник в связи с задачей повышения точности распознавания целей. В настоящее время *ROC*-анализ широко применяется при построении классификаторов различных областей.

ROC-кривая показывает соотношение между долей верно классифицированных примеров (*True Positive Rate* — *TPR*) т. е. полнотой (чувствительностью), и долей неверно классифицированных примеров (*False Positive Rate* — *FPR*)

$$FPR = \frac{N_{FP}}{N_{FP} + N_{TN}} = 1 - \frac{N_{TN}}{N_{TN} + N_{FP}} = 1 - S_p,$$

где $S_p = \frac{N_{TN}}{N_{TN} + N_{FP}}$ — доля истинно отрицательных исходов, или специфичность.

Чтобы построить *ROC*-кривую, нужно изменять порог отсечения в интервале от 0 до 1 с заданным шагом. В результате при каждом значении порога будет меняться количество правильно и неправильно распознанных примеров, а соответственно, чувствительность и долей неверно классифицированных примеров. Для каждого порога рассчитывается чувствительность и долей неверно классифицированных примеров, и строится график, по вертикальной оси которого откладывается чувствительность, а по горизонтальной — доля неверно классифицированных примеров.

Пример *ROC*-кривой представлен на рис. 4.11. Так как на один положительный исход может приходиться несколько отрицательных, и наоборот, то реальная *ROC*-кривая является не гладкой, а изрезанной.

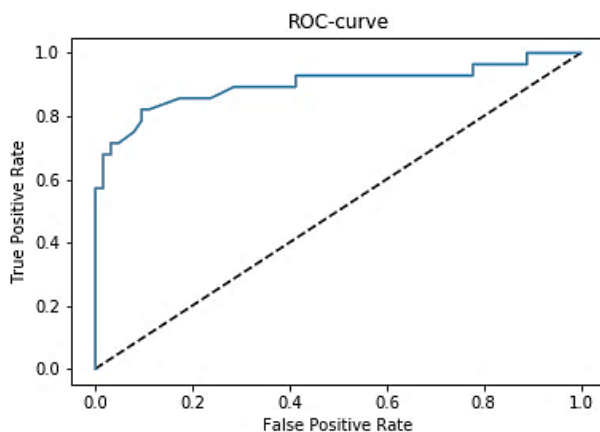


Рис. 4.11. Пример *ROC*-кривой

Каждая точка *ROC*-кривой соответствует определенному значению порога отсечения. При этом оптимальным будет значение, соответствующее точке *ROC*-кривой с координатами, максимально близкими к (0; 1), для которой и чувствительность, и специфичность равны 1, то есть как положительные, так и отрицательные примеры распознаны правильно. Диагональная линия на рис. 4.11

соответствует "бесполезному" классификатору, когда результат получается случайным угадыванием.

Площадь под *ROC*-кривой (*AUC* — *Area Under ROC Curve*) говорит о прогностической силе модели, причем $AUC=1$ соответствует идеальному классификатору. Для реальных классификаторов площадь под *ROC*-кривой больше 0,7–0,8, соответствует достаточно высокой точности классификации.

Так как площадь *AUC* под *ROC*-кривой не меньше 1/2, то вводят *критерий Gini* [34]

$$Gini = AUG * 2 - 1.$$

Критерий *Gini* является нормировочным для *AUC* и принимает значения от нуля до единицы.

Кроме основных метрик используют и другие метрики (см., например, [35]).

Доля неверно классифицированных примеров (False Positive Rate — FPR)

$$FPR = \frac{N_{FP}}{N_{FP} + N_{TN}} = 1 - \frac{N_{TN}}{N_{TN} + N_{FP}} = 1 - S_P,$$

где $S_P = \frac{N_{TN}}{N_{TN} + N_{FP}}$ — доля истинно отрицательных исходов, или специфичность.

В [35]) эта метрика называется *1- специфичность*, единица минус специфичность (1-specificity).

Сбалансированная точность (Balanced Accuracy — BA) — среднее значение чувствительности и специфичности

$$R_{BA} = \frac{S_R + S_P}{2}.$$

В условиях несбалансированности классов эта метрика лучше отражает точность.

Коэффициент Мэттьюса (Matthews correlation coefficient — MCC). *MCC*, по сути, является коэффициентом корреляции между фактической и спрогнозированной бинарной классификациями. Он возвращает значение от -1 до $+1$. Значение $+1$ соответствует идеальной согласованности между фактическими и спрогнозированными значениями, значение 0 означает, что прогноз не лучше случайного угадывания, а значение -1 указывает на полную рассогласованность между фактическими и спрогнозированными значениями. Этот коэффициент рекомендуется применять в случае несбалансированных классов. *MCC* вычисляется на основе матрицы ошибок по формуле

$$F_{MCC} = \frac{N_{TP} \cdot N_{TN} - N_{FP} \cdot N_{FN}}{\sqrt{(N_{TP} + N_{FP})(N_{TP} + N_{FN})(N_{TN} + N_{FP})(N_{TN} + N_{FN})}}.$$

Метрика *каппа Коэна (Cohen's kappa)* оценивает согласованность между двумя экспертами, классифицирующими n объектов по s взаимоисключающим категориям. В качестве экспертов выступают фактические и спрогнозированные

метки классов зависимой переменной. Ее часто используют вместо правильности (*Accuracy*) в случае несбалансированных классов

$$\kappa = \frac{p_0 - p_e}{1 - p_e},$$

p_0 — относительная наблюдаемая согласованность (равна *Accuracy*), p_e — ожидаемая вероятность случайной согласованности, κ — греческая буква "каппа", давшая название метрике

Если между экспертами наблюдается полная согласованность, то $\kappa = 1$. Если согласованность между двумя экспертами случайна, то $\kappa = 0$. Статистика может быть отрицательной, когда согласованность хуже случайной. Рассмотрим вычисление каппы Коэна для случая двухклассовой классификации.

Наблюдаемая согласованность равна

$$p_0 = R_{AC} = \frac{N_{TP} + N_{TN}}{N}.$$

Для вычисления вероятности случайной согласованности примем, что эксперт 1 (строки — фактические классы на рис. 4.10) отнес все наблюдения к первому классу. Вероятность этого события равна $(N_{TP} + N_{FN}) / (N_{TP} + N_{FN} + N_{FP} + N_{TN})$. Примем также, что эксперт 2 (столбцы — предсказанные классы) также отнес все наблюдения к первому классу. Вероятность этого события равна $(N_{TP} + N_{FP}) / (N_{TP} + N_{FN} + N_{FP} + N_{TN})$. Вероятность того, что оба эксперта отнесли наблюдения к классу 1, равна

$$p_{e1} = \frac{N_{TP} + N_{FN}}{N_{TP} + N_{FN} + N_{FP} + N_{TN}} \cdot \frac{N_{TP} + N_{FP}}{N_{TP} + N_{FN} + N_{FP} + N_{TN}}.$$

Аналогично, вероятность того, что оба эксперта отнесли все наблюдения к классу 0, равна

$$p_{e0} = \frac{N_{TN} + N_{FP}}{N_{TP} + N_{FN} + N_{FP} + N_{TN}} \cdot \frac{N_{TN} + N_{FN}}{N_{TP} + N_{FN} + N_{FP} + N_{TN}}.$$

Общая вероятность случайной согласованности равна

$$p_e = p_{e1} + p_{e0}.$$

При $\kappa > 0,75$ согласованность считается высокой, при $0,4 < \kappa < 0,75$ — хорошей, в противном случае — плохой.

В задачах многоклассовой классификации, когда необходимо выделить несколько классов, вычисление метрик качества, обычно, сводят к вычислению одной из рассмотренных выше двухклассовых метрик [8]. Для многоклассовой классификации используются две стратегии. Если проводится классификация на K классов, то стратегия "один против остальных" (one-vs-rest) предусматривает создание K двоичных классификаторов. Первый классификатор производит классификацию

объектов на два класса: относящихся к первому классу, и всем остальным. Вторым классификатором — относящихся ко второму классу и всем остальным и т.д. При классификации нового объекта признаки объекта подаются на все классификаторы и выбирается класс, чей классификатор дал наибольший результат. Стратегия "один против одного" (one-versus-one) для классификации на K классов создает $\frac{K(K-1)}{2}$ двоичных классификаторов, выделяющих по два класса. Например, при классификации на три класса первый классификатор выделяет первый и второй классы, второй — первый и третий, третий — второй и третий. Во время прогнозирования образец подается на каждый из классификаторов. В качестве класса образца принимается класс, спрогнозированный большинством классификаторов.

Для многоклассовой классификации метрики вычисляются с помощью микроусреднения, макроусреднения и взвешенного макроусреднения. Например, микроусреднение для *Precision* в случае k классов имеет вид

$$R_{PR_микро} = \frac{N_{TP1} + N_{TP2} + \dots + N_{TPk}}{N_{TP1} + N_{TP2} + \dots + N_{TPk} + N_{FP1} + N_{FP2} + \dots + N_{FPk}},$$

где N_{TPi} и N_{FPi} — число истинно положительных наблюдений и число ложно положительных наблюдений для класса i .

Макроусреднение вычисляется как среднее для значения метрик для разных классов, например

$$R_{PR_макро} = \frac{R_{PR1} + R_{PR2} + \dots + R_{PRk}}{k}.$$

В случае несбалансированных классов применяется взвешенное (нормализованное) макроусреднение, вычисляемое как сумма произведений метрик разных классов, умноженных на вес каждого класса. Вес класса равен доле примеров класса в выборке.

Литература к разделу 4

1. Гмурман В. Е. Теория вероятностей и математическая статистика. — М.: Юрайт, 2017. — 480 с.
2. Юджовски Э. Наглядное объяснение теоремы Байеса. [Электронный ресурс]. URL: https://lesswrong.ru/w/%D0%9D%D0%B0%D0%B3%D0%BB%D1%8F%D0%B4%D0%BD%D0%BE%D0%B5_%D0%BE%D0%B1%D1%8A%D1%8F%D1%81%D0%BD%D0%B5%D0%BD%D0%B8%D0%B5_%D1%82%D0%B5%D0%BE%D1%80%D0%B5%D0%BC%D1%8B_%D0%91%D0%B0%D0%B9%D0%B5%D1%81%D0%B0 (дата обращения: 13.12.2023).
3. Паклин Н. Б., Орешков В. И. Бизнес-аналитика: от данных к знаниям. — СПб.: Питер, 2013. — 704 с.
4. 6 простых шагов для освоения наивного байесовского алгоритма (с примером кода на Python). [Электронный ресурс]. URL: <http://datareview.info/article/6-prostyih-shagov-dlya-osvoeniya-naivnogo-bayesovskogo-algoritma-s-primerom-koda-na-python/> (дата обращения: 13.10.2023).
5. Топ-10 data mining-алгоритмов простым языком. [Электронный ресурс]. URL: <https://habrahabr.ru/company/itinvest/blog/262155/> (дата обращения: 13.10.2023).
6. Рашка С., Мирджалили В. Python и машинное обучение: машинное и глубокое обучение с использованием Python, scikit-learn и TensorFlow. — СПб.: ООО "Диалектика", 2020. — 948 с.
7. Элбон К. Машинное обучение с использованием Python. Сборник рецептов. — СПб.: БХВ-Петербург, 2019. — 384 с.
8. Жерон О. Прикладное машинное обучение с помощью Scikit Learn, Keras и TensorFlow: концепции, инструменты, и техники для создания интеллектуальных систем. — СПб.: ООО "Диалектика", 2020. — 1040 с.
9. Алгоритмы: анализ и построение / Т. Кормен, Ч. Лейзерсон, Р. Ривест, К. Штайн. — М.: Вильямс, 2005. — 1296 с.
10. Шахиди А. Деревья решений: общие принципы. [Электронный ресурс]. URL: <https://loginom.ru/blog/decision-tree-pl> (дата обращения: 13.10.2023).
11. Вапник В. Н., Червоненкис А. Я. Теория распознавания образов. Статистические проблемы обучения. — М.: Наука, 1974. — 416 с.
12. Cortes C., Vapnik V. Support-vector networks // Machine Learning. — 1995. — Vol. 20, no. 3. — P. 273 – 297. [Электронный ресурс]. URL: http://image.diku.dk/imagecanon/material/cortes_vapnik95.pdf (дата обращения: 21.10.2023).

13. Бишоп К. Распознавание образов и машинное обучение. — Спб.: ООО "Диалектика", 2020. — 960 с.
14. Хасты Т., Тибширани Р., Фридман Д. Основы статистического обучения: интеллектуальный анализ данных, логический вывод и прогнозирование. — СПб.: ООО "Диалектика", 2020. — 768 с.
15. Хайкин С. Нейронные сети: полный курс. — М.: Вильямс, 2006. — 1104 с.
16. Воронцов К. В. Лекции по методу опорных векторов. [Электронный ресурс]. URL: <http://www.ccas.ru/voron/download/SVM.pdf> (дата обращения: 21.10.2023).
17. Классификация данных методом опорных векторов. [Электронный ресурс]. URL: <https://habr.com/ru/post/105220/> (дата обращения: 21.10.2023).
18. Гилл Ф., Мюррей У., Райт М. Практическая оптимизация. — М.: Мир, 1985. — 509 с.
19. Васильев Ф. П. Численные методы решения экстремальных задач. — М.: Наука, 1988. — 552 с.
20. Таха Хемди А. Введение в исследование операций. — М.: Вильямс, 2005. — 912 с.
21. Bertsekas D. P. Nonlinear programming. — Belmont: Athena Scientific, 1999. — 742 p.
22. Базара М., Шетти К. Нелинейное программирование. Теория и алгоритмы. — М.: Мир, 1982. — 580 с.
23. Тихонов А. Н., Арсенин В. Я. Методы решения некорректных задач. — М.: Наука, 1986. — 288 с.
24. Cover T, M. Geometrical and Statistical Properties of Systems of Linear Inequalities with Applications in Pattern Recognition // IEEE Transactions on Electronic Computers. — 1965, Vol. 14. — P. 326–334.
25. Koutroumbas K., Theodoridis S. Pattern Recognition. — Academic Press, 2008. — 984 p.
26. Березанский Ю. М., Ус Г. Ф., Шефтель З. Г. Функциональный анализ. — Київ, Вища школа, 1990. — 600 с.
27. Mercer's theorem. [Электронный ресурс]. URL: http://en.wikipedia.org/wiki/Mercer%27s_theorem (дата обращения: 21.10.2023).
28. SVM with polynomial kernel visualization. [Электронный ресурс]. URL: <https://www.youtube.com/watch?v=OdINM96sHio> (дата обращения: 21.10.2023).

29. Пател А. Прикладное машинное обучение без учителя с использованием Python. — СПб.: ООО "Диалектика", 2020. — 432 с.
30. Паклин Н. Кластеризация категориальных данных: масштабируемый алгоритм CLOPE. [Электронный ресурс]. URL: <https://basegroup.ru/community/articles/clope> (дата обращения: 27.10.2023).
31. Анализ данных и процессов / А. А. Барсегян, М. С. Куприянов, И. И. Холод, М. Д. Тесс, С. И. Елизаров. — СПб.: БХВ-Петербург, 2009. — 512 с.
32. Брюс П., Брюс Э., Гедек П. Практическая статистика для специалистов Data Science. — СПб.: БХВ-Петербург, 2021. — 352 с.
33. ГОСТ Р ИСО 3534-1-2019. Статистические методы. Словарь и условные обозначения. Часть 1. Общие статистические термины и термины, используемые в теории вероятностей. — Москва: Стандартинформ, 2019. — 70 с.
34. Протоद्याконов А. В., Пылов П. А., Садовников В. Е. Алгоритмы Data Science и их практическая реализация на Python. — М., Вологда: Инфра-Инженерия, 2022. — 392 с.
35. Груздев А. В. Предварительная подготовка данных в Python. Том 2: План, примеры и метрики качества. — М. ДМК пресс, 2023. — 814 с.

5. АНСАМБЛИ МОДЕЛЕЙ МАШИННОГО ОБУЧЕНИЯ

5.1. Предпосылки объединения моделей машинного обучения в ансамбли

Цель объединения моделей машинного обучения в ансамбли — получить более точный результат. Причины использования ансамблей достаточно очевидны. Например, используя прогноз нескольких экспертов, можно получить более точный прогноз, чем дает каждый эксперт. Рассматривая модели машинного обучения, будем, прежде всего, иметь в виду классификаторы, но многие результаты справедливы и для регрессионных моделей. Недаром в [1] при рассмотрении ансамблей используется термин *прогнозатор*, понимая под ним классификаторы и регрессоры. Построенная модель машинного обучения может допускать большое число ошибок. Говорят, что построена слабая модель [2]. Следует попытаться усовершенствовать модель, например, изменяя структуру нейронной сети, алгоритмы и параметры обучения. Но зачастую из-за сложности задачи и низкого качества обучающих данных не удастся достичь удовлетворительного качества работы модели. Тогда целесообразно применить набор моделей, применяемых совместно для решения единственной задачи. Такой набор моделей называется *ансамблем (комитетом)* моделей [1–3].

Можно выделить три причины, по которым ансамбль моделей может быть лучше отдельных моделей, входящих в ансамбль [4]:

1. *Статистическая причина.* Применение ансамбля моделей усредняет ошибку каждой отдельной модели и уменьшает влияние нестабильностей и случайностей при формировании гипотез. Решение задач классификации и регрессии представляет собой поиск гипотез о свойствах системы или о следующем состоянии системы. Для этого выдвигаются и оцениваются гипотезы. Выдвижение и оценка гипотез зависит от случайных факторов: разброс значений в наборе данных, выбор первой гипотезы, выбор последовательности обучающих примеров и т.д. Отдельно взятая модель часто находит и "доказывает" на обучающем множестве гипотезу, которая затем не показывает точности на примерах, не вошедших в обучающую выборку. Однако если использовать достаточно большое количество моделей, обученных примерно на одном и том же множестве примеров, то можно уменьшить нестабильность и случайность полученного результата путем комбинирования результатов (например, путем их усреднения, голосования и т.д.). Теоретически можно показать [5], что усреднение по множеству моделей, построенных на основе независимых обучающих множеств, всегда уменьшает ожидаемое значение среднеквадратической ошибки.

2. *Вычислительная причина.* Часто модели машинного обучения обучаются градиентными алгоритмами, которые могут не попадать в точку глобального

минимума функционала качества, а попадать в точки локального минимума. Ансамбли моделей, обученных на различных подмножествах исходных данных, имеют бóльший шанс найти глобальный оптимум, так как ищут его из разных начальных точек.

3. *Репрезентативная причина*. Комбинированная гипотеза может не находиться во множестве возможных гипотез для базовых классификаторов, т. е. строя комбинированную гипотезу, мы расширяем множество возможных гипотез.

Применяется несколько подходов к построению ансамблей моделей. Для формирования выхода ансамбля при определенных состояниях выходов моделей используют три подхода [1–3].

1. *Голосование (мажоритарное голосование)*. Применяется в задачах классификации. Выбирается тот класс, который был выдан простым большинством моделей ансамбля.

2. *Взвешенное голосование*. Отличается от простого голосования назначением весов (баллов) для результатов разных моделей. Баллы учитывают точность работы разных классификаторов.

3. *Усреднение (взвешенное или невзвешенное)*. Применяется при решении с помощью ансамбля задачи регрессии, когда выходы моделей будут числовыми. Выход всего ансамбля может определяться как простое среднее значение выходов всех моделей. Если производится взвешенное усреднение, то выходы моделей умножаются на соответствующие веса.

5.2. Классификаторы с голосованием

Это самый простой ансамбль. Несколько различных классификаторов, например, логистическая регрессия, дерево решений, классификатор на основе SVM обучаются на одном и том же наборе данных. Результат формируется на основе голосования (рис. 5.1).

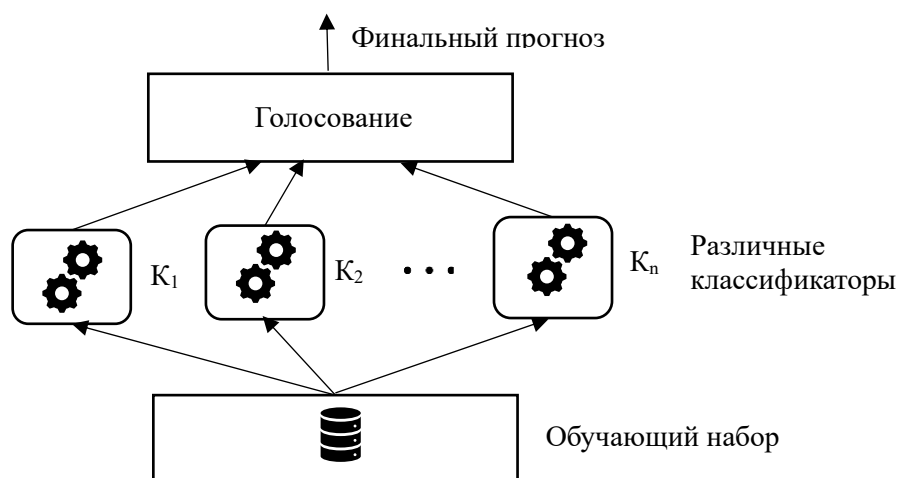


Рис. 5.1. Классификатор с голосованием

Различные классификаторы обучаются на одном и том же наборе данных. При распознавании примера $\mathbf{x}$ на обученных классификаторах пример подается на вход каждого классификатора. Финальный прогноз определяется по формуле

$$C = \text{mode}\{C_1(\mathbf{x}), C_2(\mathbf{x}), \dots, C_n(\mathbf{x})\},$$

где mode — мода — наиболее часто встречающееся значение в наборе данных, $C_1(\mathbf{x}), C_2(\mathbf{x}), \dots, C_n(\mathbf{x})$ — метки классов, спрогнозированных каждым классификатором.

Недостатком ансамбля классификаторов с голосованием является необходимость построения и обучения нескольких классификаторов.

5.3. Бэггинг и перестановка

Другой подход использует одну модель машинного обучения, но обучение производится на разных случайных поднаборах обучающего набора (рис. 5.2). Подход основан на том, что многие классификаторы, параметры которых формируются в процессе обучения, являются неустойчивыми [2]. Небольшие изменения в обучающем множестве могут привести к изменению модели (в случае нейронной сети — к изменению весов сети). Таким образом, обучая одинаковые модели на различных обучающих множествах, можно получить разные классификаторы. Объединяя результаты классификаторов с помощью голосования (или усреднения в случае регрессии) можно получить более точный результат, чем в случае одного классификатора.

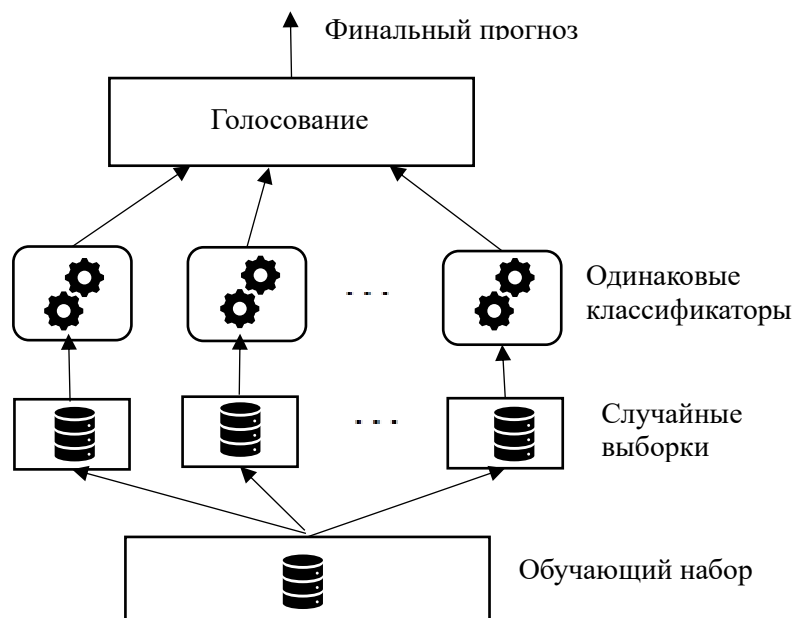


Рис. 5.2. Бэггинг и перестановка

Почему подход уменьшает ошибки отдельных классификаторов? Эффект можно объяснить, представляя ошибку классификатора в виде суммы двух

слагаемых: смещения и дисперсии [4]. Смещение — это ошибка конкретного обучающего алгоритма на бесконечном числе независимых обучающих примеров. Смещение не может равняться нулю, так как не существует идеальных классификаторов. Дисперсия — это ошибка, обусловленная конечными размерами обучающей выборки. Использование нескольких обучающих наборов позволяет уменьшить ошибку за счет уменьшения дисперсии.

При наличии большого обучающего набора можно случайным образом выбрать из него необходимое количество выборок для обучения классификаторов. Но такая возможность встречается довольно редко. Обычно мы имеем ограниченный обучающий набор. Тогда можно использовать два подхода: бэггинг и перестановку [1, 3, 6].

Чаще используется *бэггинг*. Термин "бэггинг" (bagging) происходит от английского словосочетания bootstrap aggregating (бутстреп агрегирование), так как для формирования выборок в этом методе применяется бутстреп. Алгоритм бэггинга достаточно прост. Сначала из исходного множества данных с использованием бутстрепа формируется столько обучающих выборок, сколько предполагается использовать классификаторов. Примеры выборок извлекаются случайным образом с возвратом из общего обучающего множества. То есть извлеченный в выборку пример возвращается в исходный набор. Полученные выборки содержат такое же количество примеров, что и исходное множество. Наборы примеров в этих выборках будут различными: одни примеры могут быть отобраны по нескольку раз, а другие — ни разу. Доказано [7], что в каждой обучающей выборке отсутствует примерно треть примеров исходного обучающего множества. Далее каждый классификатор обучается на своем обучающем множестве. Результат классификации получается путем голосования или простого усреднения (в случае регрессии).

Перестановка реализуется следующим образом [6]. Пусть необходимо из одной выборки получить три подвыборки того же размера, что и исходная выборка. Три экземпляра исходного набора объединяется в единый набор и примеры этого набора перемешиваются случайным образом. Из полученного большого набора случайным образом *без возврата* извлекаются примеры первого поднабора. Из оставшихся примеров случайно выбираются примеры второго поднабора. Наконец, из оставшихся случайно выбирается третий поднабор.

При использовании бэггинга некоторые примеры из обучающей выборки будут выбраны несколько раз, а некоторые — не разу. Доказано [1, 7], что для каждого классификатора в среднем выбирается примерно 63% примеров исходной выборки. Оставшиеся 37% примеров называются *неиспользуемыми* (oob — out-of-bag) и могут использоваться для проверки переобучения классификаторов в качестве проверочного набора [1].

На практике широко применяются *случайные леса* (Random Forest) — ансамбли деревьев принятия решений, обучаемые методом бэггинга. Алгоритм случайного леса имеет важное отличие от алгоритма построения одиночного дерева — при построении дерева при расщеплении каждого узла он ищет лучший признак расщепления не среди всех признаков, а в случайном поднаборе признаков, своем для каждого дерева. Это делает деревья еще более различными.

Кроме выборки примеров для формирования подвыборок при большом числе признаков можно применять выборку признаков. Тогда каждый классификатор обучается на свой подвыборке, отличающейся на разных примерах с разными наборами признаков. Такой подход называется *методом случайных участков* (Random Patches method) [1].

5.4. Бустинг

5.4.1. AdaBoost

Бустинг (англ. *boosting* — усиление) — это процедура *последовательного* построения классификаторов, когда каждый следующий классификатор стремится компенсировать недостатки предыдущих классификаторов и исправить своего предшественника. Известно много методов бустинга, одними из популярных методов является *AdaBoost* — сокращение от Adaptive Boosting (адаптивный бустинг) и *градиентный бустинг* (Gradient Boosting).

Рассмотрим метод AdaBoost [1]. Пусть дан набор примеров $\{\mathbf{x}_i, y_i\}$, где $\mathbf{x}_i$ $i=1, 2, \dots, m$ — обучающие примеры, $y_i = \pm 1$ —метки классов. Алгоритм состоит из ряда итераций. На каждой итерации строится классификатор, который обучается на исходном наборе примеров. В качестве функционала ошибки используется взвешенная ошибка

$$L = \sum_{i=1}^m w_i e_i, \quad (5.1)$$

где w_i — вес i -го примера, вычисленный на предыдущей итерации, $e_i = [y_i \neq \hat{y}_i]$ — *индикатор ошибки*, равный 0, если выход классификатора $\hat{y}_i$ равен целевому y_i значению и 1 в противном случае.

Причем веса правильно классифицированных примеров уменьшаются, а веса неправильно классифицированных примеров остаются без изменения. В результате на каждой итерации классификатор будет больше "внимания" уделять примерам, неправильно распознанным на предыдущих итерациях. Перед началом итераций веса примеров одинаковы: $w_i = 1/m$, где m — количество примеров. Итерации заканчиваются, когда функционал ошибки (5.1) станет малым. Выход ансамбля

формируется в результате взвешенного голосования. Причем веса классификаторов также зависят от ошибки. В результате алгоритм имеет вид.

Положить номер классификатора $j=1$. В цикле, пока не выполнится критерий останова $L \leq \varepsilon$, где $\varepsilon > 0$ — малое число или пока не выполнится заданное число шагов, выполнить шаги 1–5:

1. Обучить j -й классификатор, вычислить ошибку классификатора (5.1).

Вычислить взвешенную частоту ошибок r_j классификатора. Взвешенная частота ошибок j -го классификатора имеет вид

$$r_j = \frac{\sum_{i=1}^m w^{(i)} \mathbb{I}_{\hat{y}_j^{(i)} \neq y^{(i)}}}{\sum_{i=1}^m w^{(i)}},$$

где i — номер примера, $y^{(i)}$ — метка i -го примера, $\hat{y}_j^{(i)}$ — прогноз j -го классификатора для i -го примера.

2. Вычислить вес классификатора

$$\alpha_j = \eta \ln \left(\frac{1 - r_j}{r_j} \right), \quad (5.2)$$

где η — подбираемая скорость обучения (обычно $\eta = 1$).

Чем точнее работает классификатор, тем выше будет его вес. Если классификатор угадывает всего лишь случайно, тогда его вес будет близок к нулю. Однако если он почти всегда ошибается (т.е. правильность ниже случайного угадывания), то его вес будет отрицательным.

3. Пересчитать веса всех примеров ($i = 1, 2, \dots, m$)

$$w^{(i)} = \begin{cases} w^{(i)}, & \text{если } \hat{y}_j^{(i)} = y^{(i)}, \\ w^{(i)} e^{\alpha_j}, & \text{если } \hat{y}_j^{(i)} \neq y^{(i)}. \end{cases}$$

4. Нормировать веса примеров

$$w^{(i)} = \frac{w^{(i)}}{\sum_{i=1}^m w^{(i)}}.$$

5. Положить $j = j + 1$. Переход на шаг 1.

После обучения классификаторов результат классификации подсчитывается по формуле

$$\hat{y} = \text{sign} \left(\sum_{j=1}^K \alpha_j \hat{y}_j \right),$$

где K — число классификаторов (итерационных циклов).

5.4.2. Градиентный бустинг

В настоящее время популярен *градиентный бустинг* (*gradient boosting*) [8, 9]. рассмотрим идеи градиентного бустинга [9]. Рассмотрим *задачу регрессии*, при решении которой минимизируется квадратичный функционал — функция потерь

$$L = \frac{1}{2} \sum_{i=1}^l (a(\mathbf{x}_i) - t_i)^2 \rightarrow \min,$$

где $a(\mathbf{x}_i)$ — выход модели (регрессора) на обучающем примере $\mathbf{x}_i$, $i = 1, 2, \dots, l$, t_i — целевое значение для примера $\mathbf{x}_i$.

Будем искать итоговый алгоритм в виде *суммы слабых моделей* $b(\mathbf{x})$

$$a_N(\mathbf{x}) = \sum_{n=1}^N b_n(\mathbf{x}),$$

где $b_n(\mathbf{x})$ — выход n -й модели, базовые алгоритмы b_n принадлежат некоторому семейству алгоритмов A .

Первую модель можно описать следующим образом

$$b_1(\mathbf{x}) = \arg \min_{b \in A} \frac{1}{2} \sum_{i=1}^l (b(\mathbf{x}_i) - t_i)^2.$$

Наш алгоритм, естественно, не обеспечит нулевую функцию потерь. Рассчитаем для каждого примера *невязку* (*остаток*)

$$r_i^{(1)} = t_i - b_1(\mathbf{x}_i).$$

Если прибавить эти невязки к ответам построенного алгоритма, то он не будет допускать ошибок на *обучающей выборке*. Значит, будет разумно построить второй алгоритм так, чтобы его ответы были как можно ближе к невязкам. Результирующий ответ будем строить как сумму ответов моделей

$$b_2(\mathbf{x}) = \arg \min_{b \in A} \frac{1}{2} \sum_{i=1}^l (b(\mathbf{x}_i) - r_i^{(1)})^2.$$

Каждый следующий алгоритм будем настраивать на невязках предыдущих алгоритмов:

$$r_i^{(N)} = t_i - a_{N-1}(\mathbf{x}_i) = t_i - \sum_{n=1}^{N-1} b_n(\mathbf{x}_i), \quad i = 1, \dots, l,$$

$$b_N(\mathbf{x}) = \arg \min_{b \in A} \frac{1}{2} \sum_{i=1}^l (b(\mathbf{x}_i) - r_i^{(N)})^2.$$

Важно, что *невязки могут быть найдены как антиградиент функции потерь по ответу построенной композиции моделей*

$$r_i^{(N)} = t_i - a_{N-1}(\mathbf{x}_i) = -\frac{\partial}{\partial z} \frac{1}{2} (z - t_i)^2 \Big|_{z=a_{N-1}(\mathbf{x}_i)}.$$

Получается, что мы делаем один шаг *алгоритма градиентного спуска*, двигаясь в сторону скорейшего убывания ошибки на обучающей выборке. На практике, следуя алгоритму градиентного спуска, компоненты вектора антиградиента прибавляют, умноженными на коэффициент. Этот коэффициент или подбирается или находится как решение задачи одномерной оптимизации.

Алгоритм градиентного бустинга имеет вид:

1. Создаем первую модель (дерево решений или регрессор).
2. Обучаем на всей выборке.
3. Делаем предсказание.
4. Считаем антиградиент от функционала ошибки (например, от среднеквадратичной ошибки).
5. Создаем следующую модель и обучаем ее на полученных в п. 4 антиградиентах и том же наборе параметров.
6. Суммируем предсказания полученных моделей.
7. Повторяем с п. 4 по п. 6, пока сумма предсказаний не перестанет меняться или функция потерь не станет меньше заданной величины.

Градиентный бустинг склонен к переобучению. Это явление можно объяснить одной из двух причин:

- Если базовые алгоритмы очень простые (например, решающие деревья небольшой глубины), то они плохо приближают вектор антиградиента. По сути, добавление такого базового алгоритма будет соответствовать шагу вдоль направления, сильно отличающегося от направления наискорейшего убывания. Соответственно, градиентный бустинг может свестись к случайному блужданию в пространстве.
- Если базовые алгоритмы сложные (например, глубокие решающие деревья), то они способны за несколько шагов бустинга идеально настроиться под обучающую выборку что, также является переобучением, связанным с излишней сложностью семейства алгоритмов.

Для ослабления эффекта переобучения используют два основных приема *регуляризации*. Первый — уменьшение шага на каждой итерации. Для этого достаточно добавлять каждый новый алгоритм с меньшим весом, чем предыдущий. Второй прием — изменение объема выборки. Второй прием реализуется в *стохастическом градиентном бустинге*. В этом варианте бустинга на каждой итерации берется для обучения алгоритма не вся выборка, а ее случайная часть. На следующей итерации берется уже другая случайная часть выборки. После каждой итерации части выборки возвращаются обратно в выборку.

Для предотвращения переобучения можно использовать кросс-валидацию и останавливать обучение, когда ошибка на тестовом наборе начинает расти.

Мы рассмотрели задачу регрессии. При решении задачи классификации возникает проблема: при использовании многих классификаторов, например, деревьев решений, обычно применяемые функции потерь, например, процент верных ответов, не являются дифференцируемыми. Проблема решается использованием в классификаторах дифференцируемых функций потерь. Например [9], используется *логистическая функция потерь*

$$L(t, a) = \ln(1 + e^{-at}),$$

где at — произведения целевого значения t и прогноза модели a

Производная от этой функции потерь равна

$$\frac{\partial L}{\partial a} = -\frac{t}{1 + e^{-at}}.$$

Применяют также экспоненциальную функцию потерь $L(t, a) = e^{-at}$ с производной

$$\frac{\partial L}{\partial a} = -te^{-at}.$$

При использовании функций потерь, их значения не будут нулевыми даже при правильной классификации. Но чем увереннее алгоритм дает ответ, тем меньше значение функции потерь.

Литература к разделу 5

1. Жерон О. Прикладное машинное обучение с помощью Scikit Learn, Keras и TensorFlow: концепции, инструменты, и техники для создания интеллектуальных систем. — СПб.: ООО "Диалектика", 2020. — 1040 с.

2. Паклин Н. Б., Орешков В. И. Бизнес аналитика: от данных к знаниям. — СПб.: Питер, 2013. — 704 с.

3. Рашка С., Мирджалили В. Python и машинное обучение: машинное и глубокое обучение с использованием Python, Scikit learn и TensorFlow 2. — СПб.: ООО "Диалектика", 2020. — 848 с.

4. Гончаров М. Ансамбли моделей. [Электронный ресурс].
URL: <http://www.businessdataanalytics.ru/download/ModelEnsembles.pdf>
(дата обращения: 22.11.2025).

5. Терехов С. А. Гениальные комитеты умных машин // Научная сессия МИФИ-2007. IX Всероссийская научно-техническая конференция "Нейроинформатика-2007": Лекции по нейроинформатике. Часть 2. — М.: МИФИ, 2007. — С. 11–42.

6. Брюс П., Брюс Э., Гедек П. Практическая статистика для специалистов Data Science. — СПб.: БХВ-Петербург, 2021. — 352 с.

7. Флах П. Машинное обучение. Наука и искусство построения алгоритмов, которые извлекают данные из знаний. — М.: ДМК Пресс, 2015. — 400 с.

8. Хасты Т., Тибширани Р., Фридман Дж. Основы статистического обучения: интеллектуальный анализ данных, логический вывод и прогнозирование. — СПб.: ООО "Диалектика", 2020. — 768 с.

9. Соколов Е. А. Лекция 10. Градиентный бустинг. [Электронный ресурс].
URL: <https://github.com/esokolov/ml-course-hse/blob/master/2021-fall/lecture-notes/lecture10-ensembles.pdf>
(дата обращения: 22.11.2025).

6. МНОГОСЛОЙНЫЙ ПЕРСЕПТРОН

6.1. Математическая модель искусственного нейрона

Искусственные нейронные сети (в дальнейшем, просто *нейронные сети*) — это модели машинного обучения, построенные из простых нелинейных элементов — искусственных нейронов, соединенных в сети. Сначала рассмотрим *многослойный перцептрон* (MLP — Multi-layer Perceptron) [1–3]. Более правильное название этой модели — полносвязная нейронная сеть. Многослойный перцептрон — это определенная структура сети и алгоритм обучения с учителем, который обучает нелинейную функцию (нейронную сеть) на наборах данных решению задач классификации или регрессии.

Искусственный нейрон строится на простейшем представлении о работе нейронов головного мозга: входные сигналы усиливаются или ослабевают (умножаются на некоторые коэффициенты — веса), взвешенные сигналы суммируются и после нелинейного преобразования поступают на выход нейрона. Учтите, что мы занимаемся прикладным искусственным интеллектом. В вычислительной нейронауке (Computational neuroscience) [4], изучающей нервную систему человека с точки зрения процессов обработки информации, строятся значительно более точные модели нейронов. В спайковых нейронных сетях и нейроморфных процессорах [5] также используются импульсные модели нейронов.

Математическая модель искусственного нейрона описывается уравнением

$$y = f(s) = f\left(\sum_{i=1}^n w_i x_i + w_0\right), \quad (6.1)$$

где y — выходной сигнал нейрона, x_i — входные сигналы, n — количество входных сигналов, $f(s)$ — *функция активации*, w_i — *синаптический вес* (весовой коэффициент, вес), w_0 — *смещение* (порог).

Сумма

$$s = \sum_{i=1}^n w_i x_i + w_0 \quad (6.2)$$

называется *взвешенной суммой*.

Пример структуры искусственного нейрона представлен на рис. 6.1. *Адаптивный сумматор* осуществляет взвешенное суммирование входов. "*Смещение*" получило свое название из-за того, что изменяет (смещает) взвешенную сумму. Часто взвешенную сумму (6.2) записывают в форме

$$s = \sum_{i=0}^n w_i x_i, \quad (6.3)$$

где $x_0 = 1$.

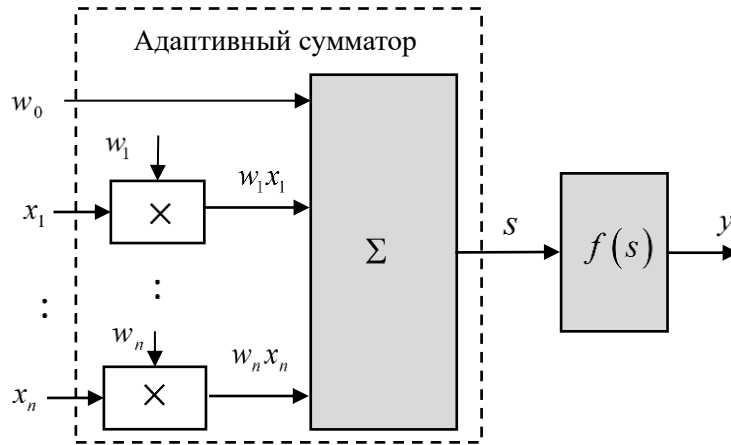


Рис. 6.1. Структура искусственного нейрона

Тогда произведение $w_0 x_0$ представляет собой смещение. Запись (6.3) формально содержит только веса и удобна для использования.

Нейроны входят в состав нейронных сетей. Выход (6.1) i -го нейрона сети с учетом (6.3) записывается в виде

$$y_i = f_i(s_i) = f_i\left(\sum_{j=0}^n w_{ij} x_j\right), \quad (6.4)$$

где индекс i является номером нейрона, индекс j — номером входа нейрона, вес w_{ij} — это вес, соответствующий входу j нейрона i .

Множество входных сигналов можно представить как вектор-столбец $\mathbf{x} = [1 \ x_1 \ x_2 \ \dots \ x_n]^T$, где T — символ транспонирования. Аналогично можно записать вектор весов в виде $\mathbf{w} = [w_0 \ w_1 \ w_2 \ \dots \ w_n]^T$. При использовании векторных обозначений взвешенная сумма (6.3) запишется в виде скалярного произведения

$$s = \sum_{i=0}^n w_i x_i = (\mathbf{w}, \mathbf{x}) = \mathbf{w}^T \mathbf{x} = \|\mathbf{w}\| \|\mathbf{x}\| \cos \alpha,$$

где $(\mathbf{w}, \mathbf{x}) = \mathbf{w}^T \mathbf{x} = \|\mathbf{w}\| \|\mathbf{x}\| \cos \alpha$ — скалярное произведение векторов, $\|\mathbf{w}\| = \sqrt{w_0^2 + w_1^2 + \dots + w_n^2}$ и $\|\mathbf{x}\| = \sqrt{x_0^2 + x_1^2 + \dots + x_n^2}$ — евклидовы нормы (длины) векторов, α — угол между векторами $\mathbf{w}$ и $\mathbf{x}$.

6.2. Функции активации

Функция активации производит преобразование взвешенной суммы s . Причем, обычно производится нелинейное преобразование. Это принципиальный момент, обеспечивающий уникальные свойства нейронных сетей. Простейшей функцией активации является *линейная функция*

$$f(s) = s.$$

Линейная функция активации применяется в выходном слое нейронной сети при решении задач регрессии. В скрытых слоях линейная функция применяется в простых сетях, в которых существует линейная зависимость между входными и выходными переменными. Однако основные преимущества нейронных сетей обусловлены нелинейными характеристиками нейрона. Легко показать, что любая многослойная линейная сеть эквивалентна однослойной. А однослойные сети имеют ограниченные возможности. Поэтому наибольшее распространение получили нелинейные функции активации.

Исторически первыми в нейронных сетях стали применяться *пороговые функции*, например, *смещенная пороговая функция активации* (рис. 6.2а)

$$f(s) = \begin{cases} 0, & \text{если } s < 0, \\ 1, & \text{если } s \geq 0. \end{cases} \quad (6.5)$$

Используется также *антисимметричная пороговая функция* (рис. 6.2б)

$$f(s) = \begin{cases} -1, & \text{если } s < 0, \\ 1, & \text{если } s \geq 0. \end{cases} \quad (6.6)$$

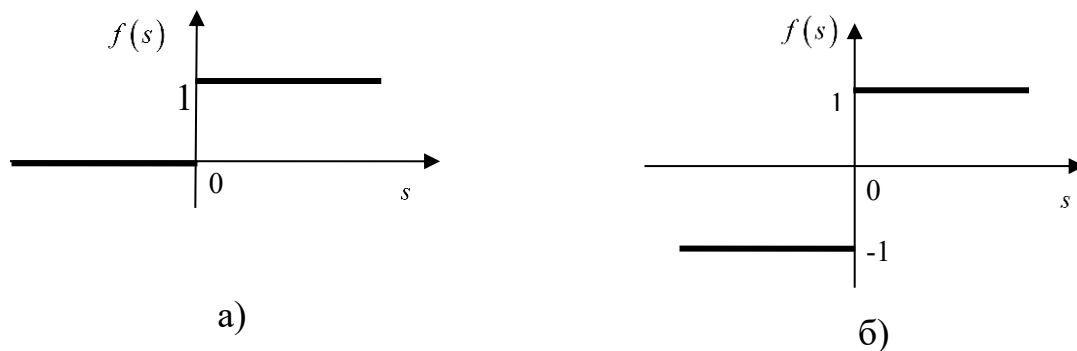


Рис. 6.2. Пороговые функции активации
а — смещенная функция; б — антисимметричная функция

Для обучения нейронных сетей обычно применяют градиентные методы обучения. Как мы увидим далее, градиентные методы требуют вычисления производных от выходов нейронов. Однако производные от пороговых функций равны нулю, кроме нулевого значения выхода, где производная не определена. Поэтому многослойную сеть со ступенчатыми функциями активации невозможно обучить градиентными методами. Пороговая функция не обеспечивает достаточной гибкости при работе сети, так как незначительное изменение взвешенной суммы приводит к срабатыванию нейрона. Этого недостатка лишена линейная функция активации, но все достоинства нейронных сетей проявляются при использовании нелинейных функций активации. Недостатки ступенчатых функций преодолены в *сигмоидальных функциях активации*.

Сигмоидальной функцией, или *сигмóидой* называется гладкая монотонно возрастающая дифференцируемая ограниченная функция. Такие функции

называют еще *S*-образными функциями из-за сходства графиков этих функций с растянутой латинской буквой *S*. Дифференцируемость сигмоидальных функций позволяет реализовать самые распространенные градиентные алгоритмы обучения сетей. Сигмоидальные функции в области малых сигналов почти линейны, что решает проблему обработки слабых сигналов, а в области больших сигналов обеспечивают резко выраженную нелинейную характеристику, близкую к пороговой.

В искусственных нейронах получили распространение несколько разновидностей сигмоидальных функций. *Логистическая* функция активации описывается выражением

$$f(s) = \frac{1}{1 + e^{-s}}, \quad (6.7)$$

антисимметричная логистическая функция (биполярная логистическая функция) описывается выражением

$$f(s) = \frac{1 - e^{-s}}{1 + e^{-s}}. \quad (6.8)$$

Широко используют в качестве функции активации *гиперболический тангенс*

$$\text{th}(s) = \frac{e^s - e^{-s}}{e^s + e^{-s}}. \quad (6.9)$$

Функция $\text{th}(s)$ совпадает с антисимметричной логистической функцией (6.8) от $2s$. В этом легко убедиться, разделив числитель и знаменатель (6.9) на e^s . Примеры функций активации приведены на рис. 6.3.

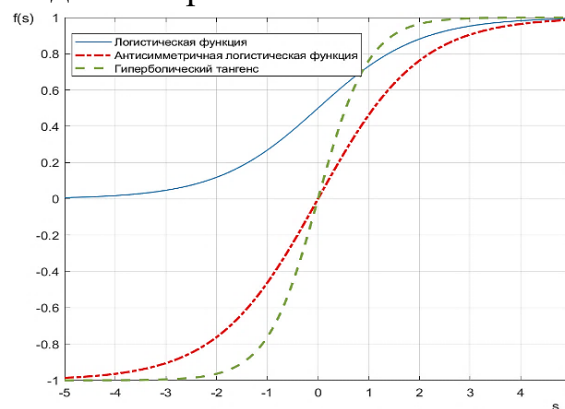


Рис. 6.3. Примеры функций активации

При решении на нейронной сети задач многоклассовой классификации для выходного слоя сети удобна *конкурирующая функция активации с мягким максимумом* — функция *softmax*

$$f(s_i) = \frac{e^{s_i}}{\sum_{j=1}^n e^{s_j}}, \quad (6.10)$$

где $f(s_i)$ — выход i -го нейрона, s_i — взвешенная сумма i -го нейрона, n — число нейронов выходного слоя.

Суммирование ведется по всем нейронам выходного слоя сети. Такая функция обеспечивает сумму выходов слоя, равную единице при любых значениях взвешенных сумм нейронов данного слоя. Это позволяет трактовать выходы сети как вероятности событий, совокупность которых образует полную группу. Кроме того, функция *softmax* является дифференцируемой, что позволяет применять градиентные методы обучения.

Недостатком сигмоидальных функций активации является насыщение функции при больших входных значениях. В градиентных алгоритмах обучения вычисляются производные df/ds . Эти производные равны:

— для логистической функции активации

$$\frac{df(s)}{ds} = \frac{e^{-s}}{(1 + e^{-s})^2},$$

— для антисимметричной логистической функции

$$\frac{df(s)}{ds} = \frac{2e^{-s}}{(1 + e^{-s})^2},$$

— для гиперболического тангенса

$$\frac{df(s)}{ds} = \frac{4}{(e^s + e^{-s})^2}.$$

Даже при относительно небольших значениях s эти производные становятся близкими к нулю (рис. 6.4) и градиентные алгоритмы обучения перестают работать (наступает "паралич" сети).

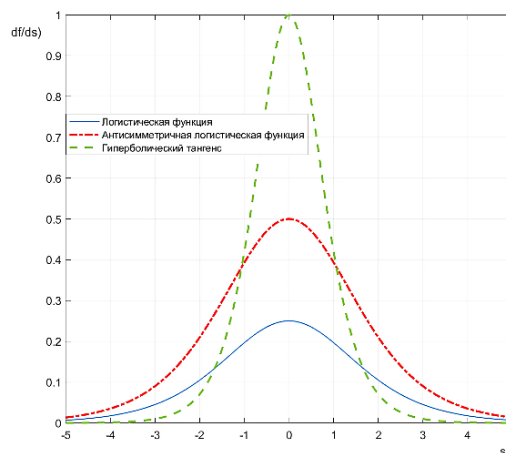


Рис. 6.4. Производные от функций активации

В сетях с малым числом слоев это явление практически не проявляется, но сильно проявляется в сетях глубокой архитектуры, т. е. содержащих большое число

слоев. Кроме того, вычисление сигмоидальных функций активации и производных от них требует многократного вычисления экспоненты, что в сетях глубокой архитектуры с большим количеством нейронов приводит к большим затратам машинного времени. В [7] показано, что проблемы с обучением сетей глубокой архитектуры обусловлены сигмоидальными функциями активации и неудачной инициализацией (заданием начальных значений) весов. Об инициализации мы поговорим, когда будем рассматривать обучение сетей, а сейчас рассмотрим функции активации. От недостатков сигмоидальных функций свободны простые функции активации *rectified linear unit (ReLU)* [7]. Общепринятого перевода этого термина на русский язык нет. Используются термины: усеченное линейное преобразование, выпрямленный линейный элемент, линейная ректификация. Это целый класс ненасыщаемых функций активации. Наибольшее распространение получила *положительная линейная функция активации*, или *жесткий максимум (max)*

$$f(s) = \max(0, s). \quad (6.11)$$

Выражение (6.11) можно записать по-другому

$$f(s) = \begin{cases} 0, & s < 0, \\ s, & s \geq 0. \end{cases}$$

По сравнению с сигмоидальными функциями функция (6.11) легко вычисляется, что очень важно для больших сетей. Производная от этой функции равна либо 1, либо 0, что исключает "паралич" сети. Но если взвешенная сумма нейрона является отрицательной, то выходной сигнал равен нулю — нейрон "умирает" [7]. Обычно такие нейроны больше не активируются. Но при этом возникает разреженность связей между скрытыми слоями сети — исчезает связь выхода этого нейрона с другими нейронами. Это упрощает вычисления. Кроме того, считается, что разреженность связей лучше соответствует сущности биологических нейронов, так как в мозге одновременно связано относительно небольшое число нейронов [8]. Появление "мертвых" нейронов регулируется настройкой скорости обучения сети.

Известна непрерывная аппроксимация функции (6.11) — "мягкая положительная" функция *softplus* [9]

$$f(s) = \ln(1 + e^s).$$

Сравнение логистической функции (6.7) и функций *max* и *softplus* показано на рис. 6.5. Функция *softplus* не получила распространения из-за бóльшей сложности вычислений, чем функция *max*.

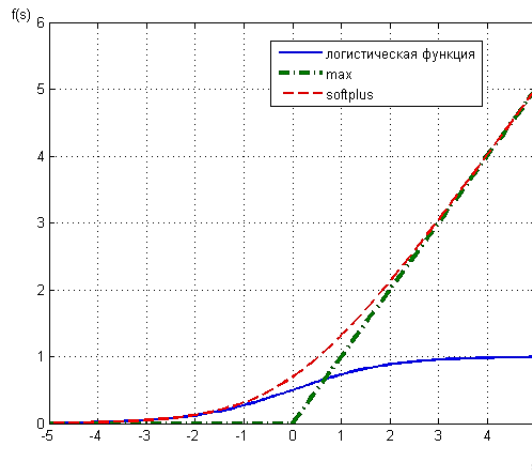


Рис. 6.5. Сравнение логистической функции с функциями *max* и *softplus*

Существует несколько модификаций функции *ReLU*, в которых решается проблема "умирания" нейронов *ReLU*. Функция *Leaky ReLU* (дословно — *ReLU* с утечкой), или *LReLU* — в отличие от *ReLU* (6.11) при $s < 0$ принимает небольшое отрицательное значение. Функция *LReLU* [10] имеет вид

$$f(s) = \begin{cases} \alpha s, & s < 0, \\ s, & s \geq 0, \end{cases}$$

где α — малая константа, зачастую $\alpha = 0.01$.

При отрицательных значениях взвешенной суммы выход нейрона будет не равен нулю, а небольшой отрицательной величиной. Часто функция *LReLU* дает лучшие результаты обучения сети, чем *ReLU* [10], но результаты не всегда стабильны. График функции показан на рис. 6.6.

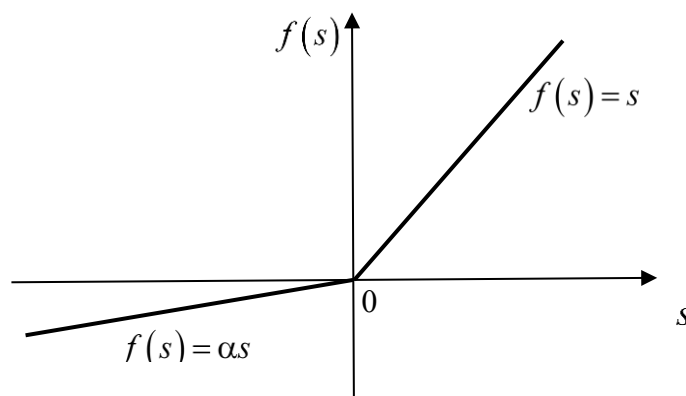


Рис. 6.6. Функция *LReLU*

В *параметрическом ReLU* (*parametric ReLU*, *PReLU*) [11] коэффициент α на отрицательном интервале не задается предварительно, а определяется для каждого нейрона с помощью алгоритма обратного распространения ошибки. В статье [11] утверждается, что применение данной функции позволило превзойти уровень человека в задаче распознавания изображений.

Для случайного (рандомизированного) *ReLU* (*randomized ReLU*, *RReLU*) коэффициент на отрицательном интервале во время обучения генерируется случайным образом, а при работе с обученной сетью остается постоянным [12]. Так в работе [12] использовался коэффициент $\alpha = 1/U(l, u)$, где $U(l, u)$ — случайное число, равномерно распределенное от l до u . При работе с обученной сетью использовался коэффициент $\alpha = 2/(l + u)$. Лучший результат был получен при $l = 3$ и $u = 8$.

В [13] предложена экспоненциальная линейная функция (*Exponential Linear Unit*, *ELU*). При положительных значениях взвешенной суммы функция *ELU* является линейной, а при отрицательных — экспоненциальной:

$$f(s) = \begin{cases} \alpha(e^s - 1), & s < 0, \\ s, & s \geq 0. \end{cases}$$

Функция имеет ненулевую производную при отрицательной взвешенной сумме, что препятствует исчезновению градиента и образованию "мертвых" нейронов. Обеспечивается насыщение при больших по абсолютной величине отрицательных значениях взвешенной суммы, что обеспечивается разреженность сети. Обычно $\alpha = 1$, но может принимать другие значения. Недостаток *ELU* — большее время вычисления, чем *ReLU*.

Функция *SELU* (*Scaled exponential linear unit*) [14] является масштабированным вариантом функции *ELU*

$$f(s) = \lambda \begin{cases} s, & s \geq 0, \\ \alpha(e^s - 1), & s < 0, \end{cases}$$

где $\alpha = 1,67326$, $\lambda = 1,0507$.

Функция *SELU* обладает эффектом самонормализации, т.е. выходы нейронов после функции активации имеют среднее значение равное 0 и стандартное отклонение равное 1. Это обеспечивает сети более быструю сходимость. При использовании этой функции активации следует использовать начальную инициализацию весов, соответствующую нормальному распределению, для обеспечения самонормализующих свойств. Самонормализация позволяет избежать взрывных и исчезающих градиентов.

Функция *GELU* (*Gaussian Error Linear Units*) [15] имеет вид

$$f(s) = s \frac{1}{2} \left[1 + \operatorname{erf} \left(\frac{s}{\sqrt{2}} \right) \right],$$

где $\operatorname{erf}(s) = \frac{2}{\sqrt{\pi}} \int_0^s e^{-t^2} dt$ — функция ошибок (функция Лапласа или интеграл Вероятности).

Функция *GELU* можно аппроксимировать более простыми выражениями

$$f(s) \approx 0,5x\left(1 + \tanh\left(\sqrt{2/\pi}\left(s + 0,044715s^3\right)\right)\right),$$

или $f(s) = s\sigma(1,702s)$, где $\sigma(s) = \frac{1}{1 + e^{-s}}$.

Основным отличием функции *GELU* от рассмотренных модификаций *ReLU* является то, что *GELU* является невыпуклой немонотонной функцией, нелинейной в положительной области и имеет кривизну во всех точках. Увеличенная кривизна и немонотонность позволяют *GELU* легче аппроксимировать сложные функции, и в отличие от *ReLU*, *GELU*. Но *GELU* сложна для вычислений.

Команда исследовательского проекта Google по изучению искусственного интеллекта (Google Brain Team) [16] предложила новую функцию активации *Swish* (свиш — всплеск) [17]

$$f(s) = \frac{s}{1 + e^{-\beta s}}.$$

Swish, по существу, это *ReLU* с небольшим отрицательным всплеском (рис. 6.7). Применение функции *Swish* в глубоких нейронных сетях повышает точность классификации, но подбор гиперпараметра β трудоемок.

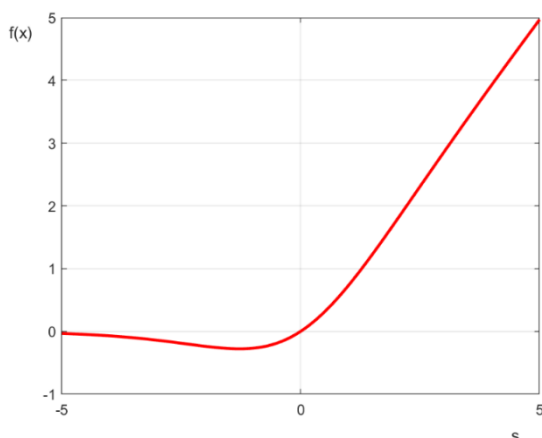


Рис. 6.7. Функция *Swish*

Идеи функции *Swish* развиты в функции *Mish* [18]. Функция *Mish* не требует задания гиперпараметров, но сложнее в вычислительном плане

$$f(s) = \text{th}(\text{softplus}(s)).$$

Вид функции *Mish* и производные в сравнении с другими функциями показаны на рис. 6.8.

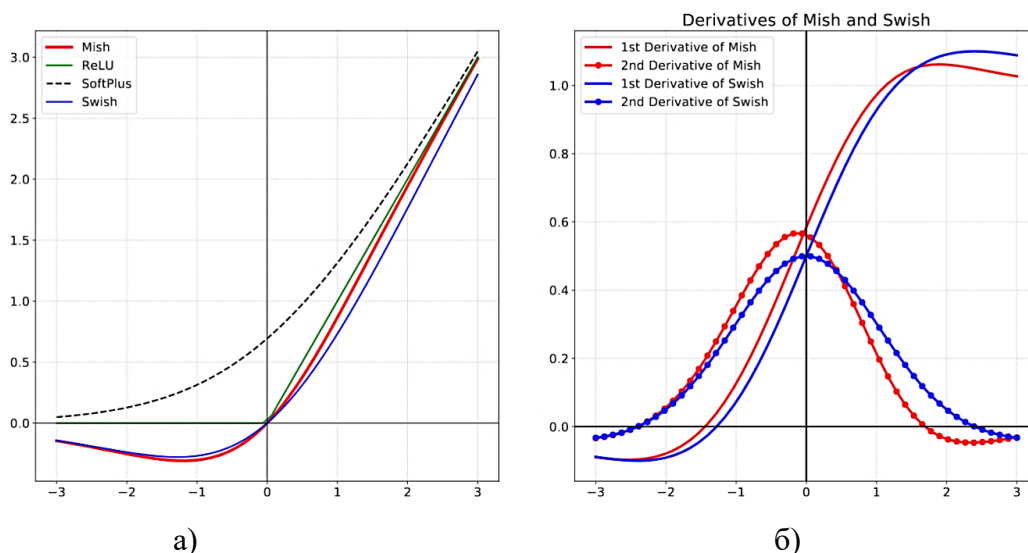


Рис. 6.8. Функция *Mish* [16]

а) графики функций активации Mish, ReLU, SoftPlus и Swish б) 1-я и 2-я производная функций активации Mish и Swish.

Мы рассмотрели далеко не все виды функций активации. Это интенсивно развиваемое направление. Причем выбор функции активации требует большой экспериментальной работы.

6.3. Инициализация нейронных сетей

На результаты обучения сети большое влияние оказывает выбор начальных значений весов сети. Казалось бы, проще всего взять нулевые начальные значения весов. Но при использовании алгоритма обратного распространения ошибки только веса выходного слоя можно инициализировать нулевыми значениями. Для других слоев компоненты вектора градиента будут равны нулю и веса не будут изменяться и останутся нулевыми. Поэтому используются случайные начальные значения весов. Обычно применяют *псевдослучайный выбор весов* с равномерным распределением значений в заданном интервале. Другой вариант — использование нормально распределенных случайных начальных значений с нулевым математическим ожиданием и единичным средним квадратическим отклонением. Однако слишком большие значения весов могут привести к *"параличу" сети*. В ситуации "паралича" выход сигмоидальной функции активации близок к насыщению. Формируемые в методе обратного распространения ошибки компоненты вектора градиента в точке насыщения близки к нулю, поэтому значения весов практически не меняются и нейрон, попавший в насыщение, остается в этом состоянии. В результате процесс обучения сети останавливается. Слишком маленькие начальные значения весов приводят к прекращению обучения сети. Поэтому выбирают такие начальные значения весов, при которых выходы адаптивных сумматоров достаточно далеки от зоны насыщения функции

активации и попадают на практически линейный участок сигмоидальной функции активации. Известны различные алгоритмы инициализации весов. Все они носят эвристический характер и основаны на экспериментальных исследованиях. Для неглубоких сетей долгое время наиболее популярным был *алгоритм инициализации весов Д. Нгуена–Б. Уидроу (D. Nguyen–B. Widrow)* [19]. В работе [19] исследовалась аппроксимация функций с помощью двухслойной нейронной сети с функциями активации гиперболический тангенс в первом слое и линейной функцией активации в выходном слое. Входные данные изменялись в диапазоне от -1 до $+1$. Рекомендации давались, исходя из предположения, что выходные сигналы нейронов первого слоя должны располагаться в линейной части функции активации. При целом ряде эвристических допущений показано, что евклидова норма вектора начальных значений весов должна удовлетворять уравнению

$$\|\mathbf{w}^{(0)}\| = \sqrt[n]{p},$$

где p — число нейронов первого слоя, n — число входов сети.

Причем практически рекомендовано использовать уравнение "с запасом"

$$\|\mathbf{w}^{(0)}\| = 0,7\sqrt[n]{p}. \quad (6.12)$$

Но уравнение (6.12) не позволяет вычислить конкретные веса. Для такого вычисления формируется вспомогательный вектор $\mathbf{w}$ равномерно распределенных в $[-1, 1]$ значений весов w_i , $i = 1, 2, \dots, n$. Вычисляется коэффициент β

$$\beta = 0,7\sqrt[n]{p},$$

начальные значения весов вычисляются по формуле

$$w_i^{(0)} = \frac{\beta w_i}{\|\mathbf{w}\|},$$

где $\|\mathbf{w}\|$ — евклидова норма вспомогательного вектора.

Смещения рекомендовано формировать как случайные числа, равномерно распределенные в $[-\|\mathbf{w}^{(0)}\|, \|\mathbf{w}^{(0)}\|]$. Экспериментально показано, что начальные значения весов линейного выходного слоя можно формировать как случайные числа, равномерно распределенные в $[-0,5, 0,5]$.

Для сетей глубокой архитектуры характерны *проблемы исчезновения градиентов и взрывного роста градиентов* [6]. Это означает, что в алгоритме обратного распространения ошибки градиенты или становятся все меньше и меньше и веса практически не изменяются или градиенты растут и веса становятся огромными и алгоритм расходится. Эти проблемы практически не позволяли обучать сети глубокой архитектуры.

В работе [20] был предложен подход к инициализации весов, сильно упростивший обучение глубоких сетей. В [20] доказано, что сложность обучения глубоких сетей обусловлена плохой инициализацией и применением сигмоидальных функций активации. В работе предложен простой способ инициализации весов глубоких сетей. Предложенный способ назван по имени Глоро: *инициализация Ксавье* (*Xavier initialization*), или *инициализация Глоро*. В [20] показано, что для исключения проблем исчезновения и взрывного роста градиента необходимо, чтобы дисперсия выходов каждого слоя была равна дисперсии его входов, и градиенты имели равную дисперсию до и после протекания через слой в обратном направлении. Гарантировать соблюдение обоих условий можно только в случае равенства количества входов и выходов слоя, что бывает редко. Для *логистической функции активации* в [20] предложены формулы для инициализации весов, являющиеся компромиссом для требования равенства дисперсий:

- инициализация весов каждого слоя сети нормально распределенными значениями с нулевым средним значением и стандартным отклонением

$$\sigma = \sqrt{\frac{2}{n_{\text{входов}} + n_{\text{выходов}}}},$$

- инициализация весов каждого слоя сети значениями равномерно распределенными между $-r$ и $+r$, где

$$r = \sqrt{\frac{6}{n_{\text{входов}} + n_{\text{выходов}}}},$$

где $n_{\text{входов}}$ и $n_{\text{выходов}}$ — количество входов и выходов слоя.

В [21] представлены параметры инициализации для различных функций активации, названные *инициализацией Хе* (He) [6] (табл. 6.1)

Таблица 6.1. Параметры инициализации для различных функций активации

Функция активации	Равномерное распределение $[-r, +r]$	Нормальное распределение
Логистическая	$r = \sqrt{\frac{6}{n_{\text{входов}} + n_{\text{выходов}}}}$	$\sigma = \sqrt{\frac{2}{n_{\text{входов}} + n_{\text{выходов}}}}$
Гиперболический тангенс	$r = 4 \sqrt{\frac{6}{n_{\text{входов}} + n_{\text{выходов}}}}$	$\sigma = 4 \sqrt{\frac{2}{n_{\text{входов}} + n_{\text{выходов}}}}$
ReLU и разновидности	$r = \sqrt{2} \sqrt{\frac{6}{n_{\text{входов}} + n_{\text{выходов}}}}$	$\sigma = \sqrt{2} \sqrt{\frac{2}{n_{\text{входов}} + n_{\text{выходов}}}}$

6.4. Структура многослойного персептрона

Рассмотрим *слой нейронов* на примере однослойной нейронной сети (рис. 6.9). Говоря о количестве слоев сети, необходимо определить, что считать первым слоем сети. Например, в Scikit-Learn первым слоем считается распределительный слой, который не производит никакой обработки, а только передает входные сигналы на обрабатывающий слой. То есть первым слоем считается входной вектор. При таком подходе сеть на рис. 6.9 является двухслойной. Обратите внимание, что внутри слоя нейроны не связаны.

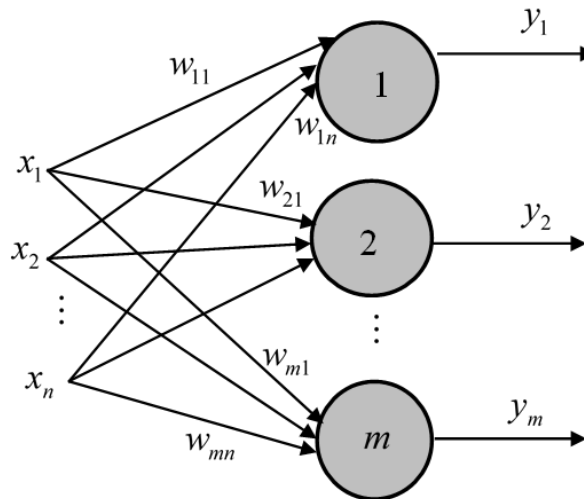


Рис. 6.9. Однослойная сеть

Выход y_i нейрона номер i формируется по формуле (6.4), где индекс i является номером нейрона, индекс j — номером входа нейрона, вес w_{ij} — это вес, соответствующий входу j нейрона i . В матричной форме работа сети описывается следующим образом

$$\mathbf{y} = \mathbf{f}(\mathbf{W}\mathbf{x}),$$

где $\mathbf{y} = [y_1 \ y_2 \ \dots \ y_m]^T$ — вектор выходов сети, $\mathbf{f} = [f_1 \ f_2 \ \dots \ f_m]^T$ — вектор-функция активации (как правило, все нейроны одного слоя имеют одинаковую функцию активации), $\mathbf{x} = [x_1 \ x_2 \ \dots \ x_n]^T$ — вектор входов сети, $\mathbf{W}$ — матрица весов, имеющая вид

$$\mathbf{W} = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1n} \\ w_{21} & w_{22} & \dots & w_{2n} \\ \dots & \dots & \dots & \dots \\ w_{m1} & w_{m2} & \dots & w_{mn} \end{bmatrix}. \quad (6.13)$$

Матрица (6.13) как правило является прямоугольной ($m \neq n$). Строки матрицы представляют собой веса соответствующих нейронов.

Структура многослойного персептрона показана на рис. 6.10. Изображенная на рис. 6.10 сеть содержит M слоев, среди которых выделяют первый слой, *промежуточные (скрытые)* слои и выходной слой. Номер слоя отображается на рисунке верхним индексом. Слои содержат, как правило, разное количество нейронов $N^{(i)}$, $i = 2, \dots, M$. Матрицы $\mathbf{W}^{(i)}$, $i = 2, \dots, M$ — это матрицы весов соответствующих слоев.

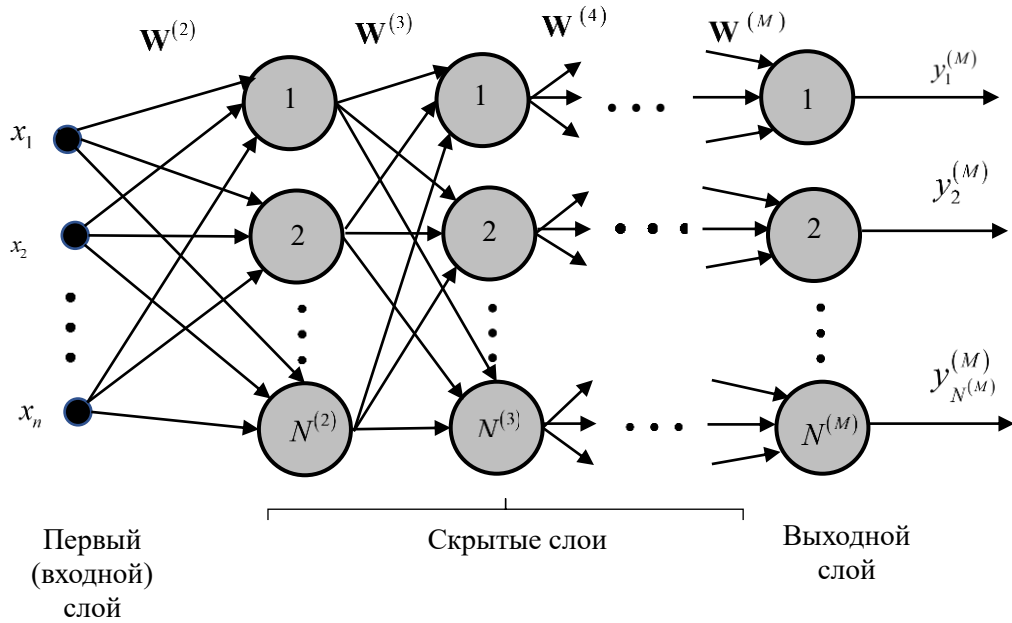


Рис. 6.10. Структура многослойного персептрона

Как видно из рис. 6.10, вектор выхода второго слоя $\mathbf{y}^{(2)} = \mathbf{f}_2(\mathbf{W}^{(2)}\mathbf{x})$ является входом третьего слоя, выход третьего слоя $\mathbf{y}^{(3)} = \mathbf{f}_3(\mathbf{W}^{(3)}\mathbf{y}^{(2)})$ является входом четвертого слоя и т. д. Выход $\mathbf{y}^{(M)}$ слоя M является выходом сети.

Глубокие, или глубинные нейронные сети (*Deep Neural Networks*) зачастую содержат слои, отличающиеся от рассмотренных слоев нейронов и выполняющие специфические функции.

Схема обработки информации в полносвязной многослойной сети, содержащей M слоев, представлена на (рис. 6.11)

$$\begin{aligned} \mathbf{x} &\rightarrow \mathbf{W}^{(2)} \xrightarrow{\mathbf{s}^{(2)}} \mathbf{f}_2(\mathbf{s}^{(2)}) \xrightarrow{\mathbf{y}^{(2)}} \dots \xrightarrow{\mathbf{y}^{(M-2)}} \mathbf{W}^{(M-1)} \xrightarrow{\mathbf{s}^{(M-1)}} \mathbf{f}_{M-1}(\mathbf{s}^{(M-1)}) \xrightarrow{\mathbf{y}^{(M-1)}} \\ &\xrightarrow{\mathbf{y}^{(M-1)}} \mathbf{W}^{(M)} \xrightarrow{\mathbf{s}^{(M)}} \mathbf{f}_M(\mathbf{s}^{(M)}) \rightarrow \mathbf{y}^{(M)} \end{aligned}$$

Рис. 6.11. Схема обработки информации в многослойной сети

На рис. 6.1. $\mathbf{x}$ — вектор входа; $\mathbf{W}^{(i)}$ — матрица весов слоя i ; $\mathbf{s}^{(i)}$ — вектор взвешенных сумм слоя i ; $\mathbf{y}^{(i)} = \mathbf{f}_i(\mathbf{s}^{(i)})$ — вектор выхода слоя i ; $\mathbf{f}_i$ — вектор-функция

активации слоя i (будем считать, что у всех нейронов одного слоя одинаковая функция активации). Отметим, что функция активации может быть различной у разных слоев, например, зачастую выходной слой имеет линейную функцию активации, а все остальные — логистическую. Первый слой называется входным. Слой M называется выходным, остальные слои — скрытыми или промежуточными. Часто используют фиктивный входной слой, число нейронов которого равно размерности входного вектора $\mathbf{x}$. В такой трактовке входной слой не производит никакой обработки, а только распределяет информацию.

С использованием обозначений рис. 6.11 вектор выходов адаптивных сумматоров нейронов первого слоя равен

$$\mathbf{s}^{(2)} = \mathbf{W}^{(2)} \mathbf{x},$$

вектор выходов нейронов второго слоя имеет вид

$$\mathbf{y}^{(2)} = \mathbf{f}_2(\mathbf{s}^{(2)}).$$

Для третьего слоя имеем

$$\mathbf{s}^{(3)} = \mathbf{W}^{(3)} \mathbf{y}^{(2)}, \quad \mathbf{y}^{(3)} = \mathbf{f}_3(\mathbf{s}^{(3)}).$$

Для произвольного слоя $i = 1, 2, \dots, M$

$$\mathbf{s}^{(i)} = \mathbf{W}^{(i)} \mathbf{y}^{(i-1)}, \quad \mathbf{y}^{(i)} = \mathbf{f}_i(\mathbf{s}^{(i)}),$$

где $\mathbf{y}^{(0)} = \mathbf{x}$, $\mathbf{y}^{(M)}$ — вектор выходов сети.

При использовании *последовательного режима обучения* настройка весов и смещений производится после подачи каждого вектора обучающей выборки, и функционал ошибки сети (функция потерь) рассчитывается по формуле

$$L = \frac{1}{2} \sum_{i=1}^{N^{(M)}} e_i^2 = \frac{1}{2} \sum_{i=1}^{N^{(M)}} \left(y_i^{(M)} - t_i \right)^2, \quad (6.14)$$

где $N^{(M)}$ — число нейронов выходного слоя, M — число слоев, $\mathbf{e} = [e_i]$ — вектор ошибок на выходе сети, $e_i = y_i^{(M)} - t_i$ — ошибка на выходе i , $\mathbf{y} = [y_i^M]$ — выходной вектор сети, $\mathbf{t} = [t_i]$ — вектор целей.

Как уже указывалось, коэффициент $1/2$ введен для упрощения последующего анализа.

Часто используется функционал

$$L = \frac{1}{N^{(M)}} \sum_{i=1}^{N^{(M)}} e_i^2.$$

В *пакетном и мини-пакетном режиме обучения* настройка весов производится после предъявления сети множества векторов, функционал ошибки имеет вид

$$L = \frac{1}{2} \sum_{q=1}^Q \sum_{i=1}^{N^{(M)}} \left(e_i^{(q)} \right)^2 = \frac{1}{2} \sum_{q=1}^Q \sum_{i=1}^{N^{(M)}} \left(y_i^{(qM)} - t_i^{(q)} \right)^2, \quad (6.15)$$

где Q — объем (количество примеров) в обучающей выборке или мини-пакете, q — номер обучающего примера, $e_i^{(q)} = y_i^{(qM)} - t_i^{(q)}$ — ошибка на выходе i при подаче на вход примера q , $y_i^{(qM)}$ — i -ый выход выходного слоя M для входного примера q , $t_i^{(q)}$ — i -е значение вектора цели для входного примера q .

При использовании в выходном слое функции активации *softmax* i -й выход сети описывается выражением

$$y_i = f(s_i) = \frac{e^{s_i}}{\sum_{j=1}^{N^{(M)}} e^{s_j}}, \quad 0 < y_i < 1. \quad (6.16)$$

В качестве функционала ошибки в этом случае используется *перекрестная энтропия*

$$L = - \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} t_k^{(q)} \ln y_k^{(qM)}, \quad (6.17)$$

где $t_k^{(q)}$ — требуемое (целевое) значение выхода k сети при подаче примера q , $y_k^{(qM)}$ — реальное значение выхода k сети при подаче примера q , n — число выходов сети, Q — число примеров в обучающей выборке.

В дальнейшем рассмотрим пакетный режим обучения, так как последовательный режим является частным случаем при объеме обучающей выборки $Q = 1$. Правило функционирования слоя m ($m = 2, \dots, M$) описывается формулой

$$y_k^{(qm)} = f_m \left(\sum_{l=0}^{N^{(m-1)}} w_{kl}^{(m)} y_l^{(q,m-1)} \right), \quad k = 1, 2, \dots, N^{(m)}, \quad m = 2, \dots, M. \quad (6.18)$$

В (6.18) учтено, что входным вектором слоя m является выходной вектор слоя $m-1$. Входом первого слоя является входной вектор, т. е. формально $y^{(0)} = x$. Будем считать, что сеть является полносвязной. Каждый нейрон слоя m имеет столько входов, сколько выходов имеет слой $m-1$, j -й выход $y_j^{(q,m-1)}$ слоя $m-1$ поступает на j -й вход каждого нейрона слоя m . Индекс $l=0$ соответствует смещению. Смещение будем рассматривать как единичный вход, соединенный весом $w_{k0}^{(m)}$. Вес w_{ij} — это вес, соответствующий входу j нейрона i .

6.5. Обучение многослойного персептрона

6.5.1. Алгоритм обратного распространения ошибки

При обучении многослойных сетей чаще всего используют *градиентные методы обучения*, требующие вычисления производных функционала ошибки по параметрам сети. Например, в алгоритме градиентного спуска, который будет рассмотрен позже, элементы матрицы весов w_{ij} корректируются по формуле

$$w_{ij}^{(k)} = w_{ij}^{(k-1)} - \eta \frac{\partial L^{(k-1)}}{\partial w_{ij}^{(k-1)}}, \quad (6.19)$$

где k — номер цикла обучения, η — числовой коэффициент (скорость обучения),

E — функционал ошибки (6.15), $\frac{\partial L^{(k-1)}}{\partial w_{ij}^{(k-1)}}$ — компонент градиента функционала

ошибки по вектору весов.

Но функционал ошибки и его производные по весам можно вычислить только для выходного слоя. *Алгоритм обратного распространения ошибки*, правильнее называть "алгоритм обратного распространения градиента" (*Error Back Propagation*) [1–3] позволяет вычислить эти производные для всех слоев сети, начиная с выходного слоя. Алгоритм обратного распространения ошибки используется в сочетании с различными стратегиями оптимизации, что приводит к различным методам обучения сетей.

Рассмотрим "ручное" вычисление производных при использовании *в выходном слое одной из сигмоидальных функций активации*. В настоящее время при применении специальных библиотек реализации нейронных сетей используется автоматическое дифференцирование, которое будет рассмотрено ниже, и пользователь избавлен от вычисления производных.

Выражение для производной функционала ошибки (6.15) по весам для выходного слоя M имеет вид

$$\begin{aligned} \frac{\partial L}{\partial w_{ij}^{(M)}} &= \frac{\partial}{\partial w_{ij}^{(M)}} \left(\frac{1}{2} \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \left(e_k^{(q)} \right)^2 \right) = \frac{\partial}{\partial w_{ij}^{(M)}} \left(\frac{1}{2} \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \left(y_k^{(qM)} - t_k^{(q)} \right)^2 \right) = \\ &= \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \left(y_k^{(qM)} - t_k^{(q)} \right) \frac{\partial y_k^{(qM)}}{\partial w_{ij}^{(M)}}, \end{aligned} \quad (6.20)$$

где w_{ij} — вес, связывающий нейрон i с входом j ; $i = \overline{1, N^{(M)}}$ — номер (индекс) нейрона в слое M ; $j = \overline{0, N^{(M-1)}}$ — номер входа нейрона i в слое M (входами нейрона слоя M являются выходы нейронов слоя $M-1$); $N^{(M-1)}$ — число нейронов

в слое $M-1$; $y_k^{(qM)}$ — k -й элемент вектора выхода слоя M для входного вектора примера с номером q ; $t_k^{(q)}$ — k -й элемент целевого вектора примера с номером q .

Вычислим производную $\partial y_k^{(qM)} / \partial w_{ij}^{(M)}$. Применим цепное правило дифференцирования сложных функций, учтем дифференцируемость функции активации и правило функционирования нейрона (формула (6.18) при $m = M$)

$$\frac{\partial y_k^{(qM)}}{\partial w_{ij}^{(M)}} = \frac{\partial y_k^{(qM)}}{\partial s_i^{(qM)}} \cdot \frac{\partial s_i^{(qM)}}{\partial w_{ij}^{(M)}} = \frac{\partial y_k^{(qM)}}{\partial s_i^{(qM)}} \cdot y_j^{(q, M-1)}, \quad (6.21)$$

причем

$$\frac{\partial y_k^{(qM)}}{\partial s_i^{(qM)}} = \begin{cases} 0, & \text{если } k \neq i, \\ \frac{\partial f_M(s_i^{(qM)})}{\partial s_i^{(qM)}}, & \text{если } k = i, \end{cases} \quad (6.22)$$

где $s_i^{(qM)}$ — взвешенная сумма нейрона i слоя M для вектора примера q .

Выражение (6.22) означает, что в сумме по индексу k в (6.20) остается только одно слагаемое. Подставляя (6.21) и (6.22) в (6.20), получаем

$$\frac{\partial L}{\partial w_{ij}^{(M)}} = \sum_{q=1}^Q \left(y_i^{(qM)} - t_i^{(q)} \right) \frac{\partial f_M(s_i^{(qM)})}{\partial s_i^{(qM)}} y_j^{(q, M-1)}. \quad (6.23)$$

Обозначим

$$\delta_i^{(qM)} = \left(y_i^{(qM)} - t_i^{(q)} \right) \cdot \frac{\partial f_M(s_i^{(qM)})}{\partial s_i^{(qM)}}. \quad (6.24)$$

В [2] выражение (6.24) названо *локальным градиентом (Local Gradient)*. Локальный градиент нейрона выходного слоя равен произведению ошибки выхода этого нейрона $(y_i^{(qM)} - t_i^{(q)})$ на производную функции активации этого нейрона.

Локальный градиент нейрона выходного слоя равен компоненту градиента функционала ошибки по соответствующей взвешенной сумме:

$$\delta_i^{(qM)} = \frac{\partial E}{\partial s_i^{(qM)}} = \frac{\partial E}{\partial y_i^{(qM)}} \cdot \frac{\partial y_i^{(qM)}}{\partial s_i^{(qM)}} = \left(y_i^{(qM)} - t_i^{(q)} \right) \cdot \frac{\partial f_M(s_i^{(qM)})}{\partial s_i^{(qM)}}.$$

Окончательно получаем

$$\frac{\partial L}{\partial w_{ij}^{(M)}} = \sum_{q=1}^Q \delta_i^{(qM)} y_j^{(q, M-1)}, \quad i = \overline{1, N^{(M)}}, \quad j = \overline{0, N^{(M-1)}}. \quad (6.25)$$

Соотношения для производной функционала ошибки по весам скрытого слоя выведем на примере скрытого слоя $M-1$

$$\begin{aligned}
\frac{\partial L}{\partial w_{ij}^{(M-1)}} &= \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \left(y_k^{(qM)} - t_k^{(q)} \right) \cdot \frac{\partial y_k^{(qM)}}{\partial w_{ij}^{(M-1)}} = \\
&= \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \left(y_k^{(qM)} - t_k^{(q)} \right) \cdot \frac{\partial f_M \left(s_k^{(qM)} \right)}{\partial s_k^{(qM)}} \cdot \frac{\partial s_k^{(qM)}}{\partial y_i^{(q,M-1)}} \cdot \frac{\partial y_i^{(q,M-1)}}{\partial s_i^{(q,M-1)}} \cdot \frac{\partial s_i^{(q,M-1)}}{\partial w_{ij}^{(M-1)}} = \\
&= \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \left(y_k^{(qM)} - t_k^{(q)} \right) \cdot \frac{\partial f_M \left(s_k^{(qM)} \right)}{\partial s_k^{(qM)}} \cdot w_{ki}^{(M)} \cdot \frac{\partial f_{M-1} \left(s_i^{(q,M-1)} \right)}{\partial s_i^{(q,M-1)}} \cdot y_j^{(q,M-2)} = \\
&= \sum_{q=1}^Q \delta_i^{(q,M-1)} y_j^{(q,M-2)}. \tag{6.26}
\end{aligned}$$

В (6.26) при вычислении производных сохранилась сумма по k от 1 до $N^{(M)}$. Локальный градиент нейрона i слоя $M-1$ для q -го обучающего примера равен

$$\begin{aligned}
\delta_i^{(q,M-1)} &= \sum_{k=1}^{N^{(M)}} \left(y_k^{(qM)} - t_k^{(q)} \right) \cdot \frac{\partial f_M \left(s_k^{(qM)} \right)}{\partial s_k^{(qM)}} \cdot w_{ki}^{(M)} \cdot \frac{\partial f_{M-1} \left(s_i^{(q,M-1)} \right)}{\partial s_i^{(q,M-1)}} = \\
&= \left(\sum_{k=1}^{N^{(M)}} \delta_k^{(qM)} w_{ki}^{(M)} \right) \cdot \frac{\partial f_{M-1} \left(s_i^{(q,M-1)} \right)}{\partial s_i^{(q,M-1)}}, \quad i = \overline{1, N^{(M-1)}}. \tag{6.27}
\end{aligned}$$

Для скрытого слоя $M-2$ имеем

$$\frac{\partial L}{\partial w_{ij}^{(M-2)}} = \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \left(y_k^{(qM)} - t_k^{(q)} \right) \frac{\partial y_k^{(qM)}}{\partial w_{ij}^{(M-2)}}.$$

Для вычисления $\frac{\partial y_k^{(qM)}}{\partial w_{ij}^{(M-2)}}$ применим правило вычисления полных производных:

если $y = f(v(t), w(t))$, то полная производная равна

$$\frac{\partial y}{\partial t} = \frac{\partial f}{\partial v} \cdot \frac{\partial v}{\partial t} + \frac{\partial f}{\partial w} \cdot \frac{\partial w}{\partial t}.$$

Тогда можно записать

$$\begin{aligned}
\frac{\partial y_k^{(qM)}}{\partial w_{ij}^{(M-2)}} &= \sum_{p=1}^{N^{(M-1)}} \frac{\partial y_k^{(qM)}}{\partial s_p^{(q,M-1)}} \cdot \frac{\partial s_p^{(q,M-1)}}{\partial w_{ij}^{(M-2)}}, \\
\frac{\partial L}{\partial w_{ij}^{(M-2)}} &= \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \left(y_k^{(qM)} - t_k^{(q)} \right) \frac{\partial y_k^{(qM)}}{\partial w_{ij}^{(M-2)}} = \\
&= \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \left(y_k^{(qM)} - t_k^{(q)} \right) \cdot \sum_{p=1}^{N^{(M-1)}} \frac{\partial y_k^{(qM)}}{\partial s_p^{(q,M-1)}} \cdot \frac{\partial s_p^{(q,M-1)}}{\partial w_{ij}^{(M-2)}} =
\end{aligned}$$

$$\begin{aligned}
&= \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \left(y_k^{(qM)} - t_k^{(q)} \right) \sum_{p=1}^{N^{(M-1)}} \frac{\partial y_k^{(qM)}}{\partial s_k^{(qM)}} \cdot \frac{\partial s_k^{(qM)}}{\partial y_p^{(q,M-1)}} \cdot \frac{\partial y_p^{(q,M-1)}}{\partial s_p^{(q,M-1)}} \times \\
&\quad \times \frac{\partial s_p^{(q,M-1)}}{\partial y_i^{(q,M-2)}} \cdot \frac{\partial y_i^{(q,M-2)}}{\partial s_i^{(q,M-2)}} \cdot \frac{\partial s_i^{(q,M-2)}}{\partial w_{ij}^{(M-2)}} = \\
&= \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \left(y_k^{(qM)} - t_k^{(q)} \right) \sum_{p=1}^{N^{(M-1)}} \frac{\partial f_M(s_k^{(qM)})}{\partial s_k^{(qM)}} \cdot w_{kp}^{(M)} \cdot \frac{\partial f_{M-1}(s_p^{(q,M-1)})}{\partial s_p^{(q,M-1)}} \times \\
&\quad \times w_{pi}^{(M-1)} \cdot \frac{\partial f_{M-2}(s_i^{(q,M-2)})}{\partial s_i^{(q,M-2)}} \cdot y_j^{(q,M-3)}.
\end{aligned}$$

В полученном выражении изменим порядок суммирования

$$\begin{aligned}
\frac{\partial L}{\partial w_{ij}^{(M-2)}} &= \sum_{q=1}^Q \sum_{k=1}^{N^{(M)}} \sum_{p=1}^{N^{(M-1)}} \left(y_k^{(qM)} - t_k^{(q)} \right) \frac{\partial f_M(s_k^{(qM)})}{\partial s_k^{(qM)}} \cdot w_{kp}^{(M)} \cdot \frac{\partial f_{M-1}(s_p^{(q,M-1)})}{\partial s_p^{(q,M-1)}} \times \\
&\quad \times w_{pi}^{(q,M-1)} \cdot \frac{\partial f_{M-2}(s_i^{(q,M-2)})}{\partial s_i^{(q,M-2)}} \cdot y_j^{(q,M-3)} = \\
&= \sum_{q=1}^Q \sum_{p=1}^{N^{(M-1)}} \left(\sum_{k=1}^{N^{(M)}} \delta_k^{(qM)} \cdot w_{kp}^{(M)} \right) \cdot \frac{\partial f_{M-1}(s_p^{(q,M-1)})}{\partial s_p^{(q,M-1)}} \cdot w_{pi}^{(M-1)} \cdot \frac{\partial f_{M-2}(s_i^{(q,M-2)})}{\partial s_i^{(q,M-2)}} \cdot y_j^{(q,M-3)} = \\
&= \sum_{q=1}^Q \left(\sum_{p=1}^{N^{(M-1)}} \delta_p^{(q,M-1)} \cdot w_{pi}^{(q,M-1)} \right) \cdot \frac{\partial f_{M-2}(s_i^{(q,M-2)})}{\partial s_i^{(q,M-2)}} \cdot y_j^{(q,M-3)} = \sum_{q=1}^Q \delta_i^{(q,M-2)} \cdot y_j^{(q,M-3)},
\end{aligned}$$

где

$$\delta_i^{(q,M-2)} = \left(\sum_{p=1}^{N^{(M-1)}} \delta_p^{(q,M-1)} \cdot w_{pi}^{(q,M-1)} \right) \cdot \frac{\partial f_{M-2}(s_i^{(q,M-2)})}{\partial s_i^{(q,M-2)}}.$$

В общем виде для слоя номер m получаем

$$\frac{\partial L}{\partial w_{ij}^{(m)}} = \sum_{q=1}^Q \delta_i^{(qm)} y_j^{(q,m-1)}, \quad m = \overline{2, M}, \quad i = \overline{1, N^{(m)}}, \quad j = \overline{0, N^{(m-1)}}, \quad (6.28)$$

где для второго слоя $y_j^{(q,0)}$ — j -й вход сети для примера q .

Локальные градиенты вычисляются по формуле

$$\delta_i^{(qm)} = \left(\sum_{p=1}^{N^{(m+1)}} \delta_p^{(q,m+1)} w_{pi}^{(m+1)} \right) \cdot \frac{\partial f_m(s_i^{(qm)})}{\partial s_i^{(qm)}}, \quad m = \overline{1, M-1}. \quad (6.29)$$

Из (6.28) видно, что компоненты вектора градиента вычисляются как сумма по всем примерам произведений локального градиента нейрона на его входные

сигналы, которыми для всех слоев кроме первого являются выходы предыдущего слоя. Для выходного слоя локальные градиенты вычисляются по формуле (6.24). Локальный градиент равен произведению производной от функции активации на сумму локальных градиентов нейронов следующего слоя, умноженных на веса, связывающие данный нейрон с нейроном следующего слоя (это веса следующего слоя).

В случае последовательного режима обучения функционал ошибки вычисляется по формуле (6.14), т. е. веса настраиваются после подачи каждого входного вектора, и в (6.23) и (6.28) надо положить $Q = 1$.

При использовании градиентных алгоритмов обучения применяются дифференцируемые функции активации, позволяющие вычислять производные вида $\frac{\partial f(s_i)}{\partial s_i}$.

Нейронную сеть удобно рассматривать как *вычислительный граф* [22]. Вершины (узлы) графа соответствуют нейронам, веса дуг равны весам нейронной сети. Когда программа, реализующая нейронную сеть, двигаясь по графу сети от входа к выходу, вычисляет взвешенные суммы и выходы всех нейронов. Такие вычисления называются *проходом вперед* (прямым проходом). Частные производные функционала ошибки по весам сети (компоненты градиента функционала ошибки) вычисляются в *проходе назад* (обратном проходе): вычисления начинаются с выходного слоя и происходит движение по графу сети от выходного слоя к входному. Обратный проход основан на правиле дифференцирования сложных функций и является частным случаем *автоматического дифференцирования* — вычисления компьютерной программой численных значений производных сложных функций, используя правило дифференцирования сложных функций и численные значения производных, полученные в узлах графа [23].

Вычисление локального градиента (6.29) можно записать в матричной форме. Так как веса $w_{pi}^{(m+1)}$ являются элементами i -го столбца матрицы весов $\mathbf{W}^{(m+1)}$, то вектор локальных градиентов слоя m запишем в виде

$$\delta^{(qm)} = \left[\mathbf{W}^{(m+1)} \right]^T \delta^{(q,m+1)} \odot \left[\frac{\partial f_m(s_i^{(qm)})}{\partial s_i^{(qm)}} \right],$$

где $\delta^{(qm)}$ и $\delta^{(q,m+1)}$ — векторы локальных градиентов, $\top$ — символ транспонирования, $\odot$ — поэлементное умножение, $\left[\frac{\partial f_m(s_i^{(qm)})}{\partial s_i^{(qm)}} \right]$ — вектор из

элементов $\frac{\partial f_m(s_i^{(qm)})}{\partial s_i^{(qm)}}$.

Представление нейронной сети в виде вычислительного графа и автоматическое дифференцирование используются в современных пакетах программ реализации нейронных сетей, например, TensorFlow [6].

Аналогично производятся вычисления при использовании в выходном слое функции *softmax* (6.16) и перекрестной энтропии в качестве функционала ошибок (6.17).

Для выходного слоя M получаем

$$\begin{aligned} \frac{\partial L}{\partial w_{ij}^{(M)}} &= \frac{\partial}{\partial w_{ij}^{(M)}} \left(- \sum_{q=1}^Q \sum_{k=1}^n t_k^{(q)} \ln y_k^{(qM)} \right) = - \sum_{q=1}^Q \sum_{k=1}^n t_k^{(q)} \frac{\partial \ln y_k^{(qM)}}{\partial w_{ij}^{(M)}} = \\ &= - \sum_{q=1}^Q \sum_{k=1}^n t_k^{(q)} \frac{\partial \ln y_k^{(qM)}}{\partial s_i^{(qM)}} \cdot \frac{\partial s_i^{(qM)}}{\partial w_{ij}^{(M)}}. \end{aligned} \quad (6.30)$$

$$\frac{\partial \ln y_k^{(qM)}}{\partial s_i^{(qM)}} = \begin{cases} 0, & \text{если } k \neq i, \\ \frac{\partial \ln y_i^{(qM)}}{\partial y_i^{(qM)}} \cdot \frac{\partial y_i^{(qM)}}{\partial s_i^{(qM)}} = \frac{1}{y_i^{(qM)}} \cdot \frac{\partial f_M(s_i^{(qM)})}{\partial s_i^{(qM)}}, & \text{если } k = i. \end{cases} \quad (6.31)$$

Выражение (6.31) означает, что в сумме по индексу k в (6.30) остается только одно слагаемое. Подставляя (6.31) в (6.30) и учитывая, что $\frac{\partial s_i^{(qM)}}{\partial w_{ij}^{(M)}} = y_j^{(q,M-1)}$, получаем

$$\frac{\partial L}{\partial w_{ij}^{(M)}} = - \sum_{q=1}^Q \frac{t_i^{(q)}}{y_i^{(qM)}} \cdot \frac{\partial f_M(s_i^{(qM)})}{\partial s_i^{(qM)}} \cdot y_j^{(q,M-1)}.$$

Обозначив локальный градиент

$$\delta_i^{(qM)} = - \frac{t_i^{(q)}}{y_i^{(qM)}} \cdot \frac{\partial f_M(s_i^{(qM)})}{\partial s_i^{(qM)}}, \quad (6.32)$$

окончательно получаем

$$\frac{\partial L}{\partial w_{ij}^{(M)}} = \sum_{q=1}^Q \delta_i^{(qM)} y_j^{(q,M-1)}, \quad i = \overline{1, N^{(M)}}, \quad j = \overline{0, N^{(M-1)}}.$$

Для слоя $M - 1$ получаем

$$\begin{aligned}
\frac{\partial L}{\partial w_{ij}^{(M-1)}} &= \frac{\partial}{\partial w_{ij}^{(M-1)}} \left(- \sum_{q=1}^Q \sum_{k=1}^n t_k^{(q)} \ln y_k^{(qM)} \right) = - \sum_{q=1}^Q \sum_{i=1}^n t_k^{(q)} \frac{\partial \ln y_k^{(qM)}}{\partial w_{ij}^{(M)}} = \\
&= - \sum_{q=1}^Q \sum_{k=1}^n t_k^{(q)} \frac{\partial \ln y_k^{(qM)}}{\partial y_k^{(qM)}} \cdot \frac{\partial y_k^{(qM)}}{\partial s_k^{(qM)}} \cdot \frac{\partial s_k^{(qM)}}{\partial y_i^{(q,M-1)}} \cdot \frac{\partial y_i^{(q,M-1)}}{\partial s_i^{(q,M-1)}} \cdot \frac{\partial s_i^{(q,M-1)}}{\partial w_{ij}^{(M)}} = \\
&= - \sum_{q=1}^Q \sum_{k=1}^n \frac{t_k^{(q)}}{y_k^{(qM)}} \frac{\partial f_M(s_k^{(qM)})}{\partial s_k^{(qM)}} \cdot w_{k+i}^{(M)} \cdot \frac{\partial f_{M-1}(s_k^{(q,M-1)})}{\partial s_i^{(q,M-1)}} \cdot y_j^{(q,M-2)} = \\
&= \sum_{q=1}^Q \delta_i^{(q,M-1)} y_j^{(q,M-2)},
\end{aligned}$$

где

$$\begin{aligned}
\delta_i^{(q,M-1)} &= - \sum_{k=1}^{N^{(M)}} \frac{t_k^{(q)}}{y_k^{(qM)}} \frac{\partial f_M(s_k^{(qM)})}{\partial s_k^{(qM)}} \cdot w_{ki}^{(M)} \cdot \frac{\partial f_{M-1}(s_k^{(q,M-1)})}{\partial s_i^{(q,M-1)}} = \\
&= \left(\sum_{k=1}^{N^{(M)}} \delta_k^{(qM)} w_{ki}^{(M)} \right) \cdot \frac{\partial f_{M-1}(s_k^{(q,M-1)})}{\partial s_i^{(q,M-1)}}, \quad i = \overline{1, N^{(M-1)}}.
\end{aligned}$$

Аналогичные результаты получаются для других слоев сети. Сравнивая полученные формулы с формулами для функционала ошибки (6.15), видим, что отличие заключается только в формуле локального градиента для выходного слоя (6.32).

Производная $\frac{\partial f_M(s_i^{(qM)})}{\partial s_i^{(qM)}}$ для функции активации *softmax* легко вычисляется

$$\begin{aligned}
\frac{\partial f_M(s_i^{(qM)})}{\partial s_i^{(qM)}} &= \frac{\partial}{\partial s_i^{(qM)}} \left(\frac{e^{s_i^{(qM)}}}{\sum_{j=1}^{N^{(M)}} e^{s_j^{(qM)}}} \right) = \frac{\frac{\partial e^{s_i^{(qM)}}}{\partial s_i^{(qM)}} \sum_{j=1}^{N^{(M)}} e^{s_j^{(qM)}} - e^{s_i^{(qM)}} \frac{\partial}{\partial s_i^{(qM)}} \left(\sum_{j=1}^{N^{(M)}} e^{s_j^{(qM)}} \right)}{\left(\sum_{j=1}^{N^{(M)}} e^{s_j^{(qM)}} \right)^2} = \\
&= \frac{e^{s_i^{(qM)}} \sum_{j=1}^{N^{(M)}} e^{s_j^{(qM)}} - \left(e^{s_i^{(qM)}} \right)^2}{\left(\sum_{j=1}^{N^{(M)}} e^{s_j^{(qM)}} \right)^2} = \frac{e^{s_i^{(qM)}}}{\sum_{j=1}^{N^{(M)}} e^{s_j^{(qM)}}} \left(1 - \frac{e^{s_i^{(qM)}}}{\sum_{j=1}^{N^{(M)}} e^{s_j^{(qM)}}} \right) = y_i^{(qM)} (1 - y_i^{(qM)}).
\end{aligned}$$

При использовании метода обратного распространения ошибки обучение сети производится по следующему алгоритму.

1. *Инициализация весов.* Весам сети присваиваются малые случайные значения.
2. *Проход вперед.* На вход сети подается входной вектор (или несколько входных векторов в пакетном режиме обучения), рассчитываются выходные векторы

скрытых слоев и выходного слоя. Рассчитывается функционал ошибки L . Если $L > \varepsilon$, где ε – заданная величина, то на шаг 3, иначе на конец алгоритма (шаг 5).

3. *Проход назад*. Двигаясь от выходного слоя, рассчитываются производные вида $\frac{\partial L}{\partial w_{ij}}$ (компоненты вектора градиента функционала ошибки по весам и

смещениям сети) и по формуле (6.19) корректируются веса сети.

4. На шаг 2.

5. Конец алгоритма.

Обучение сети с использованием алгоритма обратного распространения ошибки обладает недостатками, свойственными градиентным алгоритмам. Целью обучения сети является достижение глобального минимума функционала ошибки. При обучении градиентным алгоритмом достигается ближайший к начальному состоянию сети *локальный минимум*. Существует также проблема "паралича" нейронной сети.

6.5.2. Представление данных нейронных сетей в форме тензоров

Все современные библиотеки, реализующие нейронные сети, в качестве структуры данных используют *тензоры*. Это отразилось даже в названии популярной библиотеки TensorFlow (<https://www.tensorflow.org/?hl=ru>) — дословно: "Тензорный поток". Тензоры — это многомерные массивы с единым типом. Тензоры — это обобщение понятия матриц с произвольным количеством измерений: матрицы являются двумерными тензорами. В терминологии тензоров измерения часто называют *осями*. Количество осей тензора называется его *рангом*. Скаляр является тензором нулевого ранга. Векторы являются тензорами первого ранга. Матрицы являются тензорами второго ранга. Если упаковать матрицы в массив, то получится тензор третьего ранга (рис. 6.12).

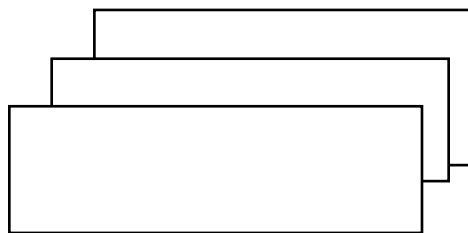


Рис. 6.12. Пример тензора третьего ранга

Упаковав трехмерные тензоры в массив, получим тензор четвертого ранга.

Тензор определяется тремя ключевыми атрибутами [24]:

- *количество осей (ранг)* — например, трехмерный тензор имеет три оси, а матрица — две;

- *форма* — кортеж целых чисел, описывающих количество измерений на каждой оси тензора. Вектор имеет форму с единственным элементом, например (5). Форма матрицы — это количество строк и столбцов;
- *тип данных* — это тип данных, содержащихся в тензоре.

Первая ось (с индексом 0, потому что нумерация начинается с 0) в тензорах называется *осью образцов* (иногда ее называют *измерением образцов*). Образцы — это примеры, на которых обучается модель машинного обучения. Например, образцы — это векторы признаков обучающих примеров, изображения, на которых обучается нейронная сеть.

Данные, которые используются в машинном обучении, обычно относятся к одной из таких категорий [24]:

Векторные данные — двумерные тензоры с формой (образцы, признаки), где каждый образец — это вектор числовых атрибутов ("признаков"), характеризующих примеры, на которых обучается модель.

Временные ряды или последовательности — трехмерные тензоры с формой (образцы, метки времени, признаки), где каждый образец для каждой метки времени содержит признаки (рис. 6.13). Например, для прогноза курса акций для некоторых моментов времени (метод времени) хранятся признаки: текущая цена акций, наибольшая и наименьшая цена акций за предыдущую метку времени.

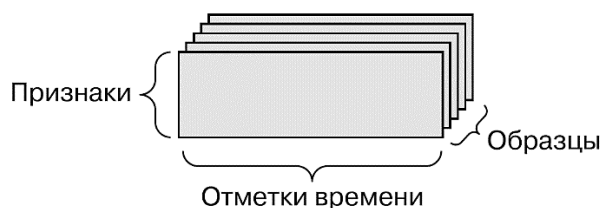


Рис. 2.13. Пример трехмерного тензора с временным рядом [7]

Изображения — четырехмерные тензоры с формой (образцы, высота, ширина, цвет), где каждый образец является двумерной матрицей пикселей, а каждый пиксель представлен вектором со значениями интенсивности цвета (рис. 6.14).

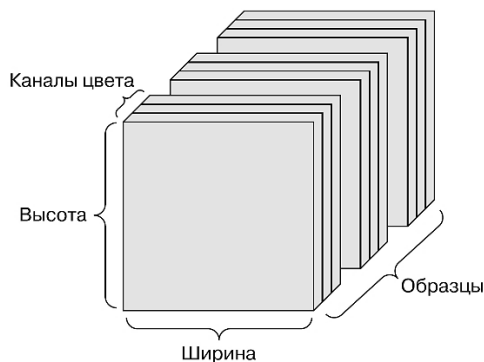


Рис. 6.14. Пример четырехмерного тензора с изображениями

Например, 128 черно-белых изображений, имеющих размер 256x256 пикселей, можно сохранить в тензоре с формой (128, 256, 256, 1), а 128 цветных изображений — в тензоре с формой (128, 256, 256, 3). В отношении форм тензоров с изображениями существует два соглашения [24]: соглашение "канал следует последним" (используется в TensorFlow) и соглашение "канал следует первым" (которое все больше теряет популярность). По соглашению "канал следует последним" ось цвета помещается в конец: (образцы, высота, ширина, цвет). По соглашению "канал следует первым" ось цвета помещается сразу после оси пакетов: (образцы, цвет, высота, ширина). При следовании соглашению "канал следует первым" примеры выглядели бы так: (128, 1, 256, 256) и (128, 3, 256, 256).

Видео — пятимерные тензоры с формой (образцы, кадры, высота, ширина, цвет), где каждый образец является последовательностью (длина равна значению кадры) изображений. Видео представляется как последовательность кадров, где каждый кадр — цветное изображение. Каждый кадр хранится в трехмерном тензоре (высота, ширина, цвет), соответственно, их последовательность можно поместить в четырехмерном тензоре (кадры, высота, ширина, цвет), а пакет разных видеороликов — в пятимерном тензоре с формой: образцы, кадры, высота, ширина, цвет.

Операции с тензорами являются обобщением операций с матрицами. Существуют поэлементные операции с тензорами: сложение, вычитание, поэлементное умножение и деление. Широко используется операция скалярного произведения, обычно применяемая к векторам и матрицам. Скалярное произведение применяют и при работе с матрицами: элементы матрицы, являющейся произведением двух матриц, это скалярные произведения строк первой матрицы и соответствующих столбцов второй матрицы. Например, произведение матрицы весов слоя нейронной сети на входной вектор слоя — это скалярное произведение тензора матрицы весов на тензор входного вектора.

6.5.3. Понятие автоматического дифференцирования

В алгоритме обратного распространения ошибки необходимо вычислять локальные градиенты, что требует многократного дифференцирования функций. Для реальных размеров нейронных сетей аналитическое вычисление производных практически нереализуемо. Казалось бы, что можно применить приближенное численное дифференцирование, например

$$\frac{\partial L}{\partial w_{ij}} \approx \frac{L(w_{ij} + \Delta w_{ij}) - L(w_{ij})}{\Delta w_{ij}},$$

где $\Delta w_{ij}^{(M)}$ — небольшое изменение веса.

Такое вычисление производных вносит погрешность, но самое главное — требует многократного вычисления выхода сети и функционала ошибки при изменении каждого веса, что нереализуемо для глубоких нейронных сетей.

Поэтому в современных библиотеках машинного обучения получило распространение автоматическое (алгоритмическое) дифференцирование [6, 23–24]. Известно несколько подходов к автоматическому дифференцированию. Рассмотрим применяемое в современных библиотеках программ *автоматическое дифференцирование в обратном режиме*, следуя [6].

Автоматическое дифференцирование основано на цепном правиле дифференцирования сложных функций и графах вычислений. Вспомним *цепное правило дифференцирования*. Пусть имеются дифференцируемые функции $f_1 = g(x)$ и $f_2 = h(f_1)$, тогда $\frac{df_2}{dx} = \frac{df_2}{df_1} \cdot \frac{df_1}{dx}$.

Граф вычислений — ориентированный граф, узлы которого соответствуют операциям или переменным. Переменные могут передавать свое значение в операции, а операции могут передавать свои результаты в другие операции. Таким образом, каждый узел в графе определяет функцию переменных. Автоматическое дифференцирование вычисляет *численные значения* производных для заданных значений аргументов.

Рассмотрим простейший пример [6]. Пусть необходимо вычислить производные от выражения

$$f(x, y) = yx^2 + y + 2.$$

Автоматическое дифференцирование в обратном сначала проходит через граф в прямом направлении (т.е. от входов к выходам), чтобы рассчитать значение каждого узла. Затем автоматическое дифференцирование в обратном режиме осуществляет второй проход, но в противоположном направлении (т.е. от выходов к входам), чтобы вычислить все частные производные. Название "обратный режим" происходит от второго прохода через граф, когда градиенты протекают в обратном направлении. На рис. 6.15 показан второй проход. Во время *первого прохода* были вычислены значения всех узлов, начиная с $x = 3$ и $y = 4$. Эти значения показаны в правом нижнем углу каждого узла (например, $x \times x = 9$). Узлы обозначены от n_1 до n_7 . Выходным узлом является n_7 : $f(3, 4) = n_7 = 42$.

Идея обратного прохода заключается в движении вниз по графу и вычислении частных производных $f(x, y)$ относительно следующих друг за другом узлов, пока не будут достигнуты узлы переменных. Автоматическое дифференцирование в обратном режиме использует на цепное правило (chain rule), например

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial n_i} \times \frac{\partial n_i}{\partial x}.$$

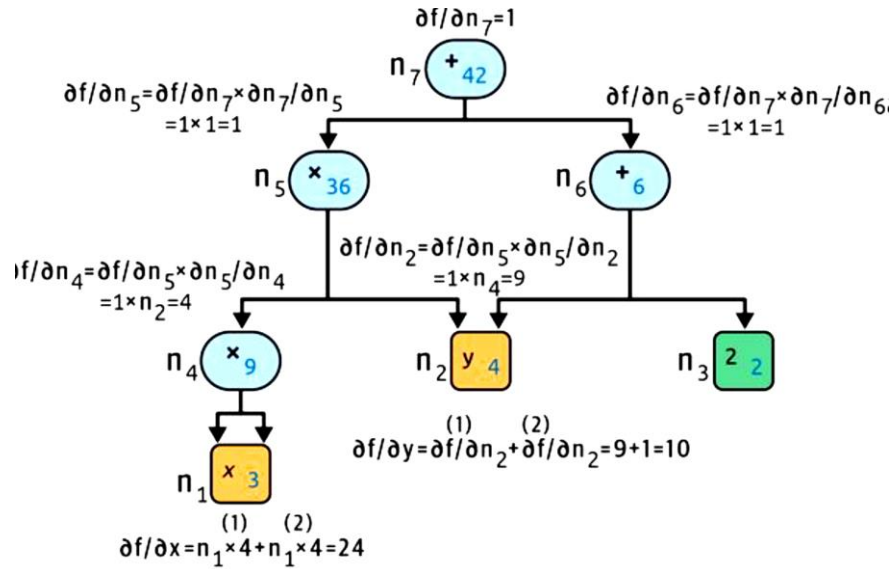


Рис. 6.15. Автоматическое дифференцирование в обратном режиме

Так как n_7 является выходным узлом ($f = n_7$), то $\frac{\partial f}{\partial n_7} = 1$.

Продолжаем двигаться вниз по графу к n_5 : $\frac{\partial f}{\partial n_5} = \frac{\partial f}{\partial n_7} \times \frac{\partial n_7}{\partial n_5}$. Мы уже получили

$$\frac{\partial f}{\partial n_7} = 1. \text{ Так как } n_7 = n_5 + n_6, \text{ то } \frac{\partial f}{\partial n_5} = \frac{\partial f}{\partial n_7} \times \frac{\partial n_7}{\partial n_5} = 1 \times 1 = 1.$$

Переходим к узлу n_4 : $\frac{\partial f}{\partial n_4} = \frac{\partial f}{\partial n_5} \times \frac{\partial n_5}{\partial n_4}$. Так как $n_5 = n_4 \times n_2$, то $\frac{\partial n_5}{\partial n_4} = n_2$,

$$\text{поэтому } \frac{\partial f}{\partial n_4} = 1 \times n_2 = 4.$$

Наконец, $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial n_1} = \frac{\partial f}{\partial n_4} \times \frac{\partial n_4}{\partial n_1}$. Так как $n_4 = n_1^2$, то $\frac{\partial n_4}{\partial n_1} = 2 \times n_1 = 6$ и

$$\frac{\partial f}{\partial x} = 4 \times 6 = 24.$$

Аналогично можно вычислить $\frac{\partial f}{\partial y} = 10$.

Программа автоматического дифференцирования должна "знать" правила дифференцирования и производные элементарных функций.

Автоматическое дифференцирование в обратном режиме является очень мощной и точной методикой, особенно когда имеется много входов и мало выходов, т.к. для вычисления частных производных всех выходов относительно всех входов оно требует только одного прямого прохода плюс одного обратного прохода на выход. При обучении нейронных сетей мы обычно хотим свести к минимуму функцию потерь, так что есть единственный выход (потеря) и потому для расчета градиентов необходимы только два прохода через граф.

6.5.4. Общие подходы к градиентным алгоритмам обучения нейронных сетей

Обучение нейронной сети с учителем — это настройка ее весов и смещений, минимизирующая ошибки между известными значениями в обучающем наборе и результатами работы сети. Величина ошибки сети характеризуется неотрицательным функционалом ошибки (функцией потерь). Для обучения нейронных сетей применяют *градиентные методы безусловной оптимизации* [25–28]. Эти методы используют вектор градиента функции потерь по параметрам нейронной сети (весам и смещениям). *Вектор градиента*, или просто градиент — это вектор, компоненты которого равны частным производным функции потерь по параметрам нейронной сети

$$\mathbf{g}(\mathbf{w}) = \text{grad } L = \nabla L = \left[\frac{\partial L(\mathbf{w})}{\partial w_1} \quad \frac{\partial L(\mathbf{w})}{\partial w_2} \quad \dots \quad \frac{\partial L(\mathbf{w})}{\partial w_n} \right]^T.$$

Вектор градиента определяет направление скорейшего возрастания функции. Так как нам необходимо найти минимум функции потерь, то будем изменять вектор параметров сети по направлению, противоположному градиенту — по направлению *антиградиента* $-\mathbf{g}(\mathbf{w})$.

Равенство нулю градиента в точке $\mathbf{w}^*$ является *необходимым условием* того, что точка $\mathbf{w}^*$ является *точкой локального минимума*

$$\mathbf{g}(\mathbf{w}^*) = 0.$$

Достаточным условием того, что точка $\mathbf{w}^*$ является точкой локального минимума, является положительная определенность *матрицы Гессе* (Гессиана)

$$\mathbf{H}(\mathbf{w}^*) = \begin{bmatrix} \frac{\partial^2 L(\mathbf{w}^*)}{\partial w_1 \partial w_1} & \frac{\partial^2 L(\mathbf{w}^*)}{\partial w_1 \partial w_2} & \dots & \frac{\partial^2 L(\mathbf{w}^*)}{\partial w_1 \partial w_n} \\ \dots & \dots & \dots & \dots \\ \frac{\partial^2 L(\mathbf{w}^*)}{\partial w_n \partial w_1} & \frac{\partial^2 L(\mathbf{w}^*)}{\partial w_n \partial w_2} & \dots & \frac{\partial^2 L(\mathbf{w}^*)}{\partial w_n \partial w_n} \end{bmatrix}. \quad (6.33)$$

В малой окрестности точки $\mathbf{w}$ гладкая целевая функция хорошо аппроксимируется суммой первых трёх слагаемых её разложения по формуле Тейлора [1]

$$L(\mathbf{w} + \mathbf{p}) = L(\mathbf{w}) + [\mathbf{g}(\mathbf{w})]^T \mathbf{p} + \frac{1}{2} \mathbf{p}^T \mathbf{H}(\mathbf{w}) \mathbf{p}, \quad (6.34)$$

где $\mathbf{p}$ — некоторый вектор.

Разложение (6.34) представляет собой квадратичное приближение целевой функции в окрестности точки $\mathbf{w}$ с погрешностью $O(\|\mathbf{p}\|^3)$ и является основой для построения градиентных методов оптимизации.

В зависимости от наивысшего порядка частных производных функции $L(\mathbf{w})$, используемых в процессе решения задачи минимизации, численные методы решения задачи безусловной минимизации принято делить на три группы:

– *Методы нулевого порядка*, использующие только информацию о значении минимизируемой функции. Эти методы используются при решении задач *одномерной минимизации*. Для многомерных задач такие методы основаны на случайном выборе вектора направления оптимизации. В обучении нейронных сетей методы нулевого порядка используются редко.

– *Методы первого порядка*, использующие информацию о первых производных минимизируемой функции. Для функции $L(\mathbf{w})$ это означает использование градиента.

– *Методы второго порядка*, требующие для своей реализации знания вторых производных минимизируемой функции. Для функции $L(\mathbf{w})$ это означает использование матрицы Гессе (6.33).

Применению градиентных методов оптимизации при обучении нейронных сетей свойственен целый ряд проблем [29]. Уже неоднократно упоминалась *проблема локальных минимумов*. В математике, обычно, рассматривается оптимизация выпуклых функций [30]. *Выпуклая функция (выпуклая вниз функция)* — функция, для которой любой отрезок между двумя любыми точками графика функции лежит не ниже соответствующей дуги графика. Для выпуклой функции может быть найден глобальный минимум. Функционал ошибки нейронной сети не является выпуклой функцией и имеет множество локальных минимумов. Градиентные методы обеспечивают попадание в один из локальных минимумов.

Другая проблема — это *плато*, седловые точки, овраги и резко растущие градиенты. Могут существовать широкие области с постоянным значением градиента — *плато*. При попадании на плато градиент равен нулю, поэтому веса сети во многих градиентных методах перестают изменяться. Градиентные методы плохо сходятся, если поверхности уровня функционала ошибки сильно вытянуты в некоторых направлениях — имеют вид *оврагов*. В этом случае градиентный спуск производится по ломаной линии, каждое звено которой перпендикулярно к соседним звеньям, а длина звена убывает (6.16). В *седловых точках* градиент равен нулю, причем в окрестности седловой точки некоторые значения выше значения в седловой точке, некоторые — ниже. График функции поверхности функции двух переменных вблизи седловой точки по форме напоминает седло (рис. 6.17), откуда и произошло название. Замечено, что для сетей глубокой архитектуры характерно наличие большого количества седловых точек, а для сетей с небольшим числом слоев — локальных минимумов. В седловых точках градиентные алгоритмы ведут себя непредсказуемо. В сетях с большим числом слоев часто встречаются *крутые*

участки, напоминающие утесы. На стене утеса градиент может быть очень большим, поэтому можно проскочить мимо минимума.

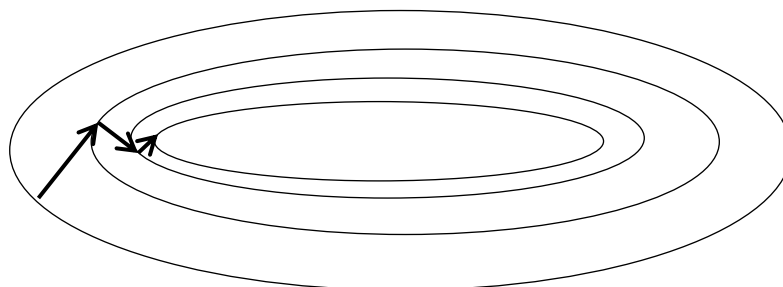


Рис. 6.16. Скорейший спуск в случае оврага

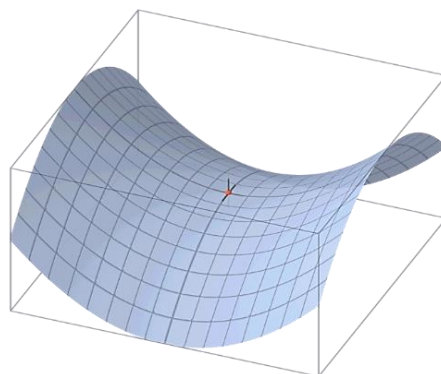


Рис. 6.17. Седловая точка функции $z = x^2 - y^2$ (рисунок из Википедии)

Другая проблема — *плохая обусловленность матрицы Гессе* функционала ошибки. Опуская математические подробности, которые можно найти в [29], отметим, что при плохой обусловленности матрицы Гессе обучение происходит очень медленно даже при больших значениях градиента. Проблема усугубляется тем, что вычислять матрицу Гессе очень сложно и чаще всего применяются градиентные методы первого порядка.

При обучении с учителем возможно несколько режимов обучения. *Пакетный режим (Batch Mode)* заключается в предъявлении сети *всего* набора обучающих данных, вычислении ошибки, усредненной по всему набору обучающих данных, и изменении параметров сети в соответствии с ошибкой. Процедура предъявления сети *всего* набора обучающих данных называется *эпохой (epoch)*. Для больших обучающих наборов пакетный алгоритм оказывается очень медленным.

В *последовательном режиме (Sequential Mode)* обучения модификация параметров сети производится после предъявления сети *каждого* из примеров обучающего набора. Процедура предъявления сети *каждого* примера называется *проходом (passe)*.

В современных нейронных сетях с большим количеством слоев последовательный режим с использованием метода градиентного спуска реализуется в виде *стохастического градиентного спуска (SGD — Stochastic Gradient Descend)*. В SGD на каждом проходе из обучающего набора случайным

образом выбирается пример и по нему настраиваются параметры сети. Каждый проход SGD реализуется быстро, но поведение функционала ошибки имеет случайный характер, поэтому требуется больше проходов, чем эпох пакетного обучения. Но случайный характер обучения уменьшает вероятность попадания в локальные минимумы — из состояния, близкого к локальному минимуму функционала ошибки, сеть на следующем проходе может оказаться в состоянии, далеком от локального минимума.

Наибольшее распространение при обучении больших нейронных сетей получил *мини-пакетный (Mini-Batch)* режим. В методе на каждом шаге обучения настройка параметров сети производится на относительно небольших наборах данных — *мини-пакетах (Mini-Batch)*, случайно выбираемых (обычно, *без возврата*) из общего набора данных. Используется также термин итерация — обучение на каждом из батчей. Мини-пакетный режим позволяет использовать различные методы градиентные оптимизации, а не только градиентный спуск. Изменение функционала ошибки носит не столь нерегулярный характер, как в случае стохастического градиентного спуска, поэтому возможности избежать локальный минимум ниже. Зачастую (см., например, [29]) *мини-пакетный градиентный спуск (Mini-Batch Gradient Descent)* называют стохастическим градиентным спуском.

6.5.5. Алгоритмы обучения первого порядка

Алгоритмы обучения первого порядка используют только градиент функции потерь. Простейшим градиентным алгоритмом является *алгоритм градиентного спуска (gradient descent algorithm)*. Тогда k -я итерация (k -й шаг обучения) для i -го веса (смещения формально рассматриваем как веса) описывается формулой

$$w_i^{(k)} = w_i^{(k-1)} - \eta \frac{\partial L(\mathbf{w}^{(k-1)})}{\partial w_i}, \quad (6.35)$$

где η — подбираемый коэффициент (learning rate — скорость обучения);

$\frac{\partial L(\mathbf{w}^{(k-1)})}{\partial w_i}$ — значение компонента вектора градиента функции потерь по весу w_i ,

функция потерь вычисляется по вектору $\mathbf{w}^{(k-1)}$ весов предыдущего шага обучения; для простоты в (6.35) использована сплошная нумерация весов.

Скорость обучения подбирают. Если задать слишком малую скорость, то процесс обучения будет медленным. Если задать слишком большую скорость, то кривая обучения будет негладкой, процесс может разойтись (функционал ошибки будет расти). Рекомендуется постепенной уменьшать скорость в процессе обучения. Известно несколько подходов к снижению скорости обучения. При

использовании *обратного масштабирования* (inverse scaling), или *обратного спада* скорость обучения на k -ом шаге обучения определяется по формуле

$$\eta_k = \frac{\eta_0}{k^p}, \quad (6.36)$$

где η_0 — задаваемая пользователем начальная скорость, p — задаваемая пользователем константа.

При использовании *адаптивного режима* (adaptive) скорость остается постоянной, равной заданной начальной скорости, пока функция потерь уменьшается. Каждый раз, когда два последовательных шага обучения не могут уменьшить функцию потерь, по крайней мере на заданную величину, текущая скорость обучения делится на заданную величину.

При использовании *экспоненциального спада* (inverse decay) скорость обучения η изменяется по формуле

$$\eta_k = \eta_0 e^{-\alpha k},$$

где k — номер шага обучения, η_0 — начальная скорость, α — подбираемый параметр.

Ограничение слагаемым первого порядка в разложении функционала ошибки по формуле Тейлора (6.34) не позволяет использовать информацию о кривизне функционала ошибки, что обуславливает малую скорость сходимости алгоритма градиентного спуска. Алгоритм градиентного спуска плохо работает на плоских участках целевой функции, а также вблизи локальных минимумов. В этих областях значение градиента близко к нулю и скорость сходимости метода градиентного спуска очень мала.

Для преодоления этих недостатков служит *алгоритм градиентного спуска с моментом* [1, 3, 29] (в русской литературе принято использовать термин "момент", в англоязычной литературе термин *градиентный спуск с импульсом* (gradient descent with momentum)). В этом алгоритме вектор весов рассчитывается по формуле

$$\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + \Delta \mathbf{w}^{(k)}$$

Приращение весов $\Delta \mathbf{w}^{(k)}$ рассчитывается по формуле

$$\Delta \mathbf{w}^{(k)} = -\eta \mathbf{g}(\mathbf{w}^{(k-1)}) + \alpha \Delta \mathbf{w}^{(k-1)}, \quad (6.37)$$

где η — скорость обучения, α — коэффициент момента, принимающий значения в интервале $[0, 1]$.

Слово "импульс" в названии алгоритма означает, что изменение весов происходит "по инерции" в направлении изменения весов предыдущего шага обучения. Выражение (6.37) содержат слагаемые, зависящие от градиента, и не зависящие от градиента. Причем чем больше значение коэффициента α , тем сильнее на подбор

весов сказывается слагаемое, не зависящее от градиента. Это влияние существенно возрастает на плоских участках целевой функции и вблизи локальных минимумов. В этих областях слагаемое, не зависящее от градиента, начинает доминировать в (6.37), что способствует к выходу из этой области. Можно сказать, что изменение весов происходит "по инерции" в направлении вектора поправки предыдущего шага обучения. Однако если поправка веса практически не будет зависеть от градиента, то это приведет к нестабильности алгоритма (алгоритм "не знает", куда идти). Выходом является допущение возрастания целевой функции в ограниченных пределах. Обычно, если $L_k > L_{k-1}$, то веса не изменяются. Например, при $E_k > E_{k-1}$, веса не изменяются, а скорость обучения уменьшается с коэффициентом, равным по умолчанию 0,7. Если новое значение функционала ошибки меньше прежнего, то принимаются новые значения весов и увеличивается скорость обучения с коэффициентом по умолчанию 1,05).

В методе ускоренного градиента Нестерова (NAG — *Nesterov Accelerated Gradient*) [3, 29] поправка вектора параметров описывается следующим образом

$$\Delta \mathbf{w}^{(k)} = \alpha \Delta \mathbf{w}^{(k-1)} - \eta \mathbf{g}(\mathbf{w}^{(k-1)} + \alpha \Delta \mathbf{w}^{(k-1)}).$$

Метод NAG отличается от градиентного спуска с импульсом более точным вычислением вектора поправки. Так как вектор градиента, как правило, указывает правильное направление минимума, то градиент, вычисленный чуть дальше в правильном направлении, будет точнее градиента, вычисленного в исходной точке. Это объясняет более высокую скорость метода Нестерова по сравнению с методом с импульсом.

Известно несколько эвристических (не имеющих строгого математического обоснования) алгоритмов, основанных на методе скорейшего спуска [1]. Простым и эффективным эвристическим алгоритмом является алгоритм *RPROP* (англ.: *Resilient Back PROPagation* — "эластичный" *Back Propagation*), предложенный М. Ридмиллером (*M. Riedmiller*) и Х. Брауном (*H. Braun*). В точках локальных минимумов и на "горизонтальных" участках функционала ошибки производные функционала ошибки по весам близки к нулю и процесс обучения сети методом скорейшего спуска практически прекращается. *RPROP* использует лишь знаки частных производных функционала ошибки по каждому весу и свою величину скорости обучения для каждого веса. Причем скорость обучения адаптируется в процессе обучения. Первый шаг алгоритма обучения выполняется по алгоритму скорейшего спуска. На последующих шагах учитывается только знак компонента градиента функционала

$$w_i^{(k)} = w_i^{(k-1)} + \Delta w_i^{(k)},$$

$$\Delta w_i^{(k)} = -\eta_i^{(k)} \operatorname{sgn} \left(\frac{\partial E(\mathbf{w}^{(k-1)})}{\partial w_i} \right),$$

где $w_i^{(k)}$ — i -й компонент вектора весов на k -ом шаге обучения, sgn — функция знака

$$\operatorname{sgn}(x) = \begin{cases} 1, & x > 0, \\ 0, & x = 0, \\ -1, & x < 0. \end{cases}$$

Коэффициент обучения рассчитывается для каждого веса с учетом изменения значения компонента градиента

$$\eta_i^{(k)} = \begin{cases} \min \{ a\eta_i^{(k-1)}, \eta_{\max} \}, & \text{если } g_i^{(k)} \cdot g_i^{(k-1)} > 0, \\ \max \{ b\eta_i^{(k-1)}, \eta_{\min} \}, & \text{если } g_i^{(k)} \cdot g_i^{(k-1)} < 0, \\ \eta_i^{(k-1)} & \text{в остальных случаях,} \end{cases} \quad (6.38)$$

где $g_i^{(k)} = \frac{\partial E(\mathbf{w}^{(k)})}{\partial w_i}$ — i -й компонент вектора градиента на k -ом шаге обучения.

Авторы алгоритма предложили следующие значения констант: $a = 1,2$, $b = 0,5$, $\eta_{\min} = 10^{-6}$, $\eta_{\max} = 50$.

Выражение (6.38) означает, что при сохранении знака компонента градиента скорость обучения соответствующего компонента вектора весов можно увеличить. Если знак компонента градиента меняется — нужно уменьшить. Если компонент градиента равен нулю, то движение не прекращается, что позволяет проходить точки локальных минимумов (по крайней мере, неглубокие локальные минимумы). Причем проверку на нуль целесообразно заменить сравнением с малым числом ε — если $|g_i^{(k)} g_i^{(k-1)}| < \varepsilon$, то скорость обучения не меняется.

Некоторые компоненты вектора градиента функции потерь по весам резко изменяются, а некоторые изменяются медленно. Градиентный спуск имеет одинаковую скорость обучения для разных компонентов вектора весов, хотя желательно иметь разную скорость. Для весов, соответствующий компонент вектора градиента которых резко уменьшается, желательно иметь меньшую скорость, чтобы не проскочить минимум, а для весов, компонент вектора градиента которых изменяется медленно, большую скорость. Это позволяют алгоритмы с *адаптивной скоростью обучения*.

Чтобы градиент изменялся по более пологой кривой применяют алгоритм *AdaGrad* [3, 6, 29], уменьшающий градиент вдоль самых крутых направлений спуска

$$\mathbf{s}^{(k+1)} = \mathbf{s}^{(k)} + \mathbf{g}(\mathbf{w}^{(k)}) \otimes \mathbf{g}(\mathbf{w}^{(k)}), \quad (6.39)$$

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta \mathbf{g}(\mathbf{w}^{(k)}) ./ \sqrt{\mathbf{s}^{(k+1)} + \varepsilon},$$

где $\mathbf{g}(\mathbf{w}^{(k)})$ — вектор градиента, $\otimes$ — поэлементное умножение, η — подбираемый коэффициент, ε — сглаживающий член, для избежания деления на ноль, обычно $\varepsilon \approx 10^{-10}$, $./$ — поэлементное деление.

В векторе $\mathbf{s}$ (6.39) накапливаются квадраты градиента, компоненты этого вектора равны $s_i^{(k+1)} = s_i^{(k)} + (\partial g_i / \partial w_i)^2$. Алгоритм уменьшает каждый компонент вектора градиента. Причем уменьшает в большей степени для больших значений компонентов. Но AdaGrad зачастую замедляет обучение слишком быстро, веса перестают практически изменяться, не достигнув минимума функционала ошибки.

Алгоритм *RMSProp (Root Mean Square Propagation)* [3, 6, 29] в значительной мере исправляет этот недостаток, используя коэффициент ослабления

$$\mathbf{s}^{(k+1)} = \beta \mathbf{s}^{(k)} + (1 - \beta) \mathbf{g}(\mathbf{w}^{(k)}) \otimes \mathbf{g}(\mathbf{w}^{(k)}),$$

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta \mathbf{g}(\mathbf{w}^{(k)}) ./ \sqrt{\mathbf{s}^{(k+1)} + \varepsilon},$$

где $\beta \approx 0,9$ — коэффициент ослабления.

Лучшим на данный момент, особенно для обучения глубоких сетей считается алгоритм *Adam (adaptive moments)* [3, 6, 29, 31]

$$\mathbf{m} = \beta_1 \mathbf{m}^{(k)} - (1 - \beta_1) \mathbf{g}(\mathbf{w}^{(k)}),$$

$$\mathbf{s} = \beta_2 \mathbf{s}^{(k)} + (1 - \beta_2) \mathbf{g}(\mathbf{w}^{(k)}) \otimes \mathbf{g}(\mathbf{w}^{(k)}),$$

$$\mathbf{m}^{(k+1)} = \frac{\mathbf{m}}{1 - \beta_1^{k+1}},$$

$$\mathbf{s}^{(k+1)} = \frac{\mathbf{s}}{1 - \beta_2^{k+1}},$$

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \eta \mathbf{m}^{(k+1)} ./ \sqrt{\mathbf{s}^{(k+1)} + \varepsilon},$$

где $\beta_1 = 0,9$, $\beta_2 = 0,999$ — параметры ослабления.

Обратите внимание, что при вычислении $\mathbf{m}^{(k+1)}$ и $\mathbf{s}^{(k+1)}$ коэффициенты ослабления возводятся в степень, равную номеру текущей итерации. Название *adaptive moments* — адаптивные моменты возникло потому, что элементы вектора $\mathbf{m}$ накапливают сумму компонентов вектора градиента — оценку среднего значения (первого момента), а элементы вектора $\mathbf{s}$ накапливают сумму квадратов компонентов вектора градиента — оценку дисперсии (второго момента).

6.5.6. Алгоритмы обучения второго порядка

а) алгоритм сопряженных градиентов

Метод сопряженных градиентов (*Conjugate-Gradient Method*) [32] является мощным методом решения систем линейных алгебраических уравнений вида

$$\mathbf{Ax} = \mathbf{b}, \quad (6.40)$$

где $\mathbf{A}$ — симметричная положительно определенная матрица размером $n \times n$, $\mathbf{b}$ — вектор-столбец размером $n \times 1$.

Решение системы (6.40) эквивалентно минимизации квадратичного функционала

$$f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{Ax} - \mathbf{b}^T \mathbf{x} + c \quad (6.41)$$

где c — скаляр.

Причем при отсутствии погрешностей вычислений решение задач (6.40) и (6.41) может быть найдено не более чем за n шагов.

В малой окрестности точки локального минимума функционал ошибки нейронной сети хорошо аппроксимируется квадратичным функционалом (6.34). Между функционалами (6.34) и (6.41) существует прямая аналогия: вектору $\mathbf{x}$ соответствует вектор $\mathbf{p}$ изменения весов сети, вектору $\mathbf{b}$ соответствует вектор градиента $\mathbf{g}$, а матрице $\mathbf{A}$ — Гессиан $\mathbf{H}$ (в точке локального минимума Гессиан положительно определен). Поэтому метод сопряженных градиентов может быть применен для минимизации функционала ошибки сети, то есть для обучения сети. Это будет метод второго порядка, так как Гессиан (6.33) содержит вторые производные функционала ошибки. А так как функционал ошибки только приближенно можно рассматривать как квадратичный, то даже в отсутствии ошибок вычислений метод становится итерационным. В таком виде алгоритм требует вычисления Гессиана, что представляет сложную задачу. Однако известны подходы [25, 33], позволяющие исключить вычисление Гессиана и ограничиться вычислением только векторов градиента.

В варианте метода сопряженных градиентов, используемом для обучения нейронных сетей, вектор направления поиска минимума $\mathbf{p}_k$ выбирается таким образом, чтобы направление было сопряжено со всеми предыдущими направлениями $\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{k-1}$. Векторы $\mathbf{p}_i$ и $\mathbf{p}_j$ называются сопряженными (сопряженными относительно матрицы $\mathbf{A}$), если $(\mathbf{Ap}_i, \mathbf{p}_j) = 0$. Вектор, удовлетворяющий указанным условиям, может быть построен по выражению

$$\mathbf{p}_k = -\mathbf{g}_{k-1} + \beta_k \mathbf{p}_{k-1}, \quad (6.42)$$

где $\mathbf{g}_{k-1} = \mathbf{g}(\mathbf{w}_{k-1})$ — вектор градиента, β_k — коэффициент, вычисляемый в каждой итерации обучения. В (6.42) принимают $\mathbf{p}_0 = -\mathbf{g}_0$, где $\mathbf{g}_0 = \mathbf{g}(\mathbf{w}_0)$, $\mathbf{w}_0$ — начальное значение вектора весов.

Вектор $\mathbf{p}_1$ вычисляется по формуле градиентного спуска. На последующих итерациях используются специальные формулы. В общем виде вычисления коэффициента β_k требует знания Гессиана. Однако известно несколько способов вычисления коэффициента β_k , не требующих знания Гессиана, наиболее популярными при обучении нейронных сетей являются два:

– формула Флетчера-Ривса (Fletcher R., Reeves C. M.) [34]

$$\beta_k = \frac{\mathbf{g}_{k-1}^T \mathbf{g}_{k-1}}{\mathbf{g}_{k-2}^T \mathbf{g}_{k-2}} = \frac{(\mathbf{g}_{k-1}, \mathbf{g}_{k-1})}{(\mathbf{g}_{k-2}, \mathbf{g}_{k-2})}, \quad (6.43)$$

– формула Полака-Рибьера (Polak E., Ribiere G.) [35]

$$\beta_k = \frac{(\mathbf{g}_{k-1} - \mathbf{g}_{k-2})^T \mathbf{g}_{k-1}}{\mathbf{g}_{k-2}^T \mathbf{g}_{k-2}} = \frac{(\mathbf{g}_{k-1} - \mathbf{g}_{k-2}, \mathbf{g}_{k-1})}{(\mathbf{g}_{k-2}, \mathbf{g}_{k-2})}. \quad (6.44)$$

Реже используется формула Хестенса-Штифеля (Hestenes-Stiefel) [36]

$$\beta_k = \frac{(\mathbf{g}_{k-1} - \mathbf{g}_{k-2})^T \mathbf{g}_{k-1}}{(\mathbf{g}_{k-1} - \mathbf{g}_{k-2})^T \mathbf{p}_{k-1}} = \frac{(\mathbf{g}_{k-1} - \mathbf{g}_{k-2}, \mathbf{g}_{k-1})}{(\mathbf{g}_{k-1} - \mathbf{g}_{k-2}, \mathbf{p}_{k-1})}.$$

При минимизации квадратичного функционала формулы Флетчера-Ривса (6.43) и Полака-Рибьера (6.44) эквивалентны. Из-за приближенного характера формулы (6.34) и неточности вычисления нового приближения вектора весов сопряженность направлений поиска постепенно теряется. Это приводит к снижению скорости сходимости метода. Формула Полака-Рибьера позволяет алгоритму выйти из этой ситуации. Можно дать следующее качественное объяснение такому свойству формулы Полака-Рибьера [1]. Обучение "застопорится", если $\mathbf{g}_{k-1} \approx \mathbf{g}_{k-2}$. Но тогда коэффициент $\beta_k \approx 0$ и как видно из (6.42) новое направление поиска становится равным антиградиенту. Это означает *рестарт процедуры обучения*: направление минимизации из точки полученного решения выбирается по алгоритму скорейшего спуска, а на последующих итерациях применяется метод сопряженных градиентов. Считается, что возникли проблемы со сходимостью, если число итераций обучения превысило число настраиваемых параметров сети. Рестарт процедуры обучения реализован во всех алгоритмах сопряженных градиентов.

Мы пока только выбрали направление поиска оптимального решения. Изменение вектора весов производится по выбранному направлению

$$\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + \eta_k \mathbf{p}_k.$$

Коэффициент η_k находится как решение задачи одномерной оптимизации. Часто применяется аппроксимация функционала ошибки по направлению $\mathbf{p}_k$ с последующим расчетом минимума полученной функции одной переменной η [1–2].

Рассмотрим идею метода на примере аппроксимации многочленом второго порядка [1]

$$L(\mathbf{w}) = P_2(\eta) = a_0 + a_1\eta + a_2\eta^2. \quad (6.45)$$

В каждом цикле обучения вычисляются коэффициенты a_0, a_1, a_2 и находится значение коэффициента η , обеспечивающее минимум функции (6.45). Для вычисления коэффициентов a_0, a_1, a_2 необходимо задать три значения коэффициента η таких, чтобы минимум функционала ошибки лежал между первым и третьим значениями. Например, первое значение коэффициента берется с предыдущего шага обучения. Далее берется значение коэффициента в два раза больше первого и так далее, пока значение функционала не начнет возрастать. После этого по трем значениям коэффициента — начальному и двум последним (обозначим их η_1, η_2 и η_3) вычисляются три вектора весов

$$\mathbf{w}_1^{(k)} = \mathbf{w}^{(k-1)} + \eta_1 \mathbf{p}_k, \quad \mathbf{w}_2^{(k)} = \mathbf{w}^{(k-1)} + \eta_2 \mathbf{p}_k, \quad \mathbf{w}_3^{(k)} = \mathbf{w}^{(k-1)} + \eta_3 \mathbf{p}_k$$

Рассчитав значения функционала $L_1 = L(\mathbf{w}_1^{(k)})$, $L_2 = L(\mathbf{w}_2^{(k)})$, $L_3 = L(\mathbf{w}_3^{(k)})$ и используя значения полинома (6.45), получаем систему из трех уравнения

$$P_2(\eta_1) = L_1, \quad P_2(\eta_2) = L_2, \quad P_2(\eta_3) = L_3,$$

из которой находим коэффициенты a_0, a_1, a_2 .

Для нахождения значения коэффициента η , обеспечивающего минимум полинома (6.45), приравняем производную от полинома нулю

$$\frac{dP_2}{d\eta} = 2a_2\eta + a_1 = 0,$$

Откуда значение коэффициента равно $\eta = -a_1/2a_2$.

После этого отбрасываем крайнее значение коэффициента η , соответствующее большему значению функционала ошибки, и проверяем, можно ли через три оставшихся значения функционала ошибки провести вогнутую кривую. Если это условие выполняется, то по оставшимся трем значениям строим параболу вида (6.45), определяем новое значение коэффициента η и т. д. Процесс заканчивается, когда изменение функционала ошибки станет меньше заданной малой величины.

Более точные результаты дает использование многочлена третьего порядка, как например, в самом популярном при обучении нейронных сетей алгоритме Чараламбуса (*Charalambous C.*) [37]. Рассмотренный метод одномерной оптимизации может занять много времени, так как требует многократного вычисления вектора весов, выхода сети и функционала ошибки. В [38] предложен *масштабируемый алгоритм сопряженных градиентов (scaled conjugate gradient*

algorithm), в котором в одномерной оптимизации используется одномерная форма алгоритма Левенберга-Марквардта.

Алгоритм метода сопряженных градиентов строится следующим образом (в алгоритме не отражены рестарты, при каждом рестарте алгоритм повторяется).

На "нулевой" итерации выполняются подготовительные действия:

1. Полагается $k = 0$.
2. Задается начальное приближение вектора весов $\mathbf{w}^{(0)}$ и, используя алгоритм обратного распространения ошибки, вычисляется вектор градиента $\mathbf{g}_0$.
3. В качестве направления движения выбирается $\mathbf{p}_1 = -\mathbf{g}_0$.

На *первой и следующих итерациях* ($k = 1, 2, \dots$) выполняются следующие действия:

4. Вычисляется номер текущей итерации $k = k + 1$.
5. С использованием одномерной оптимизации вычисляется коэффициент η_k .
6. Находится новое приближение вектора весов

$$\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + \eta_k \mathbf{p}_k.$$

7. Вычисляются выход сети и функционал ошибки $E(\mathbf{w}_k)$.
8. Проверяется условие окончания процесса обучения $E(\mathbf{w}_k) \leq \varepsilon$, где ε — малая величина. Если условие выполняется, то алгоритм завершен (переход на шаг 13), иначе — переход на следующий шаг алгоритма.
9. Используя алгоритм обратного распространения ошибки, вычисляется вектор градиента $\mathbf{g}_k$.
10. По формулам (6.43) или (6.44) вычисляется коэффициент β_{k+1} .
11. Определяется новое направление движения

$$\mathbf{p}_{k+1} = -\mathbf{g}_k + \beta_{k+1} \mathbf{p}_k$$

12. Переход на шаг 4.
13. Конец алгоритма.

Метод сопряженных градиентов имеет сходимость, близкую к линейной [1], и работает значительно быстрее метода градиентного спуска. Метод уступает методу Левенберга-Марквардта (см. далее), но менее требователен к памяти. Особо следует отметить, что в сетях глубокого обучения из-за очень большого числа весов, как правило, применяют более простые методы первого порядка.

б) квазиньютоновские алгоритмы

Эти методы называют ещё *алгоритмами переменной метрики* [1].

Используем разложение (6.34)

$$L(\mathbf{w} + \mathbf{p}_k) = L(\mathbf{w}) + [\mathbf{g}(\mathbf{w})]^T \mathbf{p}_k + \frac{1}{2} \mathbf{p}_k^T \mathbf{H}(\mathbf{w}) \mathbf{p}_k, \quad (6.46)$$

где k — номер приближения.

Для достижения минимума функционала (6.46) требуется, чтобы

$$\frac{dL(\mathbf{w}^{(k-1)} + \mathbf{p}_k)}{d\mathbf{p}_k} = 0. \quad (6.47)$$

Это необходимое условие экстремума функционала (6.46).

Выражение (6.47) — это производная от функционала L в точке $(\mathbf{w}^{(k-1)} + \mathbf{p}_k)$ по направлению $\mathbf{p}_k$ [39]. Производной от функционала F в точке $\mathbf{x}$ по направлению $\mathbf{y}$ называется выражение⁵

$$\frac{dF(\mathbf{x})}{d\mathbf{y}} = \lim_{t \rightarrow 0} \frac{F(\mathbf{x} + t\mathbf{y}) - F(\mathbf{x})}{t}.$$

Из (6.46), обозначив $\mathbf{g}_{k-1} = \mathbf{g}(\mathbf{w}^{(k-1)})$ и $\mathbf{H}_{k-1} = \mathbf{H}(\mathbf{w}^{(k-1)})$, получаем

$$\begin{aligned} \frac{dL(\mathbf{w}^{(k-1)} + \mathbf{p}_k)}{d\mathbf{p}_k} &= \frac{d(\mathbf{g}_{k-1}, \mathbf{p}_k)}{d\mathbf{p}_k} + \frac{1}{2} \frac{d(\mathbf{H}_{k-1}\mathbf{p}_k, \mathbf{p}_k)}{d\mathbf{p}_k}. \\ \frac{d(\mathbf{g}_{k-1}, \mathbf{p}_k)}{d\mathbf{p}_k} &= \lim_{t \rightarrow 0} \frac{(\mathbf{g}_{k-1}, \mathbf{p}_k + t\mathbf{p}_k) - (\mathbf{g}_{k-1}, \mathbf{p}_k)}{t} = \lim_{t \rightarrow 0} \frac{(\mathbf{g}_{k-1}, \mathbf{p}_k) + t(\mathbf{g}_{k-1}, \mathbf{p}_k) - (\mathbf{g}_{k-1}, \mathbf{p}_k)}{t} = \\ &= (\mathbf{g}_{k-1}, \mathbf{p}_k), \\ \frac{d(\mathbf{H}_{k-1}\mathbf{p}_k, \mathbf{p}_k)}{d\mathbf{p}_k} &= \lim_{t \rightarrow 0} \frac{(\mathbf{H}_{k-1}(\mathbf{p}_k + t\mathbf{p}_k), \mathbf{p}_k + t\mathbf{p}_k) - (\mathbf{H}_{k-1}\mathbf{p}_k, \mathbf{p}_k)}{t} = \\ &= \lim_{t \rightarrow 0} [2(\mathbf{H}_{k-1}\mathbf{p}_k, \mathbf{p}_k) + t(\mathbf{H}_{k-1}\mathbf{p}_k, \mathbf{p}_k)] = 2(\mathbf{H}_{k-1}\mathbf{p}_k, \mathbf{p}_k). \end{aligned}$$

Тогда

$$\frac{dL(\mathbf{w}^{(k-1)} + \mathbf{p}_k)}{d\mathbf{p}_k} = (\mathbf{g}_{k-1}, \mathbf{p}_k) + (\mathbf{H}_{k-1}\mathbf{p}_k, \mathbf{p}_k) = ((\mathbf{g}_{k-1} + \mathbf{H}_{k-1}\mathbf{p}_k), \mathbf{p}_k)$$

Выражение (6.47) примет вид

$$\frac{dL(\mathbf{w}^{(k-1)} + \mathbf{p}_k)}{d\mathbf{p}_k} = ((\mathbf{g}_{k-1} + \mathbf{H}_{k-1}\mathbf{p}_k), \mathbf{p}_k) = 0,$$

откуда с учетом, что $\mathbf{p}_k \neq 0$, получаем

$$\mathbf{g}(\mathbf{w}^{(k-1)}) + \mathbf{H}(\mathbf{w}^{(k-1)})\mathbf{p}_k = 0.$$

Таким образом, вектор $\mathbf{p}_k$ определяется из решения системы линейных алгебраических уравнений

⁵ С точки зрения теории функций и функционального анализа — это дифференциал Гато (см., например, Колмогоров А. Н., Фомин С. В. Элементы теории функций и функционального анализа. — М.: ФИЗМАТЛИТ, 2004. — 572 с.).

$$\mathbf{H}(\mathbf{w}^{(k-1)})\mathbf{p}_k = -\mathbf{g}(\mathbf{w}^{(k-1)}), \quad (6.48)$$

или формально

$$\mathbf{p}_k = -\left[\mathbf{H}(\mathbf{w}^{(k-1)})\right]^{-1} \mathbf{g}(\mathbf{w}^{(k-1)}). \quad (6.49)$$

Алгоритм минимизации, в котором направление $\mathbf{p}_k$ определяется системой (6.48), называют *методом Ньютона*, а решение этой системы — *ньютоновским направлением*.

Известно [26], что в задаче поиска минимума произвольной квадратичной функции с положительно определенной матрицей вторых производных метод Ньютона дает решение за одну итерацию, причем независимо от выбора начальной точки. При этом при вычислении нового приближения $\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + \eta_k \mathbf{p}_k$ шаг должен быть $\eta_k = 1$. Поэтому, если матрица Гессе минимизируемой функции в точке минимума положительно определена, начальное приближение $\mathbf{w}_0$ достаточно близко к точке минимума, а последовательность коэффициентов η_k сходится к единице, то метод Ньютона будет быстро сходиться. Вычисление Гессиана представляет собой вычислительно сложную задачу. Также нет гарантий, что Гессиан для функционала ошибок многослойного персептрона будет положительно определенным [2]. Кроме того, Гессиан может быть плохо обусловлен. Поэтому Гессиан приближенно заменяют симметричной положительно определенной матрицей $\mathbf{B}$. Методы, использующие такие матрицы, называются *квазиньютоновскими методами*.

В квазиньютоновских методах матрица приближения к Гессиану рассчитывается на каждом шаге обучения. Самой популярной формулой пересчета матриц является *формула Бройдена-Флетчера-Гольдфарба-Шенно (Broyden-Fletcher-Goldfarb-Shanno — BFGS)* [26]. Обозначим

$$\begin{aligned} \mathbf{s}_{k-1} &= \mathbf{w}^{(k-1)} - \mathbf{w}^{(k-2)} = \eta_{k-1} \mathbf{p}_{k-1}, \\ \mathbf{y}_{k-1} &= \mathbf{g}(\mathbf{w}^{(k-1)}) - \mathbf{g}(\mathbf{w}^{(k-2)}). \end{aligned}$$

Тогда формула *BFGS* имеет вид (доказательство можно найти в [26])

$$\mathbf{B}_k = \mathbf{B}_{k-1} - \frac{\mathbf{B}_{k-1} \mathbf{s}_{k-1} \mathbf{s}_{k-1}^T \mathbf{B}_{k-1}}{\mathbf{s}_{k-1}^T \mathbf{B}_{k-1} \mathbf{s}_{k-1}} + \frac{\mathbf{y}_{k-1} \mathbf{y}_{k-1}^T}{\mathbf{y}_{k-1}^T \mathbf{s}_{k-1}}. \quad (6.50)$$

В качестве начального приближения берется единичная матрица $\mathbf{B}_0 = \mathbf{E}$. В квазиньютоновском методе на каждом шаге обучения вычисляется матрица $\mathbf{B}_k$ (6.50) и в результате решения системы линейных алгебраических уравнений

$$\mathbf{B}_k \mathbf{p}_k = -\mathbf{g}_{k-1} \quad (6.51)$$

определяется вектор изменения весов. Новое приближение вектора весов рассчитывается по формуле $\mathbf{w}^{(k)} = \mathbf{w}^{(k-1)} + \eta_k \mathbf{p}_k$, в которой шаг η_k определяется в результате решения одномерной задачи оптимизации. Систему (6.51) с положительно определенной матрицей можно решить методом Холецкого [32].

Менее эффективной считается формула Девидона-Флетчера-Пауэлла (*Davidon-Fletcher-Powell — DFP*) [26]

$$\mathbf{B}_k = \mathbf{B}_{k-1} - \frac{\mathbf{B}_{k-1} \mathbf{s}_{k-1} \mathbf{s}_{k-1}^T \mathbf{B}_{k-1}}{\mathbf{s}_{k-1}^T \mathbf{B}_{k-1} \mathbf{s}_{k-1}} + \frac{\mathbf{y}_{k-1} \mathbf{y}_{k-1}^T}{\mathbf{y}_{k-1}^T \mathbf{s}_{k-1}} + \left(\mathbf{s}_{k-1}^T \mathbf{B}_{k-1} \mathbf{s}_{k-1} \right) \mathbf{q}_{k-1} \mathbf{q}_{k-1}^T,$$

где

$$\mathbf{q}_{k-1} = \frac{1}{\mathbf{y}_{k-1}^T \mathbf{s}_{k-1}} \mathbf{y}_{k-1} - \frac{1}{\mathbf{s}_{k-1}^T \mathbf{B}_{k-1} \mathbf{s}_{k-1}} \mathbf{B}_{k-1} \mathbf{s}_{k-1}.$$

В качестве начального приближения берется единичная матрица $\mathbf{B}_0 = \mathbf{E}$.

На каждом шаге обучения сети необходимо решать новую систему (6.51). Поэтому популярен прием, когда строятся приближения не самой матрицы Гессе, а обратной к ней. Вычисление нового направления поиска производится по формуле

$$\mathbf{p}_k = -\mathbf{V}_k \mathbf{g}(\mathbf{w}^{(k-1)}),$$

где $\mathbf{V}_k$ — приближение к обратной матрице $[\mathbf{H}(\mathbf{w}_{k-1})]^{-1}$. Известны [1, 25] формулы пересчета обратных матриц, которые носят те же имена, что и формулы пересчета прямых матриц: формула Бройдена-Флетчера-Гольдфарба-Шенно (*Broyden-Fletcher-Goldfarb-Shanno — BFGS*)

$$\mathbf{V}_k = \mathbf{V}_{k-1} + \left(1 + \frac{\mathbf{y}_{k-1}^T \mathbf{V}_{k-1} \mathbf{y}_{k-1}}{\mathbf{s}_{k-1}^T \mathbf{y}_{k-1}} \right) \frac{\mathbf{s}_{k-1} \mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{y}_{k-1}} - \frac{\mathbf{s}_{k-1} \mathbf{y}_{k-1}^T \mathbf{V}_{k-1} \mathbf{y}_{k-1} \mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{y}_{k-1}},$$

часто записываемая в виде

$$\mathbf{V}_k = \left(\mathbf{E} - \frac{\mathbf{s}_{k-1} \mathbf{y}_{k-1}^T}{\mathbf{y}_{k-1}^T \mathbf{s}_{k-1}} \right) \mathbf{V}_{k-1} \left(\mathbf{E} - \frac{\mathbf{y}_{k-1} \mathbf{s}_{k-1}^T}{\mathbf{y}_{k-1}^T \mathbf{s}_{k-1}} \right) + \frac{\mathbf{s}_{k-1} \mathbf{s}_{k-1}^T}{\mathbf{y}_{k-1}^T \mathbf{s}_{k-1}},$$

формула Девидона-Флетчера-Пауэлла (*Davidon-Fletcher-Powell — DFP*)

$$\mathbf{V}_k = \mathbf{V}_{k-1} + \frac{\mathbf{s}_{k-1} \mathbf{s}_{k-1}^T}{\mathbf{s}_{k-1}^T \mathbf{y}_{k-1}} - \frac{\mathbf{V}_{k-1} \mathbf{y}_{k-1} \mathbf{y}_{k-1}^T \mathbf{V}_{k-1}}{\mathbf{y}_{k-1}^T \mathbf{V}_{k-1} \mathbf{y}_{k-1}}$$

В качестве начального приближения в обеих формулах берется единичная матрица $\mathbf{B}_0 = \mathbf{E}$. Известно [1], что формула *BFGS* менее чувствительна к погрешностям вычислений, чем *DFP*. Матрица Гессе является плотной и может занимать много места в памяти. Метод *L-BFGS* [40] является модификацией метода *BFGS* для случаев, когда не удаётся разместить матрицу $\mathbf{V}$ в память. Для этого, в методе сохраняется история вычисления $\mathbf{V}$ в виде набора векторов $\mathbf{s}$ и $\mathbf{y}$, полученных в

некотором числе последних итераций. По этим векторам вычисляется приближение матрицы V .

В одношаговом алгоритме секущих (*One Step Secant — OSS*) [41] на каждом шаге обучения при вычислении матрицы, аппроксимирующей обратный Гесссиан, используется не приближение предыдущей итерации V_{k-1} , а единичная матрица. То есть на каждом шаге вычисления производятся с начального приближения. В результате исключаются операции с матрицами, и новое направление поиска вычисляется через векторы, определенные на предшествующем шаге. В результате число арифметических операций в *OSS* будет пропорционально числу весов сети n , а не n^2 , как в *BFGS*. При этом направления поиска получаются взаимно сопряженными. Алгоритм *OSS* требует меньших объемов памяти и вычислений, чем алгоритм *BFGS*, но больше, чем алгоритм сопряженных градиентов.

в) алгоритмы Левенберга-Марквардта и доверительных областей

Популярным квазиньютоновским методом обучения нейронных сетей является алгоритм (метод) Левенберга-Марквардта (*Levenberg-Marquardt method*) [1, 25], являющийся реализацией известного метода безусловной оптимизации [27–27]. В этом алгоритме функционал ошибки представляется в виде

$$L(\mathbf{w}) = \frac{1}{2} \sum_{i=1}^M [e_i(\mathbf{w})]^2 = \frac{1}{2} (\mathbf{e}, \mathbf{e}) = \frac{1}{2} \mathbf{e}^T \mathbf{e}, \quad (6.52)$$

где $e_i = y_i(\mathbf{w}) - t_i$, $y_i(\mathbf{w})$ — выход сети, t_i — целевое значение, $\mathbf{e}$ — вектор ошибок

$$\mathbf{e}(\mathbf{w}) = \begin{bmatrix} e_1(\mathbf{w}) \\ e_2(\mathbf{w}) \\ \dots \\ e_M(\mathbf{w}) \end{bmatrix}.$$

В формуле (6.52) использован один индекс ошибки. В случае последовательного обучения функционал ошибки вычисляется как сумма квадратов ошибок по всем выходам сети (6.14). Тогда в (6.52) e_i — ошибка i -ого выхода сети, а M — число нейронов выходного слоя. В случае пакетного обучения функционал ошибки вычисляется как сумма квадратов ошибок по всем выходам сети и по всем обучающим примерам (6.15). Тогда для применения (6.52) необходимо последовательно пронумеровать ошибки по всем примерам и выходам сети. В этом случае $M = Q \cdot N$, где Q — количество примеров обучающей выборки, N — число нейронов выходного слоя. Например, вектор ошибок сети можно представить в виде

$$\mathbf{e} = \left[e_1^{(1)} e_2^{(1)} \dots e_N^{(1)} \mid e_1^{(2)} e_2^{(2)} \dots e_N^{(2)} \mid \dots \mid e_1^{(Q)} e_2^{(Q)} \dots e_N^{(Q)} \right]^T, \quad (6.53)$$

где $e_i^{(q)}$ — ошибка нейрона i выходного слоя ($i = \overline{1, N}$, N — число нейронов в выходном слое) на примере q ($q = \overline{1, Q}$, если $Q = 1$ — последовательный режим обучения, если $Q > 1$ пакетный режим обучения).

Последовательно пронумеровав элементы вектора (6.53), получим нужное представление. Будем также использовать единый вектор весов, последовательно пронумеровав веса, начиная с первого слоя.

Изменим вектор весов на величину $\Delta \mathbf{w}$, приводящую функционал (6.52) к минимуму. Используем линейное приближение вектора ошибки

$$\mathbf{e}(\mathbf{w} + \Delta \mathbf{w}) = \mathbf{e}(\mathbf{w}) + \mathbf{J} \cdot \Delta \mathbf{w},$$

где $\mathbf{J}$ — матрица Якоби вектора $\mathbf{e}$ — матрица размером $M \times n$, где n — размерность вектора весов (количество весов в сети)

$$\mathbf{J}(\mathbf{w}) = \begin{bmatrix} \frac{\partial e_1}{\partial w_1} & \frac{\partial e_1}{\partial w_2} & \dots & \frac{\partial e_1}{\partial w_n} \\ \frac{\partial e_2}{\partial w_1} & \frac{\partial e_2}{\partial w_2} & \dots & \frac{\partial e_2}{\partial w_n} \\ \dots & \dots & \dots & \dots \\ \frac{\partial e_M}{\partial w_1} & \frac{\partial e_M}{\partial w_2} & \dots & \frac{\partial e_M}{\partial w_n} \end{bmatrix}$$

Функционал ошибки после изменения весов имеет вид

$$L(\mathbf{w} + \Delta \mathbf{w}) = \frac{1}{2}(\mathbf{e}(\mathbf{w}) + \mathbf{J}\Delta \mathbf{w}, \mathbf{e}(\mathbf{w}) + \mathbf{J}\Delta \mathbf{w}). \quad (6.54)$$

Используем необходимое условие минимума функционала (6.54)

$$\frac{\partial L(\mathbf{w} + \Delta \mathbf{w})}{\partial \Delta \mathbf{w}} = 0.$$

Опуская громоздкие, но несложные преобразования при вычислении производной по направлению $\Delta \mathbf{w}$, аналогичные вычислению производной (6.47), получаем

$$\mathbf{J}^T \mathbf{J} \Delta \mathbf{w} + \mathbf{J}^T \mathbf{e} = 0,$$

то есть поправка $\Delta \mathbf{w}$ определяется из решения системы линейных алгебраических уравнений

$$\mathbf{J}^T \mathbf{J} \Delta \mathbf{w} = -\mathbf{J}^T \mathbf{e}. \quad (6.55)$$

Из (6.52) видно, что компоненты вектора градиента равны $\frac{\partial L}{\partial w_j} = \sum_{i=1}^M \frac{\partial e_i}{\partial w_j} e_i$, а вектор

градиента равен $\mathbf{g} = \mathbf{J}^T \mathbf{e}$.

Тогда (6.55) можно записать в виде

$$\mathbf{J}^T \mathbf{J} \Delta \mathbf{w} = -\mathbf{g}. \quad (6.56)$$

Сравнивая уравнения (6.56) и (6.48), видно, что мы пришли к квазиньютоновскому методу, в котором использована аппроксимация Гессiana: $\mathbf{H} = \mathbf{J}^T \mathbf{J}$. Такой метод называется *методом Ньютона-Гаусса* [26], а аппроксимация Гессiana называется аппроксимацией Гаусса-Ньютона.

Так как число обусловленности матрицы $\mathbf{J}^T \mathbf{J}$ равно квадрату числа обусловленности матрицы $\mathbf{J}$, матрица $\mathbf{J}^T \mathbf{J}$ будет плохо обусловлена. Поэтому вводится параметр регуляризации μ (параметр Левенберга-Марквардта)

$$(\mathbf{J}^T \mathbf{J} + \mu \mathbf{E}) \Delta \mathbf{w} = -\mathbf{g},$$

где $\mathbf{E}$ — единичная матрица.

Так как используется приближенная аппроксимация Гессiana, то необходимо реализовать итерационный процесс обучения сети, на каждом шаге которого поправка вектора весов находится из решения системы линейных алгебраических уравнений

$$(\mathbf{J}_{k-1}^T \mathbf{J}_{k-1} + \mu_k \mathbf{E}) \Delta \mathbf{w}^{(k)} = -\mathbf{g}_{k-1}. \quad (6.57)$$

В формуле (6.57) нижние индексы у векторов и матриц означают, что они вычисляются по значению вектора весов, полученному на предыдущем шаге обучения. Причем параметр регуляризации изменяется на каждом шаге обучения.

В начале процесса обучения, когда вектор весов далек от оптимального, используется относительно большое значение μ . В этом случае Гессиан практически заменяется приближенным значением $\mathbf{H} \approx \mu \mathbf{E}$, а вектор поправки определяется методом градиентного спуска с малым шагом

$$\Delta \mathbf{w}^{(k)} = -\frac{1}{\mu_k} \mathbf{g}_{k-1}.$$

По мере уменьшения погрешности параметр μ уменьшается и метод приближается к методу Ньютона с аппроксимацией Гессiana $\mathbf{H} \approx \mathbf{J}^T \mathbf{J}$. Это обеспечивает высокую скорость сходимости, так как метод Ньютона вблизи минимума функционала ошибки имеет хорошую сходимость.

Алгоритм Левенберга-Марквардта требует малого числа шагов, но при большом количестве весов сети требует больших затрат памяти на хранение матриц и больших затрат времени на решение систем (6.57).

Матрица Якоби вычисляется на основе метода обратного распространения ошибки (мы рассматривали вычисление производных вида $\frac{\partial L}{\partial w_i}$, а здесь аналогично

надо найти производные вида $\frac{\partial e_j}{\partial w_i}$).

Можно показать [42], что метод Левенберга-Марквардта является вариантом *метода доверительных областей* (*trust-region methods*) [43]. Рассмотрим идею применения метода доверительных областей для обучения нейронных сетей [44].

Пусть необходимо минимизировать целевую функцию $f(\mathbf{x})$. Метод является итерационным. На каждой итерации для заданной области B_k , называемой *доверительной областью*, и начальной точки $\mathbf{x}_k \in B_k$ строится модель $m_k(\mathbf{x})$, аппроксимирующая функцию $f(\mathbf{x})$, причем $m_k(\mathbf{x}_k) = f(\mathbf{x}_k)$. Доверительная область B_k задается множеством точек

$$B_k = \{ \mathbf{x} \mid \|\mathbf{x} - \mathbf{x}_k\| \leq \Delta_k \},$$

где Δ_k — радиус доверительной области, $\|\cdot\|$ — произвольная норма вектора, наилучшим способом соответствующая решаемой задаче.

Тогда задача сводится к задаче минимизации m_k в области B_k . Полученный в результате ее решения шаг $\mathbf{s}_k$, такой, что $\|\mathbf{s}_k\| \leq \Delta_k$, и m_k принимает минимальное значение в точке $\mathbf{x}_k + \mathbf{s}_k$ для заданной области B_k , становится пробным шагом, минимизирующим f . Если значение $f(\mathbf{x}_k + \mathbf{s}_k) - f(\mathbf{x}_k)$ сопоставимо с $m_k(\mathbf{x}_k + \mathbf{s}_k) - m_k(\mathbf{x}_k)$, т.е. уменьшение, предсказанное моделью, подтверждено целевой функцией, то пробная точка становится начальной точкой для следующей итерации. В противном случае точка $\mathbf{x}_k + \mathbf{s}_k$ отклоняется, а доверительная область сужается в надежде на то, что модель будет точнее аппроксимировать целевую функцию в меньшей области. На следующей итерации выбирается модель и радиус доверительной области и проводится минимизация новой модели. Итерации продолжаются, пока не будет получен минимум функции f (по крайней мере — малое значение функции). Различные алгоритмы метода доверительных областей отличаются моделями, алгоритмами минимизации моделей и выбором радиуса доверительной области.

Если использовать в качестве модели разложение (6.34) и применить метод Ньютона, то метод доверительных областей будет отличаться от метода Ньютона только ограничением на шаг. Марквардт доказал [45], что эта задача эквивалентна классическому методу Ньютона (6.48), в котором матрица Гессе $\mathbf{H}$ заменена скорректированной матрицей $\mathbf{H} + \mu \mathbf{E}$, где μ — положительная константа, $\mathbf{E}$ — единичная матрица. Причем, если исходная матрица Гессе была знаконеопределена, то скорректированная матрица будет положительно определенной, что стабилизирует процесс поиска минимума. Если матрицу Гессе аппроксимировать с помощью матрицы Якоби: $\mathbf{H} = \mathbf{J}^T \mathbf{J}$, то получаем метод Левенберга-Марквардта. То есть можно сказать, что в методе Левенберга-Марквардта соотношение между шагом и радиусом доверительной окрестности регулируется параметром μ .

6.5.7. Регуляризация в обучении нейронных сетей

Для предотвращения проблемы переобучения применяются различные приемы. Модификации алгоритма обучения, предпринимаемые с целью уменьшения ошибки обобщения, не увеличивая ошибку обучения, называются *регуляризацией* [3]. Регуляризация накладывает некоторые ограничения на алгоритм обучения с целью предотвратить переобучение.

Практически наибольшее распространение получил прием, называемый *ранней остановкой*. При использовании ранней остановки в обучающем множестве выделяют обучающее множество (*Train Set*) и контрольное (валидационное) множество (*Validation Set*). Обучение сети производится на обучающем множестве, а на валидационном множестве производится непрерывная проверка ошибки. Непрерывность проверки ошибки означает, что после одного шага обучения сети (на пакете обучающих примеров в случае пакетного обучения или на одном примере в случае последовательного обучения) и вычисления ошибки на вход сети подается пакет (или один пример) из контрольного множества и вычисляется ошибка на контрольном множестве. Как только ошибка на валидационном множестве, которое не используется для обучения сети, перестает уменьшаться, это означает, что ошибка обобщения не уменьшается и необходимо остановить обучение. Ошибка сети, как правило, ведет себя нерегулярно, поэтому рекомендуется [3] прекращать обучение, когда ошибка не уменьшается на некотором небольшом числе шагов обучения.

Другой подход к регуляризации состоит в ограничении значений весов. Переобучение означает повышенную чувствительность сети к небольшим изменениям входных данных. Такая чувствительность обычно означает большие значения весов [3]. Поэтому другая идея регуляризации состоит во введении в функционал ошибки *штрафа* за рост абсолютных величин весов. Для этого в функционал ошибки добавляют *регуляризатор*, равной норме вектора весов. Обычно применяют два вида регуляризаторов:

- L_2 -регуляризатор, использующий вторую (евклидову) норму вектора весов

$$L_{L_2}(\mathbf{w}) = L_D(\mathbf{w}) + \lambda \|\mathbf{w}\|^2 = L_D(\mathbf{w}) + \lambda \sum_{i=1}^n w_i^2, \quad (6.58)$$

- L_1 -регуляризатор, использующий первую норму вектора весов

$$L_{L_1}(\mathbf{w}) = L_D(\mathbf{w}) + \lambda \|\mathbf{w}\|_1 = L_D(\mathbf{w}) + \lambda \sum_{i=1}^n |w_i|, \quad (6.59)$$

В (6.58) и (6.59) L_D — одна из функций потерь, например, сумма квадратов ошибок

$$L_D = \sum_{i=1}^M e_i^2, \quad (6.60)$$

где $e_i = t_i - y_i$ — ошибка i -го выхода сети, t_i — желаемая величина (целевое значение) i -го выхода сети, y_i — полученное значение выхода, λ — параметр регуляризации, n — общее число весов сети.

Рассмотрим влияние L_2 -регуляризатора на процесс обучения сети. Без использования регуляризатора вес w_i на $k+1$ -ом шаге обучения корректируется по формуле

$$w_i^{(k+1)} = w_i^{(k)} - \eta \frac{\partial L_D^{(k)}}{\partial w_i^{(k)}},$$

где η — подбираемая скорость обучения, $\frac{\partial L_D^{(k)}}{\partial w_i^{(k)}}$ — компонент градиента функционала (6.60), вычисленный при значениях весов на k -ом шаге обучения.

При использовании регуляризатора (6.58) веса корректируются по формуле

$$w_i^{(k+1)} = w_i^{(k)} - \eta \frac{\partial L_D^{(k)}}{\partial w_i^{(k)}} - 2\eta\lambda w_i^{(k)} = w_i^{(k)}(1 - 2\eta\lambda) - \eta \frac{\partial L_D^{(k)}}{\partial w_i^{(k)}}.$$

При использовании L_2 -регуляризатора вес сначала умножается на фактор затухания $(1 - 2\eta\lambda)$, а затем корректируется с помощью компонента вектора градиента. Нейронная сеть "забывает" значения весов на предыдущих шагах обучения. Только наиболее значимые и повторные обновления будут отражаться на весах.

При использовании L_1 -регуляризатора основной проблемой является то, что регуляризатор содержит абсолютные величины весов, которые не имеют производной при нулевом значении веса. В случае ненулевых весов w_{ij} можно непосредственно использовать обновление, вычисляемое с помощью частной производной. Дифференцируя функционал ошибки (6.59), можно определить формулу вычисления весов для случаев, когда w_{ij} отличны от нуля

$$w_{ij}^{(k+1)} = w_i^{(k)} - 2\eta\lambda s_i - \eta \frac{\partial L_D^{(k)}}{\partial w_i^{(k)}},$$

где s_i — частная производная от $|w_i|$ по w_i , равная

$$s_i = \begin{cases} -1, & \text{если } w_i < 0, \\ +1, & \text{если } w_i > 0. \end{cases}$$

Остается проблема определения s_i при $w_i = 0$. Практически из-за погрешностей вычислений строгое равенство нулю для w_i встречается крайне редко. Если значение веса в точности равно нулю, то можно опустить регуляризацию и положить $s_i = 0$.

Сравним L_2 -регуляризатор и L_1 -регуляризатор. Регуляризаторы отличаются механизмом затухания. В L_2 -регуляризаторе используется мультипликативное

затухание — вес умножается на фактор затухания. В L_1 -регуляризаторе используется аддитивное затухание — от значения веса вычитается фактор затухания. При использовании L_1 -регуляризатора веса могут становиться нулевыми, т. е. входные соответствующие входные сигналы не будут поступать на нейроны — сеть становится разреженной.

С точки зрения точности L_2 -регуляризатор обычно превосходит L_1 -регуляризатор, поэтому L_2 -регуляризатор применяется чаще. При больших количествах входов и элементов различия в производительности невелики.

Разновидностью регуляризации является добавление к входным данным шума — небольших нормально распределенных случайных чисел с нулевым математическим ожиданием. Можно показать, добавление шума в определенной степени эквивалентно применению L_2 -регуляризатора [3]. Покажем это на простейшем примере одного линейного нейрона, т.е. нейрона с линейной функцией активации. В этом случае матрица весов является вектором-строкой. Пусть на вход нейрона подан вектор признаков, к каждому компоненту которого добавлено нормально распределенное случайное число с нулевым математическим ожиданием и средним квадратическим отклонением σ , то есть вектор $\mathbf{x} + \sigma\boldsymbol{\varepsilon}$, где $\boldsymbol{\varepsilon}$ — вектор нормально распределенных случайных чисел с нулевым математическим ожиданием и единичным средним квадратическим отклонением. Выход линейного нейрона описывается выражением

$$y = \mathbf{w}(\mathbf{x} + \sigma\boldsymbol{\varepsilon}) = \mathbf{w}\mathbf{x} + \sigma\mathbf{w}\boldsymbol{\varepsilon}.$$

Рассмотрим математическое ожидание функционала ошибки при подаче на вход нейрона случайного вектора (математическое ожидание обозначим M)

$$\begin{aligned} M[(t - y)^2] &= M[(t - \mathbf{w}\mathbf{x} - \sigma\mathbf{w}\boldsymbol{\varepsilon})^2] = \\ &= M[(t - \mathbf{w}\mathbf{x})^2 - 2\sigma(t - \mathbf{w}\mathbf{x})\mathbf{w}\boldsymbol{\varepsilon} + \sigma^2(\mathbf{w}\boldsymbol{\varepsilon})^2] = \\ &= (t - \mathbf{w}\mathbf{x})^2 - 2\sigma(t - \mathbf{w}\mathbf{x}) \cdot \mathbf{w} \cdot M[\boldsymbol{\varepsilon}] + \sigma^2 \cdot M[(\mathbf{w}\boldsymbol{\varepsilon})^2]. \end{aligned}$$

По определению вектора $\boldsymbol{\varepsilon}$ имеем $M[\boldsymbol{\varepsilon}] = 0$. Можно показать, что

$$M[(\mathbf{w}\boldsymbol{\varepsilon})^2] = \sum_{i=1}^n w_i^2, \quad (6.61)$$

где n — число входов нейрона.

Покажем это на примере нейрона с двумя входами. Тогда

$$\begin{aligned} M[(\mathbf{w}\boldsymbol{\varepsilon})^2] &= M[(w_1\varepsilon_1 + w_2\varepsilon_2)^2] = M[w_1^2\varepsilon_1^2 + 2w_1w_2\varepsilon_1\varepsilon_2 + w_2^2\varepsilon_2^2] = \\ &= w_1^2 M[\varepsilon_1^2] + 2w_1w_2 M[\varepsilon_1\varepsilon_2] + w_2^2 M[\varepsilon_2^2]. \end{aligned}$$

Математическое ожидание произведения двух случайных величин равно

$$M[\varepsilon_1 \cdot \varepsilon_2] = M[\varepsilon_1] \cdot M[\varepsilon_2] + \text{cov}(\varepsilon_1, \varepsilon_2),$$

где $\text{cov}(\varepsilon_1, \varepsilon_2)$ — ковариация (корреляционный момент).

Тогда

$$M[\varepsilon_1^2] = M[\varepsilon_1 \cdot \varepsilon_1] = M[\varepsilon_1] \cdot M[\varepsilon_1] + \text{cov}(\varepsilon_1, \varepsilon_1) = 1,$$

так как $M[\varepsilon_1] = 0$, $\text{cov}(\varepsilon_1, \varepsilon_1) = 1$, так как случайная величина коррелирована с собой. Аналогично $M[\varepsilon_2^2] = 1$, $M[\varepsilon_1 \varepsilon_2] = 0$, так как ε_1 и ε_2 некоррелированные величины. Таким образом (6.61) справедливо.

Таким образом

$$M[(t - y)^2] = (t - \mathbf{w}\mathbf{x})^2 + \sigma^2 \sum_{i=1}^n w_i^2.$$

То есть усредненное значение функционала ошибки при использовании зашумленных примеров эквивалентно применению L_2 -регуляризатора.

В случае многослойных сетей с нелинейными функциями активации полной эквивалентности между применением шума и L_2 -регуляризатора нет. Но положительный эффект наблюдается. Рекомендуется [3] обучать несколько сетей одинаковой архитектуры с использованием зашумленных данных, а при использовании сети усреднять результат. Но это весьма затратно, поэтому на практике просто обучают сеть на зашумленных данных.

При обучении глубоких нейронных сетей широко используется относительно новый и мощный метод регуляризации — *dropout* (исключение, отключение, прореживание) [3, 6, 46]. В этом методе для каждого нейрона (кроме нейронов выходного слоя) устанавливается вероятность p , с которой нейрон будет исключен из сети. Эта вероятность называется *уровнем отключения (dropout rate)*. На практике часто принимают $p = 0,5$. Алгоритм обучения строится следующим образом: на каждой эпохе обучения (для глубоких сетей эпоха — это, как правило, настройка параметров сети на примерах одного мини-пакета) для каждого нейрона определяется вероятность удаления. При применении обученной сети (например, при классификации одного примера) участвуют все нейроны, но выход каждого нейрона умножается на *вероятность сохранения* $1 - p$, с которой нейрон оставляли при обучении [6]. Умножение выходных сигналов нейронов на вероятность сохранения объясняется тем, что при применении обученной сети каждый нейрон получает входные сигналы от всех нейронов предыдущего слоя, а не от их части, как вовремя обучения сети. Известен подход, когда при применении сети выход каждого нейрона делится на вероятность сохранения [6]. Эффект от применения *dropout* объясняется тем, что на каждом шаге (на каждом мини-пакете) используется сеть со своей архитектурой. То есть обучается ансамбль сетей. Применение обученной с использованием *dropout* сети эквивалентно усреднению по ансамблю сетей [46], а применение ансамбля моделей лучше применения одной модели. В

сетях глубокой архитектуры обычно процедуру dropout реализуют как отдельный слой нейронной сети, следующий за слоем нейронов, и удаляющий с некоторой вероятностью связи нейронов со следующим слоем.

6.6. Пакетная нормализация

В процессе обучения нейронной сети статистическое распределение входных сигналов каждого слоя изменяется, так как вход каждого слоя зависит от параметров предшествующих слоев, а в процессе обучения изменяются веса предшествующих слоев сети. Изменение распределения входов слоя приводит к изменению выходов адаптивных сумматоров. При этом сигмоидальные функции активации могут войти в насыщение. Насыщение функции активации приводит к очень малому значению градиента, что приводит к замедлению или полному прекращению обучения градиентными методами. Такой эффект носит название *внутренний ковариационный сдвиг (internal covariate shift)* [47]. Особенно сильно такой эффект наблюдается на более глубоких слоях сети, даже небольшие изменения на входе нейронной сети сильно влияют на последующие слои. Одним из способов решения проблемы внутреннего ковариационного сдвига является применение в качестве функции активации различных разновидностей функции ReLU. Другой подход, называемый *пакетной нормализацией (batch normalization)* [48], состоит в поддержании распределения входов функций активации в пределах зоны ненасыщения. Известно [49], что процесс обучения сходится быстрее, если входные данные сети нормализованы (имеют нулевое математическое ожидание и единичную дисперсию). Идея пакетной нормализации состоит в использовании нормализации, связанной с каждым слоем.

Рассмотрим пакетную нормализацию [3, 6, 50]. В нейронную сеть добавляются слои нормализации. Возможны два способа расположения таких слоев.

1. Каждый слой сети содержит взвешенный сумматор и функции активации. Слой нормализации располагается после слоя сети (после функции активации). Это так называемая *постактивационная нормализация* [3].

2. Каждый слой разделяется на два слоя: слой взвешенных сумматоров и слой функций активации. Слой нормализации располагается между ними. Это *преактивационная нормализация*.

В [48] показано, что преактивационная нормализация лучше, чем постактивационная. Поэтому ограничимся преактивационной нормализацией. Пусть сеть обучается методом градиентного спуска с использованием мини-пакетов. Рассмотрим i -ый элемент (узел) BN_i слоя пакетной нормализации. Каждый BN_i связан с выходом одного адаптивного сумматора слоя адаптивных сумматоров. То есть на вход BN_i поступает число, являющееся выходом i -го адаптивного сумматора. При использовании пакетной нормализации рассматриваются все

примеры мини-пакета. Пусть $v_i^{(r)}$ — вход BN_i , формируемый при подаче r -го примера пакета на вход сети. Пусть пакет состоит из m примеров, а $v_i^{(1)}, v_i^{(2)}, \dots, v_i^{(m)}$ — входы BN_i , соответствующие примерам пакета. Для каждого элемента BN_i слоя нормализации вычисляются среднее значение μ_i и стандартное отклонение σ_i по всем элементам пакета:

$$\begin{aligned}\mu_i &= \frac{\sum_{r=1}^m v_i^{(r)}}{m}, \\ \sigma_i^2 &= \frac{\sum_{r=1}^m (v_i^{(r)} - \mu_i)^2}{m}.\end{aligned}\quad \forall i, \quad (6.62)$$

Входы для каждого примера пакета нормализуются

$$\hat{v}_i^{(r)} = \frac{v_i^{(r)} - \mu_i}{\sqrt{\sigma_i^2 + \varepsilon}}, \quad \forall i, r. \quad (6.63)$$

Маленькое значение ε добавлено к дисперсии σ_i^2 , чтобы исключить деление на ноль. Следует отметить, что нормированные входы слоя пакетной нормализации зависят от статистических свойств признаков примеров пакета.

Если сразу подать нормализованные значения (6.63) на входы слоя функций активации, то в случае сигмоидальных функций активации значения входов будут поступать на практически линейный участок функции активации. Это практически устраняет главное достоинство нейронных сетей — нелинейность нейронов. В случае функции ReLU на выходе функции активации будет нулевое значение. Поэтому после нормализации значения (6.63) масштабируются и смещаются. Тогда выходы слоя пакетной нормализации для каждого примера пакета описываются формулой

$$a_i^{(r)} = \gamma_i \hat{v}_i^{(r)} + \beta_i, \quad \forall i, r. \quad (6.64)$$

Параметры β_i, γ_i настраиваются в процессе обучения сети на каждом мини-пакете. В процессе настройки будут изменяться и входы $v_i^{(1)}, v_i^{(2)}, \dots, v_i^{(m)}$ каждого слоя пакетной нормализации. Для реализации градиентного спуска необходимо вычислять вектор градиента функционала ошибки E . Компоненты вектора градиента представляют собой частные производные функционала ошибки по весам сети, параметрам β_i, γ_i и входам слоев нормализации $v_i^{(1)}, v_i^{(2)}, \dots, v_i^{(m)}$. Частные производные по весам вычисляются методом обратного распространения ошибки так же, как и в сети без пакетной нормализации. Поэтому рассмотрим только настройку параметров β_i, γ_i и входов слоя нормализации $v_i^{(1)}, v_i^{(2)}, \dots, v_i^{(m)}$

[3]. Следует учесть, что при использовании современных пакетов реализации нейронных сетей, например, TensorFlow, не нужно вручную рассчитывать производные. Пакеты реализуют автоматическое дифференцирование, используя графы вычислений производных сети.

Предположим, что обратное распространение ошибки уже выполнено вплоть до выхода узла BN_i . Поэтому имеются все производные $\frac{\partial E}{\partial a_i^{(r)}}$ функционала ошибки по выходам слоя нормализации. Тогда можно вычислить производные по параметрам β_i, γ_i :

$$\frac{\partial L}{\partial \beta_i} = \sum_{r=1}^m \frac{\partial L}{\partial a_i^{(r)}} \cdot \frac{\partial a_i^{(r)}}{\partial \beta_i}, \quad (6.65)$$

$$\frac{\partial L}{\partial \gamma_i} = \sum_{r=1}^m \frac{\partial L}{\partial a_i^{(r)}} \cdot \frac{\partial a_i^{(r)}}{\partial \gamma_i} = \sum_{r=1}^m \frac{\partial L}{\partial a_i^{(r)}} \cdot \hat{v}_i^{(r)}. \quad (6.66)$$

В (6.65) и (6.66) при вычислении производных от $a_i^{(r)}$ учитывается (6.64).

При вычислении $\frac{\partial E}{\partial v_i^{(r)}}$ учтем зависимость v_i от $v_i^{(r)}$, μ_i и σ_i как следует из (6.63)

$$v_i^{(r)} = \mu_i + \hat{v}_i^{(r)} \sqrt{\sigma_i^2 + \varepsilon}.$$

Важно, что μ_i и σ_i не являются константами, а настраиваются для *каждого* мини-пакета. Поэтому их нельзя рассчитывать по всей обучающей выборке. Используя правило дифференцирования функции многих переменных, получаем

$$\begin{aligned} \frac{\partial L}{\partial v_i^{(r)}} &= \frac{\partial L}{\partial \hat{v}_i^{(r)}} \cdot \frac{\partial \hat{v}_i^{(r)}}{\partial v_i^{(r)}} + \frac{\partial L}{\partial \mu_i} \cdot \frac{\partial \mu_i}{\partial v_i^{(r)}} + \frac{\partial L}{\partial \sigma_i^2} \cdot \frac{\partial \sigma_i^2}{\partial v_i^{(r)}} = \\ &= \frac{\partial L}{\partial \hat{v}_i^{(r)}} \left(\frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} \right) + \frac{\partial L}{\partial \mu_i} \left(\frac{1}{m} \right) + \frac{\partial L}{\partial \sigma_i^2} \left[\frac{2(v_i^{(r)} - \mu_i)}{m} \right]. \end{aligned} \quad (6.67)$$

Из (6.64) получаем

$$\frac{\partial L}{\partial \hat{v}_i^{(r)}} = \frac{\partial L}{\partial a_i^{(r)}} \cdot \frac{\partial a_i^{(r)}}{\partial \hat{v}_i^{(r)}} = \gamma_i \frac{\partial L}{\partial a_i^{(r)}}. \quad (6.68)$$

Вычислим производную функционала ошибки по дисперсии, учитывая, что функционал ошибки зависит от всех $\hat{v}_i^{(r)}$. Учтем также (6.63) и (6.68)

$$\begin{aligned}
\frac{\partial L}{\partial \sigma_i^2} &= \sum_{q=1}^m \frac{\partial L}{\partial \hat{v}_i^{(q)}} \cdot \frac{\partial \hat{v}_i^{(q)}}{\partial \sigma_i^2} = -\frac{1}{2\left(\sqrt{\sigma_i^2 + \varepsilon}\right)^3} \sum_{q=1}^m \frac{\partial L}{\partial \hat{v}_i^{(q)}} \left(v_i^{(q)} - \mu_i\right) = \\
&= -\frac{\gamma_i}{2\left(\sqrt{\sigma_i^2 + \varepsilon}\right)^3} \sum_{q=1}^m \frac{\partial L}{\partial a_i^{(q)}} \left(v_i^{(q)} - \mu_i\right).
\end{aligned} \tag{6.69}$$

Вычислим производную функционала ошибки по среднему, учитывая, что функционал ошибки зависит от всех $\hat{v}_i^{(r)}$, а σ_i^2 зависит только от μ_i . Учтем также (6.63), (6.68) и (6.69)

$$\begin{aligned}
\frac{\partial L}{\partial \mu_i} &= \sum_{q=1}^m \frac{\partial L}{\partial \hat{v}_i^{(q)}} \cdot \frac{\partial \hat{v}_i^{(q)}}{\partial \mu_i} + \frac{\partial L}{\partial \sigma_i^2} \cdot \frac{\partial \sigma_i^2}{\partial \mu_i} = -\frac{1}{\sqrt{\sigma_i^2 + \varepsilon}} \sum_{q=1}^m \frac{\partial L}{\partial \hat{v}_i^{(q)}} - 2 \frac{\partial L}{\partial \sigma_i^2} \cdot \frac{\sum_{q=1}^m \left(v_i^{(q)} - \mu_i\right)}{m} = \\
&= -\frac{\gamma_i}{\sqrt{\sigma_i^2 + \varepsilon}} \sum_{q=1}^m \frac{\partial L}{\partial a_i^{(q)}} + \frac{\gamma_i}{\left(\sqrt{\sigma_i^2 + \varepsilon}\right)^3} \left[\sum_{q=1}^m \frac{\partial L}{\partial a_i^{(q)}} \left(v_i^{(q)} - \mu_i\right) \right] \cdot \left[\frac{\sum_{q=1}^m \left(v_i^{(q)} - \mu_i\right)}{m} \right].
\end{aligned} \tag{6.70}$$

Подставляя в (6.67)–(6.70), получаем выражение для $\frac{\partial L}{\partial v_i^{(r)}}$ (производные по входам слоя пакетной нормализации) в терминах $\frac{\partial L}{\partial a_i^{(r)}}$ (производные по значениям выходов слоя пакетной нормализации). Настроив слой пакетной нормализации, можно настроить предшествующий слой и т.д.

На этапе тестирования, когда обученная нейронная сеть используется для предсказания по одному входному вектору, значения μ и σ^2 должны быть рассчитаны по *всему набору обучающих данных* и считаются постоянными. Набор обучающих данных может быть очень большим. В этом случае μ и σ^2 можно взять, как средние значения по всем пакетам (для всех пакетов μ и σ^2 уже подсчитаны).

Пакетная нормализация играет роль регуляризатора [3], так как один и тот же пример в разных пакетах при использовании пакетной нормализации по-разному влияет на обучение сети. Это эквивалентно добавлению шума к обучающим данным. А мы уже отмечали, что добавление небольшого шума в обучающие данные играет роль регуляризатора.

Литература к разделу 6

1. Осовский С. Нейронные сети для обработки информации. 2-е изд., перераб. и доп. — М.: Горячая линия-Телеком, 2016. — 448 с.
2. Хайкин С. Нейронные сети: полный курс. — М.: Вильямс, 2006. — 1104 с.
3. Аггарвал Ч. Нейронные сети и глубокое обучение: учебный курс. — СПб.: ООО "Диалектика", 2020. — 752 с.
4. Abbot M. Computational Neuroscience. — Clanrye International, 2018. — 261 p.
5. Neuromorphic engineering. [Электронный ресурс]. URL: https://en.wikipedia.org/wiki/Neuromorphic_engineering (дата обращения: 17.10.2023).
6. Жерон О. Прикладное машинное обучение с помощью Scikit Learn, Keras и TensorFlow: концепции, инструменты, и техники для создания интеллектуальных систем. — СПб.: ООО "Диалектика", 2020. — 1040 с.
7. Glorot X., Bordes A., Bengio Y. Deep Sparse Rectifier Neural Networks // Proceedings of the 14th International Conference on Artificial Intelligence and Statistics, 2011, Vol. 15. — P. 315–323.
8. CS231 n Convolutional Neural Networks for Visual Recognition. [Электронный ресурс]. URL: <http://cs231n.github.io/neural-networks-1/#actfun> (дата обращения: 17.10.2023).
9. Incorporating second-order functional knowledge for better option pricing / C. Dugas, Y. Bengio, F. Belisle, C. Nadeau, R. Garcia // Advances in Neural Information Processing Systems 13 (NIPS'00), MIT Press, 2001. — P. 472–478.
10. Maas A. L., Hannun A. Y., Ng A. Y. Rectifier nonlinearities improve neural network acoustic models // Proceedings of the 30-th International Conference on Machine Learning, 2013. [Электронный ресурс]. URL: https://ai.stanford.edu/~amaas/papers/relu_hybrid_icml2013_final.pdf (дата обращения: 17.10.2023).
11. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification / K. He, X. Zhang, S. Ren, J. Sun. // Proceeding ICCV'15 Proceedings of the 2015 IEEE International Conference on Computer Vision (ICCV). — 2015. — P. 1026–1034.
12. Empirical Evaluation of Rectified Activations in Convolutional Network / B. Xu, N. Wang, T. Chen, M. Li. [Электронный ресурс]. URL: <http://arxiv.org/pdf/1505.00853v2.pdf> (дата обращения: 17.10.2022).
13. Clevert D., Unterthiner T., Hochreiter S. Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs) // [Электронный ресурс]. URL: <http://arxiv.org/abs/1511.07289> (дата обращения: 17.10.2023).

14. Self-Normalizing neural networks / G. Klambauer, T. Unterthiner, A. May, S. Hochreiter // [Электронный ресурс]. URL: <https://arxiv.org/abs/1706.02515v5> (дата обращения: 17.10.2023).
15. Hendrycks D., Gimpel K. Gaussian Error Linear Units (GELUs) // URL: <https://arxiv.org/abs/1606.08415> (дата обращения: 17.10.2023).
16. Google Brain Team. [Электронный ресурс]. URL: <https://research.google/teams/brain/> (дата обращения: 17.10.2023).
17. Ramachandran P., Zoph B., Le Q. V. Swish: A Self-Gated Activation Function. [Электронный ресурс]. URL: <https://arxiv.org/abs/1710.05941/> (дата обращения: 17.10.2022).
18. Misra D. Mish: A Self Regularized Non-Monotonic Activation Function. [Электронный ресурс]. URL: <https://arxiv.org/abs/1908.08681> (дата обращения: 17.10.2023).
19. Nguyen D., Widrow B. Improving the learning speed of 2-layer neural networks by choosing initial values of the adaptive weights // 1990 IJCNN International Joint Conference on Neural Networks. — 1990, Vol. 3. — P. 21–26.
20. Glorot X., Bengio Y. Understanding the Difficulty of Training Deep Feedforward Neural Networks // International conference on artificial intelligence and statistics, 2010. — P. 249–256.
21. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification / K. He, X. Zhang, S. Ren, J. Sun. // Proceeding ICCV'15 Proceedings of the 2015 IEEE International Conference on Computer Vision (ICCV). — 2015. — P. 1026–1034.
22. Стэнфордский курс: лекция 4. Введение в нейронные сети. [Электронный ресурс]. URL: <https://www.reg.ru/blog/stehnfordskij-kurs-lekciya-4-vvedenie-v-nejronnye-seti/> (дата обращения: 17.10.2023).
23. Automatic Differentiation in Machine Learning: a Survey / Bavdin A. G., Pearlmutter B. A., Radul A. A., Siskind J. M. // Journal of Machine Learning Research, 2018, Vol. 18. — No. 1. — P. 1–43.
24. Шолле Ф. Глубокое обучение на Python. — СПб.: Питер, 2023. — 576 с.
25. Кохендерфер М. Дж., Уилер Т. А. Алгоритмы оптимизации. — СПб.: ООО "Диалектика", 2020. — 528 с.
26. Гилл Ф., Мюррей У., Райт М. Практическая оптимизация. — М.: Мир, 1985. — 509 с.
27. Дэннис Дж., Шнабель Р. Численные методы безусловной оптимизации и решения нелинейные уравнений. — М.: Мир, 1988. — 440 с.
28. Пантелеев А. В., Летова Т. А. Методы оптимизации. Практический курс. — М.: Логос, 2011. — 424 с.

29. Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение. — М.: ДМК Пресс, 2018. — 652 с.
30. Нестеров Ю. Е. Методы выпуклой оптимизации. — М.: Издательство МЦНМО, 2010. — 281 с.
31. Kingma D., Ba J. Adam: A method for stochastic optimization. [Электронный ресурс]. URL: <https://arxiv.org/abs/1412.6980> (дата обращения: 15.15.2022).
32. Горбаченко В. И. Вычислительная линейная алгебра с примерами на MATLAB. — СПб.: БХВ-Петербург, 2011. — 320 с.
33. Nocedal J., Wright S. Numerical Optimization. — Springer, 2006. — 664 p.
34. Fletcher R., C.M. Reeves C. M. Function minimization by conjugate gradients // Computer Journal, 1964, Vol. 7. — P. 149–154.
35. Polak E., Ribière G. Note sur la convergence de méthodes de directions conjuguées // Revue française d'informatique et de recherche opérationnelle, série rouge, 1969, tome 3, n° 1. — P. 35–43.
36. Hestenes M. R., Stiefel E. Methods of conjugate gradients for solving linear systems // Journal of Research of the National Bureau of Standards, 1952, Vol. 49. — No. 6. — P. 409–436.
37. Charalambous C. Conjugate gradient algorithm for efficient training of artificial neural networks // Circuits, Devices and Systems, IEE Proceedings G, 1992, Vol. 139. — No 3. — P. 301–310.
38. Moller M. F. A scaled conjugate gradient algorithm for fast supervised learning // Neural Networks, 1993, Vol. 6. — Issue 4. — P. 525–533.
39. Фаддеев Д. К., Фаддеева В. Н. Вычислительные методы линейной алгебры. — СПб.: "Лань", 2002. — 736 с.
40. Liu D. C., Nocedal J. On the Limited Memory Method for Large Scale Optimization // Mathematical Programming. Series B, 1989, Vol. 45 (3). — P. 503–528.
41. Battiti R. First and second order methods for learning: Between steepest descent and Newton's method // Neural Computation, 1992, Vol. 4. — Issue 2. — P. 141–166.
42. Гладких Б. А. Методы оптимизации и исследование операций для бакалавров информатики. Ч. II. Нелинейное и динамическое программирование. — Томск: Изд-во НТЛ, 2011. — 264 с.
43. Conn A. R., Gould N. I. M., Toint P. L. Trust regions methods. — Society for Industrial and Applied Mathematics, 2000. — 959 p.
44. Горбаченко В. И., Жуков М. В. Решение краевых задач математической физики с помощью сетей радиальных базисных функций // Журнал вычислительной математики и математической физики. — 2017, том 57. — № 1. — С. 133–143.

45. Marquardt D. W. An Algorithm for Least-Squares Estimation of Nonlinear Parameters // Journal of the Society for Industrial and Applied Mathematics. — 1963, Vol. 11. — No. 2. — P. 431–441.
46. Dropout: A Simple Way to Prevent Neural Networks from Overfitting / N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov // Journal of Machine Learning Research. — 2014, Vol. 15. — No. 1. — P. 1929–1958.
47. Shimodaira H. Improving predictive inference under covariate shift by weighting the log-likelihood function // Journal of statistical planning and inference, 2000, Vol. 90. — No 2. — P. 227–244.
48. Ioffe S., Szegedy C. Batch normalization: Accelerating deep network training by reducing internal covariate shift // arXiv preprint arXiv:1502.03167 (2015).
49. Efficient backprop / Y. LeCun, L. Bottou, G. B. Orr, K.-R. Müller // Neural networks: Tricks of the trade. — Springer. — P. 9–48 p.
50. Паттанаяк С. Глубокое обучение и TensorFlow для профессионалов. Математический подход к построению систем искусственного интеллекта на Python. — СПб.: ООО "Диалектика", 2019. — 480 с.